ZERO TO EXPERT · INTERVIEW READY

The Complete Postman Book

APIs & HTTP · Scripting & Testing · Automation & CI/CD · Security · Troubleshooting · Enterprise Practices · 300+ Interview Questions

Covers Postman v10 – v12 · Code verified with Newman 6.2.3
Edition: October 2026

Contents

About This Book

- 🧭 How to use this book
- 🗓️ Suggested 3-week study plan (add week 4 for Ch 21–26 if you work on large/enterprise projects)
- ✅ Accuracy policy

Chapter 1 — Foundations: APIs, HTTP, and the Web (Before You Touch Postman)

- 1.1 What is an API? (The 5-year-old version)
- 1.2 Types of APIs
- 1.3 Client–Server Model
- 1.4 What happens when you hit "Send" (end to end)
- 1.5 Anatomy of a URL
- 1.6 HTTP Methods (Verbs) — In Depth
- 1.7 HTTP Status Codes — Complete Guide
- 1.8 HTTP Headers — In Depth
- 1.9 Content Types (MIME Types)
- 1.10 JSON — In Depth
- 1.11 XML Basics (for SOAP and legacy APIs)
- 1.12 REST — Principles In Depth
- 1.13 REST vs SOAP vs GraphQL vs gRPC (comparison table — interview favourite)
- 1.14 Authentication vs Authorization
- 1.15 Cookies and Sessions
- 1.16 CORS (Cross-Origin Resource Sharing)
- 1.17 Caching basics
- 1.18 HTTP versions
- 1.19 HTTPS, TLS and Certificates (so SSL errors never scare you)
- 1.20 Where Postman fits in the Software Development Life Cycle
- 1.21 Chapter 1 Quick Revision

Chapter 2 — Getting Started with Postman

- 2.1 What is Postman?
- 2.2 Postman versions — a short history (good for interviews and for reading old tutorials)
- 2.3 Postman plans (as of 2026)
- 2.4 Installing Postman
- 2.5 Signing in, accounts and teams
- 2.6 The Postman Interface — A Guided Tour
- 2.7 Workspaces
- 2.8 Settings you should configure on day 1
- 2.9 Keyboard shortcuts (most useful)
- 2.10 Your First Request — Step by Step
- 2.11 What you can import into Postman
- 2.12 History, Trash, and recovering lost work
- 2.13 Chapter 2 Quick Revision

Chapter 3 — Building and Sending Requests in Depth

- 3.1 The URL bar
- 3.2 Query Parameters
- 3.3 Path Variables
- 3.4 Headers
- 3.5 Request Body — All Types In Depth
- 3.6 Request Settings tab (per request)
- 3.7 Understanding the Response
- 3.8 Cookies in Postman
- 3.9 Code Snippets
- 3.10 Sending and Downloading
- 3.11 Capturing requests: Proxy and Interceptor
- 3.12 Working with files, encodings and special cases
- 3.13 A complete CRUD walkthrough (Restful-Booker)
- 3.14 Chapter 3 Quick Revision

Chapter 4 — Collections and Folders

- 4.1 What is a collection?
- 4.2 Creating and organising
- 4.3 Collection-level settings (inherited by everything inside)
- 4.4 Documentation in collections
- 4.5 Examples (Saved responses) in depth
- 4.6 Sharing collections
- 4.7 Collection file formats
- 4.8 Importing / exporting best practices
- 4.9 Forks, Pull Requests, Merges (collection version control inside Postman)
- 4.10 Collection runs order and control (preview)
- 4.11 Common collection problems & fixes
- 4.12 Chapter 4 Quick Revision

Chapter 5 — Variables and Environments (The Heart of Real-World Postman)

5.1 Why variables matter
5.2 Syntax
5.3 Variable scopes (the 5 + Vault)
5.4 Values: Shared vs Local (formerly Initial vs Current)
5.5 Variable types: default vs secret
5.6 Environments in depth
5.7 Collection variables vs environment variables (common interview question)
5.8 Global variables — when (not) to use
5.9 Data variables (data-driven testing preview)
5.10 Local variables
5.11 Postman Vault (secrets)
5.12 Dynamic variables (built-in random data)
5.13 Scripting API for every scope
5.14 Variables in the Runner, Newman and CI
5.15 Debugging variables (systematic checklist)
5.16 Naming conventions (company standard)
5.17 Chapter 5 Quick Revision

Chapter 6 — Authorization & Authentication in Postman (Every Type, In Depth)

6.1 The Authorization tab
6.2 API Key
6.3 Bearer Token
6.4 JWT — what it is and how Postman handles it
6.5 Basic Auth
6.6 Digest Auth
6.7 OAuth 1.0
6.8 OAuth 2.0 — The Big One
6.9 Hawk Authentication
6.10 AWS Signature (SigV4)
6.11 NTLM
6.12 Akamai EdgeGrid & ASAP
6.13 Automating login & token refresh (THE pattern every company uses)
6.14 Cookie/session-based auth and CSRF tokens
6.15 Mutual TLS (client certificates)
6.16 Auth testing checklist (QA & security)
6.17 Chapter 6 Quick Revision

Chapter 7 — Scripting in Postman: The Complete `pm` API

7.1 Where scripts live
7.2 Execution order (memorise this!)
7.3 The Sandbox — what scripts can and can't do
7.4 `pm.info` — where am I?
7.5 `pm.request` — read and modify the request (pre-request) or inspect it (post-response)
7.6 `pm.response` — the response (post-response scripts)
7.7 Variables API (recap — full details in Chapter 5)
7.8 `pm.sendRequest` — call other APIs from a script
7.9 `pm.execution` — control the run
7.10 `pm.cookies`
7.11 `pm.test` and `pm.expect` (testing — full chapter next)
7.12 `pm.visualizer` — custom visual responses
7.13 Console logging (debugging)
7.14 Error handling inside scripts
7.15 Built-in libraries available via `require()`
7.16 Package Library and `pm.require` (reusable code)
7.17 Legacy script syntax (you WILL see this in old company collections)
7.18 Script snippets panel
7.19 Practical script recipes
7.20 Script quality rules (company standards)
7.21 Chapter 7 Quick Revision

Chapter 8 — Testing and Assertions in Depth

8.1 Anatomy of a Postman test
8.2 Postman's response assertions (`pm.response.to.*`)
8.3 Chai `pm.expect` — the complete toolkit
8.4 The essential test suite for any endpoint
8.5 JSON Schema validation (contract testing)
8.6 Testing arrays, lists, sorting, pagination
8.7 Testing headers, cookies, and security headers
8.8 Negative testing patterns
8.9 Response time & performance assertions
8.10 Testing across requests (stateful assertions)
8.11 Comparing responses / snapshot testing
8.12 Tests in collection/folder scripts (DRY)
8.13 Asynchronous tests
8.14 Skipping and conditional tests
8.15 Postbot & AI-assisted tests
8.16 Test design principles (what seniors know)
8.17 Common testing bugs and fixes
8.18 Chapter 8 Quick Revision

Chapter 9 — Request Chaining and Workflows

9.1 Basic chaining with variables
9.2 Default execution order & controlling it
9.3 Workflow patterns
9.4 Infinite loops — how they happen and how to prevent
9.5 Why is `setNextRequest` "not working"? (Troubleshooting)
9.6 End-to-end workflow example: E-commerce checkout
9.7 Setup & teardown strategy
9.8 Postman Flows (visual workflows) — brief
9.9 Chapter 9 Quick Revision

Chapter 10 — Collection Runner, Data-Driven Testing & Performance Testing

10.1 Opening the Runner
10.2 Run modes
10.3 Runner configuration (functional mode)
10.4 Run results
10.5 Data-driven testing (DDT)
10.6 Running a folder with different data than the rest
10.7 Common Runner problems
10.8 Performance testing in Postman
10.9 Scheduled collection runs (cloud)
10.10 Chapter 10 Quick Revision

Chapter 11 — Newman, Postman CLI and CI/CD

11.1 Why run collections from the command line?
11.2 Newman — Installation
11.3 Newman — Running collections
11.4 Newman as a Node.js library
11.5 Newman in Docker
11.6 Postman CLI
11.7 CI/CD integration examples
11.8 Organising Postman assets in a Git repo (export-based approach)
11.9 Newman vs Postman app differences (bugs that only show up in CI)
11.10 Chapter 11 Quick Revision

Chapter 12 — Mock Servers, Monitors, API Design (Spec Hub), Documentation, Flows, Postman API, Insights & AI

PART A — MOCK SERVERS
PART B — MONITORS
PART C — API DESIGN: SPEC HUB / API BUILDER
PART D — DOCUMENTATION
PART E — POSTMAN FLOWS
PART F — POSTMAN API (programmatic control)
PART G — INSIGHTS, API CATALOG, API NETWORK
PART H — AI IN POSTMAN (Postbot & Agent Mode)
12.29 Chapter 12 Quick Revision

Chapter 13 — Beyond REST: GraphQL, gRPC, WebSocket, Socket.IO, MQTT, SOAP, SSE, JSON-RPC, MCP & AI Requests

13.1 GraphQL
13.2 gRPC
13.3 WebSocket
13.4 Socket.IO
13.5 MQTT (IoT)
13.6 SOAP
13.7 Server-Sent Events (SSE)
13.8 JSON-RPC
13.9 MCP (Model Context Protocol) requests
13.10 AI (LLM) requests
13.11 Chapter 13 Quick Revision

Chapter 14 — Collaboration, Version Control & Organising Large Projects

14.1 Collaboration features
14.2 Version control options
14.3 Workspace strategy for a big organisation
14.4 Naming conventions
14.5 Collection design standards (template for a team "Postman Guidelines" doc)
14.6 Environment management at scale
14.7 Onboarding a new team member (checklist)
14.8 Handling common team situations
14.9 Governance (Enterprise-scale)
14.10 Chapter 14 Quick Revision

Chapter 15 — API Security Testing with Postman

15.1 OWASP API Security Top 10 (2023) — with Postman tests
15.2 Injection testing with data files
15.3 Security regression suite structure
15.4 Postman's own security hygiene
15.5 Postman vs dedicated security tools
15.6 Chapter 15 Quick Revision

Chapter 16 — Debugging & Troubleshooting: Every Common Error and How to Fix It

16.1 The debugging method (use for EVERY problem)
16.2 "Could not send request" / network errors
16.3 SSL/TLS errors
16.4 Proxy problems
16.5 Status-code-driven debugging
16.6 Response is OK, but something's wrong
16.7 Script errors
16.8 Variable problems (quick recap)
16.9 Cookies & session problems
16.10 "It works in Postman but not in my app" (and the reverse)
16.11 Runner, Newman & CI problems
16.12 OAuth problems
16.13 Postman app problems
16.14 Writing a great bug report from Postman
16.15 Chapter 16 Quick Revision

Chapter 17 — Organisational Best Practices & Situational Playbook

17.1 The 40 golden rules (print this)
17.2 Situational playbook — "What do I do if…?"
17.3 Postman vs other tools (know when to use what)
17.4 A maturity model for API testing with Postman

Chapter 18A — Interview Questions & Answers: Fundamentals (APIs, HTTP, Postman Basics)

Part 1 — API & HTTP concepts
Part 2 — Postman basics

Chapter 18B — Interview Questions & Answers: Scripting, Testing, Automation, CI/CD & Advanced Features

Part 3 — Scripts & the `pm` object
Part 4 — Testing & assertions
Part 5 — Chaining, Runner, data-driven
Part 6 — Newman, Postman CLI & CI/CD
Part 7 — Mocks, monitors, docs, design, API, AI

Chapter 18C — Scenario-Based, Live-Coding, Architecture & Behavioural Interview Questions

Part 8 — Scenario-based questions
Part 9 — Live-coding questions (write the script)
Part 10 — Architecture / senior questions
Part 11 — Behavioural & HR-style (API/QA roles)
Part 12 — Rapid-fire (one-liners)

Chapter 18D — Researched Interview Patterns, Company-Tagged Questions & Verified Gotchas

18D.1 What interviewers ask most (recurring themes across sources)
18D.2 Company-tagged questions (tagged by prep sites, unverified)
18D.3 Scenario questions that recur across sources (full model answers)
18D.4 Verified gotchas (from running this book's code in Newman 6.2.3)
18D.5 Mock interview script (60 minutes)
Sources consulted for 18D (prep sites; quality varies)

Chapter 19 — Cheat Sheets, Snippet Library & Glossary

19.1 `pm` API cheat sheet
19.2 Snippet library (copy-paste ready)
19.3 Newman / Postman CLI cheat sheet
19.4 Status codes cheat sheet
19.5 Variable precedence & values
19.6 Script execution order
19.7 Glossary (A–Z)
19.8 Exam-night revision list (top 30 facts)

Chapter 20 — Hands-On Labs & Capstone Project

Lab 1 — First requests (🟢 15 min)
Lab 2 — Bodies (🟢 20 min)
Lab 3 — Status codes & redirects (🟢 15 min)
Lab 4 — Auth types (🟡 30 min)
Lab 5 — Environments & variables (🟡 30 min)
Lab 6 — Tests (🟡 40 min)
Lab 7 — Chaining & dynamic data (🟡 40 min) — Restful-Booker
Lab 8 — Runner & data-driven (🟡 30 min)
Lab 9 — Workflows (🔴 40 min)
Lab 10 — Pre-request magic (🔴 30 min)
Lab 11 — Mock server (🟡 30 min)
Lab 12 — Newman & CI (🔴 45 min)
Lab 13 — OpenAPI import (🟡 20 min)
Lab 14 — Other protocols (🟡 30 min)
Lab 15 — Debugging drills (🔴 30 min)
🎓 Capstone Project — "Restful-Booker Regression Suite" (production-grade)

Chapter 21 — Native Git & the Postman v12 Workflow (Deep Dive)

21.1 Why Native Git exists
21.2 What gets created on disk
21.3 Local View vs Cloud View
21.4 The daily workflow
21.5 CLI commands you'll use
21.6 Monorepos
21.7 Pull, conflicts, and mixed situations
21.8 Migrating an existing team to Native Git (playbook)
21.9 Interview questions on Native Git

Chapter 22 — Postman Flows & FQL (Deep Dive)

22.1 When to use Flows (and when not)
22.2 Building blocks
22.3 FQL: Flows Query Language
22.4 Worked example: failed-payment alert flow
22.5 Flows as MCP servers (AI agents)
22.6 Debugging flows
22.7 Interview questions on Flows

Chapter 23 — Enterprise Administration, API Governance, Integrations & the VS Code Extension

PART A — Identity & access
PART B — API governance (style guides as code)
PART C — Integrations
PART D — The Postman VS Code extension
23.12 Interview questions (admin & governance)

Chapter 24 — Contract Testing with Pact (and How It Fits with Postman)

24.1 Two styles of contract testing
24.2 How Pact works
24.3 Working example (Pact JS, verified)
24.4 Pact concepts to know for interviews
24.5 Where Postman fits in a contract-testing strategy
24.6 Interview questions

Chapter 25 — Extending Newman: Custom Reporters & Programmatic Runs

25.1 When you need a custom reporter
25.2 How Newman finds a reporter
25.3 A complete reporter (verified)
25.4 Useful emitter events & data
25.5 Programmatic Newman patterns (recap and extensions)
25.6 Interview questions

Chapter 26 — Migration Guides, Certifications & Career Growth

PART A — Migrating to (and from) Postman
PART B — Certifications & learning paths
PART C — Career growth with Postman skills

Chapter 27 — Verification Report: What Was Tested, and What Changed Because of It

27.1 Environment
27.2 What was executed
27.3 Documented but NOT executed (needs your environment)
27.4 Corrections made to the book because of verification
27.5 Re-run it yourself

About This Book

A full, book-length guide to Postman, written for **total beginners** ("explain like I'm 5" analogies) that goes all the way to **senior or lead level** (CI/CD, governance, security, large-team practices). Interview preparation is built in, and the code has been executed and checked (see Chapter 27).

> **Version note (October 2026):** Covers Postman v10 → v11 → **v12**: Native Git with YAML Collection v3, Agent Mode, API Catalog. It also covers the March 2026 plan change to Free (single user), Solo, Team and Enterprise. Where Postman renamed something, the book gives both names. For example, "Tests" became "Post-response", and "Initial/Current value" became "Shared/Local value". That way you can work with any version your company uses.

🧭 How to use this book

If you are a beginner (0–1 year)

1. Read Chapters **1 → 5** in order and do **Labs 1–6** (Chapter 20).
2. Then Chapters **6–8** with **Labs 7–8**.
3. Revise with **18A** and the **Chapter 19** cheat sheet.

If you have experience (2+ years) and want interview-ready depth

1. Skim Chapters 1–5 (the version notes in Ch. 2 and 5 matter: v11/v12 changes).
2. Deep-read **7, 8, 9, 11, 14, 16, 17**.
3. Practise **18B + 18C** out loud, and solve all live-coding questions without looking.
4. Build the **Capstone** and talk about it in interviews.

If you are working on a large company project right now

- Setting standards → **Chapter 14 + 17**
- Secrets & environments → **Chapter 5 (5.11) + 15.4**
- CI/CD → **Chapter 11**
- Something is broken → **Chapter 16**
- Need a code snippet fast → **Chapter 19.2**

🗓️ Suggested 3-week study plan (add week 4 for Ch 21–26 if you work on large/enterprise projects)

Week	Days 1–2	Days 3–4	Days 5–6	Day 7
1	Ch 1–2	Ch 3–4	Ch 5 + Labs 1–5	18A revision
2	Ch 6	Ch 7	Ch 8–9 + Labs 6–10	18B revision
3	Ch 10–11 + Lab 12	Ch 12–13	Ch 14–17	18C + Capstone + mock interview

---

✅ Accuracy policy

- **The code was executed, not just written.** The book's scripts were run in Newman 6.2.3 against a local stand-in API: 180/180 assertions pass. The Pact, custom-reporter, Spectral and FQL examples were also run, and all 12 diagrams were rendered. Running them led to 11 corrections, listed in Chapter 27. You can re-run everything from `verification/` (folder `postman-book/verification/` in the repository).
- Facts come from Postman's official documentation and widely used, established behaviour. Recent changes (v11/v12, the 2026 plans, Vault limits, Native Git) were checked against current Postman docs and coverage.
- Postman releases updates often. Plan limits, prices, UI labels and new features can change. Where something is version- or plan-dependent, the text says so. Check the Postman Learning Center or release notes for your exact version before committing to numbers in a company proposal.

Chapter 1 — Foundations: APIs, HTTP, and the Web (Before You Touch Postman)

> **Why this chapter exists:** Postman is a tool for talking to APIs. If you don't deeply understand *what* an API is and *how* HTTP works, Postman will feel like a box of random buttons. Every interviewer — from fresher level to architect level — starts with these basics. Master this chapter and half of every Postman interview is already won.

1.1 What is an API? (The 5-year-old version)

Imagine you are in a **restaurant**.

- **You** (the customer) want food.
- **The kitchen** makes the food.
- You are **not allowed** to walk into the kitchen.
- So there is a **waiter**. You tell the waiter what you want, the waiter tells the kitchen, the kitchen makes it, and the waiter brings it back.

In software:

Restaurant	Software
You (customer)	The **client** (a mobile app, a website, Postman)
Kitchen	The **server** (where data and logic live)
Waiter	The **API**
The menu	The **API documentation** (what you are allowed to ask for)
Your order ("1 pizza, no onions")	The **request**
The food that comes back	The **response**

API = Application Programming Interface. It is a set of rules that lets one program ask another program for something, without knowing how that other program works inside.

The grown-up definition

An API is a **contract** between a provider and a consumer. It defines:

1. **What** you can ask for (endpoints / operations),
2. **How** to ask (method, URL, headers, body format),
3. **What** you will get back (response structure, status codes),
4. **Who** is allowed to ask (authentication / authorization),
5. **How much** you can ask (rate limits, quotas).

Real examples

- When a weather app on your phone shows "28°C in Bengaluru", it called a weather API.
- When you click "Login with Google", the website calls Google's OAuth APIs.
- When you pay with UPI, multiple banking APIs talk to each other.

1.2 Types of APIs

By who can use them

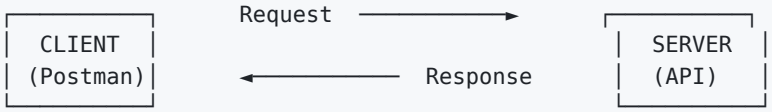
Type	Meaning	Example
Public / Open API	Anyone can use (sometimes with a free key)	GitHub API, OpenWeatherMap
Partner API	Shared only with business partners	A payment gateway API given to merchants
Internal / Private API	Used only inside a company	Order-service calling inventory-service
Composite API	One call that combines many calls	"Get user + orders + cart" in one go

By architectural style / protocol

Style	Data format	Transport	Key idea
REST	Usually JSON (also XML, etc.)	HTTP	Resources (nouns) + HTTP methods (verbs)
SOAP	XML only (an "envelope")	Usually HTTP, can be SMTP etc.	Strict contract (WSDL), WS-Security
GraphQL	JSON	HTTP (usually one POST endpoint)	Client asks for exactly the fields it needs
gRPC	Protocol Buffers (binary)	HTTP/2	Fast, strongly typed, service methods defined in `.proto` files
WebSocket	Any (text/binary frames)	Starts as HTTP, upgrades to WS	Two-way, long-lived connection (chat, live prices)
Server-Sent Events (SSE)	Text events	HTTP	Server pushes a stream to client, one-way
MQTT	Binary, lightweight	TCP	Publish/subscribe, used in IoT
Webhooks	Usually JSON	HTTP	"Don't call us, we'll call you" — server calls *your* URL when something happens

Postman supports all of these (HTTP/REST, SOAP, GraphQL, gRPC, WebSocket, Socket.IO, MQTT, SSE via HTTP, and newer AI/MCP request types). We cover each in Chapter 13.

1.3 Client–Server Model

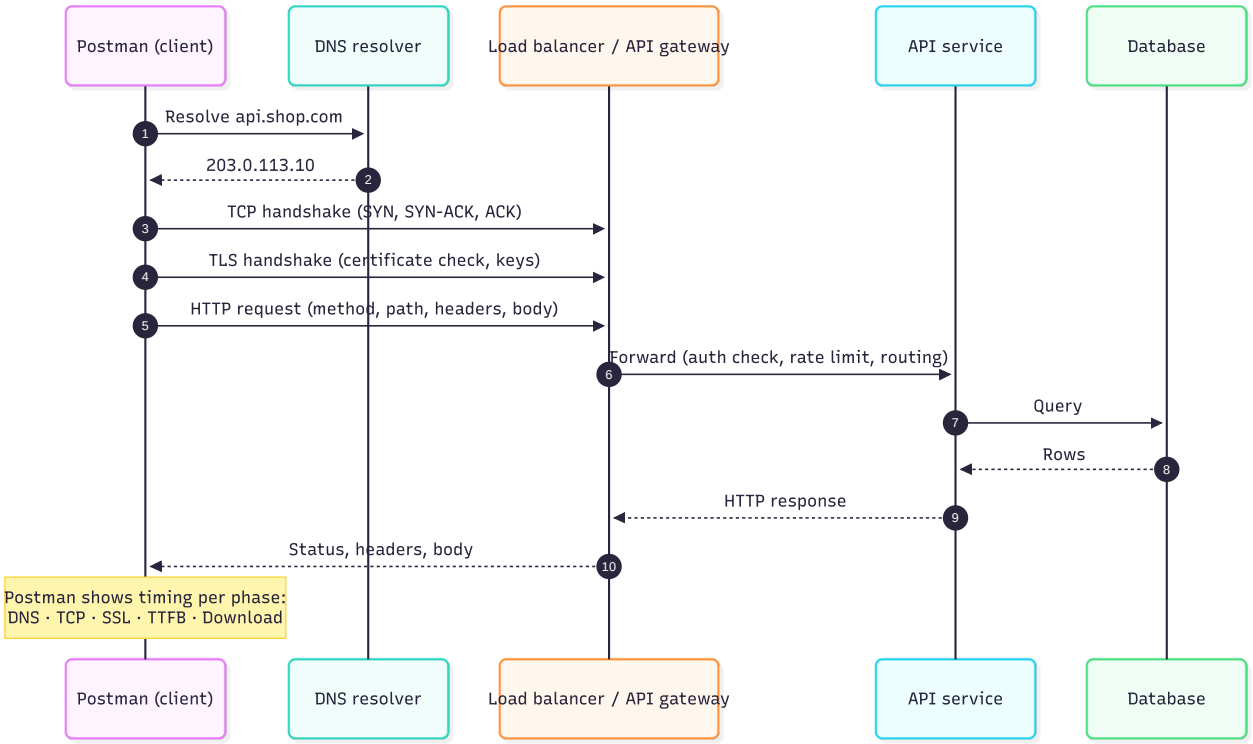


- The **client always starts** the conversation in HTTP (request–response). The server never "calls" the client in plain HTTP (that's why webhooks, WebSockets, and SSE exist).
- HTTP is **stateless**: each request is independent. The server does **not** remember the previous request by itself. That's why we send tokens/cookies **every time** — to remind the server who we are.

> 🧒 **5-year-old version of stateless:** The server is like a goldfish. Every time you talk to it, it forgets you. So every time, you must show your ID card (token).

1.4 What happens when you hit "Send" (end to end)

This is a **favourite interview question**: *"What happens when you type a URL and press Enter / Send?"*



1. **URL parsing** — the client splits `https://api.shop.com:443/v1/orders?status=paid` into scheme, host, port, path, query.

2. **DNS lookup** — `api.shop.com` is converted to an IP address (e.g. `203.0.113.10`). Checks: local cache → OS cache → router → ISP resolver → root → TLD (`.com`) → authoritative name server.
3. **TCP connection** — 3-way handshake: `SYN` → `SYN-ACK` → `ACK`.
4. **TLS handshake** (only for HTTPS) — the client and server agree on encryption, the server shows its **certificate**, the client verifies it against trusted Certificate Authorities (CAs), and session keys are created.
5. **HTTP request sent** — method, path, headers, body.
6. **Server processing** — load balancer → API gateway → application → database → back.
7. **HTTP response** — status line, headers, body.
8. **Connection reuse or close** — `Connection: keep-alive` lets the same TCP connection be reused.

Postman shows you these phases! Hover over the response **time** in Postman's response pane and you see a breakdown: *Prepare, Socket Initialization, DNS Lookup, TCP Handshake, SSL Handshake, Waiting (TTFB), Download, Process*. This is hugely useful when debugging "why is my API slow?" (Chapter 16).

1.5 Anatomy of a URL

```
https://user:pass@api.shop.com:8443/v1/users/42/orders?status=paid&page=2#summary
└──┘   └──────┘ └──────────┘ └──┘└──────────────────┘└──────────────────┘└─────┘
scheme  userinfo     host     port        path              query        fragment
```

Part	Meaning	Notes
Scheme	`http` or `https`	Default ports: 80 for http, 443 for https
Userinfo	`user:pass@`	Deprecated/insecure; avoid.
Host	Domain or IP	`localhost`, `127.0.0.1`, `api.shop.com`
Port	Network door number	Omit if default
Path	Which resource	`/v1/users/42/orders`
Path parameter	A variable piece of the path	`42` is the user ID. In Postman written as `/users/:userId`
Query parameters	Key-value filters after `?`, joined by `&`	`status=paid&page=2`
Fragment	After `#`	**Never sent to the server.** Only used by browsers.

URL encoding (percent-encoding)

URLs can only safely contain certain characters. Others must be encoded:

Character	Encoded
space	`%20` (or `+` in form/query encoding)
`&`	`%26`
`=`	`%3D`
`?`	`%3F`
`/`	`%2F`
`#`	`%23`
`@`	`%40`

Example bug: You search for `name=Tom & Jerry`. Without encoding, the server sees `name=Tom` and a weird second param `Jerry`. Encoded: `name=Tom%20%26%20Jerry`. In Postman, select text in the URL, right-click → **EncodeURIComponent**; in scripts use `encodeURIComponent()`.

1.6 HTTP Methods (Verbs) — In Depth

> 🧒 **Toy box analogy:** Your toy box is the server. **GET** = look at a toy. **POST** = put a new toy in. **PUT** = replace a toy with a new one. **PATCH** = fix one part of a toy (change its colour only). **DELETE** = throw a toy away.

Method	Purpose	Has body?	Safe?	Idempotent?	Cacheable?
GET	Read a resource	Should not (semantics undefined)	✅	✅	✅
POST	Create / trigger processing	✅	❌	❌	Only with explicit freshness info (rare)
PUT	Replace a resource completely (or create at a known URL)	✅	❌	✅	❌
PATCH	Partial update	✅	❌	❌ (not guaranteed)	❌
DELETE	Remove a resource	Optional	❌	✅	❌
HEAD	Same as GET but **no body** in response (only headers)	❌	✅	✅	✅
OPTIONS	Ask what methods/features are allowed (used in CORS preflight)	Optional	✅	✅	❌

Method	Purpose	Has body?	Safe?	Idempotent?	Cacheable?
TRACE	Echo back the request (debugging; usually disabled for security)	❌	✅	✅	❌
CONNECT	Create a tunnel (used by proxies for HTTPS)	—	❌	❌	❌

Key definitions (interviewers LOVE these)

- **Safe** = the method does not change anything on the server (read-only by intent). GET, HEAD, OPTIONS, TRACE.
- **Idempotent** = doing it **once or 10 times** leaves the server in the **same state**.
  - `DELETE /users/5` → first call deletes, next calls find nothing; the **state** is the same (user 5 gone). The **status code can differ** (200/204 first, then 404) — idempotency is about server state, *not* identical responses.
  - `PUT /users/5 {name:"Asha"}` 10 times → still name = Asha.
  - `POST /users {name:"Asha"}` 10 times → 10 users created. **Not idempotent.**
  - `PATCH /counter {op:"increment"}` → not idempotent. `PATCH /users/5 {name:"Asha"}` happens to be idempotent, but PATCH *in general* is not guaranteed.
- **Cacheable** = the response may be stored and reused.

PUT vs PATCH vs POST (classic interview question)

Existing user:

```
{ "id": 5, "name": "Asha", "email": "asha@x.com", "city": "Pune" }
```

PUT /users/5 with body `{ "name": "Asha K" }` → Strict REST meaning: **replace the whole resource**. `email` and `city` may become null/removed. (Many real APIs don't follow this strictly — always check docs.)

PATCH /users/5 with body `{ "name": "Asha K" }` → Only `name` changes. Email and city remain.

POST /users with body `{ "name": "Ravi" }` → Creates a **new** user. Server decides the ID. Typically returns **201 Created** with a `Location: /users/6` header.

Can GET have a body?

Technically HTTP doesn't forbid it, but RFC 9110 says a body in GET has **no defined semantics**, and many servers, proxies and caches drop or reject it. Postman **allows** you to send a GET with a body (some APIs like Elasticsearch use it), but it's not good practice for your own APIs.

1.7 HTTP Status Codes — Complete Guide

> 🧒 Status codes are like **traffic signals and faces**: 2xx = 😀 "All good", 3xx = 👉 "Go over there", 4xx = 🤦 "**You** made a mistake", 5xx = 🤒 "**I** (the server) am sick".

1xx — Informational

Code	Name	Meaning
100	Continue	Server got the headers; client may send the body (used with `Expect: 100-continue`)
101	Switching Protocols	e.g. upgrading HTTP → WebSocket
102	Processing	(WebDAV) Still working on it
103	Early Hints	Preload hints before the final response

2xx — Success

Code	Name	When to use
200	OK	Generic success, body included
201	Created	New resource created (POST/PUT). Should include `Location` header
202	Accepted	Request accepted for **async** processing; not done yet
203	Non-Authoritative Information	Modified by a proxy
204	No Content	Success, **no body** (common for DELETE / PUT)
205	Reset Content	Tell client to reset the form
206	Partial Content	Range requests (video streaming, resumable downloads)

3xx — Redirection

Code	Name	Notes
300	Multiple Choices	
301	Moved Permanently	Browsers/clients may change POST to GET when following
302	Found	Temporary; historically clients change POST→GET
303	See Other	Explicitly "go GET this other URL" (after a POST)
304	Not Modified	Cached version is still valid (used with `If-None-Match` / `ETag`, `If-Modified-Since`)

Code	Name	Notes
307	Temporary Redirect	Like 302 but **method and body must be preserved**
308	Permanent Redirect	Like 301 but **method and body must be preserved**

> **Postman setting:** "Automatically follow redirects" (Settings → General, and per-request in the request's **Settings** tab). Turn it **off** to see the real 3xx response and its `Location` header — very useful for testing login flows.

4xx — Client Errors (the most-tested group)

Code	Name	Typical cause	How to fix (from Postman)
400	Bad Request	Malformed JSON, missing required field, validation failed	Check body syntax, `Content-Type`, required fields
401	Unauthorized	**Not authenticated**: missing/expired/invalid token	Add/refresh token, check Authorization tab
402	Payment Required	Reserved; some APIs use for billing	—
403	Forbidden	**Authenticated but not allowed** (no permission/role, IP blocked)	Check user role/scopes, API key permissions
404	Not Found	Wrong URL/path, resource doesn't exist, (sometimes used to hide 403)	Check base URL, path variables, environment selected
405	Method Not Allowed	Using POST on a GET-only endpoint	Check method dropdown; response should include `Allow` header
406	Not Acceptable	Server can't produce format in `Accept` header	Fix `Accept` header
407	Proxy Authentication Required	Corporate proxy needs credentials	Postman Settings → Proxy
408	Request Timeout	Client took too long to send request	Network, big uploads
409	Conflict	Duplicate (e.g. email already exists), version conflict	Use unique data (dynamic variables!)
410	Gone	Permanently removed	
411	Length Required	Missing `Content-Length`	
412	Precondition Failed	`If-Match` ETag didn't match (optimistic locking)	
413	Content Too Large (Payload Too Large)	Body/file too big	Reduce size, check server limits
414	URI Too Long	Too many query params	Move data to body (POST)

Code	Name	Typical cause	How to fix (from Postman)
415	Unsupported Media Type	Wrong/missing `Content-Type` (e.g. sent text, server expects JSON)	In Postman raw body, choose **JSON** from dropdown
416	Range Not Satisfiable	Bad `Range` header	
418	I'm a teapot	Joke code (RFC 2324), sometimes used playfully	
422	Unprocessable Content (Entity)	Syntax OK but semantic validation failed	Fix field values
423/424	Locked / Failed Dependency	WebDAV	
425	Too Early		
426	Upgrade Required		
428	Precondition Required	Server requires `If-Match`	
429	Too Many Requests	Rate limit hit. Check `Retry-After` header	Add delay in Runner, back-off logic
431	Request Header Fields Too Large	Huge cookies/headers	Clear cookies in Postman's cookie manager
451	Unavailable For Legal Reasons		

5xx — Server Errors

Code	Name	Meaning
500	Internal Server Error	Unhandled exception in server code (a bug!)
501	Not Implemented	Server doesn't support the functionality
502	Bad Gateway	Proxy/gateway got an invalid response from upstream
503	Service Unavailable	Server overloaded/maintenance; may include `Retry-After`
504	Gateway Timeout	Proxy/gateway didn't get a response from upstream in time
505	HTTP Version Not Supported	
507	Insufficient Storage	
511	Network Authentication Required	Captive portals (hotel Wi-Fi)

401 vs 403 — the #1 asked difference

- **401** = "**Who are you?**" → you didn't prove identity (no token, wrong token, expired token). Response typically includes `WWW-Authenticate` header.
- **403** = "**I know who you are, but you can't do this.**" → you're logged in as a normal user trying to access admin endpoint.

400 vs 422

- **400** = request is broken (invalid JSON, wrong types, can't parse).
- **422** = request is well-formed, but business rules fail (e.g. `"age": -5`). (Many APIs just use 400 for both. Always follow your API's contract.)

502 vs 503 vs 504

- **502** — gateway reached upstream but got garbage/crash.
- **503** — service is intentionally unavailable / overloaded.
- **504** — gateway waited and upstream never answered in time.

1.8 HTTP Headers — In Depth

Headers are **metadata**: "information about the information". Like the label on a parcel (sender, weight, fragile) versus the item inside (body).

Request headers you must know

Header	Example	Purpose
`Host`	`api.shop.com`	Which website on the server (mandatory in HTTP/1.1). Postman adds automatically
`Content-Type`	`application/json`	Format of **the body you are sending**
`Accept`	`application/json`	Format you **want back**
`Authorization`	`Bearer eyJhbGci...`	Credentials
`User-Agent`	`PostmanRuntime/7.x.x`	Who is calling. Postman sends `PostmanRuntime/...` by default
`Accept-Encoding`	`gzip, deflate, br`	Compression the client supports
`Content-Length`	`348`	Body size in bytes (Postman calculates automatically)
`Cookie`	`session=abc123`	Cookies sent to server
`Cache-Control`	`no-cache`	Caching directives
`If-None-Match`	`"etag-value"`	Conditional GET (304 if unchanged)
`If-Match`	`"etag-value"`	Optimistic locking on updates
`Origin`	`https://myapp.com`	Used in CORS
`Referer`	`https://myapp.com/page`	Page that made the request (yes, misspelled in the spec!)

Header	Example	Purpose
`X-Request-ID` / `X-Correlation-ID`	`uuid`	Trace a request across microservices — great practice to set with `{{$guid}}`
`Idempotency-Key`	`uuid`	Makes POST safe to retry (Stripe-style payments)
`X-API-Key`	`abcd1234`	Custom API key header (common convention)
`Connection`	`keep-alive`	Reuse TCP connection

Response headers you must know

Header	Purpose
`Content-Type`	Format of the response body (`application/json; charset=utf-8`)
`Content-Length`	Size of response body
`Set-Cookie`	Server asks client to store a cookie
`Location`	URL of created resource (201) or redirect target (3xx)
`Cache-Control` , `ETag` , `Expires` , `Last-Modified`	Caching
`WWW-Authenticate`	How to authenticate (with 401)
`Retry-After`	When to retry (429/503)
`X-RateLimit-Limit` , `X-RateLimit-Remaining` , `X-RateLimit-Reset`	Common (non-standard) rate limit info. Newer standard drafts use `RateLimit` / `RateLimit-Policy`
`Access-Control-Allow-Origin` etc.	CORS
`Strict-Transport-Security`	Force HTTPS (HSTS)
`Content-Security-Policy` , `X-Content-Type-Options: nosniff` , `X-Frame-Options`	Security headers
`Server` , `X-Powered-By`	Server info (security best practice: hide these)

Header facts interviewers check

- Header **names are case-insensitive** (`content-type` = `Content-Type`). In HTTP/2 they are sent lowercase. Postman's `pm.response.headers.get()` is case-insensitive.
- Header **values** may be case-sensitive (e.g. tokens).
- Custom headers historically started with `X-` ; RFC 6648 (2012) deprecated that convention, but it's still widely used.
- Postman auto-adds some headers (**hidden headers**): `Postman-Token` , `Host` , `User-Agent` , `Accept` , `Accept-Encoding` , `Connection` , `Content-Length` , `Content-Type` (based on body). Click "hidden" in the Headers tab to see/disable them.

1.9 Content Types (MIME Types)

Content-Type	Used for	Postman body option
`application/json`	JSON APIs (most common)	raw → JSON
`application/xml` / `text/xml`	XML, SOAP 1.1 uses `text/xml`	raw → XML
`application/soap+xml`	SOAP 1.2	raw → XML + header
`application/x-www-form-urlencoded`	HTML form submits: `a=1&b=2`	x-www-form-urlencoded
`multipart/form-data; boundary=...`	File uploads + fields	form-data
`text/plain`	Plain text	raw → Text
`text/html`	Web pages	raw → HTML
`application/octet-stream`	Raw binary	binary
`application/graphql` / `application/json`	GraphQL (usually JSON envelope)	GraphQL
`application/javascript`	JS	raw → JavaScript
`image/png`, `application/pdf`, etc.	Files	binary

> **Very common bug:** You send JSON but forgot to choose "JSON" in Postman's raw dropdown, so Postman sends `Content-Type: text/plain`. Server says **415 Unsupported Media Type** or treats body as empty → **400**. Fix: select **raw → JSON**.

1.10 JSON — In Depth

JSON (JavaScript Object Notation) is the language most APIs speak.

Data types in JSON (exactly 6 kinds of values)

Type	Example
String	`"hello"` (always **double** quotes)
Number	`42`, `3.14`, `-7`, `1e5` (no `NaN`, no `Infinity`, no leading zeros like `07`)
Boolean	`true`, `false` (lowercase)
Null	`null`
Object	`{ "key": "value" }`
Array	`[1, 2, 3]`

Example

```
{
  "id": 101,
  "name": "Asha",
  "isActive": true,
  "manager": null,
  "skills": ["Postman", "SQL"],
  "address": {
    "city": "Pune",
    "pin": "411001"
  },
  "projects": [
    { "id": 1, "title": "Payments" },
    { "id": 2, "title": "Search" }
  ]
}
```

JSON rules that cause real bugs

1. **No trailing commas**: `{"a":1,}` ❌
2. **Keys must be double-quoted strings**: `{name:"x"}` ❌, `{'name':'x'}` ❌
3. **No comments in standard JSON** (`//` or `/* */`). (Postman's editor may tolerate/strip them in some versions, but servers usually don't — don't rely on it.)
4. Numbers too big for JavaScript (`> 2^53 - 1 = 9007199254740991`) lose precision in Postman scripts when parsed with `pm.response.json()`. Example: `{"id": 12345678901234567890}` parses to `12345678901234567000`. Fix: ask API to send IDs as strings, or parse the raw text carefully (regex / a BigInt-aware approach).

Navigating JSON in Postman scripts (preview of Chapter 7)

```
const body = pm.response.json();
body.name;                  // "Asha"
body.skills[0];             // "Postman"
body.address.city;          // "Pune"
body.projects[1].title;     // "Search"
body.projects.length;       // 2
body["address"]["pin"];     // "411001" (bracket notation — needed for keys with spaces/dashes)
```

1.11 XML Basics (for SOAP and legacy APIs)

```
<?xml version="1.0" encoding="UTF-8"?>
<user id="101">
  <name>Asha</name>
  <skills>
    <skill>Postman</skill>
    <skill>SQL</skill>
  </skills>
</user>
```

- Elements (`<name>`), attributes (`id="101"`), must be properly closed and nested, case-sensitive.
- In Postman scripts, use `xml2Json(responseBody)` (legacy global, still available) or the `xml2js` module to convert XML to a JS object (Chapter 7).

1.12 REST — Principles In Depth

REST = **RE**presentational **S**tate **T**ransfer, defined by Roy Fielding (2000 dissertation). It is an **architectural style**, not a protocol or standard.

The 6 constraints

1. **Client–Server** — separation of concerns.
2. **Stateless** — each request contains all info needed; no session stored on server between requests (tokens carry state).
3. **Cacheable** — responses must say whether they can be cached.
4. **Uniform Interface** — resources identified by URIs, manipulated via representations, self-descriptive messages, and **HATEOAS** (Hypermedia As The Engine Of Application State: responses include links to next actions).
5. **Layered System** — client can't tell if it's talking to the actual server or a proxy/load balancer/gateway.
6. **Code on Demand** (optional) — server can send executable code (e.g. JavaScript).

REST resource naming best practices

✅ Good	❌ Bad	Why
`GET /users`	`GET /getAllUsers`	Use nouns; method is the verb
`GET /users/42`	`GET /user?id=42` (acceptable but less RESTful)	Identity in path
`GET /users/42/orders`	`GET /ordersOfUser/42`	Hierarchy for relationships
`POST /users`	`POST /users/create`	Method implies create
`GET /users?status=active&sort=-createdAt&page=2&limit=20`	—	Filtering, sorting, pagination via query
`/v1/users`	—	Versioning
plural nouns, lowercase, hyphens: `/order-items`	`/Order_Items`	Consistency

API versioning strategies

- **URI path**: `/v1/users` (most common, easy to test in Postman with `{{baseUrl}}/v1`)
- **Query param**: `/users?version=1`

- **Header**: `Accept: application/vnd.shop.v1+json` or `X-API-Version: 1`

Pagination styles (you'll test these a lot)

Style	Example	Notes
Offset/limit	`?offset=40&limit=20`	Simple; slow on huge tables; duplicates when data shifts
Page/size	`?page=3&size=20`	Same as offset
Cursor	`?cursor=eyJpZCI6MTAwfQ&limit=20`	Stable; response gives `next_cursor`
Keyset	`?after_id=100&limit=20`	Fast
Link header	`Link: <...?page=3>; rel="next"`	GitHub style

1.13 REST vs SOAP vs GraphQL vs gRPC (comparison table — interview favourite)

Feature	REST	SOAP	GraphQL	gRPC
Type	Architectural style	Protocol	Query language + runtime	RPC framework
Format	JSON/XML/any	XML only	JSON	Protobuf (binary)
Transport	HTTP	HTTP, SMTP, etc.	HTTP	HTTP/2
Contract	OpenAPI (optional)	WSDL (mandatory)	Schema (SDL)	`.proto`
Endpoints	Many (one per resource)	One per service, operation in body	Usually one (`/graphql`)	Service methods
Over/under-fetching	Common	Common	Solved (ask exact fields)	N/A (defined messages)
Caching	Easy (HTTP caching)	Hard	Harder (all POST)	Hard
Error handling	HTTP status codes	SOAP Fault	Usually 200 with `errors` array	gRPC status codes (0 OK, 5 NOT_FOUND, 16 UNAUTHENTICATED…)
Streaming	No (except SSE/chunked)	No	Subscriptions (via WS)	Yes: unary, server, client, bidirectional
Best for	Public web APIs	Banking/enterprise legacy, strict contracts	Flexible frontends, mobile	Internal microservices, low latency

---

1.14 Authentication vs Authorization

> 🧒 **Authentication** = Showing your face at the school gate so the guard knows it's **you**.
> **Authorization** = The guard checking if you're allowed into the **staff room**.

- **Authentication (AuthN)**: *Who are you?* — passwords, tokens, certificates, OTP.
- **Authorization (AuthZ)**: *What are you allowed to do?* — roles, scopes, permissions.
- Failed AuthN → **401**. Failed AuthZ → **403**.

Common mechanisms (all covered in Chapter 6): API Key, Basic Auth, Bearer Token, JWT, OAuth 1.0, OAuth 2.0 (+ OpenID Connect), Digest, Hawk, AWS Signature v4, NTLM, Mutual TLS (client certificates), Akamai EdgeGrid, ASAP.

1.15 Cookies and Sessions

- A **cookie** is a small key-value text the server asks the client to store using `Set-Cookie`, and the client sends it back with `Cookie` header on future requests to that domain.
- **Session-based auth**: Login → server creates a session in memory/DB → sends `Set-Cookie: sessionId=abc` → client sends cookie on every request.
- **Token-based auth**: Login → server returns a token (often JWT) → client sends `Authorization: Bearer <token>`.

Cookie attributes:

Attribute	Meaning
`Domain`	Which domains get the cookie
`Path`	Which paths get it
`Expires` / `Max-Age`	Lifetime; none = session cookie
`Secure`	Only over HTTPS
`HttpOnly`	JS in the browser cannot read it (protects from XSS)
`SameSite=Strict/Lax/None`	Controls cross-site sending (CSRF protection). `None` requires `Secure`

Postman automatically stores cookies in its **cookie jar** (click **Cookies** under the Send button) and sends them on later requests to matching domains — just like a browser. This is why "logging in once" makes later requests work, and also why a stale cookie can cause confusing bugs (Chapter 16).

1.16 CORS (Cross-Origin Resource Sharing)

- **Origin** = scheme + host + port. `https://app.com` and `https://api.app.com` are **different origins**.
- Browsers block JavaScript on one origin from reading responses from another origin **unless** the server allows it with headers like `Access-Control-Allow-Origin`.
- For "non-simple" requests (e.g. `PUT`, custom headers, `Content-Type: application/json`), the browser first sends a **preflight** `OPTIONS` request.

Crucial interview point: *CORS is enforced by browsers, not servers and not Postman.* The **Postman desktop app is not a browser page**, so CORS doesn't block it. This is why "**It works in Postman but not in my frontend!**" is a classic situation — the API is missing CORS headers. (Postman **web** app sends requests via the **Postman Desktop Agent** or **Cloud Agent** to avoid browser CORS limits.) To test CORS from Postman, manually add an `Origin` header and send an `OPTIONS` request, then check the `Access-Control-Allow-*` response headers.

1.17 Caching basics

- `Cache-Control: no-store` — never store.
- `Cache-Control: no-cache` — store but **revalidate** before using.
- `Cache-Control: max-age=3600` — fresh for 1 hour.
- `ETag: "v5"` + client `If-None-Match: "v5"` → server replies **304 Not Modified** with empty body if unchanged.

Postman setting: **"Send no-cache header"** (Settings → General) adds `Cache-Control: no-cache` to requests to avoid getting cached responses during testing.

1.18 HTTP versions

Version	Key features
HTTP/1.0	One request per connection
HTTP/1.1	Keep-alive, `Host` header mandatory, chunked transfer, pipelining (rarely used)
HTTP/2	Binary framing, multiplexing many streams over one connection, header compression (HPACK), server push (now deprecated in browsers)
HTTP/3	Runs over **QUIC** (UDP), faster connection setup, no TCP head-of-line blocking

Postman's HTTP requests can use HTTP/1.x and HTTP/2 (there is a per-request **Settings → HTTP version** option in recent versions, with "Auto" defaulting based on server support). gRPC in Postman uses HTTP/2.

1.19 HTTPS, TLS and Certificates (so SSL errors never scare you)

- **HTTPS = HTTP + TLS encryption.**
- The server has a **certificate** signed by a **Certificate Authority (CA)**. The client trusts a list of root CAs.
- **Self-signed certificates** (common in dev/QA environments) are not signed by a trusted CA → Postman shows errors like `self signed certificate`, `unable to get local issuer certificate`, `UNABLE_TO_VERIFY_LEAF_SIGNATURE`.
- Fixes in Postman (Chapter 16): add your company **CA certificate** (Settings → Certificates → CA certificates), or for local dev only, turn off **SSL certificate verification**.
- **Mutual TLS (mTLS)**: the **client also shows a certificate**. In Postman: Settings → Certificates → **Add client certificate** (host, port, CRT/KEY or PFX file, passphrase).

1.20 Where Postman fits in the Software Development Life Cycle

Role	How they use Postman
Backend developer	Build & debug endpoints, quick manual tests, share collections with frontend
Frontend / mobile developer	Understand API responses, use mock servers before backend is ready
QA / SDET	Functional, regression, negative, data-driven, contract and integration testing; automation with Newman/Postman CLI in CI/CD
DevOps / SRE	Monitors for uptime/health checks, CI pipelines
Architect / API product manager	Design-first with OpenAPI in Spec Hub, governance rules, documentation
Security tester	Manual auth/authorization testing, fuzzing with data files, OWASP API checks
Technical writer	Publish API documentation
Partners / customers	Fork public collections ("Run in Postman") to onboard quickly

Types of API testing you'll do with Postman

- **Functional testing** — does each endpoint do what it should?
- **Smoke testing** — are the most critical endpoints alive?
- **Sanity testing** — after a small change, is the related area OK?

- **Regression testing** — did new code break old features? (Collection runs in CI)
- **Integration testing** — do services work together? (request chaining)
- **End-to-end (E2E) workflow testing** — signup → login → add to cart → pay → order status
- **Negative testing** — invalid inputs, missing fields, wrong auth
- **Boundary value testing** — min/max lengths, limits
- **Data-driven testing** — same request with many data rows (CSV/JSON)
- **Contract testing** — response matches the agreed schema (JSON Schema / OpenAPI)
- **Performance/load testing** — Postman's built-in performance runner (basic; for heavy load use JMeter/k6/Gatling)
- **Security testing** — auth, IDOR/BOLA, injection, headers
- **Monitoring** — scheduled production health checks

1.21 Chapter 1 Quick Revision

- API = contract/waiter between client and server.
- HTTP is stateless request–response; methods have safe/idempotent properties.
- 2xx success, 3xx redirect, 4xx client mistake, 5xx server mistake. 401 = not authenticated, 403 = not authorized.
- `Content-Type` = what I send; `Accept` = what I want.
- CORS is a browser thing; Postman desktop ignores it.
- JSON: double quotes, no trailing commas, big numbers lose precision in JS.
- Postman shows timing breakdown: DNS, TCP, SSL, TTFB, download.

➡️ Next: **Chapter 2 — Getting Started with Postman**.

Chapter 2 — Getting Started with Postman

> 🧒 **5-year-old version:** Postman is like a **remote control for APIs**. Instead of building a whole app just to see if the "kitchen" (server) works, you press buttons in Postman, it sends your order, and it shows you exactly what came back.

2.1 What is Postman?

Postman is an **API platform** used to **design, build, test, document, mock, monitor, and collaborate on APIs**. It started (2012) as a simple Chrome extension built by Abhinav Asthana to make API testing easier, became a desktop app, and grew into a full platform used by tens of millions of developers and the majority of Fortune 500 companies.

What Postman can do (the big map)

Capability	Feature in Postman
Send any API request	HTTP, GraphQL, gRPC, WebSocket, Socket.IO, MQTT, SOAP, AI/MCP requests
Organise requests	**Collections**, folders, workspaces
Reuse values	**Variables**: global, collection, environment, data, local; **Vault** secrets
Authenticate	Auth tab: API Key, Bearer, Basic, Digest, OAuth 1.0/2.0, JWT, AWS Sig v4, NTLM, Hawk, Akamai EdgeGrid, ASAP
Automate & test	**Pre-request scripts**, **post-response scripts** (tests), `pm.*` API, Chai assertions
Run many requests	**Collection Runner**, data files (CSV/JSON), performance tests
Command line / CI	**Newman** (open-source npm package), **Postman CLI** (official binary)
Fake APIs	**Mock servers**
Scheduled checks	**Monitors**
Design APIs	**Spec Hub / API Builder** with OpenAPI, AsyncAPI, GraphQL schemas
Document	Auto-generated **documentation**, publishing
Visual workflows	**Postman Flows** (low-code)
Capture traffic	**Proxy**, **Interceptor**
AI	**Postbot** and **Agent Mode**

Capability	Feature in Postman
Collaborate	Team workspaces, roles, comments, forks, pull requests, version control (Git integration, Native Git in v12)
Discover APIs	**Postman API Network** (public APIs), **API Catalog** (organisation-level)
Programmatic control	**Postman API** (`api.getpostman.com`)

2.2 Postman versions — a short history (good for interviews and for reading old tutorials)

Era	What changed
Chrome App (2012–2018)	Postman ran inside Chrome. Deprecated.
Native desktop app (Electron)	Windows/macOS/Linux. Same app is still used today.
v7–v8 (2019–2020)	Workspaces, APIs tab, mock servers & monitors matured; v8 introduced the new UI and the **Postman Agent** for the web app.
v9 (2021)	API Builder, Flows (beta), gRPC early access, Collection format improvements.
v10 (2022)	Git integration for APIs, **Scratch Pad deprecated** (offline mode replaced later by lightweight client), new workspace experience.
v11 (2024)	**Postbot** AI everywhere, **Postman Vault**, **Package Library** (`pm.require`), **Scripts tab** split into *Pre-request* and *Post-response* (the old "Tests" tab was renamed), **Spec Hub**, Insights. Later in v11: variable values model changed to a single local value with an optional **shared value** (previously "current value" / "initial value").
v12 (2026)	**Native Git / Git-native workspaces**: collections stored as **YAML files (Collection format v3)** in your repo, local-first development, `postman workspace push` CLI sync; **Agent Mode** expanded; **API Catalog**.

> ⚠️ **Version awareness rule (very important in a company):** Postman changes UI labels often. Old tutorials say "Tests tab", new UI says "Scripts → Post-response". Old docs say "Initial value / Current value", newer docs say "Shared value / Local value" (or a single "Value" column). **The concepts are the same.** In this book we give both names wherever a rename happened, so you can work with any version your company uses.

2.3 Postman plans (as of 2026)

In **March 2026**, Postman restructured its plans:

Old plans (before March 2026)	New plans (from March 2026)
Free, Basic, Professional, Enterprise	**Free, Solo, Team, Enterprise**

- **Free** — now limited to **a single user** (team collaboration removed from free tier).
- **Solo** — paid, individual, more AI credits & automation.
- **Team** — paid per user; replaces Basic; unlocks collaboration and role-based access control.
- **Enterprise** — advanced governance, SSO/SCIM, audit logs, Partner workspaces, private API network, etc.

> Prices and limits (monitor runs, mock calls, AI credits, Flows, etc.) change frequently. **Always check postman.com/pricing** before quoting numbers in a company proposal. In interviews it's enough to know: *free tier = individual, collaboration needs paid; Enterprise adds SSO, governance, audit logs, advanced roles.*

2.4 Installing Postman

Desktop app (recommended for work)

- **Windows**: download the installer from `postman.com/downloads`. Installs per-user into `%LOCALAPPDATA%\Postman`.
- **macOS**: download `.zip` / `.dmg` (Apple Silicon or Intel build), move to Applications. Or `brew install --cask postman`.
- **Linux**: download the `tar.gz`, extract (e.g. to `/opt/Postman`) and create a desktop entry; or use Snap: `sudo snap install postman`. Flatpak packages exist too (community-maintained).

Web app

- Go to `go.postman.co` / `web.postman.co` and sign in.
- Because browsers enforce CORS and can't reach `localhost` freely, the web app needs an **agent** to send requests:

Agent	What it does	When to use
Cloud Agent	Requests go via Postman's cloud servers	Public APIs only; can't reach `localhost` or private networks
Desktop Agent	A small local app that sends requests from your machine	Calling `localhost`, VPN/intranet APIs from the web app

Agent	What it does	When to use
Browser Agent	Sends from the browser itself	Limited; subject to CORS
Auto-select	Postman picks the best	Default

Agent selection is at the bottom-right of the web app.

Lightweight API Client (no sign-in)

If you open the desktop app without signing in, you get a **lightweight API client**: you can send requests (HTTP, GraphQL, gRPC, WebSocket…), but you can't save collections to the cloud, use environments the same way, run collections, mocks, monitors etc. (It replaced the old **Scratch Pad** offline mode.) In v12, local/Git-native workflows make offline work much more capable.

VS Code extension

Postman has an official **VS Code extension** (send requests, collections, environments inside the editor). Useful for developers who don't want to leave the IDE.

Postman CLI & Newman

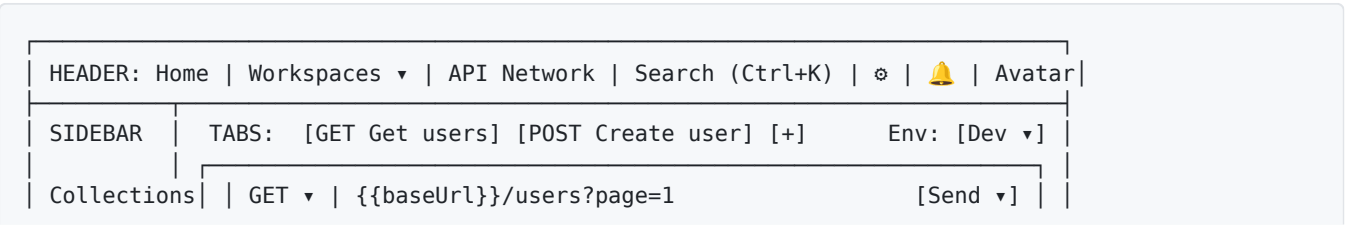
Command-line tools to run collections in CI. Covered in Chapter 11.

2.5 Signing in, accounts and teams

- A **user account** belongs to a **team** (organisation). Companies usually enforce **SSO** (SAML/OIDC: Okta, Azure AD/Entra ID, Google) on Enterprise.
- Your data (collections, environments' shared values, etc.) **syncs to Postman cloud** in normal cloud workspaces. That's why companies care about what goes into shared values (never real secrets — use Vault or local values).
- **Local values** of variables are stored **only on your machine** and are not synced to teammates.

2.6 The Postman Interface — A Guided Tour

(Labels are from v11/v12 desktop. Positions can move slightly between versions.)



```
| Environments| ├──────────────────────────────────────────────────────────────┤ |
| History  | | Params | Authorization | Headers (8) | Body | Scripts | Settings| |
| APIs/Specs| | ...request editor...                                         | |
| Mock servers| ├──────────────────────────────────────────────────────────────┤ |
| Monitors | | RESPONSE: Body | Cookies | Headers | Test Results              | |
| Flows    | |   Status: 200 OK   Time: 120 ms   Size: 1.2 KB   [Save Response]| |
|          | |   Pretty | Raw | Preview | Visualize     JSON ▾                | |
|          | └──────────────────────────────────────────────────────────────┘ |
├──────────┴──────────────────────────────────────────────────────────────────┤
| FOOTER: Online | Find & replace | Console | Postbot/Agent | Runner | Cookies |
|         Vault | Trash | Capture (Proxy/Interceptor) | Bootcamp                |
└─────────────────────────────────────────────────────────────────────────────┘
```

Header bar

- **Home** — recent workspaces and activity.
- **Workspaces** — switch/create workspaces.
- **API Network** — explore public APIs (Stripe, Twilio, Salesforce, etc. publish official collections).
- **Search** — search across your team's collections, requests, workspaces.
- **Settings (gear)** — app settings (General, Themes, Shortcuts, Data, Add-ons, Certificates, Proxy, Update, About).
- **Notifications**, **Invite**, **Upgrade**, profile.

Sidebar (left)

Items you can pin/hide via "Configure sidebar":

- **Collections** — your saved requests.
- **Environments** — sets of variables (Dev/QA/Prod).
- **History** — every request you've sent (great for "what did I send yesterday?").
- **APIs / Specs (Spec Hub)** — API definitions.
- **Mock servers**, **Monitors**, **Flows**, **Insights** (depending on version and plan).

Workbench (centre)

- **Tabs** — multiple open requests. Unsaved changes show a dot. Tabs can be pinned.
- **Environment selector** (top right) — choose which environment's variables are active. **"No environment"** is a frequent cause of `{{baseUrl}}` not resolving.
- **Eye icon / variables quick look** — view and edit current variable values (in newer versions there's a variables pane on the right).

Request builder tabs

Tab	Purpose
Params	Query params and path variables in a table (auto-syncs with URL)

Tab	Purpose
Authorization	Auth type & credentials (or "Inherit auth from parent")
Headers	Request headers; toggle to see auto-generated hidden headers
Body	none / form-data / x-www-form-urlencoded / raw / binary / GraphQL
Scripts	**Pre-request** and **Post-response** JavaScript (older UI: separate "Pre-request Script" and "Tests" tabs)
Settings	Per-request settings: SSL verification, follow redirects, follow original HTTP method, follow Authorization header on redirect, remove referer on redirect, strict HTTP parser, encode URL automatically, disable cookie jar, max redirects, HTTP version, TLS protocols/cipher suites, server certificate verification

Right side panel (newer UI): **Documentation**, **Comments**, **Code snippet** (`</>`), **Request info**, **Related collections**, **Variables**.

Response pane

- **Status** (e.g. `200 OK`), **Time** (hover = timing breakdown), **Size** (hover = headers vs body size), and network info (IP addresses, TLS details on hover).
- **Body** views:
  - **Pretty** — formatted & syntax-highlighted (choose JSON/XML/HTML/Text/Auto).
  - **Raw** — exact text.
  - **Preview** — renders HTML (e.g. an error page) in a sandboxed iframe.
  - **Visualize** — custom charts/tables built with `pm.visualizer` (Chapter 7).
- **Cookies** — cookies set by this response.
- **Headers** — response headers.
- **Test Results** — pass/fail of `pm.test` assertions.
- **Save Response** → **Save as example** (used by mocks & docs) or **Save to a file**.
- **Search** inside the response (Ctrl+F), **filter** with JSONPath in newer versions, **copy** response.

Footer

- **Console** — the debugger: every request/response with raw headers, body, scripts `console.log` output, errors. **Your best friend.** Shortcut: `Ctrl+Alt+C` (Windows/Linux) / `Cmd+Option+C` (macOS).
- **Runner** — Collection Runner.
- **Cookies** — cookie manager.
- **Capture requests** — Proxy/Interceptor.
- **Vault** — secrets.
- **Trash** — recently deleted collections (restorable for a plan-dependent period).
- **Postbot / Agent Mode** — AI assistant.

2.7 Workspaces

> 🧒 A **workspace** is a **room**. Your personal bedroom (Personal workspace), a classroom shared with classmates (Team workspace), a playground open to the whole world (Public workspace), and a meeting room where you invite a partner company (Partner workspace).

Workspace type	Who can see	Typical use
Personal	Only you	Experiments, learning
Private	You + people you invite (paid plans)	Restricted team work
Team	Members of your Postman team (organisation)	Default for company projects
Partner	Your team + invited external partners (Enterprise)	Sharing APIs with vendors/customers
Public	Everyone on the internet	Public API onboarding, community

Workspace contents

Collections, environments, API specs, mock servers, monitors, flows, and (in v12) a linked Git repository for Native Git workspaces.

Workspace roles (common)

Role	Can do
Admin	Manage workspace settings, members, roles; delete workspace
Editor	Create/edit/delete elements
Viewer	View and run; can fork; cannot edit originals

Element-level roles exist too (e.g. a collection **Editor** vs **Viewer**). Team-level roles (Enterprise/Team) include Super Admin, Admin, Billing, Developer, Community Manager, Partner Manager etc. depending on plan.

> 🏢 **Company practice:** Create **one team workspace per product/domain** (e.g. "Payments – API", "Orders – API"), not one per person. Give developers Editor, external stakeholders Viewer. Never put production secrets as shared values in a team workspace.

2.8 Settings you should configure on day 1

Settings → **General**:

Setting	Recommendation	Why
SSL certificate verification	ON (turn OFF only for local self-signed dev servers, or better, add the CA cert)	Security
Request timeout in ms	`0` = infinite (default). Set e.g. `30000` to fail fast	Hanging requests
Max response size in MB	Default 50; `0` = no limit	Big downloads
Automatically follow redirects	ON by default; turn OFF to debug redirects	
Send no-cache header	ON	Avoid cached responses
Send Postman Token header	ON (default). Some strict servers reject unknown headers — turn off then	
Trim keys and values in request body	Optional	Avoid trailing space bugs
Use next generation URL processing	Default ON in newer versions	
Working directory	Set a folder; files for form-data/binary are read from here (important for Newman portability)	
Allow reading files outside working directory	OFF for safety	
Language detection, **Default headers**, editor font, indentation	Personal preference	

Settings → **Proxy**: system proxy vs custom proxy (corporate networks), proxy auth, bypass list.

Settings → **Certificates**: CA certificates (PEM) and client certificates (mTLS).

Settings → **Data**: export all your data (backup), import.

Settings → **Themes**: Light/Dark.

Settings → **Shortcuts**: view/disable keyboard shortcuts.

2.9 Keyboard shortcuts (most useful)

Action	Windows/Linux	macOS
Send request	`Ctrl + Enter`	`Cmd + Enter`
Send and download	`Ctrl + Alt + Enter`	`Cmd + Option + Enter`
Save request	`Ctrl + S`	`Cmd + S`
Save as	`Ctrl + Shift + S`	`Cmd + Shift + S`
New tab	`Ctrl + T`	`Cmd + T`

Action	Windows/Linux	macOS
Close tab	`Ctrl + W`	`Cmd + W`
Force close tab	`Ctrl + Alt + W`	`Cmd + Option + W`
Next / previous tab	`Ctrl + Tab` / `Ctrl + Shift + Tab`	`Cmd + Shift + ]` / `[`
Search	`Ctrl + K`	`Cmd + K`
Open Console	`Ctrl + Alt + C`	`Cmd + Option + C`
Toggle sidebar	`Ctrl + \`	`Cmd + \`
Import	`Ctrl + O`	`Cmd + O`
Settings	`Ctrl + ,`	`Cmd + ,`
Focus URL bar	`Ctrl + L`	`Cmd + L`
Zoom in/out/reset	`Ctrl + +` / `Ctrl + -` / `Ctrl + 0`	`Cmd + +` / `-` / `0`
Find (in editor/response)	`Ctrl + F`	`Cmd + F`
Shortcuts list	`Ctrl + /`	`Cmd + /`

(Check **Settings → Shortcuts** for your exact version.)

2.10 Your First Request — Step by Step

We'll use free practice APIs:

- **JSONPlaceholder** — `https://jsonplaceholder.typicode.com` (fake posts/users/todos; writes are faked and not persisted).
- **httpbin** — `https://httpbin.org` (echoes what you send — perfect for learning headers, auth, status codes).
- **Restful-Booker** — `https://restful-booker.herokuapp.com` (CRUD with token auth; built for testers).
- **Swagger Petstore** — `https://petstore.swagger.io/v2`.
- **Postman Echo** — `https://postman-echo.com` (Postman's own echo service).

> Note: `reqres.in` was very popular in tutorials; since 2025 it requires an `x-api-key` header (a free key is shown on their site). If old tutorials fail with 401, that's why.

Step 1 — GET

1. Click **New → HTTP** (or `+` tab).
2. Method: **GET**. URL: `https://jsonplaceholder.typicode.com/users/1`
3. Click **Send**.
4. Response: `200 OK`, JSON with the user.

Step 2 — Query params

URL: `https://jsonplaceholder.typicode.com/posts?userId=1` Look at the **Params** tab — Postman auto-filled `userId = 1`. Untick the checkbox to temporarily disable a param without deleting it.

Step 3 — Path variables

URL: `https://jsonplaceholder.typicode.com/posts/:postId` Params tab shows a **Path Variables** table → set `postId = 5`. Postman sends `/posts/5`.

Step 4 — POST with JSON

1. Method **POST**, URL `https://jsonplaceholder.typicode.com/posts`
2. Body → **raw** → dropdown **JSON**:

```
{
  "title": "Learning Postman",
  "body": "This is fun",
  "userId": 1
}
```

3. Send → `201 Created`, response includes `"id": 101`.

Step 5 — See what you actually sent

`https://httpbin.org/anything` echoes back your method, headers, args, and body. Send any request there to see exactly what Postman transmitted. Or open the **Console**.

Step 6 — Save it

`Ctrl+S` → name "Create post" → create a **collection** "JSONPlaceholder Practice" → Save.

Step 7 — Generate code

Click `</>` (Code) → choose cURL, Python `requests`, JavaScript `fetch`, Java OkHttp, C# `HttpClient`/RestSharp, Go, PHP, Ruby, Swift, etc. Developers use this to turn a working Postman request into application code.

Step 8 — Import from cURL

Import (or just paste into the URL bar) a cURL command:

```
curl -X POST https://httpbin.org/post -H "Content-Type: application/json" -d '{"a":1}'
```

Postman converts it into a full request. Huge time-saver when a developer sends you a cURL from browser DevTools ("Copy as cURL").

2.11 What you can import into Postman

Format	Notes
Postman Collection v1, v2.0, v2.1 (JSON); Collection v3 (YAML, v12 Native Git)	v2.1 is the most common export
Postman Environment / Globals / data dump	
cURL	paste or file
OpenAPI 2.0 (Swagger), 3.0, 3.1 (JSON/YAML)	Generates a collection, optionally linked to a spec
RAML, **WSDL** (SOAP), **GraphQL schema**, **AsyncAPI**	Support varies by version
HAR (HTTP Archive from browser DevTools)	Convert a browser session into requests
Insomnia, **Thunder Client**?	Insomnia export supported; others via OpenAPI/HAR
Code from GitHub/GitLab/Bitbucket/Azure repos, API gateways (AWS API Gateway, Azure APIM, Apigee)	Integrations (plan-dependent)
A folder / zip	Bulk import
Link (URL)	Import from a URL to a raw file

2.12 History, Trash, and recovering lost work

- **History** (sidebar) — every sent request, grouped by date; click to reopen; save into collection. Can be cleared. History syncs if enabled.
- **Trash** — deleted collections can be restored for a limited time (longer on paid plans).
- **Collection changelog / version history** — on team plans, view who changed what and **restore** a collection to an older state.
- **Forks** — your fork is a safe copy; experiment there, then **pull request** to merge (Chapter 14).
- **Export** — regularly export important collections to Git (or use Git integration/Native Git) as a backup and source of truth.

2.13 Chapter 2 Quick Revision

- Postman = full API platform; desktop app preferred for work (no CORS, localhost access).
- Web app needs an agent (Desktop/Cloud/Browser) to send requests.
- UI: sidebar (collections, envs, history), request tabs (Params, Auth, Headers, Body, Scripts, Settings), response (Pretty/Raw/Preview/Visualize, Headers, Cookies, Test Results), footer (Console, Runner, Cookies, Vault).
- Workspaces: Personal, Private, Team, Partner, Public; roles Admin/Editor/Viewer.

- March 2026: Free (1 user), Solo, Team, Enterprise.
- v11 renamed "Tests" → "Post-response"; variable "initial/current" → "shared/local"; v12 adds Native Git with YAML collections.

➡️ Next: **Chapter 3 — Building and Sending Requests in Depth**.

Chapter 3 — Building and Sending Requests in Depth

> 🧒 A request is a **letter**. The **method** says what you want ("please give / please take / please change"). The **URL** is the address. **Headers** are what you write on the envelope. The **body** is the letter inside.

3.1 The URL bar

- Choose the **method** from the dropdown (GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS, plus COPY, LINK, UNLINK, PURGE, LOCK, UNLOCK, PROPFIND, VIEW). You can also **type a custom method** name — useful for APIs with non-standard verbs.
- Type or paste the URL. Variables are written `{{variableName}}` and turn **green** (resolved) or **red** (unresolved) — hover to see value and scope.
- If you omit the scheme (`api.example.com/users`), Postman defaults to `http://`. **Always type** `https://` explicitly to avoid surprises (redirects, 301s, or insecure calls).
- `localhost` / `127.0.0.1` work from the desktop app.

3.2 Query Parameters

Two ways to add:

1. Type in URL: `{{baseUrl}}/products?category=shoes&sort=price&order=asc`
2. Fill the **Params** table (Key / Value / Description). Both stay in sync.

Details:

- **Disable** a param with its checkbox — great for testing "what if this filter is missing?".
- **Bulk Edit** — switch the table to text mode `key:value` per line (fast copy-paste).
- **Repeated keys** (arrays): `?id=1&id=2&id=3` — add the same key multiple times. Some APIs expect `id[]=1&id[]=2` or `id=1,2,3`; check the API's convention.
- **Empty value** `?debug=` vs **no value** `?debug` — different on some servers; Postman supports both.
- **Encoding**: Postman encodes certain characters automatically (Settings → "Encode URL automatically" per request). If the API needs a *literal* character (e.g. a pre-encoded signature), turn auto-encoding off for that request.
- **Descriptions** appear in generated documentation — fill them in for team collections.

3.3 Path Variables

Write a colon prefix in the path: `{{baseUrl}}/users/:userId/orders/:orderId` → Params tab shows **Path Variables** table: `userId`, `orderId`.

Path variable `:id`	**Collection/env variable** `{{id}}`
Defined per request, in the Params table	Defined in a variable scope
Good for documenting the endpoint shape	Good for reusing dynamic values
Value can itself be a variable: `:userId` = `{{userId}}`	

Best practice: Use `:userId` in the path with value `{{userId}}`. Docs show the clean template, and chaining still works.

3.4 Headers

- Add Key/Value pairs; autocomplete suggests standard headers.
- Values can use variables: `Authorization: Bearer {{accessToken}}`, `X-Correlation-ID: {{$guid}}`.
- **Hidden (auto-generated) headers**: Postman automatically adds `Postman-Token`, `Host`, `User-Agent`, `Accept: */*`, `Accept-Encoding: gzip, deflate, br`, `Connection: keep-alive`, and for bodies `Content-Type` and `Content-Length`. Click **hidden** to view them; most can be unticked or overridden by adding your own header with the same name.
- **Header presets**: save a reusable group of headers (e.g. "JSON API headers": `Accept`, `Content-Type`) and insert in one click (Headers tab → Presets).
- **Bulk edit** like params.
- **Order of precedence**: The **Authorization tab** generates the `Authorization` header. If you *also* add a manual `Authorization` header, you can get confusing behaviour — **use one or the other**.

3.5 Request Body — All Types In Depth

3.5.1 none

No body (GET, DELETE usually).

3.5.2 form-data (`multipart/form-data`)

Used for **file uploads** and forms with files.

- Each row: Key, Value, type **Text** or **File** (hover the key cell to switch).

- For a file, pick one or multiple files. Postman builds the multipart body with a random **boundary** and sets `Content-Type: multipart/form-data; boundary=--------------------------123...` **automatically**.
- You can set a per-field **Content-Type** (show "Content-Type" column) — e.g. send a JSON part: key `metadata`, value `{"name":"x"}`, content type `application/json`.

> ❌ **Classic bug:** Manually setting `Content-Type: multipart/form-data` in Headers **without the boundary** → server can't parse → 400/500. **Fix: delete your manual Content-Type header and let Postman generate it.**

> ❌ **Newman/CI bug:** File paths are stored relative to Postman's working directory. In CI the file isn't there → `ENOENT`/empty upload. **Fix:** set a working directory, keep test files in the repo, run Newman with `--working-dir`, and enable "allow files outside working dir" only if needed.

3.5.3 x-www-form-urlencoded

- Body like `username=asha&password=secret%21` with `Content-Type: application/x-www-form-urlencoded`.
- Used by classic HTML forms and **OAuth 2.0 token endpoints** (the token request is form-encoded per the RFC).
- No files.

3.5.4 raw

Free text with a language selector: **Text, JavaScript, JSON, HTML, XML**. Selecting JSON sets `Content-Type: application/json`; XML sets `application/xml`; Text sets `text/plain`, etc.

JSON with variables:

```
{
  "email": "{{userEmail}}",
  "age": {{userAge}},
  "isAdmin": false,
  "requestId": "{{$guid}}",
  "createdAt": "{{$isoTimestamp}}"
}
```

- `{{userAge}}` without quotes → number in the final JSON. The editor may underline it red (it's not valid JSON before substitution) — **that's fine**; substitution happens before sending.
- Use the **Beautify** button to format JSON/XML.

3.5.5 binary

Send a single file as the whole body (`application/octet-stream` or set your own `Content-Type: image/png`). Used for S3 pre-signed URL uploads, image processing APIs.

3.5.6 GraphQL

Two editors: **Query** and **GraphQL Variables** (JSON). Postman sends:

```
{ "query": "query ($id: ID!) { user(id: $id) { name email } }", "variables": { "id": "1" } }
```

Postman can fetch the **schema via introspection** for autocomplete. (Full GraphQL chapter: 13.)

3.6 Request Settings tab (per request)

Setting	Meaning / when to change
HTTP version	Auto / HTTP/1.x / HTTP/2
Enable SSL certificate verification	Override global for this request
Automatically follow redirects	Off to inspect 3xx
Follow original HTTP method	By default, on 301/302 Postman (like browsers) may redirect POST as GET; turning this on keeps the method
Follow Authorization header	Keep auth header on cross-origin redirect (security risk—off by default)
Remove referer header on redirect	
Enable strict HTTP parser	Reject malformed responses
Encode URL automatically	Turn off for pre-encoded/signed URLs
Disable cookie jar	Don't send/store cookies for this request (test unauthenticated behaviour)
Use server cipher suite during handshake	TLS tuning
Maximum number of redirects	Default 10
TLS/SSL protocols disabled during handshake	Test TLS 1.0/1.1 rejection
Cipher suite selection	Advanced TLS testing

3.7 Understanding the Response

Status, Time, Size

- **Status**: code + reason phrase. Hover shows the meaning.

- **Time**: total ms; hover for phases (Prepare → Socket init → DNS → TCP → SSL → Waiting/TTFB → Download → Process). High **TTFB** = slow server; high **DNS** = DNS issue; high **SSL** = TLS issue; high **Download** = big payload/slow network.
- **Size**: response headers + body bytes (compressed vs uncompressed info on hover).
- **Network info** (globe icon): local & remote IP, TLS protocol, cipher, certificate details.

Body formats

Pretty / Raw / Preview / Visualize, language auto-detected from `Content-Type`. If an API returns JSON with `Content-Type: text/html`, Pretty may show text — switch the dropdown to JSON manually (and report the wrong header to the developers!).

Saving responses as Examples

Click **Save Response → Save as example**. An **example** stores a request + response pair. Examples are used by:

- **Mock servers** (they return examples),
- **Documentation** (shows sample responses),
- **Team understanding** (what a 404 looks like, what a 422 validation error looks like).

> 🏢 Save examples for **every important status code** (200, 201, 400, 401, 403, 404, 409, 422, 500) — this turns your collection into living documentation.

3.8 Cookies in Postman

- After a response with `Set-Cookie`, Postman stores the cookie in its **cookie jar** (per domain).
- Next request to the same domain → Postman sends the `Cookie` header automatically.
- **Manage Cookies** (link under Send button, or footer → Cookies): view, add, edit, delete cookies by domain. Add a domain **whitelist/allowlist** for scripts to access cookies programmatically.
- Disable per request with Settings → **Disable cookie jar**.

Common cookie-related bugs:

Symptom	Cause	Fix
Logged-out API still returns data	Old session cookie still in jar	Delete cookies for domain
431 Request Header Fields Too Large	Too many/big cookies accumulated	Clear cookie jar
Works in Postman, fails in CI (Newman)	Newman starts with an empty jar each run; you relied on a manually created cookie	Add a login request that sets cookies, or set Cookie header via variable

Symptom	Cause	Fix
CSRF token mismatch (Django/Laravel/Spring)	Need to read CSRF cookie and send it in header	Script: read cookie, set header (Chapter 6)

3.9 Code Snippets

`</>` panel converts the request into code in many languages/libraries: cURL, wget, HTTPie, PowerShell `Invoke-RestMethod`, Python (`requests`, `http.client`), JavaScript (`fetch`, jQuery, XHR, Axios), Node.js (`http`, `request`, Axios, Unirest), Java (OkHttp, Unirest, `java.net.http`), C# (`HttpClient`, RestSharp), Go, PHP (cURL, Guzzle, `HTTP_Request2`), Ruby, Swift, Kotlin, Dart, C (libcurl), R, Rust, OCaml, Objective-C, Shell, etc.

- Snippet settings: indentation, follow redirects, trim body, request timeout.
- Variables get **resolved** into the snippet using current values — be careful not to paste **secrets** into chats/tickets!

3.10 Sending and Downloading

Send dropdown → **Send and Download** saves the response body to a file (PDF reports, CSV exports, images).

3.11 Capturing requests: Proxy and Interceptor

Why

You want to see what a **mobile app** or **website** really sends, then recreate/test it in Postman.

Postman built-in proxy

1. Footer → **Capture requests** → **Proxy** (or "via Postman proxy").
2. Postman listens on a port (default **5555**).
3. Configure your device/browser to use your computer's IP:5555 as HTTP proxy.
4. For HTTPS capture, install and trust the **Postman CA certificate** on the device.
5. Requests appear in History or a chosen collection; filter by URL/method.

Postman Interceptor

- A **Chrome browser extension** that captures browser requests and **syncs cookies** from the browser into Postman (so you can call APIs with the browser's logged-in session).
- Requires the Interceptor bridge (installed automatically by the app on first use in recent versions).

- Choose domains to capture to avoid noise.

Alternative

Browser DevTools → Network → right-click a request → **Copy as cURL** → paste into Postman. Or export **HAR** → Import.

3.12 Working with files, encodings and special cases

- **Unicode**: JSON bodies are UTF-8. Emojis and Indian-language text work; ensure server declares `charset=utf-8`.
- **Large responses**: if response > max size setting → error "Maximum response size reached". Increase limit or use Send and Download.
- **Gzip/Brotli**: Postman decompresses automatically; raw view shows decoded text.
- **Chunked / streaming responses**: Postman shows final body; for **SSE** (`text/event-stream`) newer versions display the event stream live.
- **Binary responses** (images, PDF): Preview shows image/PDF; otherwise Send and Download.

3.13 A complete CRUD walkthrough (Restful-Booker)

Base URL: `https://restful-booker.herokuapp.com`

#	Request	Details	Expected
1	`POST /auth`	raw JSON `{"username":"admin","password":"password123"}`	`200`, `{"token":"abc123..."}`
2	`POST /booking`	raw JSON booking body (below), `Accept: application/json`	`200` with `bookingid`
3	`GET /booking/:id`	path var id	`200`, booking details
4	`PUT /booking/:id`	full body + header `Cookie: token={{token}}`	`200` updated
5	`PATCH /booking/:id`	`{"firstname":"Asha"}` + Cookie	`200`
6	`DELETE /booking/:id`	Cookie header	`201` (yes, this API returns 201 "Created" for delete — a great example that **real APIs don't always follow conventions**; your tests must follow the **API contract**, not assumptions)

#	Request	Details	Expected
7	`GET /booking/:id` again		`404`

Booking body:

```
{
  "firstname": "Jim",
  "lastname": "Brown",
  "totalprice": 111,
  "depositpaid": true,
  "bookingdates": { "checkin": "2026-01-01", "checkout": "2026-01-05" },
  "additionalneeds": "Breakfast"
}
```

We'll automate this exact flow with variables and scripts in Chapters 5, 7, 8, and 9.

3.14 Chapter 3 Quick Revision

- Params tab syncs with URL; path variables use `:name`.
- Let Postman set `Content-Type` for form-data (boundary!).
- OAuth token endpoints usually need x-www-form-urlencoded.
- Unquoted `{{number}}` in JSON is fine.
- Save examples for every status code → mocks + docs.
- Cookie jar persists cookies; clear it when debugging auth weirdness.
- Proxy (port 5555) and Interceptor capture real traffic; "Copy as cURL" is the fastest import.

➡️ Next: **Chapter 4 — Collections and Folders**.

Chapter 4 — Collections and Folders

> 🧒 A **collection** is a **school bag**. **Folders** are the pockets inside. **Requests** are your books. You can give the whole bag to a friend (share/export), and anything you put on the bag itself (like your name tag = auth, or a lunch box = variables) is available to everything inside.

4.1 What is a collection?

A **collection** is a group of saved requests (plus folders, scripts, variables, auth, documentation and examples) that can be **run together, shared, versioned, documented, mocked and monitored**.

Collections are the **unit of everything** in Postman:

- **Runner / Newman / Postman CLI** run collections (or folders).
- **Mock servers** are created from collections (their examples).
- **Monitors** run collections on schedule.
- **Documentation** is generated per collection.
- **Forks/pull requests** happen on collections.

4.2 Creating and organising

- **New → Collection**, or `+` in sidebar, or "Save" a request into a new collection.
- Inside: **Add request**, **Add folder** (folders can be nested to any depth).
- **Drag & drop** to reorder — **order matters** for the Runner (default sequential top-to-bottom execution).
- **Duplicate** a request/folder/collection; **rename**; **delete** (Trash).
- **Star/favourite** collections; **watch** to get notifications on changes.

Recommended structure for a big company project

```
📦 Orders Service API  (collection)
 ├── 📁 00 - Setup                      (auth, create test data)
 │    ├── POST Get access token
 │    └── POST Create test customer
 ├── 📁 01 - Orders
 │    ├── 📁 Positive
 │    │    ├── POST Create order
 │    │    ├── GET  Get order by id
 │    │    ├── GET  List orders (pagination)
 │    │    ├── PATCH Update order status
 │    │    └── DELETE Cancel order
 │    └── 📁 Negative
```

```
│          ├── POST Create order - missing items (400)
│          ├── POST Create order - no token (401)
│          ├── GET  Order of another user (403 / BOLA)
│          └── GET  Non-existent order (404)
├── 📁 02 - Payments
├── 📁 03 - E2E Flows
│    └── 📁 Checkout happy path
└── 📁 99 - Teardown                    (delete test data)
```

Alternative organisation patterns:

Pattern	When
By resource (Users, Orders, Products)	API reference / dev collections
By test type (Smoke, Regression, Negative, E2E)	QA automation collections
By user journey (Signup flow, Checkout flow)	E2E / business flows
By microservice — one collection per service	Microservice orgs; matches team ownership
Hybrid: one *reference* collection (generated from OpenAPI) + separate *test* collections	Large orgs (very common & recommended)

4.3 Collection-level settings (inherited by everything inside)

Click the collection → tabs:

Tab	What it does
Overview	Description (Markdown), stats, owner, links
Authorization	Default auth for all requests that use "Inherit auth from parent"
Scripts	Collection-level **Pre-request** and **Post-response** scripts — run for **every request** in the collection
Variables	Collection variables
Runs	Past runs
Settings	Collection-level settings (in some versions)

Inheritance — the most important concept here

```
Collection  (auth: Bearer {{token}}, pre-request: log, vars: baseUrl)
   └── Folder (auth: inherit,   pre-request: set header)
         └── Request (auth: inherit,   pre-request: build body)
```

- **Auth**: a request set to "Inherit auth from parent" walks **up**: request → folder → parent folder → collection. The **first non-inherit** auth found wins. (If the collection is "No Auth", the request gets no auth.)
- **Scripts**: they **don't override** — they **all run**, in order (see Chapter 7):
  - Pre-request: **Collection → Folder(s) → Request**
  - Post-response: **Collection → Folder(s) → Request**
- **Variables**: collection variables are visible to every request in the collection.

4.4 Documentation in collections

- Every collection, folder and request has a **description** field (Markdown supported: headings, tables, code blocks, links, images).
- **Examples** attached to requests show up in docs.
- Click **View documentation** → auto-generated docs with code snippets in chosen language.
- **Publish** docs (public) with custom domain/branding (plan-dependent), or keep private for your team.

> 🏢 **Company practice:** Description template for each request:

```
**Purpose:** Creates an order for the authenticated customer.
**Auth:** Bearer token (scope `orders:write`)
**Preconditions:** customerId exists (see Setup folder)
**Sets variables:** orderId
**Owner:** @payments-team | **Jira:** PAY-123
```

4.5 Examples (Saved responses) in depth

- An example = **request snapshot + response** (status, headers, body).
- Create: Send → **Save Response → Save as example**, or "Add example" from the request's `...` menu and type it manually (useful before the backend exists — design-first).
- Multiple examples per request: `200 OK`, `404 Not Found`, `400 Validation error`.
- Examples power **mock servers** (Chapter 12): the mock picks the best-matching example.
- Examples can contain variables.

4.6 Sharing collections

Method	How	Notes
Move/create in a team workspace	Everyone in workspace sees it	Recommended
Share link	"Share" → copy link / invite by email	Respects roles
Fork	Copy linked to parent; can pull changes and open pull requests	Safe experimentation
Run in Postman button	Embed on website/README	Public APIs
Export JSON	`...` → Export → v2.1	Git, backup, Newman
Postman API	`GET https://api.getpostman.com/collections/{uid}` with `X-API-Key`	Always-latest in CI
Native Git (v12)	Collections live as YAML files in your repo	Code review via Git PRs

4.7 Collection file formats

Collection v2.1 (JSON) — the classic export

```
{
  "info": {
    "_postman_id": "1b2c...",
    "name": "Orders Service API",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "item": [
    {
      "name": "Orders",
      "item": [
        {
          "name": "Get order by id",
          "event": [
            {
              "listen": "test",
              "script": {
                "type": "text/javascript",
                "exec": [
                  "pm.test('Status is 200', () => pm.response.to.have.status(200));"
                ]
              }
            }
          ],
          "request": {
            "method": "GET",
            "header": [{ "key": "Accept", "value": "application/json" }],
            "url": {
              "raw": "{{baseUrl}}/orders/:orderId",
              "host": ["{{baseUrl}}"],
```

```
            "path": ["orders", ":orderId"],
            "variable": [{ "key": "orderId", "value": "{{orderId}}" }]
          }
        },
        "response": []
      }
    ]
  }
 ],
 "auth": {
  "type": "bearer",
  "bearer": [{ "key": "token", "value": "{{accessToken}}", "type": "string" }]
 },
 "event": [
  { "listen": "prerequest", "script": { "type": "text/javascript", "exec": [""] } }
 ],
 "variable": [{ "key": "baseUrl", "value": "https://api.example.com" }]
}
```

Key facts for interviews:

- In JSON, the post-response script is still stored as `"listen": "test"` and pre-request as `"listen": "prerequest"` (historical names).
- `item` is recursive: an item with `item` inside is a **folder**; an item with `request` is a **request**.
- **v2.0 vs v2.1**: the main difference is how `auth` (and some URL fields) are represented — v2.0 uses objects keyed by parameter name; v2.1 uses **arrays of key/value objects**. **Export v2.1** unless a tool needs v2.0.
- `response` array = **examples**.
- `variable` at root = collection variables.

Collection v3 (YAML) — v12 Native Git

- In Postman v12, Git-native workspaces store collections as **YAML files** (Collection format v3), split into **multiple files** (e.g. one per request) inside a `postman/` folder in your repository, which makes **Git diffs, code reviews and merge conflicts far easier** than one giant JSON file.
- The Postman CLI can **run collections by path** (`postman collection run ./postman/collections/<name>`), and `postman workspace push` syncs local changes to the cloud workspace.
- Caveat from docs: when working Git-native, **local files are the source of truth**; edits made directly in the cloud view can be overwritten by the next `postman workspace push`.
- Some reporters/options may be limited for v3 YAML in early v12 releases (e.g. only `cli` reporter for some request types) — check the CLI docs of your version.

4.8 Importing / exporting best practices

1. **Export v2.1** for Newman.
2. Remove secrets before exporting (they're only in shared values if someone put them there — check!).

3. Environments export separately (`*.postman_environment.json`) — commit only **templates** without secrets.
4. Keep exported files **pretty-printed** and **sorted** for readable diffs (some teams use a script to normalise and strip `_postman_id` / `id` churn).
5. Prefer **Postman API** or **Native Git** over manual exports to avoid "who has the latest JSON?" problems.

4.9 Forks, Pull Requests, Merges (collection version control inside Postman)

> 🧒 **Fork** = photocopy the homework so you can scribble safely. **Pull request** = asking the teacher, "Can my changes go into the real homework?" **Merge** = teacher accepts.

1. `...` → **Create a fork** → choose name & workspace (e.g. your personal workspace).
2. Edit the fork freely.
3. **Pull changes** from parent to stay updated.
4. **Create pull request** → reviewers see a **diff** (requests, scripts, examples).
5. Reviewers comment, approve; **Merge** (optionally delete fork after).
6. Parent collection can be **protected**: only certain roles can merge/edit directly (settings depend on plan).

Merge conflicts: if parent changed the same item, Postman shows conflict resolution to pick source vs destination.

4.10 Collection runs order and control (preview)

- Default order: top to bottom, depth-first through folders.
- Run a **single folder** in Runner or `newman run ... --folder "Orders"`.
- Change order dynamically with `pm.execution.setNextRequest()` (Chapter 9).
- Skip a request with `pm.execution.skipRequest()`.

4.11 Common collection problems & fixes

Problem	Cause	Fix
Request has no auth unexpectedly	Request auth = "No Auth" (not inherit)	Set to "Inherit auth from parent"
Collection script running twice-like behaviour / unexpected logs	Same code in collection **and** folder scripts	Keep shared logic at one level

Problem	Cause	Fix
Teammate doesn't see my changes	You edited a fork, or local value only, or not saved	Save; merge fork; persist/share values
Huge JSON diffs in Git	Exports include ids/order changes	Use Native Git YAML, or normalise exports
"Collection not found" in Newman with URL	Using an old/expired share link or wrong API key	Use `https://api.getpostman.com/collections/<uid>?apikey=...` or a file
Runner runs requests in weird order	`setNextRequest` left in a script	Search collection for `setNextRequest`

4.12 Chapter 4 Quick Revision

- Collections = unit of running, sharing, mocking, monitoring, documenting.
- Inheritance: auth walks up; scripts all run Collection → Folder → Request.
- Export v2.1; JSON uses `listen: "prerequest"` and `listen: "test"`.
- v12 Native Git: YAML (Collection v3) files in repo.
- Fork → PR → merge for safe collaboration.

➡️ Next: **Chapter 5 — Variables and Environments**.

Chapter 5 — Variables and Environments (The Heart of Real-World Postman)

> 🧒 A **variable** is a **labelled jar**. You write a name on the jar (`baseUrl`) and put something inside (`https://dev.shop.com`). Whenever you say "use the baseUrl jar", Postman opens the jar and uses what's inside. Change what's inside the jar once → every request using it changes. An **environment** is a **shelf of jars** — one shelf for Dev, one for QA, one for Prod. Switch the shelf, and all your requests now point to a different world.

5.1 Why variables matter

Without variables:

```
GET https://dev-api.shop.com/v1/orders/98231
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...(copied manually)
```

→ To test on QA you edit 200 requests by hand. Tokens expire → paste again. Disaster.

With variables:

```
GET {{baseUrl}}/v1/orders/{{orderId}}
Authorization: Bearer {{accessToken}}
```

→ Switch environment Dev → QA. Token is fetched by a script. `orderId` is saved automatically from the "Create order" response. **This is how professional collections work.**

5.2 Syntax

Where	Syntax
URL, params, headers, body, auth fields	`{{variableName}}`
Path variables	`:name` (separate concept, value can be `{{var}}`)
Vault secrets	`{{vault:secret-name}}`
Dynamic (random) variables	`{{$randomEmail}}` , `{{$guid}}` , `{{$timestamp}}`
Scripts	`pm.environment.get("name")` , `pm.variables.get("name")` , etc. (**no curly braces** in scripts!)

Where	Syntax
Resolve a string with variables inside a script	`pm.variables.replaceIn("Hello {{firstName}}")`

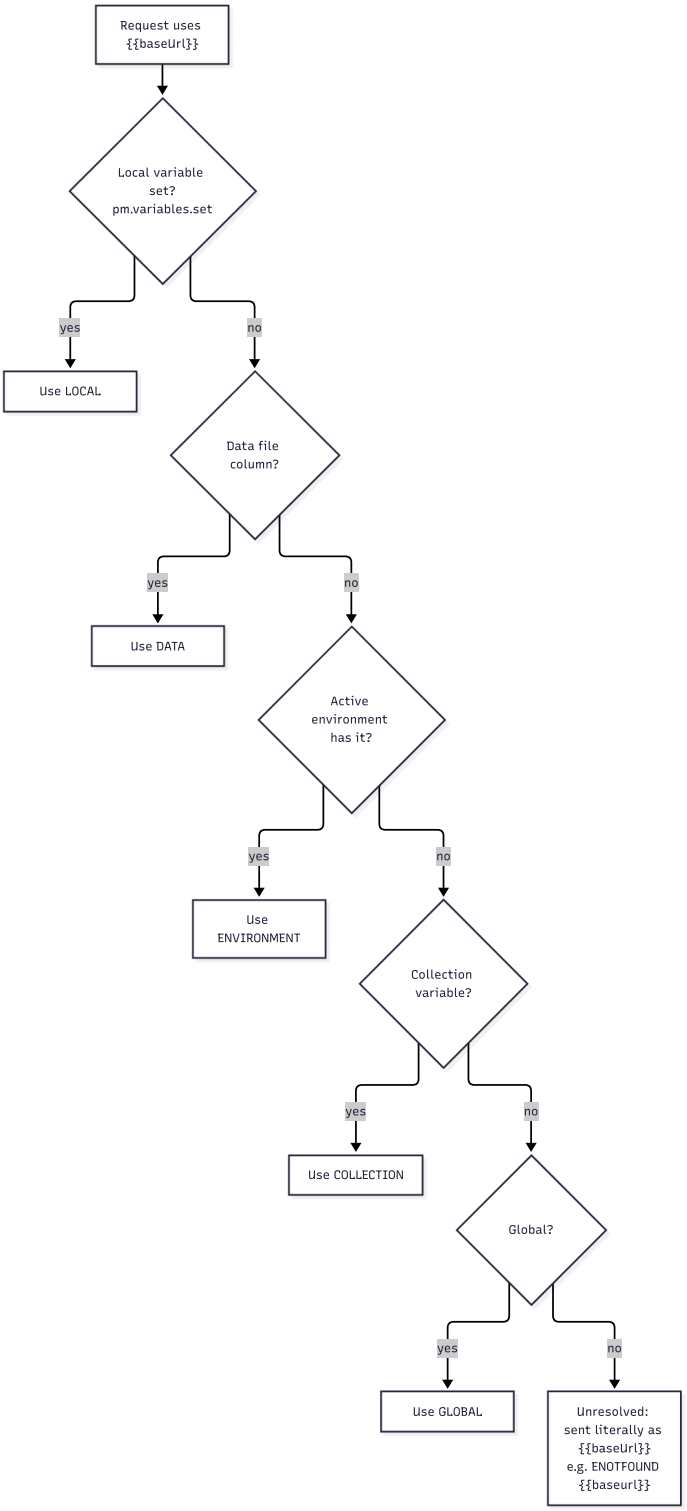
- Names are **case-sensitive**: `{{BaseUrl}}` ≠ `{{baseUrl}}`.
- Unresolved variables show **red** in the UI and are sent **literally** as `{{baseUrl}}` → errors like `getaddrinfo ENOTFOUND {{baseurl}}` or "Invalid URI".
- Variables can contain variables: `baseUrl = https://{{host}}:{{port}}` — Postman resolves nested references (avoid deep nesting; it hurts readability).

5.3 Variable scopes (the 5 + Vault)

Scope	Lives in	Visible to	Typical use
Global	Workspace globals	All requests in the workspace, regardless of collection/environment	Quick experiments; rarely in clean projects
Collection	The collection	Requests of that collection, all environments	Constants of that API: `apiVersion`, default paths, test data that doesn't vary by env
Environment	An environment (Dev/QA/Prod)	Requests in any collection while that environment is **selected**	`baseUrl`, credentials (local values), env-specific IDs
Data	A CSV/JSON data file during Runner/Newman/CLI	Only during that iteration of a run	Data-driven tests: `email`, `expectedStatus`
Local	Memory of the current request/run	Current request (or run) only	Temporary values; overriding for one run
Vault (secrets)	Encrypted **local** storage on your machine (or external vaults)	Your requests where vault access is enabled	Real API keys, passwords, client secrets

Precedence (narrowest wins)

When the **same name** exists in multiple scopes, Postman uses the **narrowest** one:




```
Local  >  Data  >  Environment  >  Collection  >  Global
(highest priority)                          (lowest priority)
```

> 🧒 Think of it like **calling your name in a house**: if someone is in **your room** (local) they answer first; then someone in the **corridor** (data); then the **floor** (environment); then the **building** (collection); then the **whole city** (global).

Example:

- Global `baseUrl = https://global.example.com`
- Collection `baseUrl = https://collection.example.com`
- Environment (QA selected) `baseUrl = https://qa.example.com`
- No data or local value. → `{{baseUrl}}` = `https://qa.example.com` (environment beats collection and global).

Now in the pre-request script: `pm.variables.set("baseUrl", "http://localhost:8080");` → `{{baseUrl}}` = `http://localhost:8080` for this request (local beats all).

> 🎯 **Interview trap:** "If I set an environment variable in a script but the value doesn't change in my request, why?" → Because a **data** or **local** variable with the same name has higher precedence, or the request already resolved before the set (e.g. you set it in a post-response script of the same request), or a **different environment** is selected.

5.4 Values: Shared vs Local (formerly Initial vs Current)

This changed in recent Postman versions, so learn both models.

Older model (v10 and earlier v11; still in the VS Code extension and many companies)

Column	Synced to Postman cloud & shared with team?	Used when sending?
Initial value	✅ Yes	Only if current value is empty (it gets copied into current)
Current value	❌ No — stays on your machine	✅ Yes, this is what requests use

- Scripts (`pm.environment.set`) change the **current** value.
- "**Persist**" / "Persist all" copied current → initial (shares it).
- "**Reset**" / "Reset all" copied initial → current.
- Exports include initial values (and optionally current values for environments).

Newer model (later v11 and v12 docs)

- A variable has a single **value** that is **local by default** (the replacement for the old *current* value).
- You may choose to **share** the value with teammates — that becomes the **shared value** (previously *initial* value), based on your local value at the time of sharing.
- Postman no longer supports maintaining multiple independent values for collection, environment and global variables in the UI the way the old two-column model did.
- Persisting/sharing needs **Editor** access.

The golden rule (both models)

> 🔐 **Never put real secrets in the shared/initial value.** Shared values sync to Postman's cloud and are visible to everyone with access (and appear in exports). Put secrets in **local/current values** or, better, in **Postman Vault**, or inject them at run time in CI (`--env-var`).

5.5 Variable types: default vs secret

Type	Behaviour
default (earlier called "any")	Shown in plain text
secret	Value masked (`••••`) in the UI; still a normal variable, still sent; still visible to anyone who unmasks or reads via script; **not** encryption

`secret` type protects against **shoulder-surfing and screenshots** — not against teammates with access. For real secrecy use **Vault**.

5.6 Environments in depth

Create

Sidebar → Environments → `+` → name: `Orders - DEV` .

Variable	Type	Shared value	Local value
baseUrl	default	`https://dev-api.shop.com`	`https://dev-api.shop.com`
clientId	default	`orders-web`	`orders-web`
clientSecret	secret	*(empty!)*	`s3cr3t...` (or `{{vault:orders-client-secret}}`)
accessToken	secret	*(empty)*	*(set by script)*

Create `Orders - QA` , `Orders - STAGING` , `Orders - PROD` with **the same variable names, different values**.

Select

Top-right dropdown. **Only one environment active at a time** (plus globals). Forgetting to select is the #1 beginner bug.

Quick look

Eye icon / variables pane: see active environment, globals, and edit inline.

Environment templates

Commit `orders.dev.template.postman_environment.json` with **empty secrets** to Git. In CI, inject secrets with `--env-var "clientSecret=$CLIENT_SECRET"`.

Protecting production

- Use a **separate PROD environment** with restricted access (only certain people can view/edit; on Enterprise use roles).
- Color/name it clearly: `🔴 PROD — Orders`.
- Add a **collection pre-request guard** that blocks destructive requests in prod:

```
// Collection-level pre-request script
const env = pm.environment.name || "";
const method = pm.request.method;
if (/prod/i.test(env) && ["POST", "PUT", "PATCH", "DELETE"].includes(method)
    && pm.environment.get("allowProdWrites") !== "true") {
    throw new Error(`Blocked ${method} on ${env}. Set allowProdWrites=true locally if you REALLY
mean it.`);
}
```

(In the Postman app, an error thrown in a pre-request script shows "There was an error in evaluating the Pre-request Script" and the request is not sent. **Newman is different (verified with Newman 6.2.3): it reports the error *and still sends the request*.** So a `throw`-based guard does not protect you in CI. For a clean, explicit skip that works in every runner, use `pm.execution.skipRequest()` in the guard instead of `throw`, and log a warning with `console.warn`.)

5.7 Collection variables vs environment variables (common interview question)

	Collection variable	Environment variable
Tied to	One collection	Not tied to a collection; tied to selected environment
Changes per environment?	No	Yes — that's the point
Travels with collection export?	✅ Yes (inside the collection JSON)	❌ Separate file
Best for	Constants: `apiVersion`, resource paths, default page size, schema objects	`baseUrl`, credentials, tokens, env-specific data

Rule of thumb: *"Does it change between Dev/QA/Prod?"* → environment. *"Is it the same everywhere for this API?"* → collection.

5.8 Global variables — when (not) to use

- Visible in **all collections** of the workspace → name collisions, hidden dependencies, "works on my machine".
- Use only for truly cross-collection values (e.g. your personal `myTestEmail`) or quick experiments.
- **In company projects: avoid globals**; prefer collection + environment.

5.9 Data variables (data-driven testing preview)

Data file `users.csv`:

```
email,password,expectedStatus
valid@x.com,Valid@123,200
wrong@x.com,bad,401
,Valid@123,400
```

In request body: `{"email":"{{email}}","password":"{{password}}"}`. In test: `pm.response.to.have.status(Number(pm.iterationData.get("expectedStatus")));` Runner/Newman runs one **iteration per row**. Data file values need care with types: in Newman 6 (verified), CSV values that look like numbers are converted to numbers (`200` → `200`, `007` → `7`, `1e3` → `1000`), while `true`/`false` stay strings. Normalise with `Number(...)` / `String(...)`, and quote-protect codes with leading zeros (ZIP codes, phone numbers) or use a JSON data file. Full details in Chapter 10.

5.10 Local variables

```
pm.variables.set("tempId", 123);   // local — gone after the request/run
pm.variables.get("tempId");        // resolves using precedence (local first)
```

Use when you need to **override** a value for a single request without touching shared state, or to pass data between pre-request script and request.

5.11 Postman Vault (secrets)

> 🧒 The Vault is a **locked diary**. Only you (on your computer) have the key. Your friends can see that you have a diary, but not what's written inside.

- Vault secrets are **stored encrypted locally** on your machine and are **not synced** to the Postman cloud.
- Reference in requests: `{{vault:secret-name}}` (e.g. Authorization → Bearer → `{{vault:github-token}}`).
- Access in scripts (requires enabling vault access for scripts in Vault settings and granting access to the collection/workspace):

```
// pm.vault methods are asynchronous — use await
const apiKey = await pm.vault.get("payments-api-key");
pm.request.headers.upsert({ key: "X-API-Key", value: apiKey });

await pm.vault.set("last-rotated", new Date().toISOString());
await pm.vault.unset("old-key");
```

- **Integrations** (Enterprise): link vault secrets to external secret managers such as **Azure Key Vault, HashiCorp Vault, AWS Secrets Manager**, and password managers (support list varies by version) — so values come from the company's official secret store.
- Limitations to remember: `pm.vault` methods are **not supported** in scheduled collection runs, monitors, the Postman CLI or Newman — in CI, inject secrets as environment variables instead.
- Vault key: you can store/rotate it; if you lose it you may need to reset the vault (secrets lost).

Secrets strategy for a company (best practice)

Where you run	Where secrets come from
Your laptop (manual testing)	Postman Vault (linked to company secret manager if available) or local values
CI/CD (Newman/Postman CLI)	CI secret store (GitHub Actions secrets, Jenkins credentials, Azure Key Vault) → `--env-var`
Monitors	Environment local/shared values used by monitor (limited options; use dedicated low-privilege monitor credentials)
Shared collection	**Nothing secret** in shared values, ever

Also: enable **Postman's secret scanner** (team setting) which detects exposed tokens in public/team workspaces and alerts/auto-masks.

5.12 Dynamic variables (built-in random data)

Postman provides **dynamic variables** (powered by the faker.js library). They generate a **new value every time** they're used.

> 🧒 Like a **magic dice**: every roll gives a new number.

Common

Variable	Example output
`{{$guid}}`	`611c2e81-2ccb-42d8-9ddc-2d0bfa65c1b4`
`{{$randomUUID}}`	`6929bb52-3ab2-448a-9796-d6480ecad36b`
`{{$timestamp}}`	`1767225600` (Unix seconds)
`{{$isoTimestamp}}`	`2026-01-01T00:00:00.000Z`
`{{$randomInt}}`	integer 0–1000
`{{$randomAlphaNumeric}}`	single char, e.g. `6`
`{{$randomBoolean}}`	`true`
`{{$randomColor}}`, `{{$randomHexColor}}`	`teal`, `#47594a`
`{{$randomAbbreviation}}`	`SQL`

Internet / network

`{{$randomIP}}`, `{{$randomIPV6}}`, `{{$randomMACAddress}}`, `{{$randomPassword}}` (15-char alphanumeric), `{{$randomLocale}}`, `{{$randomUserAgent}}`, `{{$randomProtocol}}`, `{{$randomSemver}}`, `{{$randomUrl}}`, `{{$randomDomainName}}`, `{{$randomDomainSuffix}}`, `{{$randomDomainWord}}`, `{{$randomEmail}}`, `{{$randomExampleEmail}}` (uses example.com etc. — safe for tests), `{{$randomUserName}}`.

Names & people

`{{$randomFirstName}}`, `{{$randomLastName}}`, `{{$randomFullName}}`, `{{$randomNamePrefix}}`, `{{$randomNameSuffix}}`, `{{$randomJobTitle}}`, `{{$randomJobArea}}`, `{{$randomJobDescriptor}}`, `{{$randomJobType}}`.

Phone, address, location

`{{$randomPhoneNumber}}`, `{{$randomPhoneNumberExt}}`, `{{$randomCity}}`, `{{$randomStreetName}}`, `{{$randomStreetAddress}}`, `{{$randomCountry}}`, `{{$randomCountryCode}}`, `{{$randomLatitude}}`, `{{$randomLongitude}}`.

Finance & business

`{{$randomPrice}}`, `{{$randomCurrencyCode}}`, `{{$randomCurrencyName}}`, `{{$randomCurrencySymbol}}`, `{{$randomBitcoin}}`, `{{$randomBankAccount}}`, `{{$randomBankAccountName}}`, `{{$randomCreditCardMask}}`, `{{$randomBankAccountBic}}`, `{{$randomBankAccountIban}}`, `{{$randomTransactionType}}`, `{{$randomCompanyName}}`, `{{$randomCompanySuffix}}`, `{{$randomBs}}`, `{{$randomCatchPhrase}}`.

Dates

`{{$randomDateFuture}}`, `{{$randomDatePast}}`, `{{$randomDateRecent}}`, `{{$randomWeekday}}`, `{{$randomMonth}}`.

Commerce, text, files, images

`{{$randomProductName}}`, `{{$randomProduct}}`, `{{$randomProductAdjective}}`, `{{$randomProductMaterial}}`, `{{$randomDepartment}}`, `{{$randomLoremWord}}`, `{{$randomLoremWords}}`, `{{$randomLoremSentence}}`, `{{$randomLoremSentences}}`, `{{$randomLoremParagraph}}`, `{{$randomLoremText}}`, `{{$randomLoremSlug}}`, `{{$randomFileName}}`, `{{$randomFileExt}}`, `{{$randomMimeType}}`, `{{$randomImageUrl}}`, `{{$randomAvatarImage}}`, `{{$randomDatabaseColumn}}`, `{{$randomDatabaseType}}`, `{{$randomDatabaseEngine}}`.

(Type `{{$` in Postman to see the full autocomplete list for your version.)

Using dynamic variables in scripts

```
// Generate once and REUSE (important! each {{$randomEmail}} use generates a new value)
const email = pm.variables.replaceIn("{{$randomExampleEmail}}");
pm.collectionVariables.set("newUserEmail", email);
```

Then body uses `{{newUserEmail}}` and a later test can assert the response email equals the stored value.

> ❌ **Common bug:** Body uses `{{$randomEmail}}` and the test compares `json.email === pm.variables.replaceIn("{{$randomEmail}}")` → always fails, because each call generates a **new** random value. **Generate once, store, reuse.**

Custom random data without dynamic variables

```
const uuid = require("uuid");
pm.variables.set("orderRef", "ORD-" + Date.now());
pm.variables.set("idem", uuid.v4());
pm.variables.set("qty", Math.floor(Math.random() * 10) + 1);   // 1..10
pm.variables.set("tomorrow", new Date(Date.now() + 86400000).toISOString().slice(0, 10));
```

5.13 Scripting API for every scope

```
// ===== Environment =====
pm.environment.set("token", "abc");
pm.environment.get("token");
pm.environment.has("token");            // true/false
pm.environment.unset("token");
pm.environment.clear();                  // removes ALL vars in active env (dangerous!)
pm.environment.toObject();               // { key: value, ... }
pm.environment.name;                     // active environment name
pm.environment.replaceIn("{{baseUrl}}/x");

// ===== Collection =====
pm.collectionVariables.set("orderId", 42);
pm.collectionVariables.get("orderId");
pm.collectionVariables.has / unset / clear / toObject / replaceIn

// ===== Global =====
pm.globals.set("x", 1);  pm.globals.get("x"); pm.globals.unset("x"); pm.globals.clear();

// ===== Data (read only) =====
pm.iterationData.get("email");
pm.iterationData.has("email");
pm.iterationData.toObject();

// ===== Local / resolved =====
pm.variables.set("tmp", 1);              // sets LOCAL scope
pm.variables.get("baseUrl");             // resolves with precedence across ALL scopes
pm.variables.has("baseUrl");
pm.variables.toObject();                 // merged view
pm.variables.replaceIn("{{baseUrl}}/users/{{userId}}");
```

Values are stored as... ?

- You can `set` numbers, booleans, arrays, objects. **When used in `{{ }}` they are converted to strings.**
- Older Postman stored everything as strings; to be safe and portable (Newman versions differ), **store objects with `JSON.stringify` and read with `JSON.parse`**:

```
pm.environment.set("user", JSON.stringify({ id: 7, roles: ["admin"] }));
const user = JSON.parse(pm.environment.get("user"));
```

- Comparing numbers: `pm.environment.get("count") === 5` may fail if stored as `"5"`. Use `Number(...)`.

5.14 Variables in the Runner, Newman and CI

- In the Runner, changes made by scripts during the run apply to the environment's **local/current values**; whether they remain after the run depends on the Runner's "persist variable values"/"keep variable values" option (labels vary by version).

- **Newman** works on a **copy** of the environment file — changes are **not** written back unless you use `--export-environment out.json` (and `--export-globals`, `--export-collection`).
- CLI overrides:
  - Newman: `--env-var "baseUrl=https://qa.shop.com"`, `--global-var "x=1"`
  - Postman CLI: `--env-var "key=value"`, `--global-var "key=value"`
- **Precedence in CI**: `--env-var` sets the value in the environment scope (so a data file column or a local `pm.variables.set` with the same name still wins).

5.15 Debugging variables (systematic checklist)

When `{{something}}` isn't what you expect:

1. **Is it red?** Hover → "Unresolved variable". Check spelling & case.
2. **Is the right environment selected?**
3. **Which scope is winning?** Hover over the variable in URL — Postman shows scope and value. Remember precedence.
4. **Local/current value empty?** In new model check local value; in old model current value might be empty while initial has value (or vice versa).
5. **Is a script overwriting it?** Search collection (Ctrl+Shift+F / "Find and replace") for the name.
6. **Order of execution**: variable set in a request's *post-response* script is available only to **later** requests (or later code in the same script), not the current request's URL.
7. **Console**: `console.log(pm.variables.toObject())`, and Console shows the **final resolved URL & headers** actually sent.
8. **Newman**: did you pass `-e env.json`? Is the variable in the exported file's `values` with `"enabled": true`?
9. **Trailing spaces/newlines** in values (copied tokens!) → 401s. Use `.trim()` when setting from scripts.
10. **Disabled variable** (checkbox unticked) → treated as not existing.

5.16 Naming conventions (company standard)

Kind	Convention	Examples
Base URLs	`<service>BaseUrl` or `baseUrl`	`ordersBaseUrl`, `authBaseUrl`
Credentials	descriptive, never generic	`ordersClientId`, `ordersClientSecret`
Tokens	`<audience>AccessToken`, plus expiry	`accessToken`, `accessTokenExpiresAt`
Runtime IDs (set by scripts)	`<resource>Id`	`orderId`, `customerId`
Feature flags for tests	`is<Feature>` / `allow<Action>`	`allowProdWrites`
Prefix for script-internal vars	`_`	`_retryCount`, `_pageCursor`

- camelCase consistently. No spaces. Avoid names that clash with dynamic vars (`$...`).

- Document each variable's purpose in the environment/collection description.

5.17 Chapter 5 Quick Revision

- Scopes: Global, Collection, Environment, Data, Local (+ Vault). Precedence: **Local > Data > Environment > Collection > Global**.
- Old model: Initial (shared) vs Current (local, used). New model: single local value + optional shared value.
- Secret type = masking only; Vault = encrypted local secrets (`{{vault:name}}`, `await pm.vault.get()`), not supported in Newman/CLI/monitors.
- Dynamic variables (`{{$guid}}`, `{{$randomEmail}}`) change each use → generate once with `replaceIn`, store, reuse.
- `pm.variables.set` = local; `pm.variables.get` = resolved with precedence.
- Newman doesn't write back unless `--export-environment`.

➡️ Next: **Chapter 6 — Authorization and Authentication in Postman**.

Chapter 6 — Authorization & Authentication in Postman (Every Type, In Depth)

> 🧒 Every API is a **club with a guard**. Different clubs check you differently: some want a **secret password** (API key), some want a **wristband** they gave you earlier (token), some want you to **ask your parent** to sign a permission slip (OAuth), some want a **stamp that proves the letter wasn't changed** (signatures like AWS SigV4, Hawk, OAuth 1.0).

6.1 The Authorization tab

Every request, folder and collection has an **Authorization** tab with a **Type** dropdown:

Type	Summary
Inherit auth from parent	Use folder/collection auth (default for new requests in collections)
No Auth	Send nothing
API Key	Key/value in header or query
Bearer Token	`Authorization: Bearer <token>`
JWT Bearer	Postman *generates & signs* a JWT for you
Basic Auth	`Authorization: Basic base64(user:pass)`
Digest Auth	Challenge–response with hashing
OAuth 1.0	Signed requests (Twitter/X legacy, some enterprise)
OAuth 2.0	Token flows (Authorization Code, PKCE, Client Credentials, Password, Implicit)
Hawk Authentication	HMAC-signed requests
AWS Signature	AWS SigV4 signing
NTLM Authentication	Windows/IIS challenge–response
Akamai EdgeGrid	Akamai APIs
ASAP (Atlassian)	Service-to-service JWT for Atlassian

Postman builds the needed headers/params **at send time**. You can see the final result in the **Console** or the hidden headers.

> 🏢 **Best practice:** set auth **once at collection level** (`{{accessToken}}`), make every request **Inherit**, and only override for special requests (e.g. the login request itself = **No Auth**, negative tests = **No Auth** or a wrong token).

6.2 API Key

Fields: **Key**, **Value**, **Add to**: Header or Query Params.

```
Key:   X-API-Key
Value: {{vault:weather-api-key}}
Add to: Header
→ X-API-Key: 9f8e7d...
```

or Query → `?api_key=9f8e7d...`

Notes:

- Prefer **headers** — query strings end up in server logs, browser history, proxies.
- API keys identify an **application**, not usually a **user**.
- Test cases: missing key (401), invalid key (401/403), revoked key, key for wrong environment, key in wrong place.

6.3 Bearer Token

Field: **Token** → sends `Authorization: Bearer <token>`.

- "Bearer" = whoever **bears** (holds) the token gets access — so protect it like a password.
- Tokens are often **JWTs** or opaque random strings.
- Use `{{accessToken}}` and fill it automatically via a login script (6.13).

6.4 JWT — what it is and how Postman handles it

Structure

`xxxxx.yyyyy.zzzzz` = **Header.Payload.Signature**, each **Base64URL**-encoded.

```
// Header
{ "alg": "HS256", "typ": "JWT" }
// Payload (claims)
{ "sub": "user-42", "name": "Asha", "roles": ["admin"], "iat": 1767225600, "exp": 1767229200,
"iss": "auth.shop.com", "aud": "orders-api" }
// Signature = HMACSHA256(base64url(header) + "." + base64url(payload), secret)
```

Standard claims: `iss` (issuer), `sub` (subject), `aud` (audience), `exp` (expiry, Unix seconds), `nbf` (not before), `iat` (issued at), `jti` (JWT ID).

> 🔑 JWT payload is **encoded, not encrypted**. Anyone can decode it (jwt.io). Never put passwords in a JWT. (Encrypted JWTs = **JWE**, a different thing.)

Decode a JWT in a Postman script

```
function decodeJwt(token) {
    const part = token.split(".")[1];
    const base64 = part.replace(/-/g, "+").replace(/_/g, "/")
                       .padEnd(part.length + (4 - part.length % 4) % 4, "=");
    return JSON.parse(atob(base64));
}
const claims = decodeJwt(pm.environment.get("accessToken"));
console.log(claims);
pm.test("Token has admin role", () => pm.expect(claims.roles).to.include("admin"));
pm.test("Token not expired", () => pm.expect(claims.exp * 1000).to.be.above(Date.now()));
```

JWT Bearer auth type (Postman generates the JWT)

Fields: **Algorithm** (HS256/384/512, RS256/384/512, PS256/384/512, ES256/384/512), **Secret** (HMAC) or **Private key** (RSA/EC, paste or file), **Secret Base64 encoded** checkbox, **Payload** (JSON, variables allowed), **Advanced**: request header prefix (default `Bearer`), JWT headers (e.g. `kid`), add token to Header or Query param. Use for services that accept **self-signed client JWTs** (e.g. service-to-service, Google service accounts style assertions, Apple APNs-style).

JWT security tests you can do in Postman

- Change one character of signature → expect 401.
- Expired token (`exp` in past) → 401.
- `alg: none` token → must be rejected.
- Token for a different `aud` (other service) → must be rejected.
- Tamper payload role `user` → `admin` without re-signing → 401.

6.5 Basic Auth

Fields: **Username**, **Password** → `Authorization: Basic YXNoYTpzZWNyZXQ=` (= base64 of `asha:secret`).

- **Base64 is encoding, NOT encryption** — anyone can decode. Only safe over **HTTPS**.
- Manually: `btoa("asha:secret")` in a script.
- Restful-Booker accepts Basic `admin:password123` for PUT/DELETE as an alternative to the cookie token.

6.6 Digest Auth

How it works:

1. Client sends request without credentials.
2. Server replies `401` with `WWW-Authenticate: Digest realm="...", nonce="...", qop="auth", opaque="...", algorithm=MD5`.
3. Client hashes `username:realm:password`, method/URI and nonce into a response hash and resends.

Postman fields: Username, Password; Advanced: Realm, Nonce, Algorithm (MD5, MD5-sess, SHA-256, SHA-256-sess, SHA-512-256, SHA-512-256-sess), qop, Nonce Count, Client Nonce, Opaque. Postman by default **automatically performs the two-step handshake** (it extracts nonce/realm from the 401 and retries); a checkbox lets you **disable retrying** to see the raw 401.

Try: `GET https://httpbin.org/digest-auth/auth/user/passwd` with user/passwd.

6.7 OAuth 1.0

Every request is **signed** using consumer & token secrets. Fields: **Signature Method** (HMAC-SHA1, HMAC-SHA256, HMAC-SHA512, RSA-SHA1, RSA-SHA256, RSA-SHA512, PLAINTEXT), **Consumer Key**, **Consumer Secret**, **Access Token**, **Token Secret**; Advanced: Callback URL, Verifier, Timestamp, Nonce, Version (1.0), Realm, **Include body hash**, **Add empty parameters to signature**; **Add authorization data to**: Request Body/URL or Request Headers.

Postman generates `oauth_nonce`, `oauth_timestamp`, `oauth_signature` each send. Common bugs: **clock skew** (timestamp too far from server time → 401), wrong URL encoding of params in signature base string, body params not included when they should be.

6.8 OAuth 2.0 — The Big One

> 🧒 You want to let a **photo-printing app** use your **Google Photos**, but you don't want to give it your Google password. So Google gives the app a **special ticket** (access token) that only allows "read photos", only for a while. That's OAuth 2.0.

Roles

Role	Example
Resource Owner	You, the user
Client	The app (or Postman acting as the app)

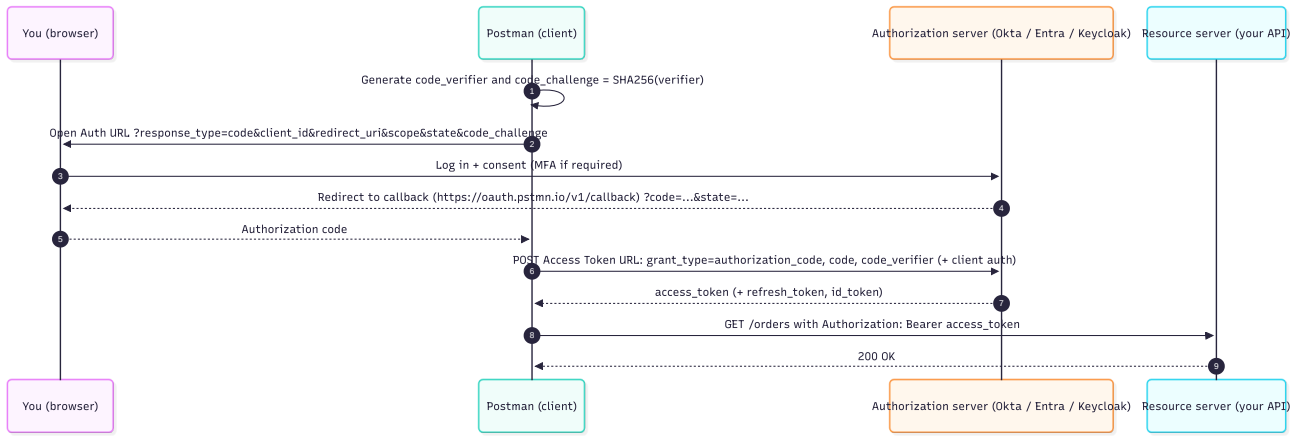
Role	Example
Authorization Server	Google/Okta/Auth0/Keycloak/Azure AD (Entra ID) — issues tokens
Resource Server	The API that accepts tokens

Tokens

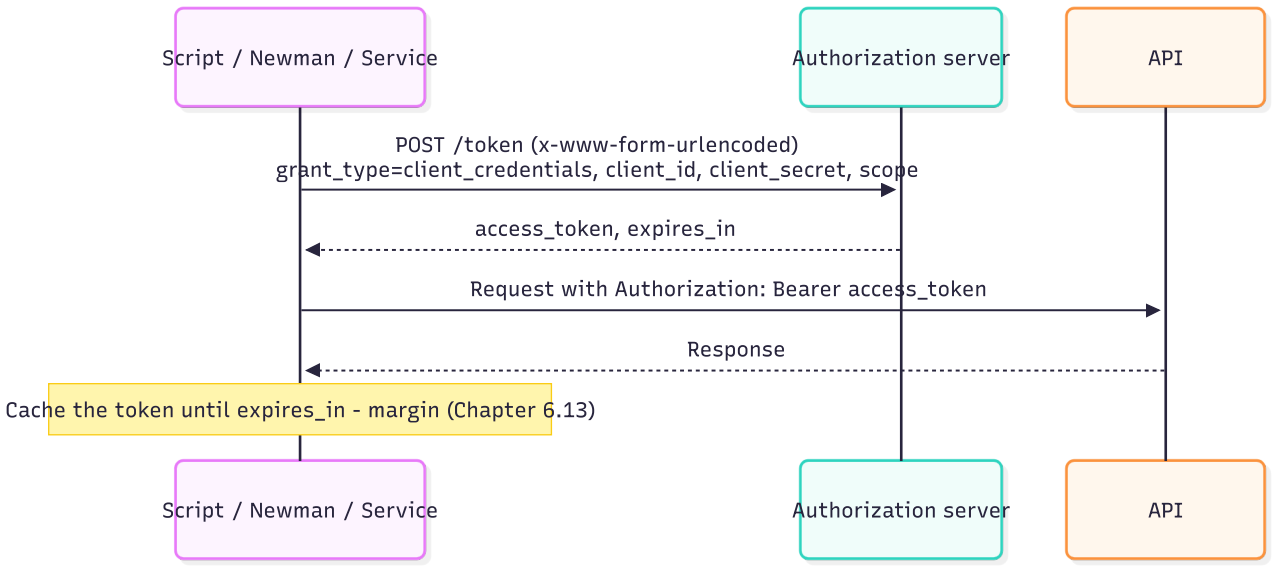
- **Access token** — short-lived (minutes–hours), sent with API calls.
- **Refresh token** — long-lived, used to get new access tokens without user login.
- **ID token** — (OpenID Connect) a JWT describing the **user's identity**; not meant for calling APIs.
- **Scopes** — permissions requested: `orders:read orders:write`.

Grant types (flows) in Postman

Authorization Code with PKCE, the flow Postman runs when you click *Get New Access Token*:



Client Credentials (machine-to-machine, the one that works in CI):



Grant type	Who uses it	Steps	Notes
Authorization Code	Web apps with a backend	Browser login → `code` → exchange code + client secret for token	Most common for user login
Authorization Code (With PKCE)	SPAs, mobile apps, and now recommended for **all** clients	Like above + `code_verifier` / `code_challenge`	OAuth 2.1 makes PKCE mandatory
Implicit	Old SPAs	Token returned directly in redirect URL fragment	**Deprecated** (insecure); avoid
Password Credentials (ROPC)	Legacy trusted first-party apps	Send username + password directly to token endpoint	**Deprecated** in OAuth 2.1; still common in internal test setups
Client Credentials	**Machine-to-machine** (service → service, cron jobs)	Send client_id + client_secret → token	No user involved; very common in microservices

(Other grants exist outside Postman's dropdown — Device Code, Refresh Token, JWT Bearer assertion, Token Exchange — you can do them with a manual POST request.)

Postman OAuth 2.0 configuration fields

Current token section:

- **Token** dropdown (select from stored tokens), **Header Prefix** (`Bearer`), **Add authorization data to**: Request Headers / Request URL, **Auto-refresh token** (uses refresh token when access token expires), **Share token** (share token with workspace collaborators — careful!).

Configure New Token section:

Field	Meaning
Token Name	Label for this token in Postman's token manager
Grant Type	one of the five above
Callback URL	Redirect URI registered with the auth server. Tick **Authorize using browser** to use `https://oauth.pstmn.io/v1/callback` (register this in your IdP!) — Postman opens your system browser and receives the code
Auth URL	e.g. `https://login.example.com/oauth2/authorize`
Access Token URL	e.g. `https://login.example.com/oauth2/token`
Client ID / Client Secret	From your app registration (use `{{vault:...}}`)
Code Challenge Method	SHA-256 (recommended) or Plain — PKCE only
Code Verifier	Auto-generated if blank — PKCE only
Scope	Space-separated

Field	Meaning
State	Random anti-CSRF string
Client Authentication	**Send as Basic Auth header** or **Send client credentials in body** — must match what the server expects!
Advanced: Refresh Token URL, Auth Request params, Token Request params, Refresh Request params, `audience`, `resource`	Auth0 needs `audience`; Azure AD v1 uses `resource`

Then **Get New Access Token** → login window/browser → token appears → **Use Token**. **Manage tokens**: delete expired tokens, view details.

Example: Client Credentials (Keycloak)

- Access Token URL: `https://keycloak.company.com/realms/shop/protocol/openid-connect/token`
- Client ID: `orders-service`
- Client Secret: `{{vault:orders-client-secret}}`
- Scope: `orders:read`
- Client Authentication: Send as Basic Auth header

Example: Azure AD / Entra ID client credentials

- Token URL: `https://login.microsoftonline.com/{{tenantId}}/oauth2/v2.0/token`
- Scope: `api://{{apiAppId}}/.default`

Common OAuth errors & fixes

Error	Meaning	Fix
`invalid_client`	Wrong client id/secret, or wrong client-auth method (header vs body)	Toggle "Client Authentication"; re-copy secret (trailing space!)
`invalid_grant`	Code expired/used, wrong redirect URI, wrong user creds, refresh token revoked	Get a fresh code; check redirect URI matches exactly
`redirect_uri_mismatch`	Callback URL not registered	Register `https://oauth.pstmn.io/v1/callback` (or your exact URL) in IdP
`unauthorized_client`	Client not allowed that grant type	Enable grant in IdP app settings
`invalid_scope`	Scope not allowed/misspelled	Check IdP config
`unsupported_grant_type`	Typo or server doesn't support	Check `grant_type` value
Token works in Postman UI, fails in Newman	OAuth 2 "Get New Access Token" is **interactive**; Newman can't do the browser step	Use client credentials/password grant via a **script or setup request** in CI
401 after ~1 hour	Access token expired	Auto-refresh, or script-based refresh (6.13)

Error	Meaning	Fix
Browser login window loops/blank	Cookies/SSO in embedded window	Use "Authorize using browser"

6.9 Hawk Authentication

Fields: Hawk Auth ID, Hawk Auth Key, Algorithm (sha256/sha1); Advanced: User, Nonce, ext, app, dlg, Timestamp, **Include payload hash**. Produces an `Authorization: Hawk id="...", ts="...", nonce="...", mac="..."` header (HMAC over request parts). Clock skew matters.

6.10 AWS Signature (SigV4)

Fields: **AccessKey**, **SecretKey**, Advanced: **AWS Region** (e.g. `ap-south-1`), **Service Name** (e.g. `execute-api`, `s3`, `es`, `lambda`), **Session Token** (for temporary STS credentials), Add authorization data to: Request Headers / URL.

Postman computes `Authorization: AWS4-HMAC-SHA256 Credential=..., SignedHeaders=..., Signature=...` plus `X-Amz-Date`, `X-Amz-Content-Sha256` (S3), `X-Amz-Security-Token`.

Common errors:

- `SignatureDoesNotMatch` — wrong secret, wrong region/service, a header changed after signing, URL encoding differences.
- `The security token included in the request is expired` — refresh STS creds.
- `Signature expired: ... is now earlier than ...` — your machine clock is off.
- Use IAM users/roles with **least privilege** for testing.

6.11 NTLM

Fields: Username, Password; Advanced: Domain, Workstation. Used for Windows-integrated IIS / SharePoint / legacy .NET APIs. Postman performs the multi-step NTLM handshake. (Kerberos/Negotiate is not offered as a built-in type.)

6.12 Akamai EdgeGrid & ASAP

- **Akamai EdgeGrid**: Access Token, Client Token, Client Secret, Nonce, Timestamp, Base URL, Headers to Sign — for Akamai's control APIs.
- **ASAP (Atlassian Service Authentication Protocol)**: Algorithm, Issuer, Audience, Key ID, Private key, Subject, additional claims, expiry — Postman issues a signed JWT for service-to-service calls.

6.13 Automating login & token refresh (THE pattern every company uses)

Pattern A — Login request + post-response script

Request `POST {{baseUrl}}/auth/login` (Auth: **No Auth**), body:

```
{ "username": "{{username}}", "password": "{{password}}" }
```

Post-response script:

```
pm.test("Login succeeded", () => pm.response.to.have.status(200));
const { accessToken, refreshToken, expiresIn } = pm.response.json();
pm.environment.set("accessToken", accessToken);
pm.environment.set("refreshToken", refreshToken);
pm.environment.set("accessTokenExpiresAt", Date.now() + (expiresIn - 30) * 1000); // 30s safety
margin
```

Collection auth: Bearer `{{accessToken}}`.

Pattern B — Collection-level pre-request script that fetches/refreshes a token only when needed (best for CI + manual)

```
// Collection > Scripts > Pre-request
const tokenUrl    = pm.environment.get("tokenUrl");
const clientId    = pm.environment.get("clientId");
const clientSecret = pm.environment.get("clientSecret");
const expiresAt   = Number(pm.environment.get("accessTokenExpiresAt") || 0);
const token       = pm.environment.get("accessToken");

// Skip for the login/token request itself
if (pm.info.requestName === "Get access token") return;

if (token && Date.now() < expiresAt) {
    return; // still valid, nothing to do
}

pm.sendRequest({
    url: tokenUrl,
    method: "POST",
    header: { "Content-Type": "application/x-www-form-urlencoded" },
    body: {
        mode: "urlencoded",
        urlencoded: [
            { key: "grant_type", value: "client_credentials" },
            { key: "client_id", value: clientId },
            { key: "client_secret", value: clientSecret },
            { key: "scope", value: pm.environment.get("scope") || "" }
        ]
    }
}, (err, res) => {
    if (err) { console.error("Token request failed", err); return; }
    if (res.code !== 200) { console.error("Token endpoint returned", res.code, res.text());
return; }
```

```
    const body = res.json();
    pm.environment.set("accessToken", body.access_token);
    pm.environment.set("accessTokenExpiresAt", Date.now() + (body.expires_in - 30) * 1000);
    console.log("New token fetched, expires in", body.expires_in, "s");
});
```

Why this works: Postman **waits for `pm.sendRequest` callbacks to finish** before sending the main request, so the main request uses the new token.

> Note: `return` at the top level of a Postman script works because scripts run inside a function wrapper. If your linter complains, wrap logic in an `if/else` instead.

Async/await version — works in recent Postman **app** versions only (top-level `await`, promise-returning `pm.sendRequest`):

```
const res = await pm.sendRequest({ url: pm.environment.get("tokenUrl"), method: "POST", /* ... */
});
pm.environment.set("accessToken", res.json().access_token);
```

Newman 6.x does not support this (verified: top-level `await` is a SyntaxError there). The callback version above was run successfully in Newman 6.2.3 — including the "fetch once, then reuse the cached token" behaviour — so prefer it for collections that run in CI.

Pattern C — Retry once on 401 (post-response)

```
if (pm.response.code === 401 && !pm.variables.get("_retried")) {
    pm.environment.unset("accessToken");           // force refresh in pre-request
    pm.variables.set("_retried", true);
    pm.execution.setNextRequest(pm.info.requestId); // rerun this request (Runner/CLI/Newman
only)
}
```

6.14 Cookie/session-based auth and CSRF tokens

Flow (Django/Laravel/Spring Security style):

1. `GET /login` → server sets `csrftoken` cookie (or returns token in HTML/meta).
2. `POST /login` with header `X-CSRFToken: <value>` + form data → `Set-Cookie: sessionid=...`.
3. Subsequent requests carry the session cookie automatically + CSRF header for unsafe methods.

Script to copy CSRF cookie into header:

```
// Pre-request of POST/PUT/DELETE requests
const csrf = pm.cookies.get("csrftoken");      // pm.cookies = cookies for the current request's
domain
if (csrf) pm.request.headers.upsert({ key: "X-CSRFToken", value: csrf });
```

For programmatic jar access to other domains:

```
const jar = pm.cookies.jar();
// The domain must be in the cookie allowlist (Cookies > Domains Allowlist)
jar.get("https://app.shop.com", "XSRF-TOKEN", (err, value) => {
    if (!err) pm.environment.set("xsrf", decodeURIComponent(value));
});
jar.set("https://app.shop.com", "feature_flag", "beta", (err, cookie) => {});
jar.unset("https://app.shop.com", "feature_flag", (err) => {});
jar.clear("https://app.shop.com", (err) => {});
```

Extract a token from HTML (e.g. hidden form field):

```
const $ = cheerio.load(pm.response.text());
pm.environment.set("csrf", $('input[name="_token"]').val());
```

6.15 Mutual TLS (client certificates)

Settings → **Certificates** → **Add certificate**:

- **Host** (e.g. `api.bank.com`, wildcards like `*.bank.com` supported) and **Port** (default 443),
- **CRT file** + **KEY file** (PEM), **or PFX file**,
- **Passphrase**. Postman sends the cert when the host matches. Newman: `--ssl-client-cert` , `--ssl-client-key` , `--ssl-client-passphrase` , or `--ssl-client-cert-list` (JSON list of host-matched certs). Postman CLI has similar options.

Errors: `socket hang up` / `ECONNRESET` / `alert handshake failure` → cert not sent (host/port mismatch), wrong format, cert not trusted by server, or expired.

6.16 Auth testing checklist (QA & security)

1. ✅ Valid credentials → 200.
2. ❌ No credentials → 401 (not 500, not 200!).
3. ❌ Invalid/expired/tampered token → 401.
4. ❌ Valid token, insufficient role/scope → 403.
5. ❌ User A's token accessing User B's resource → 403/404 (**BOLA/IDOR**).
6. ❌ Token in query string when only header allowed → rejected.
7. ❌ Brute-force login: multiple wrong passwords → lockout/429.
8. ✅ Logout invalidates token/session (server-side) → old token 401.
9. ✅ Refresh token rotation: old refresh token can't be reused.
10. ✅ Sensitive data not in responses (password hashes, secrets).
11. ✅ Error messages don't reveal whether username exists ("Invalid credentials" vs "User not found").
12. ✅ HTTPS enforced; HTTP redirected or refused.

6.17 Chapter 6 Quick Revision

- Set auth at collection level; requests inherit; override for login/negative tests.
- Basic = Base64 (not encryption). Bearer = holder gets access. JWT = header.payload.signature, encoded not encrypted.
- OAuth 2 grants: Auth Code (+PKCE recommended), Client Credentials (M2M), Password & Implicit deprecated.
- `https://oauth.pstmn.io/v1/callback` for browser-based auth in Postman.
- Interactive OAuth doesn't work in Newman → script-based token fetch.
- Pre-request token caching with expiry is the professional pattern.
- mTLS via Settings → Certificates.

➡️ Next: **Chapter 7 — Scripting in Postman (Complete `pm` API)**.

Chapter 7 — Scripting in Postman: The Complete `pm` API

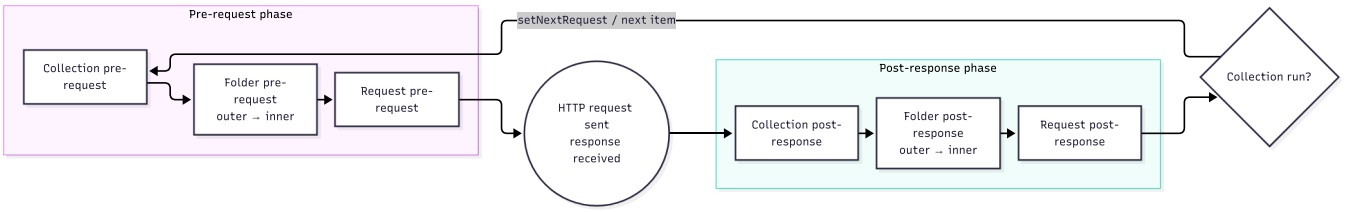
> 🧒 Scripts are **little helpers** you hire. A **pre-request helper** gets things ready *before* the letter is sent (writes today's date, fetches a key). A **post-response helper** reads the reply *after* it arrives and checks if it's correct, writes notes for the next letter, and shouts if something's wrong.

7.1 Where scripts live

Location	Pre-request	Post-response
Collection	✅	✅
Folder	✅	✅
Request	✅	✅

UI: **Scripts** tab → **Pre-request** / **Post-response** (older UI: "Pre-request Script" tab and "Tests" tab — same thing). In exported JSON: `"listen": "prerequest"` and `"listen": "test"`.

7.2 Execution order (memorise this!)



For one request inside `Collection > Folder A > Folder B > Request`:

```
1. Collection  pre-request
2. Folder A    pre-request
3. Folder B    pre-request
4. Request     pre-request
5. ──► HTTP request is sent, response received ◄──
6. Collection  post-response
7. Folder A    post-response
8. Folder B    post-response
9. Request     post-response
```

- In a Collection Run this cycle repeats for **every request**, for **every iteration**.
- Collection/folder scripts therefore run **once per request**, not once per run. To do something only once per run, guard it:

```
// Collection pre-request — run only before the first request of the first iteration
if (pm.info.iteration === 0 && !pm.variables.get("_initDone")) {
    pm.variables.set("_initDone", true);   // local: lives for the whole run
    console.log("One-time setup");
}
```

> (Local variables set with `pm.variables.set` persist for the duration of a collection run, so this flag works in Runner/Newman. For a single manual send, it's per request.)

7.3 The Sandbox — what scripts can and can't do

Postman scripts run in the **Postman Sandbox**, a JavaScript runtime (ES6+/modern JS) based on Node.js but restricted.

✅ You can:

- Use modern JS: `let/const`, arrow functions, template literals, destructuring, classes, `async/await`, Promises, `Map/Set`, optional chaining `?.`, nullish `??`.
- Use `setTimeout` / `setInterval` / `clearTimeout` (the run waits for pending timers).
- Make HTTP calls with `pm.sendRequest`.
- `require()` a fixed list of built-in libraries (7.15) and `pm.require()` packages from the **Package Library** (7.16).
- `console.log/info/warn/error` → **Postman Console**.

❌ You can't:

- Access the file system (`fs`), spawn processes, open sockets directly.
- Access the DOM/`window` / `document` (no browser). (Only Visualizer templates run in a browser-like frame.)
- `require` arbitrary npm packages directly (use Package Library / supported external registries instead).
- Share JS variables between requests — each script has its own scope. **Use Postman variables** to pass data.

7.4 `pm.info` — where am I?

```
pm.info.eventName;       // "prerequest" or "test" (post-response)
pm.info.iteration;       // current iteration index, starts at 0
pm.info.iterationCount;  // total iterations in this run
pm.info.requestName;     // name of the request
pm.info.requestId;       // unique id of the request
```

Usage: one shared collection-level script that behaves differently per event:

```
if (pm.info.eventName === "prerequest") { /* ... */ } else { /* post-response */ }
```

7.5 `pm.request` — read and modify the request (pre-request) or inspect it (post-response)

```
pm.request.method;                  // "POST"
pm.request.url.toString();          // full URL (variables may be unresolved here)
pm.request.url.getHost();           // "api.shop.com" (or "{{baseUrl}}" if unresolved)
pm.request.url.getPath();           // "/v1/orders"
pm.request.url.query.toObject();    // { page: "1" }
pm.request.headers.toObject();      // { accept: "application/json", ... }
pm.request.headers.get("Accept");
pm.request.body;                    // RequestBody object: mode, raw, urlencoded, formdata, file,
graphql
pm.request.auth;                    // auth definition
```

Modify (pre-request only — changes affect what's sent)

```
// Headers
pm.request.headers.add({ key: "X-Correlation-ID", value: pm.variables.replaceIn("{{$guid}}") });
pm.request.headers.upsert({ key: "Accept-Language", value: "en-IN" }); // add or replace
// Note: add() does NOT replace — if the header already exists you send it twice
// (servers usually see "X-A: 1, 2"). Use upsert() when a collection script may already have set
it.
pm.request.headers.remove("X-Debug");

// Query params
pm.request.url.query.add({ key: "ts", value: Date.now().toString() });
pm.request.url.query.upsert({ key: "page", value: "2" });
pm.request.url.query.remove("debug");

// Change the URL entirely
pm.request.url = pm.variables.replaceIn("{{baseUrl}}/v2/orders");

// Change method
pm.request.method = "PUT";

// Modify a raw JSON body
const body = JSON.parse(pm.request.body.raw || "{}");
body.requestedAt = new Date().toISOString();
pm.request.body.raw = JSON.stringify(body);

// ⚠️ If the request was saved WITHOUT a body (e.g. a GET), pm.request.body is undefined,
// so pm.request.body.update(...) throws "Cannot read properties of undefined (reading
'update')".
// Assigning a plain object works in that case (verified in Newman 6):
// pm.request.body = { mode: "raw", raw: JSON.stringify({ a: 1 }), options: { raw: { language:
"json" } } };

// Or replace an existing body using update()
pm.request.body.update({
    mode: "raw",
    raw: JSON.stringify({ hello: "world" }),
    options: { raw: { language: "json" } }
```

```
});

// urlencoded body
pm.request.body.urlencoded.add({ key: "grant_type", value: "client_credentials" });
```

⚠️ `pm.request.body.raw` contains **unresolved** `{{variables}}`. If you `JSON.parse` a body like `{"age": {{age}}}` it throws (invalid JSON). Use `JSON.parse(pm.variables.replaceIn(pm.request.body.raw))` to parse after resolving.

Example: HMAC request signing (very common in fintech)

```
// Pre-request: sign body with HMAC-SHA256 and add headers
const secret = pm.environment.get("hmacSecret");
const ts = Math.floor(Date.now() / 1000).toString();
const body = pm.variables.replaceIn(pm.request.body.raw || "");
const path = pm.request.url.getPathWithQuery();
const payload = `${ts}\n${pm.request.method}\n${path}\n${body}`;
const signature = CryptoJS.HmacSHA256(payload, secret).toString(CryptoJS.enc.Hex);

pm.request.headers.upsert({ key: "X-Timestamp", value: ts });
pm.request.headers.upsert({ key: "X-Signature", value: signature });
```

(`CryptoJS` is available globally, or via `require("crypto-js")`.)

Tip: if the body you sign must be **byte-identical** to what's sent, set the final resolved body back: `pm.request.body.raw = body;` so Postman doesn't re-serialise differently.

7.6 `pm.response` — the response (post-response scripts)

```
pm.response.code;            // 200
pm.response.status;          // "OK"
pm.response.responseTime;    // ms (number)
pm.response.responseSize;    // bytes
pm.response.headers;         // HeaderList
pm.response.headers.get("Content-Type");   // case-insensitive
pm.response.headers.has("ETag");
pm.response.headers.toObject();
pm.response.cookies;         // CookieList from this response (⚠️ found EMPTY in Newman 6 — use
pm.cookies or parse Set-Cookie headers)
pm.response.text();          // body as string
pm.response.json();          // body parsed as JSON (throws if not JSON!)
```

Safe JSON parsing:

```
let json;
try {
    json = pm.response.json();
} catch (e) {
    pm.test("Response is valid JSON", () => { throw new Error("Body is not JSON: " +
```

```
pm.response.text().slice(0, 200)); });
}
```

XML response:

```
const obj = xml2Json(pm.response.text());   // legacy global, still available
// or
const xml2js = require("xml2js");
xml2js.parseString(pm.response.text(), { explicitArray: false }, (err, result) => {
    pm.test("Has user name", () => pm.expect(result.user.name).to.eql("Asha"));
});
```

HTML response:

```
const $ = cheerio.load(pm.response.text());
pm.test("Title is correct", () => pm.expect($("title").text()).to.include("Dashboard"));
```

7.7 Variables API (recap — full details in Chapter 5)

```
pm.variables.get/set/has/replaceIn/toObject        // resolved / local
pm.environment.get/set/has/unset/clear/toObject/name/replaceIn
pm.collectionVariables.get/set/has/unset/clear/toObject/replaceIn
pm.globals.get/set/has/unset/clear/toObject/replaceIn
pm.iterationData.get/has/toObject                  // read-only data file row
await pm.vault.get/set/unset                       // secrets (async)
```

7.8 `pm.sendRequest` — call other APIs from a script

Use for: fetching tokens, creating prerequisite data, cleanup, polling, verifying side effects (e.g. "after POST /order, GET /inventory decreased").

Simple GET

```
pm.sendRequest("https://postman-echo.com/get?x=1", (err, res) => {
    if (err) return console.error(err);
    console.log(res.code, res.json());
});
```

Full request object

```
pm.sendRequest({
    url: pm.variables.replaceIn("{{baseUrl}}/inventory/{{sku}}"),
    method: "GET",
    header: {
        "Authorization": "Bearer " + pm.environment.get("accessToken"),
        "Accept": "application/json"
```

```
    }
}, (err, res) => {
    pm.test("Inventory decreased after order", () => {
        pm.expect(err).to.be.null;
        pm.expect(res.json().available).to.eql(Number(pm.environment.get("stockBefore")) - 1);
    });
});
```

POST with JSON body

```
pm.sendRequest({
    url: pm.variables.replaceIn("{{baseUrl}}/customers"),
    method: "POST",
    header: { "Content-Type": "application/json" },
    body: { mode: "raw", raw: JSON.stringify({ name: "Test User", email:
`qa+${Date.now()}@example.com` }) }
}, (err, res) => {
    pm.environment.set("customerId", res.json().id);
});
```

Body modes: `raw`, `urlencoded` (array of `{key,value}`), `formdata` (array of `{key,value,type}`; file uploads from sendRequest are limited), `file`, `graphql` (`{ query, variables }`).

Promise / async-await

Recent Postman **app** versions support top-level `await` and `pm.sendRequest` returning a promise when no callback is passed:

```
const res = await pm.sendRequest({ url: "https://postman-echo.com/get", method: "GET" }); //
Postman app (recent) only
```

⚠️ **Verified while writing this book: Newman 6.2.3 rejects top-level `await`** (`SyntaxError: await is only valid in async functions...`) and its `pm.sendRequest` does **not** return a promise. If your collection also runs in Newman, use this portable pattern:

```
// Works in the Postman app AND in Newman 6 (verified): wrap the callback API in a Promise
const sendRequestAsync = (req) => new Promise((resolve, reject) =>
    pm.sendRequest(req, (err, res) => err ? reject(err) : resolve(res)));

(async () => {
    const r1 = await sendRequestAsync(pm.variables.replaceIn("{{baseUrl}}/users/3"));
    const r2 = await sendRequestAsync(pm.variables.replaceIn("{{baseUrl}}/users/" +
r1.json().id));
    pm.test("chained calls work", () => pm.expect(r2.code).to.eql(200));
})().catch(err => pm.test("async block failed", () => { throw err; }));
```

Parallel requests

```
const ids = [1, 2, 3];
let done = 0;
ids.forEach(id => {
    pm.sendRequest(`https://jsonplaceholder.typicode.com/users/${id}`, (err, res) => {
```

```
        pm.test(`User ${id} exists`, () => pm.expect(res.code).to.eql(200));
        done++;
    });
});
// Postman waits for all callbacks before finishing the script.
```

Important facts:

- `pm.sendRequest` calls are **not** shown as separate items in Runner results, but **are** in the Console.
- They **do not** inherit the collection's auth or headers automatically — add them yourself.
- They don't run that request's scripts (it's a raw call).
- Variables inside the URL string are **not auto-resolved** — use `pm.variables.replaceIn()`.
- The **cookie jar** is shared, so `Set-Cookie` from a `sendRequest` may affect later requests.

7.9 `pm.execution` — control the run

```
pm.execution.setNextRequest("Request name or ID");  // jump (Runner/Newman/CLI only)
pm.execution.setNextRequest(null);                  // stop the run after this request
pm.execution.skipRequest();                          // pre-request only: don't send this request
pm.execution.location;                               // array path e.g. ["My
Collection","Folder","Request"]
pm.execution.location.current;                       // the current item's name/location in the
path
```

- `setNextRequest` works in **pre-request or post-response**; the **last value set wins**; it only applies to **collection runs** (no effect on a single manual Send).
- The target must be in the **same collection run** (if you run only a folder, it must be within that run's scope).
- Prefer **request ID** (`pm.info.requestId` of the target, or copy from the request info pane) over **name** — names change and can be duplicated.
- Legacy equivalent: `postman.setNextRequest("name")` — still works in many versions; prefer `pm.execution.setNextRequest`.
- `skipRequest()` **must** be in **pre-request**; in post-response it throws `TypeError: pm.execution.skipRequest is not a function` (exact message verified in Newman 6). When skipped, the request and its post-response scripts don't run.

Full workflow patterns in Chapter 9.

7.10 `pm.cookies`

```
pm.cookies.has("sessionId");
pm.cookies.get("sessionId");
pm.cookies.toObject();
const jar = pm.cookies.jar();   // programmatic jar (domains must be allowlisted)
jar.get(url, name, cb); jar.getAll(url, cb); jar.set(url, name, value, cb); jar.set(url, {name,
```

```
value, httpOnly:true}, cb);
jar.unset(url, name, cb); jar.clear(url, cb);
```

7.11 `pm.test` and `pm.expect` (testing — full chapter next)

```
pm.test("Status is 201", function () {
    pm.response.to.have.status(201);
});
pm.test("Has order id", () => {
    const json = pm.response.json();
    pm.expect(json).to.have.property("id");
    pm.expect(json.id).to.be.a("number");
});
pm.test.skip("Not ready yet", () => {});   // shows as skipped
```

- A test **passes** if the function doesn't throw. Any thrown error/failed assertion = **fail**.
- One failing test doesn't stop other tests or the run (unless `--bail`).
- `pm.expect` = **Chai** `expect` (BDD style).

7.12 `pm.visualizer` — custom visual responses

```
const template = `
<style>
  table { border-collapse: collapse; font-family: sans-serif; }
  td, th { border: 1px solid #ccc; padding: 6px 10px; }
  .inactive { color: #b00; }
</style>
<h3>Users ({{users.length}})</h3>
<table>
  <tr><th>ID</th><th>Name</th><th>Email</th><th>City</th></tr>
  {{#each users}}
    <tr><td>{{id}}</td><td>{{name}}</td><td>{{email}}</td><td>{{address.city}}</td></tr>
  {{/each}}
</table>`;

pm.visualizer.set(template, { users: pm.response.json() });
```

Run against `https://jsonplaceholder.typicode.com/users` → **Visualize** tab shows a table.

- Templates use **Handlebars** syntax.
- You can load external JS libraries in the template (e.g. Chart.js via `<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>`) and access data inside the template's script via:

```
<script>
  pm.getData(function (err, data) {
      // data = the object you passed to pm.visualizer.set
  });
</script>
```

- `pm.visualizer.clear()` (newer versions) removes the visualization.
- Use cases: charts of response times, tables for large arrays, rendering images/base64, comparing environments.

7.13 Console logging (debugging)

```
console.log("Order id:", pm.environment.get("orderId"));
console.info("Info"); console.warn("Careful"); console.error("Bad");
console.log(pm.request.toJSON());    // whole request object
console.log(pm.response.headers.toObject());
console.clear();                     // clear console (where supported)
```

Console shows: **each request** (incl. `pm.sendRequest`), **request headers & body actually sent**, **response headers & body**, **network info**, **script logs and errors**. In Newman logs go to stdout (unless reporter options hide them).

7.14 Error handling inside scripts

- A **runtime error** in a post-response script → shown as an error ("There was an error in evaluating the test script"), remaining code in that script stops, tests defined before still count.
- Wrap risky code in `try/catch` or inside `pm.test` (an error inside `pm.test` becomes a failed test instead of a script crash — **preferred**).

```
pm.test("Items have prices", () => {
    const items = pm.response.json().items;   // if undefined, test fails nicely
    items.forEach(i => pm.expect(i.price).to.be.above(0));
});
```

7.15 Built-in libraries available via `require()`

Library	Use	Example
`lodash` (also global `_`)	Utility functions	`_.get(json, "a.b[0].c")`, `_.uniqBy`, `_.isEqual`
`moment`	Dates	`moment().add(1, "days").format("YYYY-MM-DD")`
`crypto-js` (global `CryptoJS`)	Hashing, HMAC, AES, Base64	`CryptoJS.SHA256("x").toString()`

Library	Use	Example
`ajv`	**JSON Schema validation** (draft-07 by default; newer drafts via options)	Chapter 8
`tv4`	Older JSON Schema validator (draft-04) — **deprecated**, prefer ajv	
`chai`	Assertions (`pm.expect` is Chai)	
`cheerio`	jQuery-like HTML parsing	`cheerio.load(html)`
`uuid`	UUIDs	`require("uuid").v4()`
`xml2js`	XML ↔ JS	
`csv-parse/lib/sync`	Parse CSV text	`parse(text, { columns: true })`
`postman-collection`	Postman SDK objects	`new (require("postman-collection").Url)(...)`
`atob` , `btoa`	Base64 decode/encode (globals)	
Node built-ins	`path` , `assert` , `buffer` , `util` , `url` , `punycode` , `querystring` , `string_decoder` , `stream` , `timers` , `events`	`require("url")`

Also available as globals without `require` : `_` (lodash), `CryptoJS` , `cheerio` , `tv4` , `xml2Json` , `atob` , `btoa` , `console` , `setTimeout` / `setInterval` . For `Buffer` , use `require("buffer").Buffer` to be safe across versions.

Examples:

```
const moment = require("moment");
pm.variables.set("checkin", moment().add(7, "days").format("YYYY-MM-DD"));

// `_` is already a global — `const _ = require("lodash")` throws "Identifier '_' has already
been declared"
const lodash = require("lodash");   // or just use the global _
const json = pm.response.json();
const city = lodash.get(json, "data.user.address.city", "N/A");

const parse = require("csv-parse/lib/sync");
const rows = parse(pm.response.text(), { columns: true, skip_empty_lines: true });
pm.test("CSV has 10 rows", () => pm.expect(rows).to.have.lengthOf(10));

const sha = CryptoJS.SHA256("hello").toString(CryptoJS.enc.Hex);
const b64 = CryptoJS.enc.Base64.stringify(CryptoJS.enc.Utf8.parse("hello")); // "aGVsbG8="
```

7.16 Package Library and `pm.require` (reusable code)

Problem: Same helper (token refresh, schema validation, common assertions) copied into 40 collections. Fix one bug → 40 edits.

Old workarounds: store functions as strings in variables and `eval()` them (fragile, insecure), or put helpers in a collection-level script (only that collection).

Postman's solution (v11+): the Package Library.

1. In a workspace's **Package Library** (from the Scripts editor → "Open package library"), create a package e.g. `@shop/test-utils`.
2. Write a module with `module.exports`:

```
// Package: @shop/test-utils
function expectStatus(code) {
    pm.test(`Status is ${code}`, () => pm.response.to.have.status(code));
}
function expectJsonContentType() {
    pm.test("Content-Type is JSON", () =>
        pm.expect(pm.response.headers.get("Content-Type")).to.include("application/json"));
}
function expectFasterThan(ms) {
    pm.test(`Response time < ${ms}ms`, () =>
pm.expect(pm.response.responseTime).to.be.below(ms));
}
module.exports = { expectStatus, expectJsonContentType, expectFasterThan };
```

3. Use anywhere in your team:

```
const utils = pm.require("@shop/test-utils");
utils.expectStatus(200);
utils.expectJsonContentType();
utils.expectFasterThan(800);
```

- Packages are versioned/edited centrally; changes reflect everywhere.
- Recent versions also let `pm.require` load packages from **external registries** such as npm (syntax like `pm.require('npm:package-name@version')`) — check your version's docs; pin versions for reproducibility.
- In CI, Newman/Postman CLI need access to these packages — the Postman CLI (logged in) resolves team packages; for Newman, verify support in your version or keep helpers inline/collection-level.

7.17 Legacy script syntax (you WILL see this in old company collections)

Old (deprecated)	New
`tests["Status is 200"] = responseCode.code === 200;`	`pm.test("Status is 200", () => pm.response.to.have.status(200));`
`responseBody`	`pm.response.text()`
`JSON.parse(responseBody)`	`pm.response.json()`
`responseCode.code`	`pm.response.code`

Old (deprecated)	New
`responseTime`	`pm.response.responseTime`
`responseHeaders["Content-Type"]` / `postman.getResponseHeader("Content-Type")`	`pm.response.headers.get("Content-Type")`
`postman.setEnvironmentVariable("k", v)`	`pm.environment.set("k", v)`
`postman.getEnvironmentVariable("k")` / `environment.k`	`pm.environment.get("k")`
`postman.setGlobalVariable` / `globals.k`	`pm.globals.set` / `pm.globals.get`
`postman.clearEnvironmentVariable("k")`	`pm.environment.unset("k")`
`postman.setNextRequest("x")`	`pm.execution.setNextRequest("x")`
`data.email` (iteration data)	`pm.iterationData.get("email")`
`request.url`, `request.headers`	`pm.request.url`, `pm.request.headers`
`tv4.validate(json, schema)`	`pm.response.to.have.jsonSchema(schema)` or ajv

> 🏢 **Migration task in companies:** "Modernise legacy collection scripts." Do it folder by folder, run the collection before/after, compare results.

7.18 Script snippets panel

Next to the script editor, Postman offers **snippets** (click to insert): "Get an environment variable", "Set a global variable", "Send a request", "Status code: Code is 200", "Response body: JSON value check", "Response time is less than 200ms", "Response headers: Content-Type header check", "Use Tiny Validator for JSON data" (old) etc. Great for beginners; but learn to write them yourself for interviews.

Also: **Postbot / Agent Mode** can generate tests ("Add tests for this response", "Fix this test", "Visualize response as table").

7.19 Practical script recipes

Save an ID from a create response

```
pm.test("Created", () => pm.response.to.have.status(201));
const id = pm.response.json().id;
pm.collectionVariables.set("orderId", id);
```

Save a value from a response header (e.g., Location)

```
const location = pm.response.headers.get("Location");      // "/orders/987"
pm.collectionVariables.set("orderId", location.split("/").pop());
```

Find an item in an array

```
const users = pm.response.json();
const asha = users.find(u => u.username === "Asha");
pm.test("Asha exists", () => pm.expect(asha, "user Asha").to.exist);
pm.environment.set("ashaId", asha.id);
```

Extract with regex from text

```
const match = pm.response.text().match(/Order number: (\w+)/);
if (match) pm.environment.set("orderNo", match[1]);
```

Wait / delay (polling helper)

```
setTimeout(() => {}, 2000);   // the script (and next request) waits ~2s
```

Conditional test by environment

```
if (pm.environment.name.includes("PROD")) {
    pm.test("No debug fields in prod", () =>
pm.expect(pm.response.json()).to.not.have.property("debug"));
}
```

Remove empty params before sending

```
pm.request.url.query.each(q => { if (!q.value) pm.request.url.query.remove(q.key); });
```

Compute a date range

```
const today = new Date();
const from = new Date(today.getTime() - 7 * 86400000).toISOString().slice(0, 10);
pm.variables.set("fromDate", from);
pm.variables.set("toDate", today.toISOString().slice(0, 10));
```

Generate unique, traceable test data

```
const runId = pm.variables.get("_runId") || Date.now().toString(36);
pm.variables.set("_runId", runId);
```

```
pm.variables.set("email", `qa+${runId}-${pm.info.iteration}@example.com`);
```

7.20 Script quality rules (company standards)

1. **Put shared logic high** (collection/folder) and request-specific logic low.
2. **No secrets hardcoded** in scripts. Read from variables/Vault.
3. **Name tests clearly**: `"POST /orders → 201 and returns id"`, not `"test1"`.
4. **Wrap parsing in tests** so failures are reported, not crashes.
5. **Avoid** `pm.environment.clear()` / `pm.globals.clear()` in shared collections.
6. **Clean up** variables you create (`unset`) in teardown.
7. **Log meaningfully** but don't log tokens/PII (Console logs can be shared in screenshots, CI logs are stored).
8. **Prefer IDs over names** for `setNextRequest`.
9. **Use Package Library** for reused helpers.
10. **Keep scripts short**; if a script is 200 lines, it's an app — refactor.

7.21 Chapter 7 Quick Revision

- Order: Collection → Folder(s) → Request for both pre-request and post-response.
- Sandbox: modern JS, no fs/DOM, limited `require`, `pm.require` for team packages.
- `pm.request` modifiable in pre-request; `pm.response` in post-response.
- `pm.sendRequest` for side calls (no auto auth, no auto variable resolution).
- `pm.execution.setNextRequest` / `skipRequest` control runs.
- Visualizer = Handlebars template + `pm.getData`.
- Legacy `tests[]`, `responseBody`, `postman.*` → migrate to `pm.*`.

➡️ Next: **Chapter 8 — Testing and Assertions in Depth**.

Chapter 8 — Testing and Assertions in Depth

> 🧒 A **test** is a **question with a yes/no answer** about the reply: "Did the server say OK?" "Is the name Asha?" "Was it fast?" Postman shows a ✅ or ❌ for each question.

8.1 Anatomy of a Postman test

```
pm.test("Test name shown in results", function () {
    // assertions; if anything throws → FAIL, else → PASS
    pm.response.to.have.status(200);
});
```

- `pm.test(name, fn)` registers a test.
- Inside, use `pm.expect(...)` (Chai BDD) or `pm.response.to...` (Postman's response assertions).
- Results appear in **Test Results** tab, the Runner, Newman/CLI reports.
- Tests can be in **any** post-response script (collection/folder/request) — and even pre-request scripts (rare, e.g. validate preconditions).

Assertion messages (custom failure text)

```
pm.expect(json.total, "total should equal sum of items").to.eql(expectedTotal);
```

8.2 Postman's response assertions (`pm.response.to.*`)

Assertion	Meaning
`pm.response.to.have.status(200)`	Status code equals 200
`pm.response.to.have.status("Created")`	Reason phrase equals "Created"
`pm.response.to.be.ok`	Status is exactly **200**
`pm.response.to.be.success`	Status is **2xx**
`pm.response.to.be.accepted`	202
`pm.response.to.be.redirection`	3xx
`pm.response.to.be.clientError`	4xx
`pm.response.to.be.serverError`	5xx

Assertion	Meaning
`pm.response.to.be.error`	4xx or 5xx
`pm.response.to.be.badRequest` / `.unauthorized` / `.forbidden` / `.notFound` / `.rateLimited`	400 / 401 / 403 / 404 / 429
`pm.response.to.not.be.error`	negation works with `.not`
`pm.response.to.have.header("Content-Type")`	Header exists
`pm.response.to.have.header("Content-Type", "application/json; charset=utf-8")`	Header exact value
`pm.response.to.have.body("exact text")`	Body exact match
`pm.response.to.have.jsonBody()`	Body is JSON
`pm.response.to.have.jsonBody("data.user.name")`	JSON has path
`pm.response.to.have.jsonBody("data.user.name", "Asha")`	JSON path equals value
`pm.response.to.have.jsonSchema(schema)`	Validates against JSON Schema (uses Ajv)
`pm.response.to.be.json`	Content is JSON
`pm.response.to.be.withBody`	Has a body

> 🎯 **Interview trap:** `pm.response.to.be.ok` checks **200 only**, not "any 2xx". For "any success" use `.to.be.success` or `pm.expect(pm.response.code).to.be.within(200, 299)`.

8.3 Chai `pm.expect` — the complete toolkit

Equality

```
pm.expect(5).to.equal(5);                    // strict ===
pm.expect({a:1}).to.eql({a:1});              // deep equality
pm.expect({a:1}).to.deep.equal({a:1});       // same as eql
pm.expect("1").to.not.equal(1);              // strict: string vs number
```

> ❗ `equal` on objects/arrays compares **references** → fails for two separate but identical objects. Use `eql` / `deep.equal`.

Types

```
pm.expect(json.id).to.be.a("number");
pm.expect(json.name).to.be.a("string");
pm.expect(json.tags).to.be.an("array");
pm.expect(json.address).to.be.an("object");
pm.expect(json.active).to.be.a("boolean");
pm.expect(json.manager).to.be.null;
```

```
pm.expect(json.deletedAt).to.be.undefined;
pm.expect(json.id).to.exist;                    // not null/undefined
```

Truthiness

```
pm.expect(json.active).to.be.true;
pm.expect(json.blocked).to.be.false;
pm.expect(json.list).to.be.empty;               // [], "", {}
pm.expect(json.list).to.not.be.empty;
pm.expect(json.name).to.be.ok;                  // Chai .ok = truthy (different from
pm.response.to.be.ok!)
```

Numbers

```
pm.expect(json.price).to.be.above(0);           // >
pm.expect(json.price).to.be.below(10000);       // <
pm.expect(json.qty).to.be.at.least(1);          // >=
pm.expect(json.qty).to.be.at.most(99);          // <=
pm.expect(json.rating).to.be.within(1, 5);      // inclusive
pm.expect(0.1 + 0.2).to.be.closeTo(0.3, 0.0001);  // floating point!
```

Strings

```
pm.expect(json.email).to.include("@");
pm.expect(json.email).to.match(/^[^\s@]+@[^\s@]+\.[^\s@]+$/);
pm.expect(json.code).to.have.lengthOf(6);
pm.expect(json.status).to.be.oneOf(["PENDING", "PAID", "SHIPPED"]);
pm.expect(json.name).to.not.include("<script>");
```

Objects

```
pm.expect(json).to.have.property("id");
pm.expect(json).to.have.property("status", "PAID");          // property + value
pm.expect(json).to.have.nested.property("address.city", "Pune");
pm.expect(json).to.have.all.keys("id", "name", "email");     // exactly these keys
pm.expect(json).to.include.all.keys("id", "name");           // at least these keys
pm.expect(json).to.have.any.keys("phone", "email");
pm.expect(json).to.deep.include({ status: "PAID", currency: "INR" }); // subset match
pm.expect(json).to.not.have.property("password");            // security!
```

Arrays

```
pm.expect(json.items).to.have.lengthOf(3);
pm.expect(json.items).to.have.lengthOf.at.least(1);
pm.expect(json.tags).to.include("sale");
pm.expect(json.tags).to.have.members(["sale", "new"]);       // same members, any order
pm.expect(json.tags).to.include.members(["sale"]);           // subset
pm.expect(json.items).to.deep.include({ sku: "A1", qty: 2 });   // contains this object
pm.expect([1, 2, 3]).to.satisfy(arr => arr.every(n => n > 0));
```

Custom logic

```
pm.expect(json.total).to.satisfy(t => t === json.items.reduce((s, i) => s + i.price * i.qty, 0));
```

Negation & chaining

All can be negated with `.not`. Language chains (`to, be, been, is, that, which, and, has, have, with, at, of, same`) are just for readability.

8.4 The essential test suite for any endpoint

```
const json = pm.response.json();

// 1. Status
pm.test("Status code is 200", () => pm.response.to.have.status(200));

// 2. Headers
pm.test("Content-Type is JSON", () => {
    pm.expect(pm.response.headers.get("Content-Type")).to.include("application/json");
});

// 3. Performance (SLA)
pm.test("Response time < 800ms", () => pm.expect(pm.response.responseTime).to.be.below(800));

// 4. Structure / schema
pm.test("Body has required fields", () => {
    pm.expect(json).to.include.all.keys("id", "name", "email", "createdAt");
});

// 5. Data types
pm.test("Field types are correct", () => {
    pm.expect(json.id).to.be.a("number");
    pm.expect(json.email).to.be.a("string");
    pm.expect(new Date(json.createdAt).toString()).to.not.eql("Invalid Date");
});

// 6. Business values
pm.test("Returns the requested user", () => {
    pm.expect(json.id).to.eql(Number(pm.collectionVariables.get("userId")));
});

// 7. Security
pm.test("No sensitive fields leaked", () => {
    pm.expect(json).to.not.have.any.keys("password", "passwordHash", "ssn");
});
```

8.5 JSON Schema validation (contract testing)

> 🧒 A **schema** is a **cookie cutter**. Every cookie (response) must fit the cutter's shape. If a cookie is missing a corner (a field) or has the wrong filling (type), it doesn't fit.

Schema example

```
const schema = {
    type: "object",
    required: ["id", "name", "email", "address"],
    additionalProperties: false,          // strict contract: no unexpected fields
    properties: {
        id:    { type: "integer", minimum: 1 },
        name:  { type: "string", minLength: 1 },
        email: { type: "string", pattern: "^[^@\\s]+@[^@\\s]+\\.[^@\\s]+$" },
        phone: { type: ["string", "null"] },
        role:  { type: "string", enum: ["admin", "user", "viewer"] },
        address: {
            type: "object",
            required: ["city"],
            properties: { city: { type: "string" }, zip: { type: "string" } }
        },
        tags: { type: "array", items: { type: "string" }, uniqueItems: true }
    }
};
```

Option 1 — built-in assertion (simplest)

```
pm.test("Matches schema", () => pm.response.to.have.jsonSchema(schema));
```

Option 2 — Ajv directly (full control & readable errors)

```
const Ajv = require("ajv");
const ajv = new Ajv({ allErrors: true });
const validate = ajv.compile(schema);
const valid = validate(pm.response.json());

pm.test("Response matches schema", () => {
    pm.expect(valid, JSON.stringify(validate.errors, null, 2)).to.be.true;
});
```

Tips

- Keep schemas in a **collection variable** (as JSON string) or the **Package Library**, not copy-pasted per request:

```
const schema = JSON.parse(pm.collectionVariables.get("userSchema"));
```

- Use `$ref` / `definitions` for reused sub-schemas; register them with `ajv.addSchema()`.

- The sandbox's Ajv version determines support for `format` (e.g. `email`, `date-time`) and newer drafts. If formats aren't enforced, use `pattern` regexes, or verify the version in your sandbox docs.
- `tv4` is deprecated (draft-04 only) — migrate to Ajv.
- **Arrays of objects**: `{ type: "array", items: userSchema, minItems: 1 }`.
- Contract source of truth = the **OpenAPI spec**. Generate schemas from it rather than writing by hand. Postman's **Spec Hub** can generate collections from OpenAPI and can validate requests/responses against the spec (contract tests) — Chapter 12.

8.6 Testing arrays, lists, sorting, pagination

```
const data = pm.response.json().data;

pm.test("All items are active", () => {
    data.forEach((u, i) => pm.expect(u.status, `item ${i}`).to.eql("active"));
});

pm.test("IDs are unique", () => {
    const ids = data.map(d => d.id);
    pm.expect(new Set(ids).size).to.eql(ids.length);
});

pm.test("Sorted by price ascending", () => {
    const prices = data.map(d => d.price);
    const sorted = [...prices].sort((a, b) => a - b);
    pm.expect(prices).to.eql(sorted);
});

pm.test("Sorted by createdAt descending", () => {
    const t = data.map(d => Date.parse(d.createdAt));
    t.slice(1).forEach((v, i) => pm.expect(v).to.be.at.most(t[i]));
});

pm.test("Page size honoured", () => {
    pm.expect(data.length).to.be.at.most(Number(pm.request.url.query.get("limit")));
});

pm.test("Filter works (category=shoes)", () => {
    pm.expect(data.every(p => p.category === "shoes")).to.be.true;
});
```

> Note: `.sort()` without comparator sorts **as strings** (`[10, 9, 1] → [1, 10, 9]`). Always pass `(a,b) => a-b` for numbers and `localeCompare` for strings.

8.7 Testing headers, cookies, and security headers

```
pm.test("Has correlation id echoed", () => {
    pm.expect(pm.response.headers.get("X-Correlation-ID")).to.eql(pm.request.headers.get("X-
Correlation-ID"));
```

```
});
pm.test("Security headers present", () => {
    const h = pm.response.headers;
    pm.expect(h.get("Strict-Transport-Security")).to.exist;
    pm.expect(h.get("X-Content-Type-Options")).to.eql("nosniff");
    pm.expect(h.has("X-Powered-By")).to.be.false;
});
pm.test("Session cookie is HttpOnly & Secure", () => {
    // Parse the raw Set-Cookie headers: works in the app AND Newman
    // (pm.response.cookies was empty in Newman 6.2.3 during verification)
    const setCookies = pm.response.headers.all()
        .filter(h => h.key.toLowerCase() === "set-cookie")
        .map(h => h.value);
    const session = setCookies.find(v => v.startsWith("sessionId="));
    pm.expect(session, "sessionId cookie").to.exist;
    pm.expect(session).to.match(/;\s*HttpOnly/i);
    pm.expect(session).to.match(/;\s*Secure/i);
});
pm.test("Cookie stored in jar", () => pm.expect(pm.cookies.has("sessionId")).to.be.true);
pm.test("Cache disabled for user data", () => {
    pm.expect(pm.response.headers.get("Cache-Control")).to.match(/no-store|private/);
});
```

8.8 Negative testing patterns

Same endpoint, broken inputs — expect correct **error codes and messages**, never 500.

Case	Body	Expected
Missing required field	`{}`	400 + message naming field
Wrong type	`{"qty":"two"}`	400/422
Boundary min-1	`{"qty":0}`	400/422
Boundary max+1	`{"qty":1000}` (max 999)	400/422
Too long string	256 chars when max 255	400/422
Invalid enum	`{"status":"FLYING"}`	400/422
Malformed JSON	`{"qty":1,`	400
Wrong Content-Type	text/plain	415
Duplicate	same email twice	409
No auth	—	401
Wrong role	user token on admin endpoint	403
Non-existent ID	`/orders/999999999`	404
Wrong method	`DELETE /health`	405
SQL/NoSQL injection strings	`"' OR 1=1 --"`, `{"$gt":""}`	400 and no data leak
Huge payload	20 MB	413

Reusable negative test:

```
const expected = Number(pm.iterationData.get("expectedStatus") ||
pm.variables.get("expectedStatus"));
pm.test(`Status is ${expected}`, () => pm.response.to.have.status(expected));
if (expected >= 400) {
    pm.test("Error body has standard shape", () => {
        const err = pm.response.json();
        pm.expect(err).to.include.keys("code", "message");
        pm.expect(err.message).to.be.a("string").and.not.empty;
    });
    pm.test("No stack traces leaked", () => {
        pm.expect(pm.response.text()).to.not.match(/at\s+\w+\s+\
(.*:\d+:\d+\)|Exception|Traceback/);
    });
}
```

Data-drive this with a CSV (Chapter 10).

8.9 Response time & performance assertions

```
const sla = Number(pm.environment.get("slaMs") || 1000);
pm.test(`Response under ${sla} ms`, () => pm.expect(pm.response.responseTime).to.be.below(sla));
```

- Single-request timing in Postman includes your network; set environment-specific SLAs (QA servers are slower).
- For real load testing use Postman's **Performance tab** in the Runner or dedicated tools (Chapter 10).

8.10 Testing across requests (stateful assertions)

```
// In "Create order" post-response:
pm.collectionVariables.set("createdOrder", JSON.stringify(pm.response.json()));

// In "Get order" post-response:
const created = JSON.parse(pm.collectionVariables.get("createdOrder"));
const fetched = pm.response.json();
pm.test("GET returns what POST created", () => {
    pm.expect(fetched).to.deep.include({ id: created.id, total: created.total, status:
created.status });
});
```

8.11 Comparing responses / snapshot testing

```
// Compare this response to a saved "golden" snapshot, ignoring volatile fields
// use the global `_` (lodash); re-declaring `const _` would throw a SyntaxError
```

```
const omitVolatile = obj => _.omit(obj, ["createdAt", "updatedAt", "requestId"]);
const current = omitVolatile(pm.response.json());

const snapshot = pm.collectionVariables.get("snapshot_getProduct");
if (!snapshot) {
    pm.collectionVariables.set("snapshot_getProduct", JSON.stringify(current));
    console.warn("Snapshot created");
} else {
    pm.test("Matches snapshot", () => pm.expect(current).to.eql(JSON.parse(snapshot)));
}
```

8.12 Tests in collection/folder scripts (DRY)

Collection-level post-response, runs after **every** request:

```
// Generic checks for all requests
pm.test(`[${pm.info.requestName}] no 5xx`, () => pm.expect(pm.response.code).to.be.below(500));
pm.test(`[${pm.info.requestName}] responds < 3s`, () =>
pm.expect(pm.response.responseTime).to.be.below(3000));

if (pm.response.headers.get("Content-Type")?.includes("json") && pm.response.text()) {
    pm.test(`[${pm.info.requestName}] valid JSON`, () => pm.response.json());
}
```

Folder-level "expected status" via request description or variable convention:

```
// Requests named like "Create order - missing items (400)"
const m = pm.info.requestName.match(/\((\d{3})\)$/);
if (m) pm.test(`Expected status ${m[1]}`, () => pm.response.to.have.status(Number(m[1])));
```

8.13 Asynchronous tests

Tests inside callbacks work because Postman waits for pending callbacks/timers:

```
pm.sendRequest(pm.variables.replaceIn("{{baseUrl}}/orders/{{orderId}}"), (err, res) => {
    pm.test("Order persisted", () => pm.expect(res.code).to.eql(200));
});
```

With `async/await`: top-level `await` works only in recent Postman app versions, **not in Newman 6** (verified). For portability wrap `pm.sendRequest` in a Promise inside an async IIFE (Chapter 7.8) and always `.catch()` — an unhandled rejection can silently skip a test.

8.14 Skipping and conditional tests

```
pm.test.skip("Pending until BUG-123 fixed", () => { /* ... */ });

const feature = pm.environment.get("featureNewPricing") === "true";
(feature ? pm.test : pm.test.skip)("New pricing applied", () => {
    pm.expect(pm.response.json().pricingVersion).to.eql(2);
});
```

8.15 Postbot & AI-assisted tests

- **Postbot** (and newer **Agent Mode**) can: generate tests from a response, add schema checks, fix failing tests, write documentation, visualize responses, explain errors.
- Always **review** AI-generated tests: they tend to assert *whatever the current response is* (which might be a bug!). Tests must reflect the **specification**, not the current behaviour.

8.16 Test design principles (what seniors know)

1. **Test the contract, not the implementation.** Assert what the API promises (spec), not incidental details (field order, exact error wording unless contractual).
2. **Independent tests**: each request should create the data it needs or rely on a well-defined setup folder. Avoid "works only if run after request #7 from yesterday".
3. **Deterministic data**: unique emails/IDs per run (`runId`), cleanup after.
4. **One concern per** `pm.test` → clearer failures. But don't create 50 trivial tests per request either.
5. **Meaningful names**: `"POST /orders - 400 when items empty"` .
6. **Avoid brittle assertions**: timestamps, random IDs, ordering when unordered → use `include` , `members` , regex, ranges.
7. **Positive + negative + boundary + security** for every important endpoint.
8. **Fail fast on setup**: if login fails, stop the run (`pm.execution.setNextRequest(null)`) instead of 300 cascading failures.
9. **Keep SLAs per environment**.
10. **Review tests like code** (PRs/forks).

8.17 Common testing bugs and fixes

Symptom	Cause	Fix
`JSONError: Unexpected token '<'`	Response is HTML (login page, 502 page), not JSON	Check status first; inspect body in Console; fix auth/URL

Symptom	Cause	Fix
`AssertionError: expected { Object } to equal { Object }`	Used `equal` instead of `eql`	Use `eql` / `deep.equal`
`expected '5' to equal 5`	String vs number (variables used in `{{ }}` become strings; CSV data types vary)	`Number(...)` or `parseInt`
Test passes but should fail	Assertion outside `pm.test`, or `pm.expect` without assertion method (`pm.expect(x).to.be` with nothing)	Ensure a terminal assertion; place inside `pm.test`
Test never runs	Code path skipped, script error earlier in the script	Check Console for script errors
`Cannot read properties of undefined (reading 'x')`	Path wrong / field missing	Use optional chaining `json.data?.x`; log body
Test passes locally, fails in CI	Different env/data, timing, secrets missing	Log env name & key vars (not secrets), add waits/polling
Sorting test flaky	Equal values in non-stable order / locale	Compare with tie-breaker; use `localeCompare`
Floating point mismatch	`0.1+0.2`	`closeTo` or round to cents
Timezone failures	Server in UTC, local IST (+5:30)	Compare ISO strings in UTC; use `moment.utc()`

8.18 Chapter 8 Quick Revision

- `pm.test` + `pm.expect` (Chai) + `pm.response.to.*`.
- `pm.response.to.be.ok` = 200 only; `.success` = 2xx.
- `equal` (strict/reference) vs `eql` (deep).
- Schema validation: `pm.response.to.have.jsonSchema(schema)` or Ajv; tv4 deprecated.
- Write positive, negative, boundary, security, performance, contract tests.
- Put generic checks at collection level.

➡️ Next: **Chapter 9 — Request Chaining & Workflows**.

Chapter 9 — Request Chaining and Workflows

> 🧒 **Chaining** is a **relay race**. Runner 1 (Login) passes the baton (token) to Runner 2 (Create Order), who passes a new baton (orderId) to Runner 3 (Pay), and so on. The baton is a **variable**.

9.1 Basic chaining with variables

1. **Request A** (post-response): extract value → save to variable.
2. **Request B**: use `{{variable}}` in URL/header/body.

```
// POST /orders → post-response
const order = pm.response.json();
pm.collectionVariables.set("orderId", order.id);
pm.collectionVariables.set("orderTotal", order.total);
```

```
POST {{baseUrl}}/payments
{ "orderId": "{{orderId}}", "amount": {{orderTotal}} }
```

Which scope to store chained values in?

Scope	Pros	Cons
Collection variable	Travels with collection; independent of environment	Shared across environments (stale value when switching env)
Environment variable	Env-specific (QA orderId ≠ DEV orderId)	Pollutes env with runtime junk
Local (`pm.variables.set`)	Clean; disappears after run	Not visible across separate manual sends

Common company convention: config in **environment**, runtime IDs in **collection variables** (or environment with a `_` prefix), and clean them up in a teardown folder.

9.2 Default execution order & controlling it

- Runner executes requests **top to bottom** (depth-first through folders).
- `pm.execution.setNextRequest(nameOrId)` changes **which request runs next**.
- `pm.execution.setNextRequest(null)` **stops** the run after the current request.
- `pm.execution.skipRequest()` (pre-request) **skips** the current request.

- These only affect **collection runs** (Runner, Newman, Postman CLI, monitors, scheduled runs) — **not** a single manual Send.

9.3 Workflow patterns

Pattern 1: Conditional branching

```
// "Get user" post-response
const user = pm.response.json();
if (user.kycStatus === "VERIFIED") {
    pm.execution.setNextRequest("Create loan application");
} else {
    pm.execution.setNextRequest("Submit KYC documents");
}
```

Pattern 2: Loop over a list from a response

```
// "List products" post-response
const ids = pm.response.json().map(p => p.id);
pm.collectionVariables.set("productIds", JSON.stringify(ids));
```

```
// "Get product details" pre-request
let ids = JSON.parse(pm.collectionVariables.get("productIds") || "[]");
if (ids.length === 0) { pm.execution.skipRequest(); return; }
const current = ids.shift();
pm.collectionVariables.set("productId", current);
pm.collectionVariables.set("productIds", JSON.stringify(ids));
```

```
// "Get product details" post-response
pm.test(`Product ${pm.collectionVariables.get("productId")} OK`, () =>
pm.response.to.have.status(200));
const remaining = JSON.parse(pm.collectionVariables.get("productIds"));
if (remaining.length > 0) {
    pm.execution.setNextRequest(pm.info.requestId);   // run this request again
} else {
    pm.collectionVariables.unset("productIds");       // loop finished → continue normally
}
```

Pattern 3: Pagination — fetch all pages

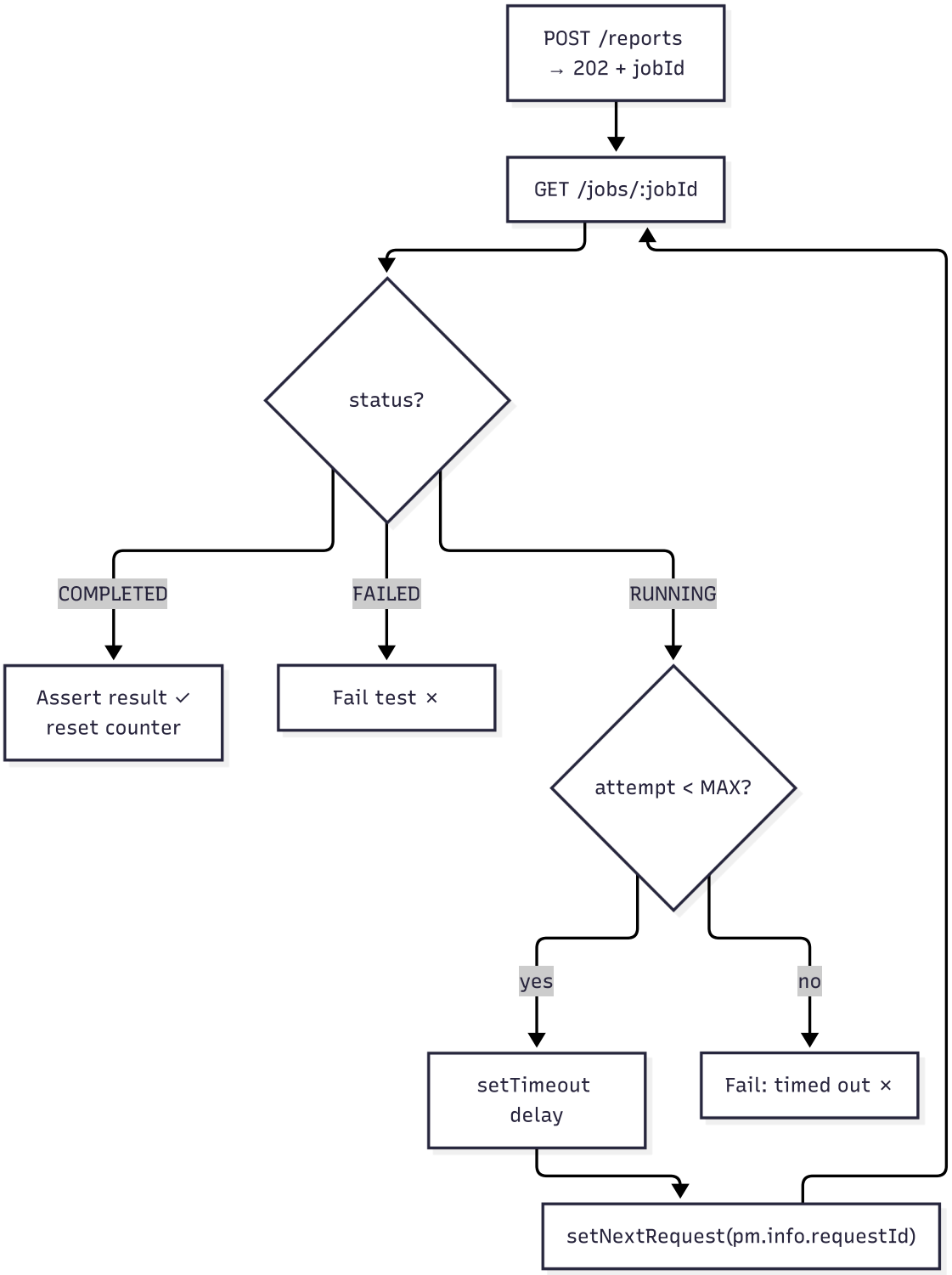
```
// "List orders" (GET {{baseUrl}}/orders?page={{page}}&limit=50)
// pre-request
if (!pm.collectionVariables.get("page")) pm.collectionVariables.set("page", 1);

// post-response
const body = pm.response.json();
let all = JSON.parse(pm.collectionVariables.get("_allOrders") || "[]");
all = all.concat(body.data);
pm.collectionVariables.set("_allOrders", JSON.stringify(all));
```

```
if (body.hasMore /* or body.page < body.totalPages */) {
    pm.collectionVariables.set("page", Number(pm.collectionVariables.get("page")) + 1);
    pm.execution.setNextRequest(pm.info.requestId);
} else {
    pm.test("Collected all orders", () => pm.expect(all.length).to.eql(body.total));
    pm.collectionVariables.unset("page");
}
```

Cursor-based: store `body.next_cursor`, stop when it's null.

Pattern 4: Polling an async job (202 Accepted → status endpoint)



```
// "Get report status" post-response
const MAX = 20;
const attempt = Number(pm.variables.get("_pollAttempt") || 0) + 1;
pm.variables.set("_pollAttempt", attempt);
const status = pm.response.json().status;

if (status === "COMPLETED") {
    pm.test("Report completed", () => pm.expect(status).to.eql("COMPLETED"));
    pm.variables.set("_pollAttempt", 0);
} else if (status === "FAILED" || attempt >= MAX) {
    pm.test("Report finished in time", () => { throw new Error(`status=${status} after ${attempt}
attempts`); });
    pm.variables.set("_pollAttempt", 0);
} else {
    setTimeout(() => {}, 3000);                        // wait 3s
    pm.execution.setNextRequest(pm.info.requestId);    // poll again
}
```

Alternatively poll **inside one script** with `pm.sendRequest` + `setTimeout` recursion.

Pattern 5: Retry on transient errors (429/502/503/504)

```
const retriable = [429, 502, 503, 504];
const tries = Number(pm.variables.get("_tries") || 0);
if (retriable.includes(pm.response.code) && tries < 3) {
    pm.variables.set("_tries", tries + 1);
    const retryAfter = Number(pm.response.headers.get("Retry-After") || 0);
    const wait = retryAfter ? retryAfter * 1000 : 1000 * Math.pow(2, tries); // exponential
backoff
    console.warn(`Retrying ${pm.info.requestName} in ${wait}ms (attempt ${tries + 1})`);
    setTimeout(() => {}, wait);
    pm.execution.setNextRequest(pm.info.requestId);
} else {
    pm.variables.set("_tries", 0);
    pm.test("Final status is 2xx", () => pm.response.to.be.success);
}
```

> Note: Postman records every attempt's tests in results. Keep assertions in the final branch so retries don't produce false failures. **Also watch collection-level tests** (e.g. a generic "no 5xx" check): they run on every attempt, so a retried 503 still shows up as a failure (seen in verification). Skip the generic check while retrying, e.g. `if (!pm.variables.get("_tries")) pm.test(...)`, or make it aware of retriable codes.

Pattern 6: Stop the run when a critical step fails

```
// "Login" post-response
if (pm.response.code !== 200) {
    console.error("Login failed — aborting run");
    pm.execution.setNextRequest(null);
}
```

Or Newman `--bail` (stops on first test failure/error).

Pattern 7: Skip a request based on a flag/environment

```
// pre-request
if (pm.environment.name.includes("PROD")) {
    console.log("Skipping destructive test in PROD");
    pm.execution.skipRequest();
}
```

Pattern 8: Run the same request with different inputs (inline data)

```
// pre-request
const cases = [
    { qty: 0, expected: 400 }, { qty: 1, expected: 201 }, { qty: 999, expected: 201 }, { qty:
1000, expected: 400 }
];
let i = Number(pm.variables.get("_case") || 0);
pm.variables.set("qty", cases[i].qty);
pm.variables.set("expectedStatus", cases[i].expected);

// post-response
pm.test(`qty=${pm.variables.get("qty")} → ${pm.variables.get("expectedStatus")}`, () =>
    pm.response.to.have.status(Number(pm.variables.get("expectedStatus"))));
i++;
if (i < 4) { pm.variables.set("_case", i); pm.execution.setNextRequest(pm.info.requestId); }
else { pm.variables.set("_case", 0); }
```

(For large data sets prefer CSV/JSON data files — Chapter 10.)

9.4 Infinite loops — how they happen and how to prevent

- `setNextRequest(pm.info.requestName)` with no exit condition.
- Exit condition depends on a variable that never changes (wrong scope: you set **local** but read **collection**).
- Two requests pointing to each other.

Prevention:

- Always include a **max counter**.
- Reset counters after the loop.
- In Runner, you can stop manually; in CI set a job **timeout** and Newman `--timeout` (overall run timeout in ms).

9.5 Why is `setNextRequest` "not working"? (Troubleshooting)

Cause	Fix
You clicked **Send** (single request)	Only works in collection runs

Cause	Fix
Name typo / duplicate names	Use request **ID**; ensure unique names
Target request outside the folder you're running	Run the whole collection or include target in the run
A later line sets it again (last call wins)	Search script for multiple calls
Called inside an async callback that fired after the script ended (rare in old versions)	Call synchronously or ensure callback completes
Using old `postman.setNextRequest` in a sandbox where it's unsupported	Use `pm.execution.setNextRequest`
Runner configured with **deselected** requests	Deselected requests can't be jumped to

---

9.6 End-to-end workflow example: E-commerce checkout

```
📁 E2E - Checkout
  1. POST  /auth/token                  → save accessToken
  2. POST  /customers                   → save customerId (unique email)
  3. GET   /products?inStock=true       → pick first product → productId, price
  4. GET   /inventory/{{productId}}     → save stockBefore
  5. POST  /carts                       → cartId
  6. POST  /carts/{{cartId}}/items      → add productId qty 2
  7. POST  /orders  {cartId}            → orderId, assert total = price*2
  8. POST  /payments {orderId, card}    → paymentId, status=CAPTURED
  9. GET   /orders/{{orderId}}          → status=PAID (poll until PAID)
 10. GET   /inventory/{{productId}}     → assert stockBefore - 2
 11. DELETE /customers/{{customerId}}   → cleanup (204)
```

Key scripts:

```
// 3. pick product
const p = pm.response.json().data.find(x => x.stock >= 2);
pm.test("Found product with stock", () => pm.expect(p).to.exist);
if (!p) { pm.execution.setNextRequest(null); }
pm.collectionVariables.set("productId", p.id);
pm.collectionVariables.set("price", p.price);

// 7. order total
const price = Number(pm.collectionVariables.get("price"));
pm.test("Total = 2 x price", () => pm.expect(pm.response.json().total).to.be.closeTo(price * 2,
0.001));

// 10. inventory decreased
const before = Number(pm.collectionVariables.get("stockBefore"));
pm.test("Stock reduced by 2", () => pm.expect(pm.response.json().stock).to.eql(before - 2));
```

9.7 Setup & teardown strategy

- **Setup folder** (first): auth, create fixtures (users, products), store IDs.
- **Test folders**: use fixtures.
- **Teardown folder** (last): delete fixtures, unset runtime variables.
- Teardown must run **even if tests fail** → don't use `setNextRequest(null)` on test failures in middle folders; instead jump to teardown: `pm.execution.setNextRequest("Teardown - delete customer")`.
- Make data unique per run (`runId`) so parallel CI runs don't collide.

9.8 Postman Flows (visual workflows) — brief

For non-coders or orchestrations, **Postman Flows** lets you connect blocks (Send Request, Evaluate (FQL/TypeScript), If, Repeat/Loop, Delay, Collect, Select, Display, AI request blocks, MCP trigger blocks…) visually. Great for demos, data processing, and API-driven automations; Flows can be **deployed** to the cloud and triggered by URL/schedule, and can even act as an **MCP server** for AI agents. Covered in Chapter 12.

9.9 Chapter 9 Quick Revision

- Chain via variables; choose scope deliberately.
- `setNextRequest(id|name|null)`, `skipRequest()` — collection runs only.
- Patterns: branch, loop, paginate, poll, retry with backoff, abort, skip, inline data cases.
- Always add loop limits; reset counters.
- Setup/teardown folders; teardown always reachable.

➡️ Next: **Chapter 10 — Collection Runner, Data-Driven & Performance Testing**.

Chapter 10 — Collection Runner, Data-Driven Testing & Performance Testing

> 🧒 The **Collection Runner** is a **robot** that presses "Send" on every request in your bag, one after another, as many times as you want, and writes a report card at the end. Give it a **list of names** (data file), and it repeats the whole thing once for each name.

10.1 Opening the Runner

- Footer → **Runner**, or collection `...` → **Run collection**, or folder → **Run folder**.
- Drag a collection/folder into the Runner.

10.2 Run modes

Mode	What
Run manually	Run now in the app (your machine, your network)
Schedule runs	Run in the **Postman cloud** on a schedule (like monitors; can't reach `localhost`/private networks unless allowed)
Automate runs via CLI	Shows the **Postman CLI** command to copy into CI
Performance	Load test with virtual users (10.8)

10.3 Runner configuration (functional mode)

Option	Meaning
Request selection/order	Tick/untick requests; drag to reorder for this run
Iterations	How many times to run the whole selection
Delay	ms between requests (avoid rate limits, give async processing time)
Data	Select CSV/JSON file → iteration data
Persist responses for a session / Save responses	Keep response bodies so you can inspect them after the run (uses memory; off for big runs)
Advanced: Stop run if an error occurs	Stop on script/network errors

Option	Meaning
Advanced: Keep variable values	Persist variable changes from the run into environment/collection after run (label varies by version)
Advanced: Run collection without using stored cookies	Start with clean cookie jar
Advanced: Save cookies after collection run	Keep cookies created during run
Turn off logs during run	Faster, less memory
Environment	Choose environment for the run

10.4 Run results

- Summary: passed/failed/skipped tests, total duration, avg response time.
- Filter: All / Passed / Failed / Skipped / Errors.
- Click a request → request/response details (if saved), test results, console logs.
- **Export results** (JSON) and **Run history** under the collection's **Runs** tab (shared in team).
- **Run again** with same config.

10.5 Data-driven testing (DDT)

CSV format

```
testCase,email,password,expectedStatus,expectedMessage
valid login,qa1@example.com,Valid@123,200,
wrong password,qa1@example.com,wrong,401,Invalid credentials
empty email,,Valid@123,400,email is required
sql injection,' OR 1=1 --,x,400,
```

- **First row = header** (variable names).
- One iteration per row. Iterations auto-set to row count.
- **Types are not reliable.** In Newman 6 (verified while writing this book), number-looking values are auto-converted to numbers and lose leading zeros (`007` → `7` , `1e3` → `1000`); `true` / `false` remain the *strings* `"true"` / `"false"` . Always normalise: `Number(pm.iterationData.get("expectedStatus"))` , `pm.iterationData.get("active") === "true"` , `String(pm.iterationData.get("zip")).padStart(6, "0")` . If exact types matter, use a **JSON** data file.
- Commas in values → wrap in double quotes: `"Smith, John"` . Quotes inside quotes → double them: `"He said ""hi"""` .
- Save as **UTF-8** (Excel may add a **BOM** — the first header becomes `testCase` , so `{{testCase}}` doesn't resolve! Save as "CSV UTF-8" carefully or strip BOM).
- Avoid trailing empty lines (some versions create an empty iteration).

JSON format

```
[
  { "testCase": "valid", "user": { "email": "qa1@example.com", "password": "Valid@123" },
"expectedStatus": 200, "roles": ["user"] },
  { "testCase": "locked", "user": { "email": "locked@example.com", "password": "Valid@123" },
"expectedStatus": 423, "roles": [] }
]
```

- Must be an **array of objects**.
- Types preserved (numbers, booleans, arrays, nested objects).
- `pm.iterationData.get("user")` returns the nested object (in `{{ }}` you can't do `{{user.email}}` reliably — flatten or set a local variable in pre-request).

Using data in the request

- `{{email}}`, `{{password}}` directly in body/URL.
- In scripts: `pm.iterationData.get("expectedStatus")`.

```
// Pre-request: map nested JSON data into simple variables
const user = pm.iterationData.get("user");
if (user) { pm.variables.set("email", user.email); pm.variables.set("password", user.password); }
```

```
// Post-response: generic data-driven assertions
const tc = pm.iterationData.get("testCase");
const expected = Number(pm.iterationData.get("expectedStatus"));
pm.test(`[${tc}] status ${expected}`, () => pm.response.to.have.status(expected));

const msg = pm.iterationData.get("expectedMessage");
if (msg) pm.test(`[${tc}] message`, () => pm.expect(pm.response.json().message).to.include(msg));
```

Iterations vs rows

- Iterations = rows by default. If you set **more** iterations than rows, the extra iterations reuse the **last row**'s values; fewer → only first N rows.
- Data variables have **higher precedence** than environment/collection/global (only local beats them). A CSV column named `baseUrl` would **override** your environment's `baseUrl` — a sneaky bug!

Running only some rows

No built-in row filter; options: separate files, or `if (pm.iterationData.get("run") === "N") pm.execution.skipRequest();`.

DDT best practices

- Include a `testCase` column → readable reports.
- Keep data files **in Git** next to exported collections.
- Separate files per purpose: `login-negative.csv`, `create-order-boundaries.json`.

- Don't put real customer data / PII in data files.
- For large combinatorial data, generate files with a script (Python/Node) rather than by hand.

10.6 Running a folder with different data than the rest

The Runner applies one data file to the whole run. Options:

- Run that folder separately with its own file (in CI: `newman run ... --folder "Login negative" -d login-negative.csv`).
- Or embed data inline in a variable (JSON string) and loop (Chapter 9, Pattern 8).

10.7 Common Runner problems

Problem	Cause	Fix
Variables not updated after run	"Keep variable values" off; or values only set locally	Enable option; check scope
Runner slow / app freezes	Saving responses for thousands of requests; huge console logs	Disable "Save responses", turn off logs, or use Newman/CLI
Request not running	Unticked in run order; `skipRequest`; `setNextRequest` jumped over it	Check selection & scripts
Runs forever	Infinite `setNextRequest` loop	Add counters; stop run
Data variable not resolving	CSV BOM, header typo, wrong delimiter (`;` in some locales)	Open in text editor; save UTF-8 comma-separated
429 errors in runs	No delay	Set Delay; add retry logic
Different results than single sends	State left from previous iterations; cookies	Use "without stored cookies"; reset variables in setup

---

10.8 Performance testing in Postman

> 🧒 Instead of 1 kid knocking on the shop door, imagine **100 kids knocking at once**. Does the shopkeeper still answer quickly? That's load testing.

Runner → **Performance** tab (exact capabilities/limits depend on your plan and version):

Setting	Meaning
Virtual users (VUs)	Number of concurrent simulated users, each running the selected requests in a loop

Setting	Meaning
Test duration	Minutes to run
Load profile	**Fixed** (constant VUs), **Ramp up** (gradual increase), **Spike** (sudden burst then down), **Peak** (ramp to a peak, hold, ramp down)
Data file	Give each VU different data (e.g. different user accounts) — assign rows sequentially or randomly depending on options
Pass/fail criteria	Thresholds e.g. avg response time < 500ms, error rate < 1% (newer versions)
Environment	Variables used

Metrics shown:

- **Average response time** and percentiles (**p90/p95/p99**) per request,
- **Throughput** (requests/second),
- **Error rate** (% failed requests, by error type),
- Graphs over time vs VUs.

Understanding the numbers (interview!)

- **Average** hides outliers; **p95 = 95% of requests were faster than this**. SLAs are usually on p95/p99.
- If response time **rises with VUs while throughput flattens** → system saturation (CPU/DB/thread pool limits).
- Errors appearing at high load: 429 (rate limit), 503 (overload), 504 (timeouts), connection resets.

Limitations & when to use other tools

- Postman's load is generated **from your machine** (local runs) → limited by your CPU/network; good for **baseline/smoke performance** and finding obvious regressions early.
- For large-scale load (thousands of VUs, distributed, complex scenarios, soak tests) use **k6, JMeter, Gatling, Locust, Artillery**, or cloud load services. Postman collections can be converted (e.g. `postman-to-k6` community tool) as a starting point.

Performance test checklist

1. Never load test **production** without approval/coordination.
2. Use realistic data and think time (Delay).
3. Warm up first.
4. Coordinate with backend/SRE to watch server metrics (CPU, memory, DB).
5. Test one change at a time; record baseline.
6. Use dedicated test accounts to avoid rate limits by user.

10.9 Scheduled collection runs (cloud)

- Runner → **Schedule runs**: frequency (hourly/daily/weekly/custom), environment, notifications (email) on failure.
- Runs in Postman cloud → **public internet only** (unless network allowlisting / private API access features on Enterprise).
- Results appear in collection **Runs** tab.
- Similar to **Monitors** (Chapter 12), which add regions, alerting integrations, and uptime graphs.

10.10 Chapter 10 Quick Revision

- Runner: iterations, delay, data file, environment, save responses, keep variable values, cookies options.
- CSV: header row, unreliable types (number-like values become numbers, leading zeros lost; booleans stay strings), beware BOM/quotes/delimiters; JSON: array of objects, types preserved.
- Data variables override environment ones (precedence!).
- Extra iterations reuse last row.
- Performance tab: VUs, duration, load profiles (fixed, ramp up, spike, peak), p90/p95/p99, throughput, error rate.
- Heavy load → k6/JMeter/Gatling.

➡️ Next: **Chapter 11 — Newman, Postman CLI and CI/CD**.

Chapter 11 — Newman, Postman CLI and CI/CD

> 🧒 The Postman app is your **toy car with a remote**. **Newman** and the **Postman CLI** are **the same car driving by itself** on a track (your build server) every time someone changes the code — no human needed to press "Send".

11.1 Why run collections from the command line?

- **CI/CD**: run API tests automatically on every commit/PR/deploy.
- **Quality gates**: fail the pipeline if tests fail (exit code ≠ 0).
- **Reports**: JUnit XML for Jenkins/GitLab/Azure, HTML for humans.
- **Scheduling & scripting**: cron jobs, Docker, Kubernetes jobs.

Two tools:

	Newman	Postman CLI
Made by	Postman (open source, MIT)	Postman (official, closed source binary)
Install	`npm install -g newman` (needs Node.js)	Install script / binary (also available via npm as `postman-cli` in recent releases)
Input	Collection/environment **files or URLs** (JSON v2.x)	Collection/environment **IDs** (from Postman cloud, needs login) **or files**; v12 also local YAML (v3) paths
Results to Postman cloud	❌ No	✅ Yes (run history visible in Postman, API Catalog CI reports)
Reporters	cli, json, junit, progress, emojitrain + community (htmlextra, allure, ...)	cli, json, junit, html
Governance/linting	❌	✅ `postman api lint`, `postman spec lint`
Programmatic Node API	✅ `require('newman')`	❌
Package Library / Vault	Limited / not supported for Vault	Team packages supported when logged in; Vault methods not supported
Best for	Fully offline/open-source pipelines, custom reporters, Node integration	Teams wanting results in Postman, governance, v12 Native Git workflows

---

11.2 Newman — Installation

```
node -v            # Newman 6.x requires a modern Node.js LTS (check README for minimum)
npm install -g newman
newman -v
# optional: HTML reporter
npm install -g newman-reporter-htmlextra
```

Project-local (recommended for reproducible CI):

```
npm install --save-dev newman newman-reporter-htmlextra
npx newman run ...
```

11.3 Newman — Running collections

```
newman run Orders.postman_collection.json \
  -e QA.postman_environment.json \
  -d data/orders.csv \
  -n 3 \
  --folder "01 - Orders" \
  --delay-request 200 \
  --timeout-request 10000 \
  --env-var "clientSecret=$CLIENT_SECRET" \
  -r cli,junit,htmlextra \
  --reporter-junit-export results/junit.xml \
  --reporter-htmlextra-export results/report.html
```

Newman options (most important)

Option	Meaning
`run <collection>`	File path **or URL** (e.g. Postman API link)
`-e, --environment <path/url>`	Environment file
`-g, --globals <path/url>`	Globals file
`-d, --iteration-data <path>`	CSV/JSON data file
`-n, --iteration-count <n>`	Iterations
`--folder <name>`	Run only this folder/request (repeat flag for multiple)
`--env-var "k=v"`	Set environment variable (repeatable)
`--global-var "k=v"`	Set global variable (repeatable)
`--working-dir <path>`	Base dir for relative file paths (form-data files)

Option	Meaning
`--no-insecure-file-read`	Block reading files outside working dir
`--export-environment <path>`	Write final environment (with script changes)
`--export-globals <path>`, `--export-collection <path>`	Same for globals/collection
`--delay-request <ms>`	Delay between requests
`--timeout <ms>`	Whole-run timeout
`--timeout-request <ms>`	Per-request timeout
`--timeout-script <ms>`	Per-script timeout
`-k, --insecure`	Disable SSL verification
`--ignore-redirects`	Don't follow 3xx
`--bail [folder,failure]`	Stop on first error/failure (modifiers: `folder` = stop on invalid folder, `failure` = stop on test failure)
`-x, --suppress-exit-code`	Always exit 0 (don't fail pipeline — use carefully)
`--color on/off/auto`, `--disable-unicode`, `--verbose`	Output control
`-r, --reporters <list>`	`cli,json,junit,progress,emojitrain` + installed ones
`--reporter-json-export <path>`	JSON report file
`--reporter-junit-export <path>`	JUnit XML
`--reporter-cli-no-summary`, `--reporter-cli-no-failures`, `--reporter-cli-no-assertions`, `--reporter-cli-no-success-assertions`, `--reporter-cli-no-console`, `--reporter-cli-silent`, `--reporter-cli-show-timestamps`	CLI reporter tuning
`--cookie-jar <path>`, `--export-cookie-jar <path>`	Load/save cookies
`--ssl-client-cert`, `--ssl-client-key`, `--ssl-client-passphrase`, `--ssl-client-cert-list`, `--ssl-extra-ca-certs`	Certificates

Exit codes

- `0` — all good.
- `1` — test failures, script errors, or run errors. CI uses this to pass/fail the job.

Run from Postman API (always latest collection, no exports)

```
newman run "https://api.getpostman.com/collections/$COLLECTION_UID?apikey=$POSTMAN_API_KEY" \
  -e "https://api.getpostman.com/environments/$ENV_UID?apikey=$POSTMAN_API_KEY"
```

> ⚠️ The API key appears in the URL → may leak into logs. Newman 6 also has `--postman-api-key <key>`: pass the plain API URLs (without `?apikey=`) and the key separately, e.g. `newman run "https://api.getpostman.com/collections/$UID" --postman-api-key "$POSTMAN_API_KEY"`. Mask it in CI, use a service account key with minimal access, rotate regularly. Alternatively download the files with `curl -H "X-API-Key: $KEY"` first, then run locally.

htmlextra reporter (very popular)

```
newman run c.json -r htmlextra \
  --reporter-htmlextra-export ./reports/index.html \
  --reporter-htmlextra-title "Orders API Regression" \
  --reporter-htmlextra-browserTitle "Orders API" \
  --reporter-htmlextra-showOnlyFails \
  --reporter-htmlextra-skipSensitiveData
```

Other options (from the reporter's README): `--reporter-htmlextra-hideRequestBody "Login"`, `--reporter-htmlextra-hideResponseBody "Login"`, `--reporter-htmlextra-showEnvironmentData`, `--reporter-htmlextra-skipHeaders "Authorization"`, `--reporter-htmlextra-template custom.hbs` (the package ships alternative templates, including a dark one), `--reporter-htmlextra-omitHeaders`, `--reporter-htmlextra-omitRequestBodies`, `--reporter-htmlextra-omitResponseBodies`, `--reporter-htmlextra-timezone "Asia/Kolkata"`, `--reporter-htmlextra-logs`, `--reporter-htmlextra-testPaging`. Also `newman-reporter-allure` for Allure reports.

> 🔐 HTML/JSON reports include requests/responses → can contain **tokens and PII**. Use skip/hide options and restrict report artifact access.

11.4 Newman as a Node.js library

```
// run-tests.js
const newman = require("newman");

newman.run({
    collection: require("./Orders.postman_collection.json"),
    environment: "./QA.postman_environment.json",
    iterationData: "./data/orders.csv",
    envVar: [{ key: "clientSecret", value: process.env.CLIENT_SECRET }],
    reporters: ["cli", "junit"],
    reporter: { junit: { export: "./results/junit.xml" } },
    timeoutRequest: 10000,
    bail: false
}, (err, summary) => {
    if (err) { console.error(err); process.exit(1); }
    const failures = summary.run.failures.length;
    console.log(`Done. Failures: ${failures}`);
    process.exit(failures ? 1 : 0);
})
.on("request", (err, args) => {
```

```
    // args.request, args.response, args.item
    if (args.response && args.response.code >= 500) {
        console.warn(`5xx on ${args.item.name}: ${args.response.code}`);
    }
})
.on("assertion", (err, args) => {
    if (err) console.log(`✗ ${args.assertion} — ${err.message}`);
});
```

Events: `start`, `beforeIteration`, `iteration`, `beforeItem`, `item`, `beforePrerequest`, `prerequest`, `beforeRequest`, `request`, `beforeTest`, `test`, `assertion`, `console`, `exception`, `beforeDone`, `done`.

Use cases: run multiple collections in parallel, custom Slack/Teams notifications, upload results to a dashboard/DB, dynamic config.

Parallel runs

```
const collections = ["users", "orders", "payments"];
Promise.all(collections.map(name => new Promise((resolve) => {
    newman.run({ collection: `./collections/${name}.json`, environment: "./qa.json", reporters:
"cli" },
        (err, summary) => resolve({ name, failures: err ? 1 : summary.run.failures.length }));
}))).then(results => {
    console.table(results);
    process.exit(results.some(r => r.failures) ? 1 : 0);
});
```

⚠️ Parallel runs must not share mutable test data.

11.5 Newman in Docker

```
docker run --rm -v "$PWD":/etc/newman -t postman/newman:alpine \
  run Orders.postman_collection.json -e QA.postman_environment.json \
  -r cli,junit --reporter-junit-export results/junit.xml
```

- Official image: `postman/newman` (tags like `latest`, `alpine`, version tags — **pin a version** in CI).
- For htmlextra, build a small custom image:

```
FROM postman/newman:alpine
RUN npm install -g newman-reporter-htmlextra
ENTRYPOINT ["newman"]
```

- To reach services on the host from the container: `--network host` (Linux) or `host.docker.internal`.

11.6 Postman CLI

Install

```
# Linux
curl -o- "https://dl-cli.pstmn.io/install/linux64.sh" | sh
# macOS (Apple Silicon)
curl -o- "https://dl-cli.pstmn.io/install/osx_arm64.sh" | sh
# macOS (Intel)
curl -o- "https://dl-cli.pstmn.io/install/osx_64.sh" | sh
```

```
# Windows (PowerShell)
powershell.exe -NoProfile -InputFormat None -ExecutionPolicy AllSigned -Command "
[System.Net.ServicePointManager]::SecurityProtocol = 3072; iex ((New-Object
System.Net.WebClient).DownloadString('https://dl-cli.pstmn.io/install/win64.ps1'))"
```

(Check the "Automate runs via CLI" panel in the Runner for the exact current command.)

Common commands

```
postman login --with-api-key "$POSTMAN_API_KEY"   # EU data region accounts: add --region eu
postman collection run 12345678-abcd-... -e 12345678-efgh-...      # IDs from Postman
postman collection run ./Orders.postman_collection.json -e ./qa.json  # local files
postman collection run <id> -d data.csv -n 2 --bail \
    --env-var "clientSecret=$CLIENT_SECRET" \
    -r cli,junit --reporter-junit-export results/junit.xml
postman api lint <api-id>          # governance/security rules on an API definition
postman spec lint <spec-file-or-id> # lint specs (Spec Hub)
postman logout
```

Useful options (similar to Newman): `-e/--environment`, `-g/--globals`, `-d/--iteration-data`, `-n/--iteration-count`, `-i/--folder` (run specific folder/request), `--env-var`, `--global-var`, `--delay-request`, `--timeout`, `--timeout-request`, `--timeout-script`, `-k/--insecure`, `--bail`, `-x/--suppress-exit-code`, `--working-dir`, `--ssl-client-cert(-list)`, `--silent`, `--color`, `-r/--reporters cli,json,junit,html` with `--reporter-<name>-export`, `--report-events` (send run results to the API Catalog CI report). Run `postman collection run --help` for your version.

v12 Native Git workflow (Git-native workspaces)

- Collections live as **YAML (Collection v3)** files in your repo (e.g. under `postman/`).
- Run by path: `postman collection run ./postman/collections/Orders` (exact path layout per your workspace).
- `postman workspace push` pushes local changes to the cloud workspace (e.g. after merge to main in CI). Local files are the source of truth — edits in the cloud view may be overwritten by the next push.
- Some reporters may be limited for v3 YAML in early v12 releases (docs note only `cli` for some request types) — verify for your version.

11.7 CI/CD integration examples

GitHub Actions — Newman

```yaml
# .github/workflows/api-tests.yml
name: API Tests
on:
  pull_request:
  push:
    branches: [main]
jobs:
  newman:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci        # newman + htmlextra pinned in package.json devDependencies
      - name: Run API tests
        env:
          CLIENT_SECRET: ${{ secrets.QA_CLIENT_SECRET }}
        run: |
          npx newman run postman/Orders.postman_collection.json \
            -e postman/env/qa.postman_environment.json \
            --env-var "clientSecret=$CLIENT_SECRET" \
            -r cli,junit,htmlextra \
            --reporter-junit-export results/junit.xml \
            --reporter-htmlextra-export results/report.html \
            --reporter-htmlextra-skipSensitiveData
      - name: Upload report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: api-test-report
          path: results/
```

GitHub Actions — Postman CLI

```yaml
      - name: Install Postman CLI
        run: curl -o- "https://dl-cli.pstmn.io/install/linux64.sh" | sh
      - name: Login
        run: postman login --with-api-key ${{ secrets.POSTMAN_API_KEY }}
      - name: Run collection
        run: |
          postman collection run "${{ vars.COLLECTION_ID }}" \
            -e "${{ vars.QA_ENV_ID }}" \
            --env-var "clientSecret=${{ secrets.QA_CLIENT_SECRET }}"
```

Jenkins (Declarative pipeline)

```groovy
pipeline {
  agent { docker { image 'postman/newman:alpine'; args '--entrypoint=' } }
  environment { CLIENT_SECRET = credentials('qa-client-secret') }
  stages {
```

```
    stage('API tests') {
      steps {
        sh '''
          newman run postman/Orders.postman_collection.json \
            -e postman/env/qa.postman_environment.json \
            --env-var "clientSecret=$CLIENT_SECRET" \
            -r cli,junit --reporter-junit-export results/junit.xml
        '''
      }
    }
  }
  post {
    always { junit 'results/junit.xml' }
  }
}
```

(`--entrypoint=` overrides the image's `newman` entrypoint so Jenkins can run `sh` steps.)

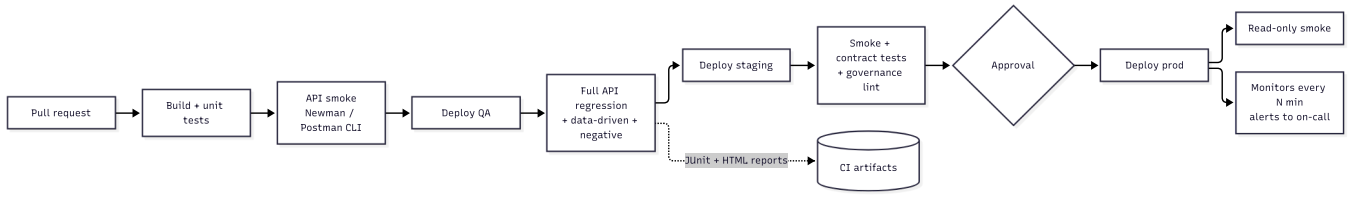
GitLab CI

```
api_tests:
  image:
    name: postman/newman:alpine
    entrypoint: [""]
  stage: test
  script:
    - newman run postman/Orders.postman_collection.json -e
postman/env/qa.postman_environment.json
        --env-var "clientSecret=$QA_CLIENT_SECRET"
        -r cli,junit --reporter-junit-export results/junit.xml
  artifacts:
    when: always
    reports:
      junit: results/junit.xml
```

Azure DevOps

```
steps:
  - task: NodeTool@0
    inputs: { versionSpec: '20.x' }
  - script: npm install -g newman
  - script: >
      newman run postman/Orders.postman_collection.json
      -e postman/env/qa.postman_environment.json
      --env-var "clientSecret=$(QA_CLIENT_SECRET)"
      -r cli,junit --reporter-junit-export $(Build.ArtifactStagingDirectory)/junit.xml
    displayName: Run API tests
  - task: PublishTestResults@2
    condition: always()
    inputs:
      testResultsFormat: JUnit
      testResultsFiles: '$(Build.ArtifactStagingDirectory)/junit.xml'
```

Where in the pipeline?




```
commit → build → unit tests → deploy to QA → [SMOKE API tests] → [REGRESSION API tests]
       → deploy to STAGING → [SMOKE + contract tests] → approval → deploy PROD → [read-only SMOKE
+ monitors]
```

- **PR builds**: fast smoke/contract subset (`--folder Smoke`).
- **Nightly**: full regression + data-driven + negative suites.
- **Post-deploy prod**: read-only health checks (never destructive).

11.8 Organising Postman assets in a Git repo (export-based approach)

```
api-tests/
├── package.json                # newman + reporters pinned
├── postman/
│   ├── collections/
│   │   ├── orders.postman_collection.json
│   │   └── payments.postman_collection.json
│   ├── environments/
│   │   ├── dev.postman_environment.json      # no secrets
│   │   ├── qa.postman_environment.json
│   │   └── staging.postman_environment.json
│   └── data/
│       ├── orders-boundaries.json
│       └── login-negative.csv
├── files/                      # upload fixtures (form-data)
├── scripts/run-all.js          # Newman Node runner
└── .github/workflows/api-tests.yml
```

Sync options: export manually (error-prone), Postman API sync script, built-in Git integration for API specs/collections (v10/v11), or **Native Git** (v12) where the repo *is* the source.

11.9 Newman vs Postman app differences (bugs that only show up in CI)

Symptom in CI	Why	Fix
`{{var}}` unresolved	Environment not passed / variable disabled / secret only in local value (not exported)	`-e` , `--env-var` from CI secrets

Symptom in CI	Why	Fix
File upload fails `ENOENT`	Path relative to your laptop	`--working-dir`, files in repo
OAuth 2 token missing	Interactive "Get New Access Token" doesn't run in CLI	Token fetch in pre-request script
Vault secrets empty	Vault not supported in Newman/CLI	Inject via `--env-var`
`pm.require` package not found	Newman can't load team packages (or not logged in for CLI)	Inline helpers / `postman login`
SSL errors (`UNABLE_TO_GET_ISSUER_CERT_LOCALLY`)	Corporate CA not trusted on runner	`--ssl-extra-ca-certs ca.pem` (Newman) or `NODE_EXTRA_CA_CERTS=ca.pem`; `-k` only as last resort
Proxy issues	CI behind corporate proxy	Set `HTTP_PROXY` / `HTTPS_PROXY` / `NO_PROXY` env vars
Timeouts	CI network slower; no per-request timeout	`--timeout-request`, retries, delays
Tests pass in app, fail in Newman	Older Newman sandbox lacks newer APIs (`pm.execution`, top-level await, `pm.response.to.have.jsonSchema` in very old versions)	Upgrade Newman; avoid bleeding-edge APIs; or use Postman CLI
Cookies missing	Fresh cookie jar per run	Login request in setup; `--cookie-jar`
Variables changed by scripts lost	Newman uses copies	`--export-environment` if needed
Unicode garbled in logs	Terminal encoding	`--disable-unicode`

11.10 Chapter 11 Quick Revision

- Newman: open-source npm CLI, file/URL inputs, rich reporters, Node API, Docker image `postman/newman`.
- Postman CLI: official binary, IDs + login, results in Postman, linting (`api lint`, `spec lint`), v12 Native Git (`workspace push`, YAML collections).
- Key flags: `-e -g -d -n --folder --env-var --bail -r --reporter-junit-export --timeout-request -k --working-dir`.
- Exit code 1 = failures → pipeline fails.
- Secrets via CI secret store + `--env-var`; never in exported files.

➡️ Next: **Chapter 12 — Mock Servers, Monitors, API Design (Spec Hub), Documentation, Flows, Postman API, AI**.

Chapter 12 — Mock Servers, Monitors, API Design (Spec Hub), Documentation, Flows, Postman API, Insights & AI

PART A — MOCK SERVERS

> 🧒 A **mock server** is a **pretend shopkeeper made of cardboard**. You ask "one apple please" and it always hands you the plastic apple you gave it earlier. Useful when the real shopkeeper (backend) hasn't arrived yet.

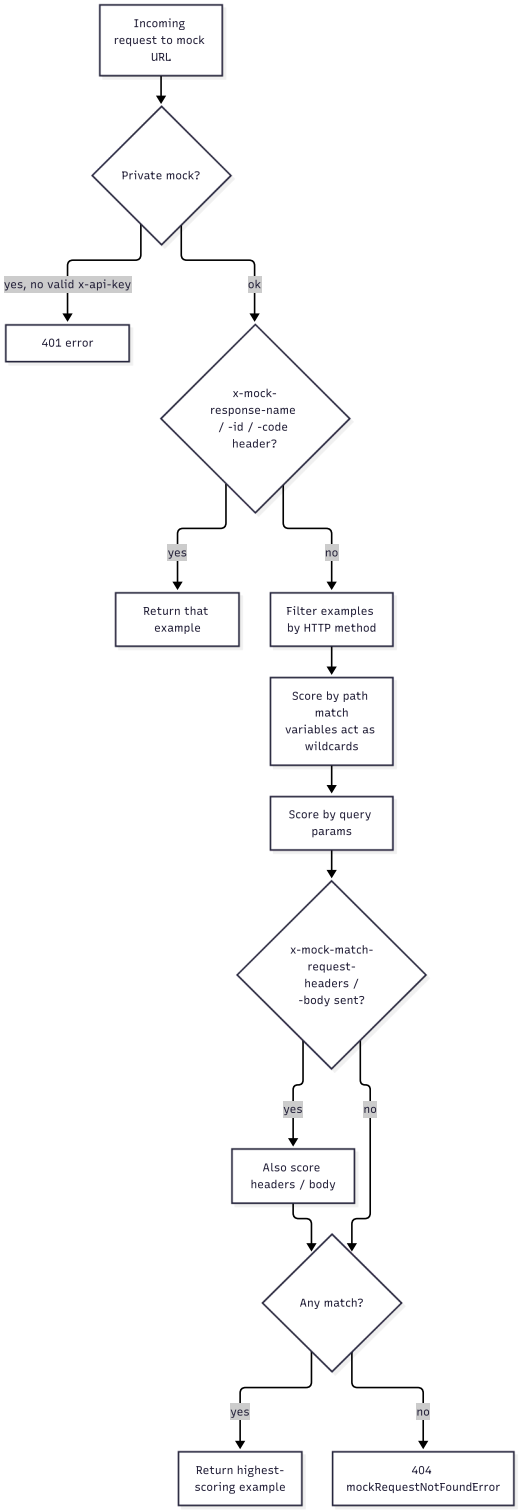
12.1 Why mocks?

- **Frontend/mobile teams** build UI before backend is ready (parallel development).
- **Design-first**: agree on the contract via examples/spec, then mock it.
- **Testing edge cases** that are hard to reproduce: 500 errors, timeouts, empty lists, huge payloads.
- **Isolating dependencies**: third-party APIs that cost money / have rate limits / no sandbox.
- **Demos and onboarding**.

12.2 Creating a mock server

1. Collection `...` → **Mock collection** (or sidebar → Mock servers → `+`), or create from a request's examples, or from a spec.
2. Choose collection (and version/spec), name, **environment** (optional; to resolve variables in examples), **Make mock server private** (requires API key), **Simulate fixed network delay**, and optionally **save the mock URL as an environment variable**.
3. Get URL like `https://<mock-id>.mock.pstmn.io`.
4. Set `{{baseUrl}}` in a "MOCK" environment to this URL → the same collection now hits the mock.

12.3 How a Postman mock decides what to return (matching algorithm)



The mock looks at **saved examples** in the collection. For an incoming request it:

1. Filters examples by **HTTP method**.
2. Matches the **URL path** (examples using variables like `{{orderId}}` or `:orderId` act as wildcards for that segment) and **query parameters** (closer matches score higher).

3. Optionally matches **headers** (if request has `x-mock-match-request-headers: header1,header2`) and **body** (if request has `x-mock-match-request-body: true`).
4. Picks the **highest-scoring** example; if none matches → 404 error JSON ("mockRequestNotFoundError").

12.4 Control headers (send these in your request to the mock)

Header	Effect
`x-api-key: <Postman API key>`	Required for **private** mocks
`x-mock-response-name: <example name>`	Return the example with this name
`x-mock-response-id: <example id>`	Return example by id
`x-mock-response-code: 404`	Return an example with this status code
`x-mock-match-request-body: true`	Include body in matching
`x-mock-match-request-headers: h1,h2`	Include those headers in matching

Example: frontend wants to see the error UI:

```
GET {{mockUrl}}/orders/123
x-mock-response-code: 500
```

12.5 Dynamic mock responses

- Examples can contain **dynamic variables** (`{{$randomFirstName}}`, `{{$guid}}`, `{{$isoTimestamp}}`) → different value each call.
- Examples can use environment variables (resolved with the mock's environment).
- Recent versions support **template helpers** in example bodies to echo request data (e.g. values from path segments, query params, headers, or body) — check your version's "dynamic mock responses" docs for exact syntax.
- **Network delay simulation** (fixed delay, e.g. 2G/3G or custom ms) to test loading spinners/timeouts.

12.6 Mock limitations & tips

Limitation	Tip
Stateless: POST doesn't really create anything; GET after POST returns the saved example	For stateful fakes use a local tool (WireMock, json-server, Prism, MockServer)
Usage limits per plan (mock calls/month)	Use local mocks for heavy automated tests
Matching surprises (wrong example returned)	Use `x-mock-response-name` / `-code`; make example URLs specific
Public mock = anyone with URL can call	Make it private if examples contain internal data
Doesn't validate request body against schema by default	Contract-validate with Spec Hub / Prism

v12 also introduced **code mocks / simulations** usable from the Postman CLI (e.g. `--simulate <file>` YAML simulations; paid plans). Check docs for availability.

PART B — MONITORS

> 🧒 A **monitor** is an **alarm clock robot** that wakes up every few minutes, runs your collection from Postman's cloud, and rings a bell (email/Slack) if anything breaks.

12.7 Creating a monitor

Collection `...` → **Monitor collection** (or sidebar → Monitors → `+`):

Setting	Meaning
Collection & environment	What to run, with which variables
Frequency	e.g. every 5/15/30 min, hourly, daily, weekly (minimum depends on plan)
Regions	Run from multiple geographic regions (multi-region on paid plans) — detects regional outages/latency
Notifications	Email recipients on failure/errors; plus integrations (Slack, Microsoft Teams, PagerDuty, Opsgenie, Datadog, New Relic, webhooks, etc.)
Retry on failure	Re-run once before alerting (reduce false alarms)
Request timeout / delay between requests	
Follow redirects / disable SSL validation	
Use static IP (Enterprise)	So firewalls can allowlist Postman's monitor IPs

12.8 Monitor results

- Dashboard: pass/fail per run, **response time graphs**, failed tests, errors, console logs, filters by region/request.
- Each run consumes **monitoring usage** (plan quota).

12.9 What to monitor (best practice)

- **Health/readiness endpoints** (`/health`, `/ready`).
- **Critical business flows** in **read-only** form: login → search → product details.
- **Third-party dependencies** you rely on (payment provider status, SMS API).
- **SSL certificate expiry**: Postman scripts don't expose the server certificate's expiry date, so use a dedicated certificate-expiry monitor, or call an internal endpoint that reports it.
- **SLA**: response time thresholds.

- **Never** run destructive operations in prod monitors; use dedicated **monitor test accounts** with least privilege.

12.10 Monitors can't reach my internal API!

Monitors run in Postman's cloud. For private/internal APIs: expose via gateway with allowlisted **static IPs** (Enterprise), use a public staging endpoint, or run the collection on a schedule **inside your network** with **Newman/Postman CLI via cron/CI** instead.

PART C — API DESIGN: SPEC HUB / API BUILDER

12.11 Design-first vs code-first

Design-first	Code-first
Write the **OpenAPI** contract first, review, mock, then implement	Write code, generate the spec from code (Swagger annotations, springdoc, FastAPI)
Parallel FE/BE work, early feedback, consistent APIs	Faster for small teams; spec often lags
Postman: Spec Hub → mock → generate collection → tests	Postman: import generated spec → collection

12.12 OpenAPI essentials (you'll read/write these in Postman)

```yaml
openapi: 3.0.3
info:
  title: Orders API
  version: 1.2.0
servers:
  - url: https://api.shop.com/v1
paths:
  /orders/{orderId}:
    get:
      summary: Get an order
      operationId: getOrder
      tags: [Orders]
      parameters:
        - name: orderId
          in: path
          required: true
          schema: { type: integer, minimum: 1 }
      security:
        - bearerAuth: []
      responses:
        '200':
          description: OK
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Order' }
              example: { id: 42, status: PAID, total: 499.0 }
        '404':
          description: Not found
```

```yaml
components:
  securitySchemes:
    bearerAuth: { type: http, scheme: bearer, bearerFormat: JWT }
  schemas:
    Order:
      type: object
      required: [id, status, total]
      properties:
        id: { type: integer }
        status: { type: string, enum: [PENDING, PAID, SHIPPED, CANCELLED] }
        total: { type: number, format: double }
```

Key parts: `openapi`, `info`, `servers`, `paths` → operations (`get/post/...`) → `parameters` (`in: path|query|header|cookie`), `requestBody`, `responses`; `components` (`schemas`, `securitySchemes`, `parameters`, `responses`, `examples`); `security`; `tags`. OpenAPI 3.1 aligns schemas fully with JSON Schema 2020-12 (e.g. `type: [string, "null"]` instead of `nullable: true`). Swagger 2.0 is the older format (`swagger: "2.0"`, `definitions`, `basePath`).

12.13 Spec Hub (v11+) / APIs (API Builder)

- Create specs in **OpenAPI 2.0, 3.0, 3.1** and **AsyncAPI 2.x** (YAML/JSON), single- or **multi-file** (with `$ref` across files).
- Editor with validation, autocomplete, **visual editor** (newer versions), preview docs.
- **Linting / governance**: built-in & custom rules (Spectral-style) for naming, security schemes, descriptions, etc. Violations shown inline; CLI `postman spec lint` / `postman api lint` in CI.
- **Generate collection** from spec; keep them **in sync** (spec → collection, and two-way sync for OpenAPI 3.0 in later v11 versions).
- **Mock** from spec, **document**, **publish versions**.
- **Git integration**: connect spec to GitHub/GitLab/Bitbucket/Azure Repos branch; v12 Native Git keeps specs and collections in your repo.
- Older **API Builder** ("APIs" sidebar): an API element linking a definition + collections (docs, tests) + mocks + monitors + environments, with **versions** and **changelog**. Many companies still use it.

12.14 Contract testing approaches in Postman

1. **Schema validation in tests** (`pm.response.to.have.jsonSchema`) using schemas taken from the spec.
2. **Generated collections** from spec with tests, kept in sync.
3. **Spec validation features** (validate requests in collections against the spec; flags mismatches like missing required params or wrong types).
4. **Consumer-driven contracts** (Pact) are a different approach — Postman is provider-contract oriented; mention Pact for microservices if asked.

PART D — DOCUMENTATION

12.15 Generating & publishing docs

- Every collection has auto-generated docs from: collection/folder/request **descriptions (Markdown)**, **params/headers descriptions**, **examples**, **auth** info, and **code snippets** in chosen language.
- **View documentation** inside workspace (private to team).
- **Publish**: public URL (`documenter.getpostman.com/...` or custom domain on paid plans), choose environment to show values (never secrets!), version, styling (logo, colours), SEO metadata, **Run in Postman** button.
- Keep docs updated by treating descriptions as part of PR review.

12.16 Writing good API docs (checklist)

- Overview: purpose, base URLs per environment, auth instructions, rate limits, error format, versioning, changelog.
- Per request: purpose, params with types/required/defaults, body fields, response fields, examples for success & errors, permissions needed.
- Getting started guide: "Fork collection → select env → get token → first call".

PART E — POSTMAN FLOWS

> 🧒 Flows are **LEGO for APIs**: snap blocks together (call this API → check result → loop → send to Slack) without writing much code.

12.17 Concepts

- A **flow** is a canvas of **blocks** connected by wires carrying data.
- Common blocks: **Start/Trigger** (manual, HTTP request trigger, schedule, **MCP** trigger), **HTTP Request / Send Request** (uses a saved collection request or inline request), **GraphQL request**, **Evaluate** (FQL expressions or TypeScript), **If/Condition**, **Repeat/Loop/For**, **Collect**, **Select** (pick field from data), **Template** (build JSON/text), **Delay**, **Display** (table/chart/output), **Log**, **AI Request** (prompt an LLM such as OpenAI, Anthropic, Google, or OpenAI-compatible providers), **Agent** blocks in newer releases.
- **FQL (Flows Query Language)**: a JSONata-like expression language to transform data (e.g. `$.data[status="active"].email`).
- **Deploy** a flow to the Postman cloud → it gets a URL; can be triggered by webhook/HTTP, schedule, or as an **MCP server** that AI agents (Claude Desktop, VS Code, Cursor) call as a tool, with tools defined via a schema.

12.18 Use cases

- Orchestrate multi-API workflows for demos/PMs.
- Data migration scripts (read from API A, transform, write to API B).
- Slack bots, scheduled reports.
- Building AI agent tools on top of existing APIs.

PART F — POSTMAN API (programmatic control)

12.19 Basics

- Base URL: `https://api.getpostman.com`
- Auth: header `X-API-Key: <your Postman API key>` (generate in Account Settings → API keys; service accounts recommended for CI). Also accepts `?apikey=` query param (avoid; leaks to logs).
- Rate limits and monthly usage limits depend on plan; responses include rate-limit headers.

12.20 Common endpoints

Endpoint	Purpose
`GET /me`	Who am I (validate key)
`GET /workspaces`, `GET /workspaces/{id}`	List workspaces & contents
`GET /collections`, `GET /collections/{uid}`	List/get collections (full JSON)
`POST /collections?workspace={id}`	Create collection
`PUT /collections/{uid}`	Replace collection (careful: full overwrite)
`PATCH /collections/{id}`	Partial update (name/description/variables/auth/events)
`GET /environments/{uid}`, `PUT /environments/{uid}`	Read/update environments (e.g. rotate a token from CI)
`GET /mocks`, `POST /mocks`	Mocks
`GET /monitors`, `POST /monitors/{uid}/run`	Monitors, trigger run
`GET /apis`, specs endpoints	API definitions / Spec Hub
`POST /webhooks?workspace={id}`	Create a webhook URL that **triggers a collection run**
`GET /collections/{uid}/transformations` / import endpoints	Convert OpenAPI ↔ collection
`/audit/logs` (Enterprise)	Audit events

12.21 Webhooks that trigger collections

Create via Postman API (`POST /webhooks` with collection UID) → get a URL. Any `POST` to that URL runs the collection in the cloud; the request payload is available to scripts via the global variable `previousRequest`:

```
const payload = JSON.parse(pm.globals.get("previousRequest"));
```

Use for: "When GitHub release is published → run smoke tests", "When deployment finishes → run checks".

12.22 Example automation: back up all collections nightly

```
for uid in $(curl -s -H "X-API-Key: $POSTMAN_API_KEY" https://api.getpostman.com/collections | jq
-r '.collections[].uid'); do
  curl -s -H "X-API-Key: $POSTMAN_API_KEY" "https://api.getpostman.com/collections/$uid" >
"backup/$uid.json"
done
```

PART G — INSIGHTS, API CATALOG, API NETWORK

12.23 Postman Insights

- An **agent** (Postman Insights Agent, from Postman's acquisition of Akita Software) installed in your infrastructure (e.g. Kubernetes, ECS, Linux hosts) passively **observes live API traffic** (eBPF-based) to **discover endpoints**, track **errors and latency**, and help keep collections/specs in sync with reality — without code changes.
- Good for: "We have 300 undocumented microservice endpoints — what's actually being called?"

12.24 API Catalog (v12) & Private API Network

- An organisation-wide **catalog of APIs**: ownership, specs, collections, CI run results (`--report-events` from Postman CLI), governance status.
- Formerly the **Private API Network** (Enterprise) — internal "app store" of APIs for discovery and reuse.

12.25 Public API Network

- Public workspaces from companies (Stripe, Twilio, Salesforce, PayPal, Microsoft Graph…) with **verified** badges. **Fork** their collection to start integrating quickly.

PART H — AI IN POSTMAN (Postbot & Agent Mode)

12.26 Postbot (v10.17+/v11)

AI assistant that can: generate tests, fix failing tests, explain responses, write documentation, create visualizations, suggest request bodies, help write scripts, summarize collection runs.

12.27 Agent Mode (2025+, central in v12)

- Agentic AI that can perform multi-step tasks across your workspace: create/modify collections and requests, write and run tests, debug failures, create environments, set up monitors and mocks, work with Spec Hub (generate/align specs), and more.
- **Skills**: reusable "recipes" for common tasks, invoked by typing `/` in the chat.
- Uses **AI credits** (plan dependent). Enterprise admins can control AI features.
- Treat output as a **draft**: review tests (they may encode current bugs as expectations), never paste secrets/PII into prompts unless your org's AI policy permits.

12.28 Postman & MCP (Model Context Protocol)

- Postman supports testing **MCP servers** (MCP client request type in recent versions), building MCP servers with **Flows (MCP trigger)**, and an official **Postman MCP server** that lets AI agents (Claude, Cursor, VS Code Copilot…) interact with your Postman workspace (collections, environments, specs) via tools.
- Interview angle: "How would you test an MCP server?" → list tools, call each tool with valid/invalid args, check JSON-RPC responses/errors, auth, timeouts — Chapter 13.

12.29 Chapter 12 Quick Revision

- Mocks return saved **examples**; matching by method → path → query → (headers/body if asked); control with `x-mock-response-name/-code/-id`, private via `x-api-key`. Stateless!
- Monitors: cloud scheduled runs, regions, alerts, retries; can't reach private networks without static IPs/allowlisting.
- Spec Hub: OpenAPI 2.0/3.0/3.1 & AsyncAPI, linting, generate & sync collections, Git.
- Docs auto-generated from descriptions/examples; publish carefully.
- Flows: visual blocks, FQL, deploy, MCP trigger, AI blocks.
- Postman API: `api.getpostman.com`, `X-API-Key`, webhooks trigger collection runs (`previousRequest`).
- Insights (traffic discovery), API Catalog (v12), Postbot & Agent Mode (AI).

➡️ Next: **Chapter 13 — GraphQL, gRPC, WebSocket, Socket.IO, MQTT, SOAP, SSE, MCP & AI requests**.

Chapter 13 — Beyond REST: GraphQL, gRPC, WebSocket, Socket.IO, MQTT, SOAP, SSE, JSON-RPC, MCP & AI Requests

Postman's **New** menu offers dedicated request types: **HTTP**, **GraphQL**, **gRPC**, **WebSocket**, **Socket.IO**, **MQTT**, and in recent versions **AI** and **MCP** requests. Each can be saved into collections.

13.1 GraphQL

> 🧒 REST is a **set menu**: you get the whole plate even if you only want the fries. GraphQL is a **buffet**: you pick exactly what goes on your plate.

Core concepts

Concept	Meaning
Schema (SDL)	Typed description of everything you can ask for
Query	Read data
Mutation	Change data
Subscription	Real-time updates (usually over WebSocket)
Variables	Parameters passed separately from the query
Fragments	Reusable field sets
Introspection	Query the schema itself (`__schema`) — used for autocomplete
Resolvers	Server functions that fetch each field

Example

```
query GetUser($id: ID!) {
  user(id: $id) {
    id
    name
    posts(first: 3) { title }
  }
}
```

Variables:

```
{ "id": "1" }
```

Mutation:

```
mutation CreatePost($input: CreatePostInput!) {
  createPost(input: $input) { id title }
}
```

In Postman — two ways

1. **GraphQL request type** (New → GraphQL): enter URL → Postman fetches schema via **introspection** → **schema explorer / query builder** (tick fields to build query), Query + Variables panes, auth, headers, scripts, **subscriptions** support over WebSocket transports in recent versions.
2. **HTTP request → Body → GraphQL**: Query + GraphQL variables editors; can import schema for autocomplete. Postman sends `POST` with JSON `{ "query": "...", "variables": {...} }`.

Use `{{variables}}` inside the GraphQL **variables JSON**, not by string-concatenating into the query.

Testing GraphQL — key differences from REST

- **Errors usually come with HTTP 200!** Always check the `errors` array:

```
const body = pm.response.json();
pm.test("HTTP 200", () => pm.response.to.have.status(200));
pm.test("No GraphQL errors", () => pm.expect(body.errors,
JSON.stringify(body.errors)).to.be.undefined);
pm.test("User returned", () => {
    pm.expect(body.data.user).to.include.keys("id", "name");
    pm.expect(body.data.user.posts).to.have.lengthOf.at.most(3);
});
```

- **Partial success**: `data` can be present with `errors` for some fields.
- Negative tests: invalid field (`Cannot query field "x"`), wrong variable type, missing required variable, unauthorized field-level access.
- **Security tests**: introspection disabled in prod? query **depth limits** (deeply nested queries), **complexity/cost limits**, batching abuse (array of queries), aliases abuse (`a1: user(id:1) a2: user(id:2) ...` for brute force), field suggestions leaking schema.
- **Performance**: N+1 resolver problems show as slow responses for list queries.

13.2 gRPC

> 🧒 gRPC is like two robots talking in a **secret, super-fast code language** (Protocol Buffers) that both robots learned from the **same dictionary** (`.proto` file).

Concepts

- **Protocol Buffers (protobuf)**: binary, strongly typed message format defined in `.proto` files.
- **Service** with **methods**; runs over **HTTP/2**.

```
syntax = "proto3";
package orders.v1;
service OrderService {
  rpc GetOrder (GetOrderRequest) returns (Order);                          // unary
  rpc ListOrders (ListOrdersRequest) returns (stream Order);               // server streaming
  rpc UploadItems (stream Item) returns (UploadSummary);                   // client streaming
  rpc Chat (stream ChatMessage) returns (stream ChatMessage);              // bidirectional
}
message GetOrderRequest { int64 id = 1; }
message Order { int64 id = 1; string status = 2; double total = 3; }
```

Four method types

Type	Request	Response
Unary	one	one
Server streaming	one	many
Client streaming	many	one
Bidirectional streaming	many	many

In Postman

1. New → **gRPC**.
2. **Server URL** (e.g. `localhost:50051` or `grpc.example.com:443`); toggle **TLS** (lock icon) for secure servers.
3. **Service definition**: use **server reflection** (if server enables it), **import a `.proto` file** (add import paths for dependencies), or use a definition from **Spec Hub/APIs**.
4. Select **method** → compose message in JSON (Postman converts to protobuf) → **Use example message** to auto-generate a template.
5. **Metadata** = gRPC's equivalent of headers (e.g. `authorization: Bearer ...`).
6. **Invoke**. For streaming: send multiple messages, **End streaming**, view message timeline.
7. **Scripts** (Before invoke / After response) and tests are supported for gRPC; the response exposes status code, metadata/trailers and messages — e.g.:

```
pm.test("gRPC OK", () => pm.response.to.have.statusCode(0));
```

(Check the gRPC scripting reference for your version for the full assertion list.)

gRPC status codes (memorise top ones)

Code	Name	HTTP-ish analogy
0	OK	200
1	CANCELLED	client cancelled
2	UNKNOWN	500
3	INVALID_ARGUMENT	400
4	DEADLINE_EXCEEDED	504
5	NOT_FOUND	404
6	ALREADY_EXISTS	409
7	PERMISSION_DENIED	403
8	RESOURCE_EXHAUSTED	429
9	FAILED_PRECONDITION	400/412
10	ABORTED	409
11	OUT_OF_RANGE	400
12	UNIMPLEMENTED	501
13	INTERNAL	500
14	UNAVAILABLE	503
15	DATA_LOSS	500
16	UNAUTHENTICATED	401

Common gRPC problems

Problem	Fix
"Could not load server reflection"	Server doesn't enable reflection → import `.proto`
Import errors in `.proto`	Add import paths (e.g. for `google/protobuf/timestamp.proto`)
`14 UNAVAILABLE` / connection failed	Wrong host/port, TLS toggle mismatch (TLS on a plaintext server or vice versa), server down
`16 UNAUTHENTICATED`	Missing `authorization` metadata
`12 UNIMPLEMENTED`	Wrong package/service/method name or version
int64 values look like strings	Protobuf JSON mapping represents 64-bit ints as **strings** — expected

Problem	Fix
`4 DEADLINE_EXCEEDED`	Increase deadline/timeout in settings

13.3 WebSocket

> 🧒 HTTP is **sending letters**: you send one, wait for a reply. WebSocket is a **phone call**: once connected, both sides can talk anytime.

Concepts

- Starts with an HTTP **Upgrade** handshake (`101 Switching Protocols`), then a persistent full-duplex connection.
- `ws://` (plain) and `wss://` (TLS).
- Frames: text, binary, ping/pong, close (close codes: 1000 normal, 1001 going away, 1006 abnormal, 1008 policy violation, 1011 server error).

In Postman

1. New → **WebSocket** → URL `wss://echo.websocket.org` (public echo server; availability can change) or your server.
2. **Headers/Params** for the handshake (auth tokens are often passed as a query param or header, or in the first message).
3. **Connect** → message composer (Text/JSON/Binary), **Send**; **Saved messages** for reuse.
4. **Messages log**: sent/received with timestamps; search/filter; connection events.
5. **Settings**: handshake timeout, reconnect attempts/intervals, max message size.
6. Raw WebSocket scripting/test support is limited compared with HTTP — verify in your version; for automated WS tests use code-based tools (Node `ws`, k6 websockets, Playwright) if needed.

What to test

- Handshake auth (no token → rejected with 401/403 during upgrade).
- Message format & schema, error messages.
- Ordering, heartbeats (ping/pong), reconnect behaviour, idle timeouts.
- Broadcast correctness (two connections: one sends, other receives).
- Limits: max message size, rate limiting.

13.4 Socket.IO

- A library **on top of** WebSocket (with long-polling fallback), using **events** and **acknowledgements**, rooms and namespaces. A plain WebSocket client **can't** talk to a Socket.IO server properly → use Postman's **Socket.IO** request type.
- In Postman: URL, **client version** (v2/v3/v4), **handshake path** (default `/socket.io`), **Events** tab to add **listeners** (event names to listen to), compose messages with **event name** + arguments, optionally **Ack**.
- Test: event names, payloads, auth middleware rejection, room join/leave behaviour.

13.5 MQTT (IoT)

- **Publish/subscribe** via a **broker** (Mosquitto, EMQX, HiveMQ, AWS IoT Core).
- **Topics** like `factory/line1/temperature`; wildcards `+` (single level), `#` (multi-level).
- **QoS**: 0 = at most once, 1 = at least once, 2 = exactly once.
- **Retain** flag (broker keeps last message), **Last Will and Testament**, **clean session**, keep-alive.
- MQTT 3.1.1 and **5.0** (user properties, reason codes, message expiry, topic aliases).

In Postman: New → **MQTT** → broker URL (`mqtt://broker.hivemq.com:1883` , `mqtts://...:8883` , or WebSocket transport), client ID, auth (username/password, certificates), version, **Topics** to subscribe (with QoS), compose & **publish** messages with QoS/retain; view message timeline.

13.6 SOAP

SOAP = XML **envelope** over (usually) HTTP POST.

SOAP 1.1 request in Postman

- Method: **POST**
- URL: `http://www.dneonline.com/calculator.asmx` (public demo calculator service; availability can change)
- Headers:
  - `Content-Type: text/xml; charset=utf-8`
  - `SOAPAction: "http://tempuri.org/Add"`
- Body → raw → XML:

```xml
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <Add xmlns="http://tempuri.org/">
      <intA>5</intA>
      <intB>7</intB>
    </Add>
```

```
  </soap:Body>
</soap:Envelope>
```

SOAP 1.2 difference

- `Content-Type: application/soap+xml; charset=utf-8; action="http://tempuri.org/Add"` (no separate SOAPAction header), envelope namespace `http://www.w3.org/2003/05/soap-envelope`.

Import WSDL

Import a WSDL URL/file → Postman generates a collection with a request per operation (sample envelopes).

Testing SOAP responses

```
const json = xml2Json(pm.response.text());
const result = json["soap:Envelope"]["soap:Body"]["AddResponse"]["AddResult"];
pm.test("5 + 7 = 12", () => pm.expect(Number(result)).to.eql(12));
```

Or with cheerio in XML mode:

```
const $ = cheerio.load(pm.response.text(), { xmlMode: true });
pm.test("Result is 12", () => pm.expect($("AddResult").text()).to.eql("12"));
```

- **SOAP Faults**: `<soap:Fault><faultcode>...</faultcode><faultstring>...</faultstring></soap:Fault>` typically with HTTP **500** (SOAP 1.1). Test for faults explicitly.
- WS-Security (UsernameToken, signatures) must be built manually into the header (or via scripts generating nonce/timestamps/digests with CryptoJS).

13.7 Server-Sent Events (SSE)

- `GET` with `Accept: text/event-stream`; server streams `event:` / `data:` / `id:` lines; one-way server → client. Common for **LLM streaming APIs** and live dashboards.
- Postman HTTP requests display SSE streams **event by event** in recent versions (the response view shows the stream, with option to view each event).
- For testing assertions on streamed data, the post-response script sees the final body text — parse lines:

```
const events = pm.response.text().split("\n").filter(l => l.startsWith("data:")).map(l =>
l.slice(5).trim());
pm.test("Stream ended with [DONE]", () => pm.expect(events[events.length - 1]).to.eql("[DONE]"));
```

13.8 JSON-RPC

Plain HTTP POST with body:

```
{ "jsonrpc": "2.0", "method": "getBalance", "params": { "account": "0xabc" }, "id": 1 }
```

Errors come as `{ "error": { "code": -32601, "message": "Method not found" }, "id": 1 }` (often HTTP 200). Codes: -32700 parse error, -32600 invalid request, -32601 method not found, -32602 invalid params, -32603 internal error. (Blockchain node APIs like Ethereum use JSON-RPC.)

13.9 MCP (Model Context Protocol) requests

- **MCP** is an open protocol (introduced by Anthropic, 2024) for connecting AI applications to tools and data. Servers expose **tools**, **resources**, and **prompts** over **JSON-RPC 2.0**, via transports like **stdio** (local process) or **Streamable HTTP** (older: HTTP+SSE).
- Recent Postman versions provide an **MCP request** type: connect to a server (stdio command or HTTP URL), list its **tools/prompts/resources**, call tools with arguments, and inspect responses.
- What to test: tool discovery (names, descriptions, input schemas), valid & invalid arguments (schema validation errors), auth (OAuth for remote MCP servers), timeouts and large outputs, error shape (`isError: true` in tool results vs JSON-RPC errors), idempotency of tools that change data, prompt-injection resilience of tool outputs.

13.10 AI (LLM) requests

- Recent versions provide an **AI request** type to send prompts to LLM providers (OpenAI, Anthropic, Google, and others), compare models/parameters, and save them in collections — useful for testing AI features and prompts like any other API.
- You can of course also call LLM APIs as normal HTTP requests (e.g. `POST https://api.anthropic.com/v1/messages` with headers `x-api-key`, `anthropic-version`, `content-type: application/json`).
- Testing LLM outputs: assert **structure** (JSON schema of tool calls/structured outputs), **latency**, **token usage**, safety; avoid asserting exact wording (non-deterministic) — use contains/regex or an evaluator.

13.11 Chapter 13 Quick Revision

- GraphQL: one endpoint, POST `{query, variables}`, errors often with **200** → check `errors`. Security: depth/complexity limits, introspection.

- gRPC: protobuf over HTTP/2, 4 method types, reflection or `.proto`, metadata = headers, status codes 0–16 (0 OK, 3 INVALID_ARGUMENT, 5 NOT_FOUND, 7 PERMISSION_DENIED, 14 UNAVAILABLE, 16 UNAUTHENTICATED).
- WebSocket: upgrade (101), full-duplex, close codes; Socket.IO needs its own client.
- MQTT: broker, topics, QoS 0/1/2, retain, LWT.
- SOAP: POST XML envelope; 1.1 `text/xml` + `SOAPAction`; 1.2 `application/soap+xml; action=`; Faults; WSDL import.
- SSE streams; JSON-RPC error codes; MCP (tools/resources/prompts over JSON-RPC); AI requests.

➡️ Next: **Chapter 14 — Collaboration, Version Control & Organising Large Projects**.

Chapter 14 — Collaboration, Version Control & Organising Large Projects

> 🧒 When you build a giant LEGO castle alone, you can be messy. When **50 kids** build it together, you need **rules**: who builds which tower, where the pieces are kept, and that nobody smashes someone else's tower without asking.

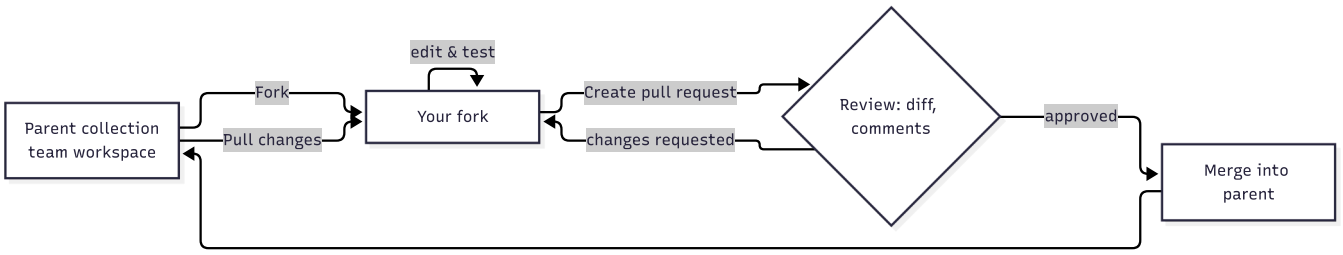
14.1 Collaboration features

Feature	Use
Team workspaces	Shared home for a product/domain
Roles (workspace & element level)	Admin / Editor / Viewer; restrict who edits prod environments
Comments	On collections, folders, requests, and even specific lines/fields; `@mention` teammates; resolve threads
Forks & pull requests	Safe changes with review, diff, approval, merge
Watch	Notifications on changes
Changelog / activity feed / version history	Who changed what; restore previous versions
Share links & invite	Give access by email/link
Partner workspaces (Enterprise)	Collaborate with external vendors securely
Groups (Enterprise)	Manage permissions by team groups synced via SCIM
API Catalog / Private API Network	Discover internal APIs
Notifications/integrations	Slack/Teams updates on collection changes, monitor failures

---

14.2 Version control options

Option A — Inside Postman (forks/PRs/version history)



- Fork → change → PR → review → merge.
- Collection **version history/changelog**: restore.
- **Tags/releases** on APIs (API Builder versions).

Option B — Git integration (v10/v11)

- Link an **API (Spec Hub / API Builder)** to a GitHub/GitLab/Bitbucket/Azure DevOps repo branch; push/pull spec and linked collections.
- Code review happens in Git; Postman shows sync status.

Option C — Native Git (v12)

- Workspace connected to a Git repository; **collections stored as YAML (Collection v3)**, split into multiple files under a `postman/` folder; specs alongside code.
- Work **locally** (even offline), commit, open Git PRs; CI runs `postman collection run <path>` and `postman workspace push` to sync to cloud.
- Pros: real diffs, branch per feature, tests live with code, code owners review API changes.
- Rule: local files are the source of truth; avoid editing the same items in cloud view.

Option D — Manual export to Git (legacy, still very common)

- Export JSON v2.1, commit; Newman runs from files.
- Problems: big unreadable diffs, merge conflicts in a single JSON file, stale copies. Mitigate with normalisation scripts and **single owner per collection**.

Option	Best for
Forks/PRs	Teams living inside Postman
Git integration	Spec-first teams on v10/v11
Native Git	Developer-centric teams on v12
Manual export	Small/legacy setups, open-source pipelines

14.3 Workspace strategy for a big organisation

```
Team: ACME Corp
├── 🏢 Platform — Auth API            (team workspace, owned by Identity team)
├── 🏢 Commerce — Orders API          (team workspace, owned by Orders team)
├── 🏢 Commerce — Payments API        (team workspace)
├── 🧪 QA — E2E Regression            (team workspace: cross-service E2E collections, references
others)
├── 🤝 Partner — Logistics Vendor X   (partner workspace)
├── 🌍 ACME Public APIs               (public workspace: published docs & collections)
└── 👤 Personal workspaces            (individual scratch work)
```

Rules:

1. **One owner team per workspace**; description lists owners, Slack channel, on-call.
2. **Reference collection per service** (generated from OpenAPI, documentation-grade) + **test collections** (smoke, regression, negative, E2E).
3. **Environments per stage per service**: `Orders — DEV/QA/STAGE/PROD` with identical variable names.
4. **Prod environments** restricted (Viewer for most; secrets in Vault/CI only).
5. **Fork for changes** to shared/critical collections; PR review by owners.
6. **No personal stuff** in team workspaces; no team assets only in personal workspaces (bus factor!).
7. **Archive** deprecated collections (rename with `[DEPRECATED]`, move to an archive workspace).

14.4 Naming conventions

Element	Convention	Example
Workspace	`<Domain> — <Service> API`	`Commerce — Orders API`
Collection	`<Service> API — <Purpose>`	`Orders API — Regression`
Folder	Number prefix for order + area	`01 - Orders`, `99 - Teardown`
Request	`<METHOD> <what> [- condition (status)]` or human sentence	`POST Create order`, `POST Create order - empty cart (400)`
Environment	`<Service> — <STAGE>`	`Orders — QA`
Variables	camelCase, descriptive	`ordersBaseUrl`, `customerId`
Examples	`<status> <scenario>`	`200 OK`, `404 Order not found`
Tests	`<Thing> <expectation>`	`Status is 201`, `total equals sum of items`

14.5 Collection design standards (template for a team "Postman Guidelines" doc)

1. **Collection description**: purpose, owners, how to run, required environment variables, test data assumptions.
2. **Auth at collection level**, requests inherit.
3. **Base URL variable** always; never hard-coded hosts.
4. **Every request has**: description, at least status + schema tests, examples for main status codes.
5. **Setup/Teardown folders**; tests independent & idempotent.
6. **No secrets** in shared values/scripts; Vault/CI secrets only.
7. **Shared helpers** in Package Library; no copy-paste of 50-line scripts.
8. **No `console.log` of tokens/PII**.
9. **Use IDs for setNextRequest**.
10. **Lint & review**: PR checklist (below).

Pull request review checklist for Postman changes

- ☐ Naming conventions followed
- ☐ No hard-coded URLs/tokens/IDs
- ☐ Tests meaningful (not only status code), negative cases included
- ☐ Variables cleaned up; no globals introduced
- ☐ Scripts readable; helpers reused
- ☐ Examples & descriptions updated
- ☐ Runs green on QA environment (attach run link/report)
- ☐ No breaking changes to shared environments

14.6 Environment management at scale

- Keep a **variables dictionary** (in collection description or wiki): name, purpose, which env sets it, secret or not.
- Use **identical keys** across environments → switching env never breaks requests.
- **Runtime variables** (IDs, tokens) set by scripts → clean them up; never share them.
- **Rotate** shared credentials; prefer per-user credentials (each tester's own account) for manual work and **service accounts** for CI.
- When a variable is renamed: search & replace across collection (Find and Replace), update all environments, communicate in channel.

14.7 Onboarding a new team member (checklist)

1. Invite to team; add to workspace(s) with right role.
2. Point to "Start here" collection description.

3. They create **Vault** secrets / local values for credentials.
4. Select environment; run **Smoke** folder → all green = setup OK.
5. Read Postman guidelines doc; first change via fork + PR.

14.8 Handling common team situations

Situation	What to do
Two people edited the same request; changes lost	Use forks + PRs for shared collections; check version history to restore
Someone leaked a token in a shared value / public workspace	**Revoke/rotate the secret immediately**, remove from Postman, check secret-scanner alerts, check public workspace visibility, review audit logs (Enterprise), post-mortem
Collection became a 2,000-request monster	Split by service/purpose; reference vs test collections; Package Library for helpers
Teammates' runs fail but yours pass	Compare environments (local vs shared values), Postman versions, cookies, data; share via Console logs
API changed (breaking) and 300 tests fail	Update spec first → regenerate/sync collection → fix tests; versioned APIs (`/v2`) → new folder/collection
Need to share collection with external vendor	Partner workspace or a sanitised public/forked copy; never share internal env
Postman outage / offline	Native Git/local files (v12), exported backups, Newman from Git
Person leaves company	Ensure assets are in team workspaces; transfer ownership; remove access; rotate creds they knew
Different Postman versions in team (UI labels differ)	Standardise version; document both label names

---

14.9 Governance (Enterprise-scale)

- **API governance rules** (style guide as lint rules): naming, required descriptions, security schemes, versioning, error format.
- **API security rules**: detect missing auth, sensitive data exposure patterns in specs.
- Run linting in CI (`postman api lint` / `postman spec lint`) — fail PRs on violations.
- **Audit logs** (Enterprise): who accessed/changed/shared what.
- **SSO + SCIM** for identity lifecycle; **domain capture** to stop employees using personal Postman accounts for company work.
- **Secret scanner**: detects exposed secrets in workspaces; configure alerts.
- **Data residency** (EU data region) where required.

14.10 Chapter 14 Quick Revision

- Collaboration: team workspaces, roles, comments, forks/PRs, version history.
- Version control: in-app forks, Git integration (v10/11), Native Git YAML (v12), or manual exports.
- One owner per workspace; reference + test collections; identical env keys; no secrets in shared values.
- PR checklist; onboarding checklist; leak response = rotate first.

➡️ Next: **Chapter 15 — API Security Testing with Postman**.

Chapter 15 — API Security Testing with Postman

> ⚖️ **Only test systems you own or are explicitly authorised to test** (your company's QA/staging, bug bounty programs within scope, CTF labs). Unauthorised security testing is illegal in most countries.

> 🧒 Security testing is **checking every door and window of your own house** to make sure a thief can't get in — trying the handle, checking if the spare key is under the mat, seeing if the window latch is broken.

Postman is not a full security scanner (like Burp Suite, OWASP ZAP), but it's excellent for **manual and automated logic-level security tests** (auth, authorization, input validation, data exposure), especially as **regression tests** in CI.

15.1 OWASP API Security Top 10 (2023) — with Postman tests

API1:2023 — Broken Object Level Authorization (BOLA / IDOR)

User A can access User B's object by changing an ID.

```
GET {{baseUrl}}/orders/{{orderIdOfUserB}}
Authorization: Bearer {{tokenUserA}}
```

```
pm.test("BOLA: cannot read other user's order", () =>
pm.expect(pm.response.code).to.be.oneOf([403, 404]));
```

Automate: setup creates resources as User B, tests access them as User A for **every endpoint with an ID** (GET/PUT/PATCH/DELETE). Data-drive IDs. Also try sequential/guessable IDs, UUIDs from other responses, IDs in body/query/headers, not only path.

API2:2023 — Broken Authentication

- No token / malformed / expired / tampered / `alg:none` JWT → 401.
- Brute force: 20 wrong passwords → lockout or 429.
- Password reset token reuse/expiry; OTP brute force; weak token entropy.
- Credential stuffing protections, no credentials in URLs.

```
// Data-driven: tokens.csv with columns token,expected
pm.test(`token case → ${pm.iterationData.get("expected")}`, () =>
    pm.response.to.have.status(Number(pm.iterationData.get("expected"))));
```

API3:2023 — Broken Object Property Level Authorization

Combines old "Excessive Data Exposure" and "Mass Assignment".

- **Excessive exposure**: response includes fields user shouldn't see (`passwordHash` , `internalNotes` , other users' emails).

```
pm.test("No sensitive props", () => {
    const text = pm.response.text();
    ["passwordHash", "ssn", "cardNumber", "secret", "internal"].forEach(k =>
pm.expect(text).to.not.include(`"${k}"`));
});
```

- **Mass assignment**: send extra fields that shouldn't be writable.

```
PATCH /users/me
{ "name": "Asha", "role": "admin", "isVerified": true, "balance": 999999 }
```

```
pm.sendRequest({ url: pm.variables.replaceIn("{{baseUrl}}/users/me"), header: { Authorization:
"Bearer " + pm.environment.get("accessToken") } },
  (e, r) => pm.test("role not escalated", () => pm.expect(r.json().role).to.not.eql("admin")));
```

API4:2023 — Unrestricted Resource Consumption

- No pagination limits (`?limit=1000000`), huge payloads, big file uploads, expensive GraphQL queries, no rate limiting.

```
pm.test("Server caps page size", () =>
pm.expect(pm.response.json().data.length).to.be.at.most(100));
```

- Burst requests in Runner (no delay) → expect **429** with `Retry-After` . (Coordinate with team; don't DoS shared environments.)

API5:2023 — Broken Function Level Authorization (BFLA)

Normal user calling admin functions.

```
DELETE {{baseUrl}}/admin/users/123      Authorization: Bearer {{userToken}}   → expect 403
GET    {{baseUrl}}/internal/metrics                                           → expect 401/403/404
```

Try changing method (`GET` allowed → try `PUT` / `DELETE`), changing path (`/api/v1/users` → `/api/v1/admin/users`), and hidden/old versions (`/v0/`).

API6:2023 — Unrestricted Access to Sensitive Business Flows

Abuse of legitimate flows at scale: buying all limited stock, mass coupon redemption, fake account creation, spam referrals. Test: same coupon applied twice, negative quantities, race conditions (run two requests in parallel with `pm.sendRequest`), skipping steps (pay → ship without payment).

API7:2023 — Server-Side Request Forgery (SSRF)

APIs that fetch URLs you provide (webhooks, avatar URL import, PDF generation).

```
{ "avatarUrl": "http://169.254.169.254/latest/meta-data/" }
{ "webhookUrl": "http://localhost:6379/" }
{ "importUrl": "file:///etc/passwd" }
```

Expect rejection (400) and no internal data in responses/time differences.

API8:2023 — Security Misconfiguration

```
const h = pm.response.headers;
pm.test("HSTS", () => pm.expect(h.get("Strict-Transport-Security")).to.exist);
pm.test("nosniff", () => pm.expect(h.get("X-Content-Type-Options")).to.eql("nosniff"));
pm.test("No server banner", () => { pm.expect(h.has("X-Powered-By")).to.be.false; });
pm.test("CORS not wildcard with credentials", () => {
    const acao = h.get("Access-Control-Allow-Origin"); const acac = h.get("Access-Control-Allow-
Credentials");
    pm.expect(!(acao === "*" && acac === "true")).to.be.true;
});
pm.test("No stack trace", () =>
pm.expect(pm.response.text()).to.not.match(/Exception|Traceback|at [\w.$]+\(/));
```

Also: `TRACE` disabled, `OPTIONS` reveals only intended methods, HTTP → HTTPS redirect, verbose errors, default credentials, debug endpoints (`/actuator`, `/swagger-ui`, `/debug`) exposed in prod.

CORS test: send `OPTIONS` with `Origin: https://evil.example` and `Access-Control-Request-Method: PUT` → server must not reflect evil origin.

API9:2023 — Improper Inventory Management

Old/undocumented versions and hosts still live (`/v1` deprecated but reachable without new protections; `beta-api.company.com`). Use Postman **Insights**/API Catalog to discover; test old versions for missing auth.

API10:2023 — Unsafe Consumption of APIs

Your API trusts third-party API data blindly. Use mocks to return malicious/malformed third-party responses (huge payloads, injection strings, redirects) and verify your API handles them safely.

15.2 Injection testing with data files

`injection.csv`:

```
payload,description
' OR '1'='1,SQLi classic
"'; DROP TABLE users;--",SQLi destructive (test env only!)
"{""$gt"":""""}",NoSQL operator
<script>alert(1)</script>,XSS stored
"{{7*7}}",SSTI
"../../../../etc/passwd",Path traversal
"; ls -la",Command injection
%00,Null byte
```

Body: `{"search": "{{payload}}"}` (careful with quoting in JSON — escape `"` in payloads or use a JSON data file). Tests: status not 500, response doesn't include DB errors (`SQL syntax`, `ORA-`, `MongoError`), XSS payload is escaped/encoded when reflected, time-based checks (`responseTime` not suspiciously high for `SLEEP(5)` payloads).

> ⚠️ Destructive payloads only in disposable test environments with permission.

15.3 Security regression suite structure

```
📦 Orders API — Security
 ├── 📁 AuthN (no token, bad token, expired, alg none, revoked)
 ├── 📁 AuthZ — BOLA (userA vs userB for each endpoint)
 ├── 📁 AuthZ — BFLA (user vs admin endpoints & methods)
 ├── 📁 Property-level (mass assignment, data exposure)
 ├── 📁 Input validation & injection (data-driven)
 ├── 📁 Rate limiting & resource limits
 ├── 📁 Headers & CORS & TLS config
 └── 📁 Business logic abuse
```

Run nightly in CI against staging; failures = security bugs → triage with security team.

15.4 Postman's own security hygiene

Risk	Mitigation
Secrets in shared values / exported files	Vault, local values, CI secrets; secret scanner
Accidentally public workspace	Review visibility; Enterprise admins restrict public workspace creation
Tokens in screenshots/Console/HTML reports	Secret type, `skipSensitiveData`, scrub logs
Over-privileged API keys for Postman API	Service accounts, least privilege, rotation

Risk	Mitigation
Using production data for tests	Synthetic data (dynamic variables), masked datasets
Disabling SSL verification globally	Add CA certs instead; disable per request only for local dev
Proxy capture certificate left trusted on devices	Remove Postman CA after capture sessions
Running untrusted collections (scripts can `sendRequest` your tokens anywhere)	Review scripts in imported/public collections before running with real credentials

That last one is important: **a collection's scripts can read your environment (including tokens) and send them to any URL**. Treat third-party collections like third-party code.

15.5 Postman vs dedicated security tools

Tool	Strength
Postman	Business-logic auth/authorization tests, regression in CI, readable for whole team
OWASP ZAP / Burp Suite	Proxy-based scanning, fuzzing, active vulnerability detection
Specialized API security platforms	Spec-based fuzzing, runtime protection
SAST/DAST in CI	Code & running app scanning

Common combo: import Postman collection into ZAP/Burp (via proxy run or OpenAPI) to scan endpoints that need complex auth flows.

15.6 Chapter 15 Quick Revision

- OWASP API Top 10 2023: BOLA, Broken Auth, Broken Object Property Level AuthZ, Unrestricted Resource Consumption, BFLA, Sensitive Business Flows, SSRF, Security Misconfiguration, Improper Inventory Management, Unsafe Consumption of APIs.
- Postman shines at authz matrices (userA vs userB, user vs admin), mass-assignment, exposure, headers/CORS, data-driven injection.
- Never run destructive payloads outside disposable environments with permission.
- Third-party collection scripts can exfiltrate your variables — review before running.

➡️ Next: **Chapter 16 — Debugging & Troubleshooting (Every Common Error)**.

Chapter 16 — Debugging & Troubleshooting: Every Common Error and How to Fix It

> 🧒 When your toy doesn't work, a good fixer doesn't randomly hit it. They check **one thing at a time**: Is it switched on? Are the batteries in? Is the wire connected? This chapter is your **checklist for broken API requests**.

16.1 The debugging method (use for EVERY problem)

1. **Read the exact error** (status code, error text, Console output). Don't guess.
2. **Open the Postman Console** (`Ctrl/Cmd + Alt/Option + C`) → expand the request → see the **actual** URL, headers, body sent and received (after variable resolution!).
3. **Isolate**: same request on httpbin/echo? from cURL? from another machine/network? in a fresh tab without scripts? with a different environment?
4. **Compare working vs non-working** side by side (the app vs Postman, colleague vs you, local vs CI) — find the difference in URL, headers, body, auth, cookies, network.
5. **Change one thing at a time**.
6. **Check the server side**: logs with correlation ID (`X-Correlation-ID: {{$guid}}` → ask backend to search logs).
7. **Write down the fix** in the collection docs/wiki so the team never loses time on it again.

Classifying the problem

Where it fails	Typical signs
Before sending (Postman/scripts/variables)	Script errors, red variables, "Invalid URL"
Network (DNS/TCP/TLS/proxy)	No status code; `ENOTFOUND` , `ECONNREFUSED` , `ETIMEDOUT` , SSL errors
Server rejects request	4xx
Server crashes	5xx
Response OK but tests fail	Assertion errors, JSON errors
Works in app, fails in CI	Environment/secrets/files/versions differences

---

16.2 "Could not send request" / network errors

Error	Meaning	Fixes
`getaddrinfo ENOTFOUND api.shop.com`	DNS can't resolve host	Typo in host; **unresolved variable** (`ENOTFOUND {{baseurl}}`); VPN not connected (internal DNS); `/etc/hosts` ; corporate DNS; try `nslookup host`
`connect ECONNREFUSED 127.0.0.1:8080`	Host reachable but nothing listening on that port	Server not running; wrong port; server listening on different interface; firewall actively rejecting
`connect ECONNREFUSED ::1:8080`	Postman resolved `localhost` to IPv6 `::1` , but your server listens only on IPv4 `127.0.0.1`	Use `127.0.0.1` instead of `localhost` , or make the server listen on `::` /both
`connect ETIMEDOUT` / `ESOCKETTIMEDOUT`	No response in time (packets dropped)	Firewall/security group, VPN, wrong IP, server overloaded; increase timeout to check
`Error: socket hang up` / `ECONNRESET`	Server/proxy closed connection abruptly	Server crash, TLS mismatch, mTLS cert missing, proxy issues, request too large, keep-alive issues; check server logs
`EHOSTUNREACH` / `ENETUNREACH`	No route to host	Network/VPN/routing
`Error: Invalid protocol: ...` / "Invalid URL"	Malformed URL, missing scheme, spaces	Add `https://` ; encode spaces; check variables
`EPROTO ... wrong version number`	Speaking **HTTPS to an HTTP port** (or vice versa)	Use `http://` for plain ports (e.g. `http://localhost:8080`)
`Request timed out` (Postman message)	Hit Postman's request timeout setting	Increase timeout (Settings → General) or investigate server slowness
`Maximum response size reached`	Response larger than limit	Settings → Max response size; Send and Download
`EACCES` / `EPERM`	OS/antivirus/permission blocks	Firewall/antivirus allow Postman; corporate endpoint security
Request hangs forever with "Sending request..."	Timeout = 0 (infinite) and server never responds; proxy waiting for auth	Set timeout; check proxy settings

`localhost` gotchas

Situation	Problem	Fix
Postman **web** app → `localhost`	Cloud agent can't reach your machine	Use **Desktop Agent** or desktop app

Situation	Problem	Fix
Server in **Docker**, Postman on host	Port not published	`docker run -p 8080:8080` ; call `localhost:8080`
Newman in **Docker** container → service on host	`localhost` = the container itself	`host.docker.internal` (Docker Desktop) or `--network host` (Linux)
Server in **WSL2**, Postman on Windows	Usually forwarded; sometimes not	Use WSL IP (`hostname -I`) or enable localhost forwarding
Server on another laptop	It listens only on 127.0.0.1	Bind server to `0.0.0.0` ; firewall allow; use its LAN IP
Android emulator calling host	`localhost` = emulator	`10.0.2.2` (not a Postman issue, but common confusion when comparing)

16.3 SSL/TLS errors

Error	Meaning	Fix
`self signed certificate` / `self-signed certificate in certificate chain`	Cert (or chain) not signed by trusted CA	Add company/dev **CA cert** in Settings → Certificates → CA Certificates (PEM); or disable SSL verification **for that request** (dev only)
`unable to get local issuer certificate` / `UNABLE_TO_GET_ISSUER_CERT_LOCALLY`	Intermediate/root CA unknown (corporate TLS inspection proxy like Zscaler re-signs traffic)	Add corporate root CA PEM to Postman; Newman `--ssl-extra-ca-certs` or `NODE_EXTRA_CA_CERTS`
`UNABLE_TO_VERIFY_LEAF_SIGNATURE`	Server didn't send full chain (missing intermediate)	Ask server team to fix chain; temporarily add intermediate CA
`certificate has expired` / `CERT_HAS_EXPIRED`	Expired cert	Renew cert (server); check your system clock
`Hostname/IP does not match certificate's altnames` / `ERR_TLS_CERT_ALTNAME_INVALID`	Calling by IP or wrong hostname	Use the hostname in the cert's SAN; fix cert
`alert handshake failure` / `SSL routines` errors	Protocol/cipher mismatch, or server requires client cert (mTLS)	Add client cert (Settings → Certificates); check TLS versions in request Settings
Works with SSL verification OFF only	Trust chain problem	Fix trust properly (CA cert) — don't leave verification off

> 🏢 **Corporate network tip:** If *everything* HTTPS fails with issuer errors on your work laptop but works at home, your company's TLS inspection proxy is re-signing traffic. Export the corporate root CA (from your OS/browser certificate store) as PEM and add it to Postman → problem solved properly.

16.4 Proxy problems

Symptom	Fix
Works on home Wi-Fi, fails in office	Configure **Settings → Proxy**: use system proxy, or custom proxy host/port, proxy auth (407 errors)
Internal hosts fail when proxy enabled	Add them to **proxy bypass** list
`407 Proxy Authentication Required`	Enter proxy credentials in proxy settings
Newman/CLI behind proxy	Set `HTTP_PROXY`, `HTTPS_PROXY`, `NO_PROXY` environment variables
Postman capture proxy (5555) left enabled on device	Device traffic fails when Postman closes → remove proxy config from device

16.5 Status-code-driven debugging

400 Bad Request

Checklist:

1. Body valid JSON? (Beautify; look for trailing commas, smart quotes `“ ”` copied from Word/Slack!)
2. `Content-Type` correct (raw → **JSON**)?
3. Required fields present, correct names (case-sensitive: `userId` vs `userid`) and types (`"5"` vs `5`)?
4. Variables resolved inside body? (Console → request body)
5. Date formats (`2026-10-16` vs `16/10/2026`), enums.
6. Read the **error message in response** — most APIs tell you exactly which field.

401 Unauthorized

1. Auth tab type correct? Request set to **Inherit** but parent has **No Auth**?
2. Token variable resolved, not expired, no extra spaces/newline, correct prefix (`Bearer ` with a space)?
3. Correct environment (QA token on DEV server = invalid)?
4. Both Auth tab **and** manual `Authorization` header present (conflict)?
5. Clock skew for signed auth (OAuth1/Hawk/AWS).
6. Decode JWT → check `aud`, `iss`, `exp`, scopes.

403 Forbidden

1. User role/scopes insufficient (decode JWT; check IdP config).
2. API key lacks permission / not subscribed to that API product.

3. IP allowlist / WAF blocking (cloud WAFs sometimes block requests with `PostmanRuntime` User-Agent or suspicious payloads — check WAF logs).
4. CSRF token missing (session-based apps).
5. Resource ownership (BOLA protection working as intended!).

404 Not Found

1. Base URL & path (version prefix `/api/v1` ?), trailing slash (`/users/` vs `/users`), path variable empty → `/users/` .
2. Wrong environment.
3. Resource really doesn't exist (deleted, different tenant).
4. Method mismatch on some frameworks returns 404 instead of 405.
5. Reverse proxy routing (gateway path rewrite).

405 Method Not Allowed

Wrong method; check `Allow` response header.

413 / 414 / 431

Payload too big / URL too long / headers too big (clear cookie jar!).

415 Unsupported Media Type

`Content-Type` missing/wrong; for form-data don't set it manually.

422 Unprocessable

Business validation; read the error details.

429 Too Many Requests

Add Runner delay; respect `Retry-After` ; use different test accounts; ask for higher limits in test env.

500 Internal Server Error

- Server bug → collect evidence: request (cURL from Code snippet, with secrets removed), response, timestamp, correlation ID → raise a bug.
- Try to find the minimal input that triggers it (often null/unexpected types).

502 / 503 / 504

Gateway/upstream issues: service down, deployment in progress, timeouts; retry later, check status pages/monitoring, inform DevOps.

3xx when you expected 200

- `http://` → `https://` redirect; login redirect (session expired → 302 to `/login`) — turn off "follow redirects" to see it.
- POST becoming GET after 301/302 → enable "Follow original HTTP method" or use the final URL directly.

16.6 Response is OK, but something's wrong

Symptom	Cause	Fix
Got HTML login page instead of JSON	Session expired / SSO redirect / wrong host	Check Preview tab; re-auth; check redirects
Response is gibberish	Binary/compressed content shown as text	Check `Content-Type` / `Content-Encoding`; use Send and Download
Old data returned	Caching (CDN/proxy)	`Cache-Control: no-cache` setting; cache-busting query `?_={{$timestamp}}`
Numbers changed (`1234567890123456789` → `1234567890123456000`)	JS precision in Pretty view / `json()`	Ask API for string IDs; inspect Raw view
Dates off by 5:30h	Timezones (UTC vs IST)	Compare in UTC
Characters broken (`Ã©`)	Encoding mismatch	Server must send UTF-8 with charset
Empty body with 200	API returns 204-like behaviour, or wrong `Accept` header	Check API contract; add `Accept: application/json`

16.7 Script errors

Error	Cause	Fix
`ReferenceError: x is not defined`	Typo/undeclared variable; using a JS variable from another script	Declare with `const/let`; pass data via Postman variables
`TypeError: Cannot read properties of undefined (reading 'id')`	Path doesn't exist in response (`json.data.id` but `data` missing)	Log `pm.response.json()`; use optional chaining; check status first
`JSONError: Unexpected token '<' at 1:1`	`pm.response.json()` on HTML	Check status/content-type before parsing
`JSONError: No data, empty input at 1:1`	Empty body (204)	Guard: `if (pm.response.text())`

Error	Cause	Fix
`SyntaxError: Unexpected token ...`	JS syntax error in your script	Check line numbers in Console; missing brackets/commas
`SyntaxError` when `JSON.parse(pm.request.body.raw)`	Body contains unresolved `{{var}}`	`JSON.parse(pm.variables.replaceIn(pm.request.body.raw))`
`TypeError: pm.execution.skipRequest is not a function`	Used in post-response	Move to pre-request
`pm.response is undefined` /not usable	Using `pm.response` in pre-request	Only in post-response
`AssertionError: expected 200 to equal '200'`	Type mismatch	Convert types
`Error: Cannot find module 'xyz'`	`require` of unsupported library	Use built-in libs or Package Library (`pm.require`)
`Error: ... package not found` (`pm.require`)	Package name/version wrong or no access; Newman/CLI context	Check package library; login for CLI
Script takes forever / run hangs	Unresolved callback, infinite `setInterval` , long `setTimeout` , infinite loops	`clearInterval` ; ensure callbacks complete; `--timeout-script` in CLI
`There was an error in evaluating the Pre-request Script`	Any uncaught error in pre-request	Console shows details; wrap in try/catch
Test results empty	Script error before any `pm.test` , or tests inside a callback that never fires	Console; ensure `sendRequest` callbacks run
Variable set in script "not saved"	Set in post-response but used in same request; wrong scope; Runner "keep values" off; local vs shared value	See Chapter 5 checklist

16.8 Variable problems (quick recap)

1. Red `{{var}}` → undefined in active scopes.
2. Wrong environment selected / "No environment".
3. Precedence (data/local overriding).
4. Local vs shared values (teammates don't see your local value).
5. Trailing whitespace.
6. Disabled variable.
7. CSV BOM / header mismatch.
8. Case sensitivity.
9. `pm.variables.set` (local) vs `pm.environment.set` .
10. Newman: env file not passed or outdated export.

16.9 Cookies & session problems

Symptom	Fix
Logged in as wrong user	Clear cookie jar for domain; check `Cookie` header in Console
Logout test fails (still authenticated)	Old cookie still sent; or Bearer token still in Auth
CSRF mismatch	Read CSRF cookie → header in pre-request script
Cookies not stored	Cookie `Domain` / `Path` / `Secure` mismatch (Secure cookie over http), "Disable cookie jar" ticked
Script can't read cookie via jar	Add domain to cookie **allowlist**

16.10 "It works in Postman but not in my app" (and the reverse)

Works in Postman, fails in browser frontend

→ Almost always **CORS** (browser enforces; Postman desktop doesn't). Check browser console for `blocked by CORS policy`. Fix on server: `Access-Control-Allow-Origin`, `-Methods`, `-Headers`, handle `OPTIONS` preflight, credentials rules. Other causes: frontend sends different headers/body (compare with DevTools "Copy as cURL"), cookies `SameSite`, mixed content (https page calling http API).

Works in Postman, fails in backend code (Java/Python/Node)

- Code doesn't follow redirects / sends different `Content-Type` / doesn't encode body properly / missing `User-Agent` (some APIs/WAFs require one) / proxy settings differ / TLS trust store (Java `cacerts`) lacks corporate CA / HTTP/2 differences.
- Use Postman's **Code snippet** for your language as a reference and compare byte by byte with a request bin (httpbin.org/anything).

Works in app/curl, fails in Postman

- Postman auto-headers (`Postman-Token`, `User-Agent: PostmanRuntime/...`) rejected by strict servers/WAF → disable/override.
- Postman auto-encoding the URL differently → disable "Encode URL automatically".
- Cookies in Postman's jar interfering → disable cookie jar for the request.
- Proxy/SSL settings in Postman differ from system.
- Body trimming/whitespace settings.

16.11 Runner, Newman & CI problems

See Chapter 10 (Runner) and Chapter 11.9 (Newman vs app). Extra:

- **Flaky tests** (pass/fail randomly): async processing not complete → poll instead of fixed delays; shared test data across parallel runs → unique data per run; environment instability → retries for transient 5xx only; time-dependent assertions → avoid.
- **Order-dependent failures**: tests rely on previous ones implicitly → make dependencies explicit (setup folder) or independent.
- **Huge runs crash app** → run via Newman/CLI, disable response saving.

16.12 OAuth problems

See Chapter 6.8 table (`invalid_client` , `invalid_grant` , `redirect_uri_mismatch` , expired tokens, browser auth issues).

16.13 Postman app problems

Problem	Fix
App slow / high memory	Close unused tabs; clear History; disable response saving in Runner; reduce Console logs; large responses → Raw view; update app
Sync issues ("Not synced", conflicts)	Check internet/proxy; sign out & in; check status.postman.com; Settings → Data export as backup before troubleshooting
Changes "disappeared"	Different workspace? fork vs parent? unsaved tab? Check Trash and version history
UI elements missing (e.g. "Tests" tab)	Renamed in newer version (Scripts → Post-response)
Can't install / blank window	Clear app data (backup first), reinstall; GPU issues: start with hardware acceleration disabled
Interceptor not connecting	Install/update Interceptor bridge, restart browser & app, check extension permissions
Desktop Agent not detected (web app)	Install/run agent, select Desktop Agent in bottom-right, allow it through firewall
Need app logs	**View → Developer** menu (Show DevTools, view logs). Log folders: Windows `%APPDATA%\Postman\logs` , macOS `~/Library/Application Support/Postman/logs` , Linux `~/.config/Postman/logs`
Status of Postman cloud	status.postman.com

16.14 Writing a great bug report from Postman

```
Title: POST /orders returns 500 when items[].qty is null

Environment: QA (orders-qa.shop.com), 2026-10-16 10:42 IST
Correlation ID: 8a3c6f2e-...
Steps:
 1. Get token (client credentials, user qa_buyer_1)
 2. POST /v1/orders with body (below)
Expected: 400 with validation error "qty is required"
Actual: 500 {"error":"Internal Server Error"}
Request (cURL from Postman, token redacted):
 curl -X POST 'https://orders-qa.shop.com/v1/orders' -H 'Authorization: Bearer <redacted>' -H
'Content-Type: application/json' --data '{"items":[{"sku":"A1","qty":null}]}'
Response headers/body: ...
Collection link: <Postman link to request/example>
Frequency: 100% reproducible
```

Tip: **Save the failing response as an example** and link it in the ticket.

16.15 Chapter 16 Quick Revision

- Console first; compare working vs failing; change one thing at a time; correlation IDs.
- Network errors: ENOTFOUND (DNS/unresolved var), ECONNREFUSED (port/IPv6 ::1), ETIMEDOUT (firewall), socket hang up (server/TLS/mTLS), EPROTO wrong version (https on http port).
- SSL: add CA certs (corporate TLS inspection!), don't leave verification off.
- Status code checklists for 400/401/403/404/415/429/5xx.
- Script errors: JSONError `<` = HTML; unresolved vars in `request.body.raw`; skipRequest in post-response.
- Postman works but browser doesn't = CORS.

➡️ Next: **Chapter 17 — Best Practices & Situational Playbook**.

Chapter 17 — Organisational Best Practices & Situational Playbook

17.1 The 40 golden rules (print this)

Structure

1. One **reference collection** per service (from OpenAPI) + separate **test collections** (smoke, regression, negative, security, E2E).
2. Folders numbered for run order; **Setup** first, **Teardown** last.
3. Requests named clearly; descriptions filled in; examples for main status codes.
4. Auth at **collection level**; requests **inherit**.
5. Shared logic at collection/folder level or **Package Library**; no copy-paste.

Variables & secrets

6. `{{baseUrl}}` **always**; never hard-code hosts.
7. **Identical variable names** across environments.
8. Config in **environment**, constants in **collection**, runtime IDs in collection/local, **avoid globals**.
9. **No secrets in shared/initial values**, scripts, examples, docs, or exports.
10. Secrets: **Vault** locally, **CI secret store +** `--env-var` in pipelines.
11. Use `secret` type to mask sensitive values on screen.
12. Clean up runtime variables in teardown.

Tests

13. Every request: status + schema/structure + key business assertions.
14. Negative + boundary + auth tests for every important endpoint.
15. Tests follow the **spec**, not current behaviour.
16. Unique, synthetic test data per run (`runId` , dynamic variables).
17. Tests independent and **idempotent**; can run in any environment (except destructive ones guarded from prod).
18. Generic checks (no 5xx, JSON valid, time SLA) at collection level.
19. Avoid brittle assertions (timestamps, ordering, exact messages unless contractual).
20. Poll instead of fixed waits for async processes.

Automation

21. Collections run in **CI** on every PR (smoke) and nightly (full).
22. JUnit + HTML reports archived; failures notify the owning team.
23. Pin Newman/CLI versions; keep runner images reproducible.

24. Quality gate: failing API tests block merge/deploy.
25. Prod: **read-only smoke tests + monitors** only.

Collaboration

26. Team workspaces per domain, clear owners.
27. Fork + PR for shared collections; review with checklist.
28. Comments for discussions; link Jira tickets in descriptions.
29. Version control via Native Git/Git integration or disciplined exports.
30. Onboarding guide in collection overview.

Security

31. Review third-party collections' scripts before running them with real credentials.
32. Keep SSL verification on; add CA certs instead.
33. Don't log tokens/PII; scrub reports.
34. Least-privilege test & service accounts; rotate keys.
35. Public workspaces reviewed by security before publishing.

Maintenance

36. Remove dead requests/collections (archive).
37. Migrate legacy `tests[]` / `postman.*` syntax.
38. Keep collections in sync with the spec (Spec Hub sync, contract tests).
39. Track flaky tests and fix root causes.
40. Document conventions in a team "Postman Guidelines" page.

17.2 Situational playbook — "What do I do if…?"

🔥 Situation 1: You join a company with 50 messy collections and no standards

1. **Inventory**: list collections, owners, last updated, usage (who runs them, CI references).
2. **Classify**: keep / merge / archive.
3. **Propose guidelines** (Chapter 14 + 17.1), get buy-in from leads.
4. Start with **one critical service**: build clean reference + smoke collection, wire into CI, show value (caught bugs, saved time).
5. Roll out service by service; pair with teams.
6. Archive the rest after a deprecation period.

🔥 Situation 2: A token/secret was committed to a public workspace or Git

1. **Revoke/rotate immediately** (assume it's compromised — bots scan public Postman workspaces and GitHub within minutes).
2. Remove from Postman (shared values, history, examples, docs) and Git history (and consider repo exposure).
3. Check logs for misuse of the leaked credential.
4. Inform security team; follow incident process.
5. Prevent: Vault, secret scanner alerts, pre-commit secret scanning (gitleaks/trufflehog), review public workspace settings.

🔥 Situation 3: Tests pass locally but fail in CI

Run through Chapter 11.9 table: environment/secrets passed? file paths? OAuth interactive? Vault? Newman version vs app features? CA certs/proxy? timeouts? data collisions? Add `--verbose` and print (non-secret) variable values to debug.

🔥 Situation 4: Backend isn't ready, frontend needs to start

1. Agree on the contract (OpenAPI in Spec Hub).
2. Add **examples** (success + errors) → **mock server**.
3. Frontend uses `{{baseUrl}}` = mock URL; error states via `x-mock-response-code`.
4. Backend implements; QA writes tests against mock first (structure), then switches env to real QA server.
5. Contract tests ensure real API matches spec.

🔥 Situation 5: An API change broke 200 tests overnight

1. Check: intentional (release notes, spec diff) or regression?
2. If regression → bug with evidence; block release.
3. If intentional → was it **breaking**? Should have been versioned; raise process issue.
4. Update spec → sync collection → bulk-fix tests (Find & Replace, shared helpers make this quick).
5. Add **contract tests + spec linting** in CI to catch breaking changes before merge.

🔥 Situation 6: Production incident — "API is slow/erroring"

1. Use **read-only** requests against prod (with approval) to reproduce; capture response times breakdown (DNS/TLS/TTFB).
2. Compare regions/environments (monitor results by region).
3. Provide correlation IDs to SRE.
4. Don't run load tests on prod during an incident!
5. After: add a **monitor** for that endpoint/SLA.

🔥 Situation 7: You need to test 10,000 combinations

1. Generate a data file (script) with **pairwise/combinatorial** reduction where possible.
2. Run via **Newman/CLI**, not the app; disable response saving; split into parallel jobs with unique data.
3. Report via JUnit/htmlextra (`showOnlyFails`).

🔥 Situation 8: Rate limits block your test runs

Delays between requests, retry with `Retry-After`, separate test accounts/API keys, ask for higher limits in test env, avoid unnecessary calls (cache tokens!).

🔥 Situation 9: Async/eventual consistency (Kafka/queues) makes tests flaky

Polling pattern with timeout and max attempts (Chapter 9), assert final state; avoid fixed `setTimeout(5000)`; check idempotency of event consumers by replaying.

🔥 Situation 10: Multi-tenant SaaS testing

Environment per tenant or tenant ID variable; cross-tenant access tests (BOLA across tenants!); data cleanup per tenant.

🔥 Situation 11: You must test a file upload API

form-data with File type; set per-part content type if needed; keep files in repo (`files/`); Newman `--working-dir`; test size limits (413), wrong types (415/400), malicious names (`../../x.sh`), zero-byte files, corrupted files.

🔥 Situation 12: Manager asks "How much of our API is tested?"

Postman doesn't produce code coverage; build **API coverage**: list endpoints from OpenAPI vs requests in test collections (Postman API + script), per method & status code; track in dashboard. Some Postman views and Insights help discover endpoints actually used.

🔥 Situation 13: Testing a legacy SOAP service alongside REST

Import WSDL; envelopes with variables; `xml2Json`/cheerio tests; fault tests; keep in same collection under `SOAP` folder or separate collection.

🔥 Situation 14: Migrating from another tool (Insomnia, SoapUI, Thunder Client, JMeter)

Export from tool (Insomnia export / OpenAPI / HAR / cURL) → import → fix auth & variables → rewrite assertions in `pm.*` (SoapUI Groovy assertions don't convert) → run side by side until confidence.

🔥 Situation 15: Migrating away from Postman (or to Native Git)

Export v2.1 JSON or use Native Git YAML; tests in `pm.*` only run in Postman-compatible runners (Newman/CLI) — plan rewrite if moving to code-based frameworks (REST Assured, Playwright API, pytest+requests, Karate).

17.3 Postman vs other tools (know when to use what)

Tool	Type	When better than Postman
cURL / HTTPie	CLI	Quick one-offs, scripts, servers without GUI
Insomnia / Bruno / Hoppscotch / Thunder Client	API clients	Bruno: Git-native, offline, plain files (Postman v12 now similar); Hoppscotch: web/open-source
SoapUI / ReadyAPI	API testing	Heavy SOAP/WS-* enterprise testing
REST Assured (Java), pytest+requests (Python), Playwright/Supertest (JS), Karate	Code frameworks	Complex logic, reuse with app code, strong typing, large-scale automation owned by SDETs
JMeter / k6 / Gatling / Locust	Load testing	Real performance/load testing
OWASP ZAP / Burp	Security	Scanning/fuzzing
WireMock / Prism / MockServer	Mocking	Stateful, local, programmable mocks
Swagger UI / Redoc / Stoplight	Docs/design	Docs embedded in your product site

Interview answer pattern: "Postman is excellent for exploratory testing, collaboration, documentation, and maintainable API regression suites runnable in CI via Newman/Postman CLI. For very complex automation logic or heavy load, I'd complement it with code frameworks or k6/JMeter."

17.4 A maturity model for API testing with Postman

Level	Characteristics
0 — Ad hoc	Random requests in personal workspaces, no tests
1 — Organised	Collections, environments, variables, basic status tests
2 — Automated	Scripts chaining, schema tests, data-driven, Runner
3 — Integrated	Newman/CLI in CI with quality gates, reports, monitors

Level	Characteristics
4 — Governed	Spec-first, contract tests, linting, Package Library, Native Git/PR reviews, security suites
5 — Optimised	Coverage metrics, Insights-driven discovery, flaky-test tracking, AI-assisted maintenance, self-service API Catalog

Use this in interviews when asked "How would you improve our API testing?"

➡️ Next: **Chapter 18 — Interview Questions & Answers (Entry → Expert)**.

Chapter 18A — Interview Questions & Answers: Fundamentals (APIs, HTTP, Postman Basics)

> These questions have been asked again and again in API testing, QA, SDET, backend developer and support-engineer interviews at service companies (TCS, Infosys, Wipro, Cognizant, Accenture, Capgemini, HCL, Tech Mahindra, LTIMindtree), product companies and startups, and in global firms. Answers are written so you can **say them out loud** — short first line, then depth.
>
> Levels: 🟢 Entry, 🟡 Intermediate, 🔴 Experienced

Part 1 — API & HTTP concepts

Q1. 🟢 What is an API? An API (Application Programming Interface) is a contract that lets one software system request data or actions from another without knowing its internals. Example: a weather app calls `GET /weather?city=Pune` and gets JSON back. It defines endpoints, methods, request/response formats, auth, and errors.

Q2. 🟢 What is API testing and why is it important? Testing the application's business layer directly by sending requests and validating responses (status, body, headers, performance, security), without a UI. It's important because it's **faster and more stable than UI tests**, catches defects earlier (shift-left), covers integrations between services, and can be automated in CI.

Q3. 🟢 What is the difference between API testing and UI testing? UI testing checks what users see and click (slower, brittle, needs the UI ready). API testing checks the logic and data layer (faster, more stable, can start before UI exists, tests edge cases the UI blocks). Test pyramid: many unit tests, a good number of API/integration tests, fewer UI tests.

Q4. 🟢 What is REST? An architectural style (Roy Fielding, 2000) using HTTP where resources are identified by URIs and manipulated with standard methods. Constraints: client–server, stateless, cacheable, uniform interface (incl. HATEOAS), layered system, optional code-on-demand.

Q5. 🟢 What is a RESTful API vs a REST API? Used interchangeably; "RESTful" usually means it follows REST constraints properly (nouns for resources, correct methods/status codes, stateless).

Q6. 🟡 REST vs SOAP? REST: style, any format (mostly JSON), lightweight, uses HTTP semantics, cacheable. SOAP: protocol, XML envelope only, strict WSDL contract, built-in standards (WS-Security, WS-ReliableMessaging), errors via SOAP Fault, usually POST only. SOAP

is common in banking/legacy enterprise; REST for web/mobile.

Q7. 🟡 REST vs GraphQL? REST has many endpoints with fixed responses (over/under-fetching). GraphQL has one endpoint where the client specifies exactly the fields; strongly typed schema; errors usually returned with HTTP 200 in an `errors` array; caching is harder. In Postman, use the GraphQL request type or GraphQL body mode.

Q8. 🟡 What is gRPC and when would you use it? A high-performance RPC framework using Protocol Buffers over HTTP/2, supporting unary and streaming calls. Used for internal microservice communication needing low latency and strong contracts. Postman supports it with server reflection or `.proto` import.

Q9. 🟢 Explain the HTTP request structure. Request line (method, path, HTTP version), headers (metadata like `Content-Type`, `Authorization`), an empty line, and optional body. Example: `POST /v1/orders HTTP/1.1`, `Host: api.shop.com`, `Content-Type: application/json`, body `{...}`.

Q10. 🟢 Explain the HTTP response structure. Status line (version, status code, reason), headers (`Content-Type`, `Set-Cookie`, `Cache-Control`...), empty line, body.

Q11. 🟢 List HTTP methods and their uses. GET read, POST create/process, PUT replace, PATCH partial update, DELETE remove, HEAD headers only, OPTIONS capabilities/CORS preflight, TRACE echo (debug), CONNECT tunnel.

Q12. 🟢 Difference between GET and POST? GET retrieves data, parameters in the URL, safe & idempotent, cacheable, bookmarkable, length-limited by URL limits. POST sends data in the body to create/process, not idempotent, not cached by default, suitable for large/sensitive data (though HTTPS is what actually protects it).

Q13. 🟡 Difference between PUT and PATCH? PUT replaces the entire resource (missing fields may be reset) and is idempotent. PATCH applies partial changes and isn't guaranteed idempotent. Example: PUT `/users/5` with only `name` may wipe `email`; PATCH changes only `name`.

Q14. 🟡 Difference between PUT and POST? POST to a collection URL creates a new resource where the server chooses the ID (not idempotent). PUT to a specific URL creates or replaces the resource at that URL (idempotent).

Q15. 🟡 What is idempotency? Which methods are idempotent? Same request repeated N times leaves the server in the same state as once. GET, HEAD, OPTIONS, TRACE, PUT, DELETE are idempotent; POST is not; PATCH is not guaranteed. Note: response codes may differ (DELETE → 204 then 404) — idempotency is about state.

Q16. 🟡 What are safe methods? Methods that don't modify server state: GET, HEAD, OPTIONS, TRACE. All safe methods are idempotent, not vice versa.

Q17. 🟡 How do you make POST idempotent? Use an `Idempotency-Key` header (unique per logical operation, e.g. UUID); the server stores the result and returns the same response on retries. Common in payments (Stripe-style). In Postman: `Idempotency-Key: {{$guid}}` generated once and reused for retries.

Q18. 🟢 What are HTTP status code classes? 1xx informational, 2xx success, 3xx redirection, 4xx client error, 5xx server error.

Q19. 🟢 Explain 200, 201, 204. 200 OK (success with body), 201 Created (resource created, usually with `Location` header), 204 No Content (success, no body — common for DELETE/PUT).

Q20. 🟢 Difference between 401 and 403? 401 Unauthorized = not authenticated (missing/invalid/expired credentials). 403 Forbidden = authenticated but not permitted (insufficient role/scope).

Q21. 🟡 Difference between 400 and 422? 400 = malformed/invalid request syntax (bad JSON, wrong types). 422 = well-formed but semantically invalid (business validation). Many APIs use 400 for both; follow the contract.

Q22. 🟡 What is 409? Conflict with current state — duplicate resource (email already registered) or version conflicts (optimistic locking).

Q23. 🟡 What is 429 and how to handle it in testing? Too Many Requests — rate limit exceeded. Respect `Retry-After`; add delays in Runner/Newman (`--delay-request`); retry with backoff; use separate test accounts; also *test* that rate limiting exists.

Q24. 🟡 Difference between 500, 502, 503, 504? 500 generic server error (bug). 502 gateway got invalid response from upstream. 503 service unavailable (overload/maintenance). 504 gateway timeout waiting for upstream.

Q25. 🟡 What's the difference between 301, 302, 307, 308? 301/308 permanent; 302/307 temporary. 307/308 must preserve method and body; 301/302 historically allow clients to change POST to GET.

Q26. 🟢 What is 304? Not Modified — conditional request (`If-None-Match` with ETag or `If-Modified-Since`) found the cached copy still valid; no body sent.

Q27. 🟢 What are headers? Give examples. Key-value metadata. Request: `Content-Type`, `Accept`, `Authorization`, `User-Agent`, `Cookie`, `Cache-Control`. Response: `Content-Type`, `Set-Cookie`, `Location`, `ETag`, `Retry-After`, `WWW-Authenticate`.

Q28. 🟢 Difference between `Content-Type` and `Accept`? `Content-Type` describes the format of the body being sent; `Accept` tells the server which formats the client wants back.

Q29. 🟢 Path parameter vs query parameter? Path identifies a specific resource (`/users/42`); query filters/sorts/paginates (`/users?role=admin&page=2`). Path params are typically required; query params optional.

Q30. 🟡 What is URL encoding? Percent-encoding special characters (`space → %20`, `& → %26`) so they're transmitted safely in URLs. In Postman, right-click → EncodeURIComponent, or `encodeURIComponent()` in scripts.

Q31. 🟢 What is JSON? Data types? Lightweight text data format; types: string, number, boolean, null, object, array. Keys must be double-quoted strings; no trailing commas or comments.

Q32. 🟡 JSON vs XML? JSON: lighter, maps to JS objects, no attributes, faster to parse, dominant in REST. XML: verbose, supports attributes, namespaces, schemas (XSD), used in SOAP/legacy.

Q33. 🟡 What is a stateless API? Server doesn't store client session state between requests; every request carries all info needed (e.g., token). Enables scaling horizontally.

Q34. 🟡 What is CORS? Does it affect Postman? Browser security mechanism restricting cross-origin requests unless the server allows them via `Access-Control-Allow-*` headers (with preflight `OPTIONS` for non-simple requests). It's enforced by browsers, so the Postman desktop app isn't blocked by CORS — which is why "works in Postman but not in the browser" usually means missing CORS headers.

Q35. 🟡 What are cookies and sessions? Cookie: small data stored by client via `Set-Cookie`, sent back in `Cookie` header. Session: server-side state referenced by a session ID cookie. Attributes: `HttpOnly`, `Secure`, `SameSite`, `Domain`, `Path`, `Expires/Max-Age`.

Q36. 🟡 Authentication vs authorization? Authentication = verifying identity (who). Authorization = verifying permissions (what). Failures: 401 vs 403.

Q37. 🟡 What is an API key? Is it secure? A secret identifier for an application, sent in a header or query. Reasonably secure over HTTPS for app identification but doesn't identify a user, is often long-lived, and leaks easily (logs, URLs). Prefer headers, rotation, scoping.

Q38. 🟡 What is a JWT? JSON Web Token: `header.payload.signature`, Base64URL-encoded, signed (HS256/RS256…). Contains claims (`sub`, `exp`, `iat`, `iss`, `aud`, roles). Encoded, not encrypted — don't put secrets in it. Server verifies signature and claims.

Q39. 🔴 What is OAuth 2.0? Explain grant types. An authorization framework letting clients obtain limited access tokens from an authorization server on behalf of a user or itself. Grants: Authorization Code (+PKCE, recommended for user flows), Client Credentials (machine-to-machine), Refresh Token, Device Code; deprecated: Implicit and Resource Owner Password. Tokens: access (short-lived), refresh (long-lived), ID token (OIDC identity).

Q40. 🔴 OAuth 2.0 vs OpenID Connect? OAuth 2.0 is for authorization (access to APIs). OIDC is an identity layer on top of OAuth 2.0 adding the ID token (JWT about the user), `userinfo` endpoint, and standard scopes (`openid profile email`).

Q41. 🔴 What is PKCE? Proof Key for Code Exchange: client generates a random `code_verifier`, sends its hash (`code_challenge`, S256) in the authorize request, and the verifier in the token request — prevents stolen authorization codes from being exchanged. Postman supports "Authorization Code (With PKCE)".

Q42. 🟡 What is Basic Auth? Is Base64 encryption? `Authorization: Basic base64(username:password)`. Base64 is encoding, not encryption; anyone can decode it — only use over HTTPS.

Q43. 🟡 What is rate limiting and throttling? Limiting requests per client over time to protect services (e.g., 100 req/min). Exceeding returns 429; headers like `X-RateLimit-Remaining` and `Retry-After` guide clients.

Q44. 🟡 What is API versioning? Methods? Managing changes without breaking clients: URI (`/v1/`), query (`?version=1`), header (`Accept: application/vnd.x.v1+json` , custom header). URI is most common.

Q45. 🟡 What is pagination? Types? Splitting large result sets: offset/limit, page/size, cursor-based, keyset, Link headers. Test: page boundaries, last page, empty page, invalid page, max limit enforcement, duplicates/missing items between pages.

Q46. 🟡 What is HATEOAS? Hypermedia As The Engine Of Application State: responses include links to related actions/resources (`"_links": {"self":..., "cancel":...}`), so clients navigate dynamically.

Q47. 🟡 What is an endpoint vs resource vs URI? Resource = the thing (orders). URI = identifier (`/orders/42`). Endpoint = URI + method combination exposed by the API (`GET /orders/42`).

Q48. 🟡 What is a webhook? A server-to-server callback: the provider sends an HTTP POST to your URL when an event happens (payment succeeded). Test with mock receivers (webhook.site, Postman mock/Flows) and verify signatures (HMAC headers).

Q49. 🟢 What is the difference between HTTP and HTTPS? HTTPS is HTTP over TLS: encrypted, integrity-protected, server authenticated by certificate; default port 443 vs 80.

Q50. 🔴 What happens when you hit an API URL? (End-to-end) URL parse → DNS → TCP handshake → TLS handshake (cert validation, key exchange) → HTTP request → load balancer/gateway → service → DB → response → client parses. Postman's response-time breakdown shows DNS, TCP, SSL, TTFB, download.

Q51. 🟡 Types of API testing? Functional, smoke, sanity, regression, integration, E2E, negative, boundary, data-driven, contract/schema, performance/load/stress/spike/soak, security, reliability, usability (DX), documentation testing, monitoring.

Q52. 🟡 What do you validate in an API response? Status code, response time, headers (content-type, caching, security), body structure/schema, data types, values/business rules, error messages/format, pagination/sorting/filtering, side effects (DB/other APIs), no sensitive data exposure.

Q53. 🟡 What are common API bugs you've found? Wrong status codes (200 for errors, 500 for validation), missing validation (negative quantities), BOLA/IDOR, sensitive data exposure, inconsistent field names/types, pagination duplicates, timezone issues, missing CORS, poor error messages, performance degradation with large inputs, broken idempotency (duplicate orders on retry), race conditions.

Q54. 🔴 What is contract testing? Verifying that provider and consumer agree on the API contract (schema, fields, types, status codes). Provider-side: validate responses against OpenAPI/JSON Schema (Postman schema tests, Spec Hub). Consumer-driven: Pact (consumers publish expectations, provider verifies).

Q55. 🟡 What is OpenAPI/Swagger? A standard, language-agnostic spec (YAML/JSON) describing REST APIs: paths, operations, parameters, request bodies, responses, schemas, security. Swagger 2.0 was renamed OpenAPI 3.x (3.0, 3.1). Postman imports it to generate collections and supports design in Spec Hub.

Part 2 — Postman basics

Q56. 🟢 What is Postman? An API platform for designing, building, testing, documenting, mocking and monitoring APIs, with collaboration features. Used to send requests (HTTP, GraphQL, gRPC, WebSocket, etc.), write JavaScript tests, run collections (Runner, Newman, Postman CLI), and share via workspaces.

Q57. 🟢 Why Postman over other tools? Easy UI, supports many protocols and auth types, scripting with JS + Chai, variables/environments, collection runner and CLI for CI, mocks, monitors, docs, collaboration, large community/API Network, AI assistance.

Q58. 🟢 What is a collection? A group of saved requests organised in folders, with shared auth, scripts, variables, documentation and examples. It's the unit for running, sharing, mocking, monitoring and documenting.

Q59. 🟢 What is a workspace? Types? A collaborative container for collections, environments, specs, mocks, monitors, flows. Types: Personal, Private, Team, Partner, Public.

Q60. 🟢 What is an environment in Postman? A named set of variables (e.g., DEV/QA/PROD `baseUrl`, credentials) that you switch to run the same requests against different servers. Only one environment is active at a time.

Q61. 🟢 What are variables in Postman? Why use them? Reusable placeholders `{{name}}` for values like base URLs, tokens, IDs — avoid duplication, enable environment switching, chaining, data-driven testing and secret separation.

Q62. 🟡 What are the variable scopes and their precedence? Global, Collection, Environment, Data, Local. Narrowest wins: Local > Data > Environment > Collection > Global. (Vault secrets referenced separately with `{{vault:name}}`.)

Q63. 🟡 Initial value vs current value? (Shared vs local value) Initial (now "shared") value syncs to the Postman cloud and is visible to teammates/exports; current (now "local") value stays on your machine and is what requests use. Put secrets only in local values (or Vault). Newer Postman uses a single local value with an option to share it.

Q64. 🟢 How do you create and use environment variables? Environments → `+` → add key/value → select the environment → use `{{key}}` in requests, or `pm.environment.get("key")` in scripts.

Q65. 🟢 What are global variables? When to use? Workspace-wide variables visible to every collection/environment. Use sparingly (quick experiments); prefer collection/environment variables to avoid collisions.

Q66. 🟡 Collection variables vs environment variables? Collection variables belong to one collection and travel with its export; same in all environments — good for constants. Environment variables vary per environment and are separate files — good for base URLs and credentials.

Q67. 🟢 What are dynamic variables? Built-in random/generated values like `{{$guid}}`, `{{$timestamp}}`, `{{$isoTimestamp}}`, `{{$randomInt}}`, `{{$randomEmail}}`, `{{$randomFirstName}}` (faker-based), new value per use. Use `pm.variables.replaceIn("{{$randomEmail}}")` to generate once in a script and store it.

Q68. 🟢 How do you send a POST request with JSON in Postman? Choose POST, enter URL, Body → raw → JSON, enter JSON, Send. Postman sets `Content-Type: application/json`.

Q69. 🟢 What body types does Postman support? none, form-data (multipart, files), x-www-form-urlencoded, raw (Text, JSON, XML, HTML, JavaScript), binary, GraphQL.

Q70. 🟡 form-data vs x-www-form-urlencoded? form-data (`multipart/form-data`) supports files and parts with their own content types, using boundaries; urlencoded sends `key=value&key2=value2` text only (classic forms, OAuth token endpoints).

Q71. 🟡 How do you upload a file in Postman? Body → form-data → key with type File → select file (or binary body for raw file APIs). Don't set Content-Type manually (boundary). For Newman, keep files in the repo and use `--working-dir`.

Q72. 🟢 What is the Postman Console? A debugging window (Ctrl/Cmd+Alt/Option+C) showing every request/response with actual headers and bodies (after variable resolution), network info, `console.log` output and script errors.

Q73. 🟢 How to generate code from a Postman request? Click the `</>` Code icon → choose language (cURL, Python requests, JS fetch/axios, Java OkHttp, C#, Go, PHP…) → copy.

Q74. 🟢 How to import a cURL command? Import → Raw text (or paste directly into the URL bar) → Postman converts it into a request.

Q75. 🟢 What can you import into Postman? Postman collections/environments (v1/v2.0/v2.1, v3 YAML in v12), cURL, OpenAPI 2.0/3.0/3.1, RAML, WSDL, GraphQL schema, HAR, Insomnia exports, from repos/gateways via integrations.

Q76. 🟡 Collection v2.0 vs v2.1? Both JSON schemas; v2.1 changes representation of auth (arrays of key-value objects) and some URL details. v2.1 is the recommended export format.

Q77. 🟢 What is History in Postman? A log of sent requests, accessible in the sidebar, to reopen/save past requests.

Q78. 🟡 What are examples in Postman? Saved request/response pairs attached to a request; used by mock servers and documentation; show different scenarios (200, 400, 404).

Q79. 🟢 How do you save a response? Response pane → Save Response → Save as example (or save to file).

Q80. 🟡 What are hidden/auto-generated headers? Headers Postman adds automatically: `Postman-Token`, `Host`, `User-Agent: PostmanRuntime/x`, `Accept`, `Accept-Encoding`, `Connection`, `Content-Type`/`Content-Length` from body. Viewable/disable-able under Headers → hidden.

Q81. 🟡 What is the Postman-Token header? A random UUID added per request (historically to prevent caching/aid debugging). Can be disabled in Settings if a strict server rejects unknown headers.

Q82. 🟡 How do you handle cookies in Postman? Postman stores cookies in a cookie jar automatically and sends them to matching domains. Manage via "Cookies" link (add/edit/delete). Scripts use `pm.cookies` and `pm.cookies.jar()` (domains must be allowlisted). Disable per request in Settings.

Q83. 🟡 What is the Postman Interceptor? A Chrome extension that captures browser requests into Postman and syncs browser cookies to Postman.

Q84. 🟡 What is the Postman proxy? Built-in proxy (default port 5555) to capture requests from devices/apps (mobile) by configuring them to use your machine as HTTP proxy (install Postman CA for HTTPS).

Q85. 🟡 Postman desktop vs web app? Desktop: native app, not subject to browser CORS, can reach localhost/internal networks directly. Web: runs in browser; needs an agent (Desktop Agent for localhost/private, Cloud Agent for public APIs, Browser Agent limited by CORS).

Q86. 🟡 What is the Lightweight API Client? Postman's signed-out mode for sending requests without an account/cloud sync (replaced Scratch Pad); limited features (no collection runs, mocks, monitors, collaboration).

Q87. 🟢 What are keyboard shortcuts you use? Ctrl/Cmd+Enter send, Ctrl/Cmd+S save, Ctrl/Cmd+T new tab, Ctrl/Cmd+W close, Ctrl/Cmd+Alt/Option+C Console, Ctrl/Cmd+K search, Ctrl/Cmd+\ sidebar, Ctrl/Cmd+O import.

Q88. 🟡 How to disable SSL verification and when? Settings → General → SSL certificate verification OFF (globally) or per-request Settings tab. Only for local/dev self-signed certificates; better to add the CA certificate in Settings → Certificates.

Q89. 🟡 How to add a client certificate? Settings → Certificates → Add client certificate: host, port, CRT+KEY or PFX, passphrase.

Q90. 🟡 How to set a request timeout? Settings → General → Request timeout (ms; 0 = infinite). Newman: `--timeout-request`.

Q91. 🟢 How do you share a collection? Move it to a team workspace, Share/invite by email/link with role, fork, export JSON, publish docs/Run in Postman button, Postman API, Native Git.

Q92. 🟡 What is a fork? Pull request? Fork = linked copy of a collection/environment for independent changes; pull request = propose merging fork changes into the parent with diff review, comments, approval and merge.

Q93. 🟡 What is the Trash in Postman? Recently deleted collections can be restored for a plan-dependent retention period.

Q94. 🟡 What changed in recent Postman versions? v11: Postbot AI, Postman Vault, Package Library (`pm.require`), Scripts tab with Pre-request/Post-response (Tests renamed), Spec Hub, Insights; variable values model (initial/current → shared/local). v12 (2026): Native Git with YAML Collection v3 files, local-first workflows, `postman workspace push`, Agent Mode, API Catalog. Plans changed in March 2026 to Free (single user), Solo, Team, Enterprise.

Q95. 🟢 What is Postbot / Agent Mode? Postman's AI assistants: Postbot generates tests, docs, visualizations, fixes scripts; Agent Mode performs multi-step tasks across collections, tests, environments, monitors and specs, with reusable "Skills" invoked via `/`.

➡️ Continue: **18B — Variables, Scripting, Testing, Automation & Advanced Features**.

Chapter 18B — Interview Questions & Answers: Scripting, Testing, Automation, CI/CD & Advanced Features

Levels: 🟢 Entry, 🟡 Intermediate, 🔴 Experienced

Part 3 — Scripts & the `pm` object

Q96. 🟢 What are pre-request scripts? JavaScript that runs before a request is sent — to set variables, generate data/timestamps/signatures, fetch tokens, modify headers/body, or skip the request.

Q97. 🟢 What are tests / post-response scripts? JavaScript that runs after the response arrives — to write assertions (`pm.test`), extract values for chaining, control workflow, visualize data. In older UI it's the "Tests" tab; in newer UI "Scripts → Post-response".

Q98. 🟡 What is the execution order of scripts? Collection pre-request → folder pre-request(s) → request pre-request → request sent → collection post-response → folder post-response(s) → request post-response. Repeats per request and iteration.

Q99. 🟡 Can you write scripts at collection/folder level? Why? Yes. For common logic: auth token refresh, common headers (correlation IDs), generic tests (no 5xx, response time, JSON validity), logging. Reduces duplication.

Q100. 🟢 What language are Postman scripts written in? JavaScript (modern ES6+), executed in the Postman Sandbox (Node-based, restricted: no filesystem/DOM).

Q101. 🟡 What is the Postman Sandbox? The JS runtime where scripts execute, providing the `pm` API, built-in libraries (lodash, moment, crypto-js, ajv, cheerio, chai, uuid, xml2js, csv-parse, tv4 deprecated), `require` , `pm.require` for Package Library, timers, and console — but no file system, network sockets (except `pm.sendRequest`), or DOM.

Q102. 🟢 What is the `pm` object? Postman's scripting API: `pm.request` , `pm.response` , `pm.info` , `pm.variables` , `pm.environment` , `pm.collectionVariables` , `pm.globals` , `pm.iterationData` , `pm.cookies` , `pm.vault` , `pm.sendRequest` , `pm.execution` , `pm.test` , `pm.expect` , `pm.visualizer` , `pm.require` .

Q103. 🟢 How do you set and get an environment variable in a script? `pm.environment.set("token", value);` and `pm.environment.get("token");` . Collection: `pm.collectionVariables.set/get` . Global: `pm.globals.set/get` .

Q104. 🟡 Difference between `pm.variables.get` and `pm.environment.get` ? `pm.variables.get` resolves across all scopes using precedence (local > data > environment > collection > global); `pm.environment.get` reads only the active environment.

Q105. 🟡 What does `pm.variables.set` do? Sets a **local** variable — temporary, highest precedence, lasts for the request (or the whole collection run), not saved to any environment/collection.

Q106. 🟡 How do you remove variables? `pm.environment.unset("k")`, `pm.collectionVariables.unset("k")`, `pm.globals.unset("k")`; `clear()` removes all in that scope (dangerous in shared envs).

Q107. 🟢 How do you read the response body? `pm.response.json()` for JSON, `pm.response.text()` for raw text; XML via `xml2Json(pm.response.text())` or `xml2js`; HTML via `cheerio.load()`.

Q108. 🟢 How do you get the status code, response time, headers? `pm.response.code`, `pm.response.status`, `pm.response.responseTime`, `pm.response.responseSize`, `pm.response.headers.get("Content-Type")`.

Q109. 🟡 How do you modify a request in a pre-request script? `pm.request.headers.upsert({key, value})`, `pm.request.url.query.add({key, value})`, `pm.request.body.raw = JSON.stringify(obj)` or `pm.request.body.update({...})`, `pm.request.method = "PUT"`, `pm.request.url = "..."`.

Q110. 🔴 How do you parse a request body that contains `{{variables}}`? `JSON.parse(pm.variables.replaceIn(pm.request.body.raw))` — because `pm.request.body.raw` holds unresolved placeholders, which may be invalid JSON (e.g. unquoted `{{age}}`).

Q111. 🟡 What is `pm.sendRequest`? Use cases? Sends an HTTP request from within a script (callback style everywhere; promise/await only in recent Postman app versions, not Newman 6). Uses: fetch auth tokens, create/cleanup test data, verify side effects, polling, call other services. It doesn't inherit collection auth/headers and doesn't auto-resolve `{{vars}}` in strings (use `replaceIn`).

Q112. 🟡 Write a `pm.sendRequest` POST example.

```
pm.sendRequest({
  url: pm.variables.replaceIn("{{baseUrl}}/auth/token"),
  method: "POST",
  header: { "Content-Type": "application/json" },
  body: { mode: "raw", raw: JSON.stringify({ user: pm.environment.get("user"), pass:
pm.environment.get("pass") }) }
}, (err, res) => {
  if (!err && res.code === 200) pm.environment.set("token", res.json().token);
});
```

Q113. 🟡 What is `pm.info`? Runtime info: `eventName` ("prerequest"/"test"), `iteration`, `iterationCount`, `requestName`, `requestId`.

Q114. 🟡 What is `pm.iterationData`? Read-only access to the current data file row during runs: `pm.iterationData.get("email")`, `.has()`, `.toObject()`.

Q115. 🔴 **What is `pm.execution`?** Run-control API: `setNextRequest(nameOrId|null)` to change/stop run flow, `skipRequest()` (pre-request only) to skip the current request, `location` to know where the script is executing.

Q116. 🟡 **`postman.setNextRequest` vs `pm.execution.setNextRequest`?** Same purpose; `postman.setNextRequest` is the legacy API; `pm.execution.setNextRequest` is the modern one. Both work only in collection runs; last call wins; `null` stops the run.

Q117. 🟡 **How do you skip a request conditionally?** In pre-request: `if (condition) pm.execution.skipRequest();` — its post-response scripts won't run either.

Q118. 🟡 **What libraries can you `require` in Postman?** lodash, moment, crypto-js, ajv, tv4 (deprecated), chai, cheerio, uuid, xml2js, csv-parse/lib/sync, postman-collection, atob/btoa, and Node built-ins like url, util, path, buffer, querystring, events, stream, timers, assert, punycode, string_decoder.

Q119. 🔴 **How do you reuse code across collections?** Modern: **Package Library** — create a team package with `module.exports` and load via `pm.require("@team/package")` (recent versions can also import from external registries like npm). Older approaches: collection-level scripts (one collection only) or storing functions as strings in variables and `eval` (discouraged).

Q120. 🟡 **What is `pm.visualizer`?** Renders custom HTML (Handlebars template + data) in the response's Visualize tab: `pm.visualizer.set(template, data)`; inside the template, `pm.getData(cb)` gets the data for JS charts (e.g., Chart.js).

Q121. 🟡 **How do you log in scripts?** `console.log/info/warn/error` → Postman Console (stdout in Newman).

Q122. 🔴 **How do you handle asynchronous code in Postman scripts?** Postman waits for pending callbacks, timers and promises (`pm.sendRequest` callbacks, `setTimeout`) before finishing the script. Tests inside callbacks register correctly. Top-level `await` works in recent Postman app versions but not in Newman 6 (verified), so for portable code wrap `pm.sendRequest` in a Promise inside an async IIFE and add `.catch()`.

Q123. 🔴 **How do you add a delay inside a script?** `setTimeout(() => {}, 2000);` — the sandbox waits for the timer. For runs, also Runner Delay / `--delay-request`.

Q124. 🟡 **What is `pm.cookies` vs `pm.cookies.jar()`?** `pm.cookies` is the cookie list for the current request's domain (`get`, `has`, `toObject`). `pm.cookies.jar()` gives programmatic access to get/set/unset/clear cookies for any allowlisted domain (async callbacks).

Q125. 🟡 **What is legacy script syntax and what replaced it?** `tests["name"] = bool`, `responseBody`, `responseCode.code`, `responseTime`, `postman.setEnvironmentVariable`, `postman.getResponseHeader`, `tv4.validate` → replaced by `pm.test`, `pm.response.text()/json()`, `pm.response.code`, `pm.response.responseTime`, `pm.environment.set`, `pm.response.headers.get`, `pm.response.to.have.jsonSchema`/ajv.

Part 4 — Testing & assertions

Q126. 🟢 How do you write a test in Postman?

```
pm.test("Status code is 200", () => pm.response.to.have.status(200));
```

Q127. 🟢 What assertion library does Postman use? Chai.js BDD (`pm.expect(x).to.eql(y)`), plus Postman response assertions (`pm.response.to.have.status` , `.to.have.header` , `.to.have.jsonBody` , `.to.have.jsonSchema` , `.to.be.ok/success/error/notFound/...`).

Q128. 🟡 Difference between `pm.expect(x).to.equal(y)` and `.to.eql(y)` ? `equal` is strict `===` (references for objects); `eql` is deep equality — use it for objects/arrays.

Q129. 🔴 Difference between `pm.response.to.be.ok` and `pm.response.to.be.success` ? `.ok` asserts status exactly 200; `.success` asserts any 2xx. (Also: Chai's `pm.expect(x).to.be.ok` means "truthy" — a different thing.)

Q130. 🟢 How to check a header exists/has value? `pm.response.to.have.header("Content-Type")` ; value: `pm.expect(pm.response.headers.get("Content-Type")).to.include("application/json")` .

Q131. 🟢 How to check response time?
`pm.expect(pm.response.responseTime).to.be.below(500);`

Q132. 🟢 How to check a value in JSON? `const j = pm.response.json();` `pm.expect(j.user.name).to.eql("Asha");` or `pm.response.to.have.jsonBody("user.name", "Asha")` .

Q133. 🟡 How to check a JSON key exists / doesn't exist?
`pm.expect(j).to.have.property("id")` , nested: `to.have.nested.property("a.b")` ; absence: `to.not.have.property("password")` .

Q134. 🟡 How to assert array length, contains item, all items match?
`pm.expect(arr).to.have.lengthOf(3)` , `pm.expect(arr).to.deep.include({id: 1})` , `arr.forEach(x =>` `pm.expect(x.active).to.be.true)` or `pm.expect(arr.every(x => x.active)).to.be.true` .

Q135. 🟡 How to validate data types? `pm.expect(j.id).to.be.a("number")` , `"string"` , `"boolean"` , `"array"` , `"object"` , `to.be.null` .

Q136. 🔴 How do you perform JSON schema validation? Define a JSON Schema and use `pm.response.to.have.jsonSchema(schema)` or Ajv:

```
const Ajv = require("ajv"); const ajv = new Ajv({ allErrors: true });
const validate = ajv.compile(schema);
pm.test("Schema valid", () => pm.expect(validate(pm.response.json()),
JSON.stringify(validate.errors)).to.be.true);
```

tv4 is deprecated. Store schemas centrally (collection variable/Package Library), derive from OpenAPI.

Q137. 🟡 How to test that a list is sorted? Map field values, compare with a sorted copy using a proper comparator: `pm.expect(vals).to.eql([...vals].sort((a,b)=>a-b))`.

Q138. 🟡 How to write a test that checks one of several status codes? `pm.expect(pm.response.code).to.be.oneOf([200, 201]);`

Q139. 🟡 How to skip a test? `pm.test.skip("name", fn)`; conditional: `(cond ? pm.test : pm.test.skip)("name", fn)`.

Q140. 🔴 Does one failing test stop other tests or the run? No — each `pm.test` is independent; the run continues. An uncaught script error stops the rest of *that script*. Newman `--bail` or `setNextRequest(null)` stops runs.

Q141. 🔴 How do you test negative scenarios? Send invalid inputs (missing/wrong-type/boundary values, invalid auth, wrong methods, non-existent IDs, duplicates) and assert proper 4xx codes, error schema, helpful messages, no 5xx, no stack traces or data leaks. Data-drive cases with CSV/JSON including `expectedStatus`.

Q142. 🔴 How do you test that data was persisted after a POST? Save the ID from POST, then GET the resource (next request or `pm.sendRequest`) and deep-compare fields; optionally verify via list/search endpoints or DB checks (through an internal API).

Q143. 🔴 How do you test idempotency? Send the same PUT/DELETE (or POST with Idempotency-Key) twice; assert the state is unchanged after the second call (GET), no duplicates created, and appropriate status codes.

Q144. 🔴 How do you test API performance in Postman? Response time assertions per request; collection runs over iterations; Runner Performance tab with virtual users and load profiles (fixed, ramp up, spike, peak) reporting avg/p90/p95/p99, throughput, error rate. For heavy load use k6/JMeter/Gatling.

Q145. 🔴 How do you avoid flaky tests? Unique data per run, independent tests, explicit setup/teardown, polling with timeouts instead of fixed sleeps, retry only transient failures, avoid asserting volatile values/order, stable environments, version-pinned runners.

Part 5 — Chaining, Runner, data-driven

Q146. 🟢 What is request chaining? Passing data from one response into later requests via variables (e.g., save `token` or `orderId` in a post-response script, use `{{orderId}}` next).

Q147. 🟢 How do you extract a token from login and use it everywhere? Login post-response: `pm.environment.set("accessToken", pm.response.json().token)`. Collection Authorization: Bearer `{{accessToken}}`; requests inherit.

Q148. 🔴 How do you auto-refresh an expired token? Collection pre-request script checks `accessTokenExpiresAt`; if missing/expired, call token endpoint via `pm.sendRequest`, store the new token and expiry; Postman waits before sending the main request. Or use OAuth 2.0 auth with Auto-refresh in the UI (not for Newman).

Q149. 🟢 What is the Collection Runner? Tool to run all/selected requests of a collection or folder in sequence with iterations, delays, data files, chosen environment; shows test results; also offers scheduled runs and performance tests.

Q150. 🟡 What is data-driven testing in Postman? Running the same requests with multiple data sets from CSV/JSON files; each row = one iteration; access via `{{column}}` or `pm.iterationData.get()`.

Q151. 🟡 CSV vs JSON data files? CSV: simple, flat, header row required, no reliable types (Newman converts number-like values to numbers and drops leading zeros; `true` / `false` stay strings). JSON: array of objects, preserves types and nesting.

Q152. 🔴 What if iterations exceed rows in the data file? The extra iterations reuse the last row's values. Fewer iterations → only first N rows.

Q153. 🔴 A CSV column has the same name as an environment variable. Which wins? The data variable (data > environment), unless a local variable with that name is set.

Q154. 🟡 How do you change request execution order? Reorder in the collection/Runner, or dynamically with `pm.execution.setNextRequest("name or id")` in collection runs.

Q155. 🔴 How do you loop a request until a condition is met? In post-response: if condition not met and attempts < max → `pm.execution.setNextRequest(pm.info.requestId)`; increment counter; optionally `setTimeout` delay; reset counter when done.

Q156. 🔴 How do you implement pagination in a run? Store page/cursor in a variable, concatenate results, and `setNextRequest` the same request until `hasMore` is false or `next_cursor` null.

Q157. 🟡 Why isn't `setNextRequest` working? It only works in collection runs (not Send), name typo/duplicates (use IDs), target not part of the run, overwritten later (last call wins), request deselected in Runner.

Q158. 🟡 How do you stop a collection run conditionally? `pm.execution.setNextRequest(null)` (stops after current request); Newman `--bail`.

Part 6 — Newman, Postman CLI & CI/CD

Q159. 🟢 What is Newman? Postman's open-source command-line collection runner (Node.js/npm). Runs exported collections/environments (files or URLs) with reporters; used in CI/CD.

Q160. 🟢 How do you install and run Newman? `npm install -g newman`; `newman run collection.json -e env.json`.

Q161. 🟡 Common Newman options? `-e` env, `-g` globals, `-d` data, `-n` iterations, `--folder`, `--env-var`, `--global-var`, `--delay-request`, `--timeout-request`, `-k` insecure, `--bail`, `-x` suppress exit code, `-r` reporters, `--reporter-junit-export`, `--export-environment`, `--working-dir`, `--ssl-

`client-cert`.

Q162. 🟡 What reporters does Newman support? Built-in: cli, json, junit, progress, emojitrain. External: htmlextra (popular), html (older), allure, teamcity, etc. (`npm install -g newman-reporter-htmlextra` then `-r htmlextra`).

Q163. 🟡 How do you generate an HTML report? `newman run c.json -r htmlextra --reporter-htmlextra-export report.html`.

Q164. 🟡 What exit codes does Newman return? 0 success, 1 when tests fail or errors occur — CI uses this to fail the job. `-x` forces 0.

Q165. 🔴 How do you integrate Postman tests in Jenkins/GitHub Actions/Azure/GitLab? Check out repo with exported collections (or fetch via Postman API / use Postman CLI with IDs), install Newman (or use `postman/newman` Docker image / Postman CLI), inject secrets from CI store via `--env-var`, run with JUnit reporter, publish JUnit results and HTML artifacts, fail build on non-zero exit. (See Chapter 11 YAML examples.)

Q166. 🔴 What is the Postman CLI and how does it differ from Newman? Official Postman binary: logs in with API key, runs collections by cloud ID or file, sends results to Postman (run history, API Catalog reports), supports governance linting (`postman api lint`, `postman spec lint`), v12 Native Git (`workspace push`, YAML collections). Newman: open-source npm, file/URL based, Node API, custom reporters, no results to Postman.

Q167. 🔴 How do you pass secrets to Newman securely? Store in CI secret store, pass via `--env-var "key=$SECRET"`; never commit them in environment files; mask in logs; use htmlextra `skipSensitiveData`.

Q168. 🟡 How do you run only one folder in Newman? `--folder "Folder Name"` (repeatable).

Q169. 🔴 How do you run Newman programmatically? `const newman = require('newman'); newman.run({ collection, environment, reporters: ['cli'] }, (err, summary) => {...}).on('request', ...)`. Useful for parallel runs and custom notifications.

Q170. 🔴 How do you run Newman in Docker? `docker run -v "$PWD":/etc/newman -t postman/newman:alpine run collection.json -e env.json`.

Q171. 🔴 Why would a collection pass in the app but fail in Newman? Missing environment/secrets (local values not exported, Vault unsupported), interactive OAuth, file paths, older Newman lacking new sandbox APIs, package library access, CA/proxy differences, fresh cookie jar, timing/network.

Q172. 🟡 How do you keep the collection in CI up to date? Fetch via Postman API (`/collections/{uid}`) at runtime, use Postman CLI with collection IDs, Git integration, or v12 Native Git (collections live in repo).

Part 7 — Mocks, monitors, docs, design, API, AI

Q173. 🟢 **What is a mock server?** A fake API that returns saved examples; lets frontend/QA work before backend is ready, simulate errors and edge cases.

Q174. 🟡 **How does a Postman mock server choose the response?** Matches incoming request against saved examples by method, path (variables as wildcards), query params, optionally headers/body (when `x-mock-match-request-headers` / `x-mock-match-request-body` are sent), picks the best score; can be forced with `x-mock-response-name`, `x-mock-response-id`, or `x-mock-response-code`. Private mocks need `x-api-key`.

Q175. 🟡 **Limitations of Postman mocks?** Stateless (no real create/update), usage limits per plan, matching surprises, no full request validation by default. Use WireMock/Prism/json-server for stateful/local needs.

Q176. 🟢 **What is a monitor?** Scheduled cloud runs of a collection (frequency, regions, environment) with alerts (email, Slack, Teams, PagerDuty…) for uptime/SLA/functional checks.

Q177. 🟡 **Monitor vs scheduled collection run vs Newman cron?** Monitors: cloud, multi-region, alerting integrations, response-time graphs. Scheduled runs: cloud collection runs on a schedule with notifications. Newman cron/CI: runs inside your network (reaches internal APIs), full control.

Q178. 🔴 **How do you monitor an internal API not reachable from the internet?** Use static IPs + allowlisting (Enterprise) or run Newman/Postman CLI on a schedule inside the network (CI, Kubernetes CronJob) and push results/alerts.

Q179. 🟡 **How do you generate API documentation in Postman?** Fill descriptions (Markdown) for collection/folders/requests/params, save examples; View documentation; Publish publicly or keep private; include environment (without secrets).

Q180. 🟡 **What is Spec Hub / API Builder?** Postman's API design area: create/edit OpenAPI (2.0/3.0/3.1) and AsyncAPI specs, lint with governance rules, generate/sync collections, mocks, docs, Git integration, versions.

Q181. 🔴 **Design-first vs code-first?** Design-first: spec first, review, mock, then build (consistent APIs, parallel work). Code-first: build then generate spec (fast, risk of drift). Postman supports both via Spec Hub and imports.

Q182. 🟡 **What are Postman Flows?** Low-code visual workflow builder: connect blocks (requests, conditions, loops, evaluate/FQL, AI requests) to orchestrate APIs; deployable to cloud with HTTP/schedule/MCP triggers.

Q183. 🟡 **What is the Postman API?** REST API at `https://api.getpostman.com` (auth `X-API-Key`) to manage collections, environments, workspaces, mocks, monitors, specs, webhooks programmatically — for CI sync, backups, automation.

Q184. 🔴 How can you trigger a collection run from an external event? Create a collection webhook (Postman API `/webhooks`); POSTing to its URL runs the collection in the cloud, with payload available as `pm.globals.get("previousRequest")`. Or trigger CI pipelines that run Newman/Postman CLI.

Q185. 🟡 What is Postman Vault? Encrypted local secret storage, not synced to cloud; referenced as `{{vault:name}}` and in scripts via async `pm.vault.get/set/unset` (once enabled); integrations with external secret managers (Azure Key Vault, HashiCorp, AWS Secrets Manager) on Enterprise; not supported in Newman/CLI/monitors.

Q186. 🟡 What is the secret variable type? Masks the value in the UI to prevent shoulder-surfing; it's not encryption and values are still accessible via scripts/exports.

Q187. 🔴 What is Postman Insights? An agent deployed in your infrastructure that passively observes live API traffic to discover endpoints, errors and latency, helping keep collections/specs aligned with actual usage.

Q188. 🟡 What are Postman's AI features? Postbot (test generation, docs, visualization, debugging) and Agent Mode (multi-step agentic tasks across workspace with Skills). Review AI-generated tests against the spec.

Q189. 🔴 What's new in Postman v12? Native Git: collections as YAML (Collection v3) files in your repo, local-first & offline development, CLI-based sync (`postman workspace push`), running collections by path; Agent Mode expansion; API Catalog; Spec Hub improvements.

Q190. 🔴 How does Postman support GraphQL/gRPC/WebSocket/MQTT/SOAP? Dedicated request types: GraphQL (introspection, query builder, variables), gRPC (reflection/proto, metadata, streaming), WebSocket and Socket.IO (connect, messages, events), MQTT (broker, topics, QoS); SOAP via HTTP POST with XML envelope and SOAPAction/WSDL import.

➡️ Continue: **18C — Scenario-Based, Coding, Architecture & Behavioural Questions**.

Chapter 18C — Scenario-Based, Live-Coding, Architecture & Behavioural Interview Questions

> Experienced-level interviews focus on **"what would you do if…"**, **"write a script for…"**, and **"design a framework for…"**. Use the **STAR** pattern for experience questions (Situation, Task, Action, Result) and **think aloud** for scenarios.

Part 8 — Scenario-based questions

S1. Your request returns 401 although you set the token. How do you debug?

1. Console: is the `Authorization` header actually sent and correctly formatted (`Bearer <token>`, no extra spaces/newlines)? 2) Variable resolved and from the right scope/environment? 3) Request auth "Inherit" but parent "No Auth", or a manual header conflicting with Auth tab? 4) Token expired → decode JWT (`exp`), check `aud` / `iss` /scopes for this API/environment. 5) Clock skew for signed schemes. 6) Try the token via cURL. 7) Fix with an auto-refresh pre-request script.

S2. API works in Postman but the frontend gets an error. Why? Most likely CORS — browsers enforce it, Postman desktop doesn't. Check browser console, preflight `OPTIONS` response headers. Also compare headers/body via DevTools "Copy as cURL", cookies with `SameSite`, mixed content (https page → http API).

S3. Tests pass locally but fail in Jenkins. What do you check? Environment file & `--env-var` secrets passed; Vault/local values not available in CI; interactive OAuth; file upload paths/ `--working-dir`; Newman version vs features; corporate CA/proxy on agents (`--ssl-extra-ca-certs`, `HTTP_PROXY`); network access from CI to the env; timing/timeouts; parallel jobs sharing data; run with `--verbose`.

S4. How would you test a login API completely? Positive (valid creds → 200, token structure/expiry, refresh token), negative (wrong password, unknown user — same generic error, empty fields, wrong types, SQLi strings, very long input), security (lockout/429 after N attempts, HTTPS only, no password in response/logs, token invalid after logout, token tampering), headers (no-cache), performance (SLA), data-driven via CSV.

S5. How would you test a "Create Order" API? Contract (schema, status 201, Location header), business rules (total = Σ price×qty, tax, discounts, stock decrement, status PENDING), validation (empty items, qty 0/negative/max+1, invalid SKU, wrong currency), auth (no token 401, other user's cart 403/404), idempotency (Idempotency-Key retry → no duplicate), concurrency (two orders for the last item), side effects (inventory, payment intent, events), performance, cleanup.

S6. How do you test an API that processes asynchronously (returns 202)? Assert 202 + job/status URL; poll the status endpoint with max attempts and delay until COMPLETED/FAILED; assert final result; test timeout behaviour and failure path; avoid fixed waits.

S7. You have to run the same request for 500 users. How? Data-driven run with a CSV/JSON file of users (Runner or Newman `-d`), `{{username}}` placeholders, `pm.iterationData` assertions with expected values per row; for concurrency, the Performance tab or k6.

S8. Backend isn't ready — how do you start testing? Get/define the OpenAPI contract; create examples and a mock server; write tests against mock (structure, flows); switch `baseUrl` to real API once deployed; contract tests ensure parity.

S9. A test is flaky. How do you handle it? Reproduce and log; identify cause (timing/async, shared data, environment instability, order dependency, volatile assertions); fix root cause (polling, unique data, isolation, robust assertions); quarantine only temporarily with a ticket; never just "retry until green" for real failures.

S10. You need to run collections against DEV, QA, STAGE with different credentials. How do you design it? Same collection, one environment per stage with identical variable names; secrets via Vault locally and CI secret store with `--env-var`; pipeline stage per environment with environment file; prod read-only tests only.

S11. How do you handle dynamic data like IDs created in one request and needed later? Extract in post-response (`pm.collectionVariables.set("orderId", json.id)`), use `{{orderId}}`; clean up in teardown; generate unique input data with dynamic variables.

S12. How do you verify that a DELETE actually deleted? Assert DELETE status (200/204), then GET the same ID → 404 (or 410); list endpoint doesn't contain it; repeat DELETE → 404 (idempotent state); related resources handled (cascade/soft delete as per spec).

S13. The response JSON is huge (5 MB). How do you validate it efficiently? Validate schema once (Ajv), sample/aggregate checks (counts, sums, unique IDs via Set), avoid logging the whole body, turn off response saving in runs, use Newman for big runs, raise max response size if needed.

S14. How do you test file upload? form-data with File; valid file → 201; size limit → 413; wrong type → 415/400; empty file; malicious filename; content verification (download and compare checksum via CryptoJS); keep fixtures in repo for CI.

S15. Your API rate-limits at 100 req/min and your regression has 400 requests. What do you do? Add delay (`--delay-request 650`), split runs across test accounts/API keys, retry on 429 with `Retry-After`, reduce unnecessary calls (token caching), request higher limits for test environment.

S16. How do you test pagination? First/middle/last pages; page size default & max; beyond last page (empty); invalid values (0, -1, "abc"); total counts consistent; no duplicates/missing items across pages (collect IDs across pages in a loop); sorting stable across pages; cursor tampering.

S17. How do you handle SSL errors in a company network? Add the corporate root CA (from TLS inspection proxy) to Postman Settings → Certificates (and `--ssl-extra-ca-certs` / `NODE_EXTRA_CA_CERTS` in CI) instead of disabling verification.

S18. How do you test an endpoint that requires a signature (HMAC)? Pre-request script computing signature with CryptoJS over method/path/timestamp/body (exactly as spec defines), set headers; tests for tampered body/old timestamp/replayed nonce rejected.

S19. A teammate overwrote your collection changes. How do you prevent it? Restore from version history; adopt fork + pull request workflow, protect the main collection (limit editors), or move to Native Git with PR reviews.

S20. How would you test BOLA/IDOR? Create resources as user B; access/modify/delete them with user A's token for every ID-based endpoint (path, query, body); expect 403/404; data-drive across endpoints; include tenant boundaries.

S21. API returns 200 with an error message in body. What's your view & test? Report as a design defect (should use proper 4xx/5xx) unless contract says so (e.g., GraphQL). Test both status and body (`errors`, `success: false`).

S22. You get `JSONError: Unexpected token '<'`. Why? The response is HTML (error page, login redirect, proxy page), not JSON. Check status/Content-Type, Preview tab; fix auth/URL; guard parsing in tests.

S23. `getaddrinfo ENOTFOUND {{baseUrl}}` — what happened? Variable not resolved: no environment selected, wrong name/case, disabled, or not defined in any scope.

S24. How do you share a collection with an external partner securely? Partner workspace (Enterprise) or a sanitised fork/public collection with a partner-specific environment template without secrets; separate credentials; least privilege; revoke on contract end.

S25. Management wants daily API health reports. Approach? Monitors for prod read-only smoke checks with alerts and response-time graphs; CI nightly Newman/CLI regression on staging with HTML/JUnit reports; dashboards via Postman run history/API Catalog; Slack summaries.

S26. You must migrate 300 legacy scripts using `tests[]` and `postman.*`. Plan by folder; map old → new APIs; use Find & Replace for simple patterns; refactor into shared Package Library helpers; run before/after comparison; PR review; document.

S27. How do you test webhooks you receive (provider calls you)? Simulate provider POSTs from Postman to your endpoint with correctly signed payloads (HMAC header), test invalid signature/replay/old timestamp rejection, retries/idempotency on duplicate events, event ordering. To inspect what a provider sends, point it at a request bin/mock or Flows.

S28. Login uses SSO (browser redirect, MFA). How do you automate? For manual testing: OAuth 2.0 Authorization Code (PKCE) with "Authorize using browser". For CI: request a non-interactive grant for test service accounts (client credentials) or test-only token endpoint/long-lived test tokens issued securely; never automate MFA bypass in prod.

S29. How do you ensure API tests don't pollute shared QA data? Unique prefixed data per run (`qa-auto-<runId>`), teardown folder, cleanup jobs for leftovers, dedicated test tenants/accounts, read-only tests where possible.

S30. How would you test API backward compatibility for v2 release? Run v1 regression suite against new build; contract diff between specs (no removed fields/required additions/type changes); old clients' examples still accepted; deprecation headers (`Deprecation`, `Sunset`); v2 suite separately.

Part 9 — Live-coding questions (write the script)

C1. Assert status 200, JSON content type, response time < 1s.

```
pm.test("200 OK", () => pm.response.to.have.status(200));
pm.test("JSON", () => pm.expect(pm.response.headers.get("Content-
Type")).to.include("application/json"));
pm.test("< 1s", () => pm.expect(pm.response.responseTime).to.be.below(1000));
```

C2. Extract `token` from login response and set as environment variable.

```
const { token } = pm.response.json();
pm.test("Token present", () => pm.expect(token).to.be.a("string").and.not.empty);
pm.environment.set("token", token);
```

C3. Verify every user in an array has a valid email and unique id.

```
const users = pm.response.json();
pm.test("Emails valid", () => users.forEach(u => pm.expect(u.email).to.match(/^[^\s@]+@[^\s@]+\.
[^\s@]+$/)));
pm.test("IDs unique", () => pm.expect(new Set(users.map(u => u.id)).size).to.eql(users.length));
```

C4. Find the user named "Clementine Bauch" (JSONPlaceholder /users) and save her id.

```
const u = pm.response.json().find(x => x.name === "Clementine Bauch");
pm.test("Found", () => pm.expect(u).to.exist);
pm.collectionVariables.set("userId", u.id);   // 3
```

C5. Count posts per userId (JSONPlaceholder /posts) and assert each user has 10.

```
const counts = pm.response.json().reduce((m, p) => (m[p.userId] = (m[p.userId] || 0) + 1, m),
{});
pm.test("Each user has 10 posts", () => Object.values(counts).forEach(c =>
pm.expect(c).to.eql(10)));
```

C6. Generate a random email and future date in pre-request.

```
pm.variables.set("email", `user_${Date.now()}@example.com`);
const d = new Date(); d.setDate(d.getDate() + 30);
pm.variables.set("futureDate", d.toISOString().split("T")[0]);
```

C7. Add a header with HMAC-SHA256 of the body.

```
const body = pm.variables.replaceIn(pm.request.body.raw || "");
const sig = CryptoJS.HmacSHA256(body,
pm.environment.get("secret")).toString(CryptoJS.enc.Base64);
pm.request.headers.upsert({ key: "X-Signature", value: sig });
```

C8. Basic auth header manually.

```
const creds = btoa(`${pm.environment.get("user")}:${pm.environment.get("pass")}`);
pm.request.headers.upsert({ key: "Authorization", value: `Basic ${creds}` });
```

C9. Fetch token only if expired (pre-request).

```
if (Date.now() >= Number(pm.environment.get("tokenExp") || 0)) {
  pm.sendRequest({
    url: pm.environment.get("tokenUrl"), method: "POST",
    header: { "Content-Type": "application/x-www-form-urlencoded" },
    body: { mode: "urlencoded", urlencoded: [
      { key: "grant_type", value: "client_credentials" },
      { key: "client_id", value: pm.environment.get("clientId") },
      { key: "client_secret", value: pm.environment.get("clientSecret") } ] }
  }, (err, res) => {
    const b = res.json();
    pm.environment.set("token", b.access_token);
    pm.environment.set("tokenExp", Date.now() + (b.expires_in - 60) * 1000);
  });
}
```

C10. Validate response against a JSON schema.

```
const schema = { type: "object", required: ["id","title","completed"],
  properties: { id: {type:"integer"}, title: {type:"string"}, completed: {type:"boolean"} } };
pm.test("Schema", () => pm.response.to.have.jsonSchema(schema));
```

C11. Loop over IDs [1..5] calling the same request.

```
// pre-request
let ids = pm.collectionVariables.get("ids");
ids = ids ? JSON.parse(ids) : [1,2,3,4,5];
pm.collectionVariables.set("id", ids.shift());
pm.collectionVariables.set("ids", JSON.stringify(ids));
// post-response
const left = JSON.parse(pm.collectionVariables.get("ids"));
if (left.length) pm.execution.setNextRequest(pm.info.requestId);
else pm.collectionVariables.unset("ids");
```

C12. Check prices sorted descending.

```
const p = pm.response.json().map(i => i.price);
pm.test("Desc", () => pm.expect(p).to.eql([...p].sort((a,b) => b - a)));
```

C13. Assert response doesn't contain sensitive fields anywhere (deep).

```
const forbidden = ["password", "ssn", "secret"];
function scan(o, path = "") {
  if (o && typeof o === "object") for (const [k, v] of Object.entries(o)) {
    pm.expect(forbidden, `found ${path}${k}`).to.not.include(k.toLowerCase());
    scan(v, `${path}${k}.`);
  }
}
pm.test("No sensitive keys", () => scan(pm.response.json()));
```

C14. Parse XML and assert a value.

```
const j = xml2Json(pm.response.text());
pm.test("Status OK", () => pm.expect(j.response.status).to.eql("OK"));
```

C15. Compare two responses (current vs previous request).

```
// Request A post-response
pm.collectionVariables.set("respA", pm.response.text());
// Request B post-response
pm.test("Same data in A and B", () =>
pm.expect(pm.response.json()).to.eql(JSON.parse(pm.collectionVariables.get("respA"))));
```

C16. Visualize users as a table.

```
pm.visualizer.set(`<table>{{#each r}}<tr><td>{{id}}</td><td>{{name}}</td></tr>{{/each}}</table>`,
{ r: pm.response.json() });
```

C17. Data-driven expected status.

```
const exp = Number(pm.iterationData.get("expectedStatus"));
pm.test(`[${pm.iterationData.get("case")}] → ${exp}`, () => pm.response.to.have.status(exp));
```

C18. Retry on 503 up to 3 times.

```
const n = Number(pm.variables.get("_n") || 0);
if (pm.response.code === 503 && n < 3) { pm.variables.set("_n", n + 1); setTimeout(() => {}, 1000
* 2 ** n); pm.execution.setNextRequest(pm.info.requestId); }
else { pm.variables.set("_n", 0); pm.test("Not 5xx", () =>
pm.expect(pm.response.code).to.be.below(500)); }
```

C19. Extract value from a Set-Cookie / header.

```
pm.environment.set("session", pm.cookies.get("SESSIONID"));
pm.environment.set("location", pm.response.headers.get("Location"));
```

C20. Assert date field is in ISO 8601 UTC and in the past.

```
const t = pm.response.json().createdAt;
pm.test("ISO UTC", () => pm.expect(t).to.match(/^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}(\.\d+)?
Z$/));
pm.test("In past", () => pm.expect(Date.parse(t)).to.be.below(Date.now()));
```

Part 10 — Architecture / senior questions

A1. How would you design an API automation framework with Postman for a 40-microservice company?

- Per service: reference collection (from OpenAPI, synced via Spec Hub) + test collections (smoke, regression, negative, security), owned by that team, in a domain workspace.
- Cross-service E2E collections in a QA workspace.
- Environments per stage with identical keys; secrets in Vault / CI secret store.
- Shared helpers in Package Library (`@acme/test-utils`: auth, schema validation, common assertions).
- Version control: Native Git (v12) or Git integration; PR reviews; governance linting in CI.
- CI: PR smoke (fast), post-deploy regression per env, nightly full + data-driven + security; Postman CLI with `--report-events` (results to API Catalog) or Newman with JUnit/htmlextra; quality gates.
- Monitors for prod read-only checks; alerts to on-call channels.
- Metrics: endpoint coverage vs spec, pass rate, flaky tests, execution time.
- Standards doc, onboarding, PR checklist.

A2. Postman vs REST Assured/Karate/Playwright — when would you choose which? Postman: fast to build, readable by whole team, collaboration/docs/mocks/monitors built-in, CI via Newman/CLI. Code frameworks: complex logic, reusing domain code/DB checks, large-scale SDET-owned suites, IDE refactoring, strong typing, test parallelisation. Many orgs use Postman for exploration, contract & smoke suites and code frameworks for deep regression.

A3. How do you manage test data at scale? Generate data via APIs in setup (not DB inserts when avoidable), unique identifiers per run, data factories in Package Library, data files for DDT in Git, teardown + periodic cleanup jobs, dedicated test tenants, synthetic PII-free data, environment-specific fixtures.

A4. How do you measure API test coverage? Endpoint×method×status-code matrix from OpenAPI vs requests/tests in collections (via Postman API parsing), plus business-scenario coverage; Insights for actually-used endpoints; track over time.

A5. How do you secure your Postman setup in an enterprise? SSO/SCIM, domain capture, roles least-privilege, restrict public workspaces, secret scanner, Vault with external secret managers, no secrets in shared values, audit logs, review imported collections, scrub reports/logs, API keys rotation via service accounts, data residency.

A6. How do you handle contract changes between teams? Design-first with spec reviews; governance lint rules; breaking-change detection in CI (spec diff); versioning and deprecation policy; consumer notification; contract tests in both provider CI and consumer CI.

A7. Describe a complex bug you found with Postman. (STAR) Example answer: *Situation*: checkout intermittently double-charged. *Task*: find root cause. *Action*: built a Postman flow sending the same payment request concurrently via `pm.sendRequest` with/without Idempotency-Key, captured correlation IDs, discovered the key was ignored when the gateway retried after 504. *Result*: fix in gateway retry logic, added idempotency regression tests to CI; zero double charges since.

A8. How do you reduce regression suite execution time? Parallelise per service/folder in CI, tag smoke vs full, avoid redundant setup (token caching), replace fixed delays with polling, mock slow third parties, run heavy DDT nightly, profile slow requests.

A9. How would you introduce Postman to a team that tests only via UI? Show quick wins (find bugs faster), templates, training sessions, pair on first collections, CI integration with visible reports, measure defects caught earlier.

A10. What's your approach to reviewing someone's Postman collection? Structure & naming, variables/secrets hygiene, auth inheritance, assertion quality (beyond status), negative coverage, independence, script readability & reuse, docs/examples, CI readiness (no local paths/interactive auth), and a green run.

Part 11 — Behavioural & HR-style (API/QA roles)

- **Tell me about your experience with Postman.** → Scope (APIs, team size), what you built (collections, scripts, CI), measurable outcomes (bugs found, time saved, coverage).
- **How do you prioritise what to test?** → Risk-based: business-critical flows, recent changes, complex logic, security-sensitive endpoints, high-traffic APIs.
- **A developer says "it works on my machine/Postman". What do you do?** → Share exact request (cURL/collection link), environment, Console logs, correlation ID; reproduce together; focus on facts.
- **How do you keep up with Postman changes?** → Release notes, Postman blog/community, learning center, POST/CON talks, Postman Academy; test new features in a sandbox workspace before adopting.
- **Have you mentored others?** → Guidelines doc, workshops, PR reviews, templates.

Part 12 — Rapid-fire (one-liners)

Question	Answer
Default Postman proxy port?	5555
Postman User-Agent?	`PostmanRuntime/<version>`

Question	Answer
Console shortcut?	Ctrl+Alt+C / Cmd+Option+C
Send shortcut?	Ctrl/Cmd + Enter
Narrowest variable scope?	Local
Broadest?	Global
Data file formats?	CSV, JSON
Newman install?	`npm install -g newman`
Newman exit code on failure?	1
Stop run?	`pm.execution.setNextRequest(null)`
Skip request?	`pm.execution.skipRequest()` (pre-request)
Deep equality?	`eql`
200 only?	`pm.response.to.be.ok`
Any 2xx?	`pm.response.to.be.success`
Schema validator now?	Ajv (`tv4` deprecated)
Mock force status?	`x-mock-response-code`
Private mock header?	`x-api-key`
Postman API base?	`https://api.getpostman.com`
Postman API auth header?	`X-API-Key`
OAuth browser callback?	`https://oauth.pstmn.io/v1/callback`
JSON script event name for tests in export?	`"listen": "test"`
Pre-request event name?	`"prerequest"`
Collection format to export?	v2.1 (v3 YAML in v12 Native Git)
Vault reference syntax?	`{{vault:name}}`
Generate once from dynamic var?	`pm.variables.replaceIn("{{$randomEmail}}")`
Import from browser?	Copy as cURL / HAR
Popular HTML reporter?	newman-reporter-htmlextra
Official Docker image?	`postman/newman`
GraphQL errors status?	Often 200 with `errors`
gRPC OK code?	0
gRPC unauthenticated?	16
OWASP API #1 (2023)?	BOLA
March 2026 plans?	Free (1 user), Solo, Team, Enterprise

➡️ Next: **Chapter 19 — Cheat Sheets, Glossary & Reference**.

Chapter 18D — Researched Interview Patterns, Company-Tagged Questions & Verified Gotchas

How this chapter was built (honest note): In October 2026, I searched interview-prep sites, QA blogs and question banks for Postman/API testing questions, including searches for specific employers (TCS, Infosys, Wipro, Cognizant, Accenture, Capgemini, HCL, Tech Mahindra, Amazon, Microsoft, Flipkart). Findings:

- **Candidate-reported, company-specific data is scarce** in what search engines index. Glassdoor/AmbitionBox/Reddit pages didn't come up in usable form.
- A few prep sites tag questions to companies (Accenture, Cognizant, Wipro, Microsoft). They **don't prove** these were asked, so they're labelled *"tagged by prep site, unverified"* below.
- The strongest signal is **which topics recur across many independent question banks**. That frequency table is in 18D.1.

Section 18D.4 adds **"gotcha" questions that came from actually running the book's code in Newman**. These separate people who have *used* Postman deeply from people who memorised lists.

18D.1 What interviewers ask most (recurring themes across sources)

Rank	Theme (seen in most question banks)	Where answered in this book
1	Variable scopes & precedence; environment vs collection vs global	Ch 5, Q62–Q66
2	Pre-request vs test (post-response) scripts; `pm.test` / `pm.expect`	Ch 7–8, Q96–Q99, Q126
3	Collections, folders, Collection Runner	Ch 4, 10
4	Data-driven testing with CSV/JSON	Ch 10, Q150–Q153
5	Newman & CI/CD (Jenkins, GitHub Actions)	Ch 11, Q159–Q172
6	Authentication: Bearer, OAuth 2.0, API keys, JWT	Ch 6, Q37–Q42
7	Request chaining (token → next request)	Ch 9, Q146–Q148
8	Status codes (401 vs 403, 400 vs 422, 5xx)	Ch 1, Q18–Q26
9	Schema validation	Ch 8.5, Q136

Rank	Theme (seen in most question banks)	Where answered in this book
10	Mock servers & monitors	Ch 12, Q173–Q178
11	Scenario: **API returns 200 but the body contains an error**	S21 + 18D.3
12	Scenario: **token expires in the middle of a collection run**	Ch 6.13, Q148, 18D.3
13	Scenario: **intermittent/flaky failures** (debug with the Console first)	S9, Ch 16
14	Scenario: **Newman passes locally but fails in CI**	Ch 11.9, S3
15	Mocks for deterministic CI / API not ready	Ch 12, S8
16	Postman vs REST Assured / when to use code frameworks	A2, Ch 17.3
17	Real defects you've found: wrong status code for negative case, authorization errors for valid users, wrong error messages	Q53, 18D.3

> 💡 **Precedence wording trap:** some question banks write the order as "Global → Collection → Environment → Data → Local". That lists **broadest to narrowest**. It means the same as this book's "Local > Data > Environment > Collection > Global" (**narrowest wins**). In an interview, say both directions explicitly: *"Local has the highest priority and global the lowest."*

18D.2 Company-tagged questions (tagged by prep sites, unverified)

These are the questions prep sites associate with specific employers. Use them as practice, not as a guarantee.

Accenture (tagged)

Q. API testing vs UI testing — why would you test at API level? API tests are faster and more stable, run before the UI exists, reach edge cases the UI blocks, and pinpoint which service failed. UI tests verify the user journey and rendering. A strong suite uses many API tests and few UI tests (the test pyramid).

Q. How would you test API performance under high traffic? Define SLAs (p95 latency, error rate, throughput). Start with Postman's performance runner (virtual users, ramp-up/spike profiles) for a baseline. For real scale use k6/JMeter/Gatling from distributed load generators against a production-like environment. Watch server metrics and look for the saturation point (latency rises while throughput flattens), plus 429/503 behaviour.

Q. How do you reduce API test execution time?

- Run collections and folders in parallel CI jobs.
- Split smoke from full regression.

- Cache tokens instead of logging in per request.
- Replace fixed delays with polling.
- Mock slow third parties.
- Move heavy data-driven suites to nightly.
- Find slow requests from run timings.

Q. How do you do data-driven API automation? Use CSV/JSON data files with Runner/Newman `-d`. Use `{{column}}` in requests and `pm.iterationData` in assertions, with expected values per row (e.g. `expectedStatus`). Normalise types: Newman converts number-like CSV values (18D.4).

Cognizant (tagged)

Q. How do you generate automated test reports? Newman reporters: `-r cli,junit,htmlextra` with `--reporter-junit-export` for CI test tabs and htmlextra for readable HTML. Or the Postman CLI's `-r junit,html`, plus run results in Postman/API Catalog. Publish artifacts in CI and notify via Slack/Teams (a custom reporter, Ch 25).

Q. How do you simulate network failures during API testing?

- Mock servers returning 500/503/504 (`x-mock-response-code`).
- Mock network delay to trigger client timeouts.
- Very low `--timeout-request` values.
- Point to unreachable hosts or closed ports (ECONNREFUSED/ENOTFOUND).
- Use proxy/fault-injection tools (Toxiproxy, service-mesh fault injection) for resets and latency.

Then verify retries, backoff, idempotency and error messages.

Wipro (tagged)

Q. How do you decide between UI automation and API automation? Automate business rules, validation, integrations and data permutations at the API level (fast, stable). Automate only critical user journeys and UI-specific behaviour at the UI level. Consider ROI, stability, and how early you can start.

(Prep sites also note Wipro rounds can mix API testing with Selenium, SQL, TestNG, Agile and CI/CD questions. Prepare to connect Postman/Newman into a TestNG/Jenkins world.)

Microsoft (tagged)

Q. Write a sample test for an API in Postman (or REST Assured).

```
pm.test("GET /users/1 returns the right user", () => {
  pm.response.to.have.status(200);
  const u = pm.response.json();
  pm.expect(u).to.include.keys("id", "name", "email");
  pm.expect(u.id).to.eql(1);
  pm.expect(u.email).to.match(/@/);
```

```
});
pm.test("responds under 500 ms", () => pm.expect(pm.response.responseTime).to.be.below(500));
```

REST Assured equivalent: `given().get("/users/1").then().statusCode(200).body("id", equalTo(1)).time(lessThan(500L));`

Fintech/product companies (tagged generically, e.g. payments firms)

Expect idempotency (`Idempotency-Key`), webhook signature verification (HMAC), retries without double charges, and rate limits. See Ch 7.5, S5, S27.

18D.3 Scenario questions that recur across sources (full model answers)

Q. The API returns 200 but the body says `{"success": false, "error": "Insufficient balance"}` . How do you test and what do you report? Never trust the status code alone. Assert the business result: `pm.expect(json.success).to.be.true` , and `pm.expect(json).to.not.have.property("error")` . For negative cases, assert the error code/message. Report it as an API design defect, because errors should use 4xx/5xx (unless the contract or protocol says otherwise, as in GraphQL), since clients, monitors and caches rely on status codes.

Q. Your token expires halfway through a 400-request run. Fix it. Add a collection-level pre-request script that stores `accessTokenExpiresAt` and refreshes the token via `pm.sendRequest` when it's missing or about to expire (with a safety margin). Postman waits for the callback before sending. This exact pattern was verified in Newman 6 for this book (Ch 6.13). Optionally add a one-time retry on 401.

Q. Tests fail randomly about one run in five. How do you investigate?

1. Open the Console/CI logs for the failing request and compare actual vs expected.
2. Classify: timing (async not finished), data collisions (shared records, parallel jobs), environment instability (5xx), order dependencies, volatile assertions (timestamps, ordering), rate limits (429).
3. Fix the root cause: polling, unique data, isolation, tolerant assertions, retry only transient codes.
4. Track flakiness over time.

Q. You found an authorization error for valid users. How do you narrow it down?

- Decode the JWT: are the roles/scopes present, is `aud` right for this API/environment, is it expired?
- Compare a working and a failing user's tokens.
- Check the gateway/WAF logs via correlation ID.
- Check caching of permissions.
- Check tenant/organisation headers.

- Reproduce with cURL to rule out client-side issues.

Q. Negative test returns the wrong error message/status. How do you report it?
Show the request (cURL, secrets redacted), the expected result per spec (e.g. 400 with `email is required`), and the actual result (500 / generic message). Include the correlation ID, environment and timestamp, and save the response as an example (Ch 16.14 template).

18D.4 Verified gotchas (from running this book's code in Newman 6.2.3)

These come from real runs while checking the book, not from memory. They're great "experienced-level" answers.

#	Question	Verified answer
G1	Your CSV has `zip` = `007` and `expectedStatus` = `200`. What arrives in the script under Newman?	Number-like values are **converted to numbers**: `7` and `200`. Leading zeros are lost, and `1e3` becomes `1000`. `true` / `false` stay **strings**. Normalise, or use a JSON data file
G2	Does top-level `await pm.sendRequest(...)` work everywhere?	**No.** It works in recent Postman app versions, but Newman 6 throws `SyntaxError: await is only valid in async functions...`, and its `pm.sendRequest` returns no promise. Wrap it in a Promise inside an async IIFE
G3	You `throw new Error()` in a pre-request "prod guard". Is the request blocked?	In the app, yes. In **Newman, the error is reported but the request is still sent.** Use `pm.execution.skipRequest()` for a guard that works everywhere
G4	`const _ = require("lodash")` fails. Why?	`_` is already a **global** in the sandbox, so redeclaring it throws `Identifier '_' has already been declared`. Use the global `_` or another name
G5	`pm.request.body.update(...)` throws `Cannot read properties of undefined`. Why?	The request was saved **without a body** (e.g. a GET), so `pm.request.body` is undefined. Assign a body object instead: `pm.request.body = { mode: "raw", raw: ... }`
G6	You add `X-Correlation-ID` in a request script and the server sees two values. Why?	A collection script already set it, and `headers.add` **appends** (the server receives `"a, b"`). Use `headers.upsert`
G7	`pm.response.cookies` is empty in CI but cookies exist. Why?	In Newman 6 `pm.response.cookies` came back empty. Use `pm.cookies.get/has` (cookie jar) or parse `Set-Cookie` headers from `pm.response.headers.all()`
G8	`pm.execution.skipRequest()` in post-response?	`TypeError: pm.execution.skipRequest is not a function`. It's pre-request only
G9	`pm.sendRequest("{{baseUrl}}/x")` fails with what?	`getaddrinfo ENOTFOUND {{baseurl}}`. Variables aren't resolved in `sendRequest` strings, so use `pm.variables.replaceIn`

#	Question	Verified answer
G10	A variable set in a request's post-response script: is it used by that request's URL?	No, the request was already sent (the server saw the literal `{{late}}`). The **next** request gets the value
G11	A retry loop retries 503s, but the run still shows failures. Why?	A **collection-level** generic test ("no 5xx") ran on every attempt. Make generic checks retry-aware
G12	Can Newman use `pm.vault` or `pm.require`?	No, both are `undefined` in Newman. Inject secrets with `--env-var` and keep helpers inline (or use the Postman CLI for team packages)
G13	Does `pm.response.to.be.ok` pass for 202?	No. `.ok` is exactly 200; `.success` is any 2xx (both verified)
G14	Pact provider verification returns 403 "request blocked" on localhost in CI. Why?	The corporate/CI HTTP proxy intercepts `127.0.0.1`. Set `NO_PROXY`, or unset proxy variables for that step (Ch 24)
G15	Can Newman run v12 YAML (Collection v3) collections?	No, only the **Postman CLI** can (Ch 21)

18D.5 Mock interview script (60 minutes)

Minutes	Interviewer asks	Practise with
0–10	Intro, your Postman experience, project architecture	26.12 CV bullets, A1
10–20	Fundamentals rapid-fire: methods, codes, idempotency, scopes	18A, 18C rapid-fire
20–35	Live coding: write tests, chain token, schema validation, loop	18C C1–C20
35–50	Scenarios: 200-with-error, token expiry, flaky, CI-only failure	18D.3, S1–S30
50–60	Design: framework for many services; your questions to them	A1–A10

Good questions to ask the interviewer: How are API tests owned (dev vs QA)? Spec-first or code-first? Which Postman version and plan: Native Git? How do you handle test data and secrets in CI? What's the biggest API-quality pain today?

Sources consulted for 18D (prep sites; quality varies)

- Hirist — Top 40+ API Testing Interview Questions (2026)
- Hirist — 20+ Postman Interview Questions (2026)
- igmGuru — Top 50 Postman Interview Questions 2026
- Simpliaxis — Postman Interview Questions 2026
- SoftwareTestPilot — Postman Interview Questions 2026

- Intervue — Postman Interview Questions
- Skilr — Top 50 Postman Interview Questions
- Hyring — Postman Interview Questions
- Postman Blog — API Testing Interview Questions
- ACTE — Wipro Software Testing Interview Questions
- Cloudsoft — Top 20 SDET Interview Questions
- Automaters — Top 80 API Testing Interview Questions
- Rahul Shetty Academy — Top 30 API Testing Interview Questions
- Indeed — Postman interview questions

Chapter 19 — Cheat Sheets, Snippet Library & Glossary

19.1 `pm` API cheat sheet

```
// ── INFO ──────────────────────────────────────────────
pm.info.eventName | iteration | iterationCount | requestName | requestId

// ── VARIABLES ─────────────────────────────────────────
pm.variables.get(k) | set(k,v) /*local*/ | has(k) | replaceIn(str) | toObject()
pm.environment.get | set | has | unset | clear | toObject | replaceIn | name
pm.collectionVariables.get | set | has | unset | clear | toObject | replaceIn
pm.globals.get | set | has | unset | clear | toObject | replaceIn
pm.iterationData.get | has | toObject
await pm.vault.get(k) | set(k,v) | unset(k)

// ── REQUEST ───────────────────────────────────────────
pm.request.method | url | headers | body | auth
pm.request.url.toString() | getHost() | getPath() | getPathWithQuery() |
query.add/upsert/remove/get/toObject
pm.request.headers.add({key,value}) | upsert(...) | remove(k) | get(k) | toObject()
pm.request.body.raw | mode | urlencoded | formdata | update({...})

// ── RESPONSE ──────────────────────────────────────────
pm.response.code | status | responseTime | responseSize | headers | cookies
pm.response.json() | text()
pm.response.to.have.status(n|reason) | header(k[,v]) | body(str) | jsonBody([path[,v]]) |
jsonSchema(s)
pm.response.to.be.ok(200) | success(2xx) | redirection | clientError | serverError | error
                | badRequest | unauthorized | forbidden | notFound | rateLimited | json |
withBody

// ── TESTS ─────────────────────────────────────────────
pm.test(name, fn) | pm.test.skip(name, fn)
pm.expect(x).to.equal | eql | deep.equal | include | have.property | nested.property
           .be.a(type) | an | null | undefined | true | false | empty | exist | ok(truthy)
           .above | below | at.least | at.most | within | closeTo
           .match(re) | oneOf([]) | lengthOf(n) | members([]) | all.keys | any.keys | satisfy(fn)

// ── CONTROL ───────────────────────────────────────────
pm.execution.setNextRequest(nameOrId | null) | skipRequest() | location
pm.sendRequest(urlOrReq, (err, res) => {}) | await pm.sendRequest(req) /* app only; not Newman 6
*/

// ── COOKIES ───────────────────────────────────────────
pm.cookies.get(n) | has(n) | toObject() ; pm.cookies.jar().get/getAll/set/unset/clear(url, ...,
cb)

// ── VISUALIZER & PACKAGES ─────────────────────────────
pm.visualizer.set(template, data) ; (in template) pm.getData(cb)
pm.require("@team/package")
```

19.2 Snippet library (copy-paste ready)

```
// 1. Standard checks
pm.test("Status 200", () => pm.response.to.have.status(200));
pm.test("JSON", () => pm.response.to.be.json);
pm.test("Fast", () => pm.expect(pm.response.responseTime).to.be.below(1000));

// 2. Save id from response
pm.collectionVariables.set("id", pm.response.json().id);

// 3. Save id from Location header
pm.collectionVariables.set("id", pm.response.headers.get("Location").split("/").pop());

// 4. Correlation id on every request (collection pre-request)
pm.request.headers.upsert({ key: "X-Correlation-ID", value: pm.variables.replaceIn("{{$guid}}")
});

// 5. Unique email generated once
if (!pm.collectionVariables.get("email")) pm.collectionVariables.set("email",
pm.variables.replaceIn("{{$randomExampleEmail}}"));

// 6. Guard against prod writes (collection pre-request)
if (/prod/i.test(pm.environment.name || "") && pm.request.method !== "GET") {
console.warn("Skipped write in PROD"); pm.execution.skipRequest(); }

// 7. Decode JWT
const claims = JSON.parse(atob(pm.environment.get("token").split(".")
[1].replace(/-/g,"+").replace(/_/g,"/")));

// 8. Safe JSON parse in tests
let body; try { body = pm.response.json(); } catch (e) { body = null; }
pm.test("Valid JSON", () => pm.expect(body).to.not.be.null);

// 9. Schema validation with Ajv
const Ajv = require("ajv"); const v = new Ajv({ allErrors: true
}).compile(JSON.parse(pm.collectionVariables.get("schema")));
pm.test("Schema", () => pm.expect(v(pm.response.json()), JSON.stringify(v.errors)).to.be.true);

// 10. Stop run on failure of critical step
if (pm.response.code >= 400) pm.execution.setNextRequest(null);

// 11. Clean up runtime variables (teardown)
["orderId", "customerId", "email"].forEach(k => pm.collectionVariables.unset(k));

// 12. Date helpers
const today = new Date().toISOString().slice(0, 10);
const plusDays = n => new Date(Date.now() + n * 864e5).toISOString().slice(0, 10);

// 13. Wait (polling helper)
setTimeout(() => {}, 2000);

// 14. Convert XML to JSON
const xml = xml2Json(pm.response.text());

// 15. Log request as sent (debug)
console.log(pm.request.method, pm.variables.replaceIn(pm.request.url.toString()));
```

19.3 Newman / Postman CLI cheat sheet

```
npm i -g newman newman-reporter-htmlextra
newman run c.json -e env.json -d data.csv -n 2 --folder "Smoke" \
  --env-var "token=$TOKEN" --delay-request 100 --timeout-request 10000 --bail \
  -r cli,junit,htmlextra --reporter-junit-export out/junit.xml --reporter-htmlextra-export
out/report.html
newman run "https://api.getpostman.com/collections/<uid>?apikey=$KEY"
docker run --rm -v "$PWD":/etc/newman postman/newman:alpine run c.json

postman login --with-api-key "$POSTMAN_API_KEY"
postman collection run <collectionId|path> -e <envId|path> -r cli,junit --reporter-junit-export
out/junit.xml
postman api lint <apiId>
postman spec lint <spec>
postman workspace push        # v12 Native Git
```

19.4 Status codes cheat sheet

2xx	3xx	4xx	5xx
200 OK	301 Moved Permanently	400 Bad Request	500 Internal Server Error
201 Created	302 Found	401 Unauthorized	501 Not Implemented
202 Accepted	303 See Other	403 Forbidden	502 Bad Gateway
204 No Content	304 Not Modified	404 Not Found	503 Service Unavailable
206 Partial Content	307 Temporary Redirect	405 Method Not Allowed	504 Gateway Timeout
	308 Permanent Redirect	409 Conflict, 413, 415, 422, 429	

19.5 Variable precedence & values

```
LOCAL  >  DATA  >  ENVIRONMENT  >  COLLECTION  >  GLOBAL
Shared value (old: initial)  → synced to cloud, visible to team, exported
Local value  (old: current)  → only on your machine, used when sending
Secrets → Vault {{vault:name}} (local, encrypted) / CI --env-var
```

19.6 Script execution order

```
Collection pre → Folder pre → Request pre → [ SEND ] → Collection post → Folder post → Request
post
```

19.7 Glossary (A–Z)

Term	Meaning
Agent (Desktop/Cloud/Browser)	Component that sends requests for the Postman web app
Agent Mode	Postman's agentic AI for multi-step tasks
Ajv	JSON Schema validator available in sandbox
API	Interface/contract for programs to communicate
API Catalog	Org-wide inventory of APIs (v12)
API Network	Public directory of APIs/workspaces
Assertion	A check that throws if false
AsyncAPI	Spec format for event-driven/async APIs
Auth inheritance	Requests using parent folder/collection auth
Bearer token	Token sent as `Authorization: Bearer`
BOLA/IDOR	Accessing another user's object by ID
Boundary	Separator in multipart bodies
Chai	Assertion library behind `pm.expect`
Collection	Group of requests with shared config
Collection Runner	Runs collections with iterations/data
Collection v2.1 / v3	JSON export format / YAML format (v12 Native Git)
Console	Postman's debugging log
Contract testing	Verifying responses match the agreed spec
CORS	Browser cross-origin rules
Data file	CSV/JSON for iterations
Dynamic variables	`{{$guid}}`, `{{$random...}}` generated values
Environment	Named set of variables per stage
Example	Saved request/response pair (mocks, docs)
Flows	Visual low-code workflow builder
FQL	Flows Query Language
Fork	Linked copy for independent changes
gRPC	Protobuf RPC over HTTP/2
HAR	HTTP Archive file from browsers
Idempotent	Repeating doesn't change final state
Insights	Traffic-observing agent for API discovery

Term	Meaning
Interceptor	Chrome extension capturing requests/cookies
Iteration	One pass through the run (one data row)
JWT	Signed JSON token `header.payload.signature`
Lightweight API Client	Signed-out mode (replaced Scratch Pad)
Local variable	Temporary, highest precedence
Mock server	Fake API returning examples
Monitor	Scheduled cloud collection runs with alerts
mTLS	Client presents a certificate too
Native Git	v12 Git-native workspaces with YAML files
Newman	Open-source CLI runner
OAuth 2.0 / OIDC	Authorization framework / identity layer
OpenAPI	REST API description standard
Package Library	Team-shared script modules (`pm.require`)
PKCE	Code verifier/challenge protection for auth code flow
Postbot	Postman AI assistant
Postman API	REST API to manage Postman resources
Postman CLI	Official CLI (login, run, lint, push)
Pre-request script	Runs before sending
Post-response script	Runs after response (formerly "Tests")
Pull request	Proposal to merge fork changes
Sandbox	Restricted JS runtime for scripts
Secret type	Masked variable value in UI
Spec Hub	Postman's API specification editor/hub
SSE	Server-Sent Events streaming
TTFB	Time to first byte (server think time)
Vault	Encrypted local secrets store
Visualizer	Custom HTML rendering of responses
Webhook	Server calls your URL on events; also Postman collection-trigger URLs
Workspace	Container for team collaboration

19.8 Exam-night revision list (top 30 facts)

1. Precedence: Local > Data > Env > Collection > Global.
2. Script order: Collection → Folder → Request (both phases).
3. `pm.response.to.be.ok` = 200; `.success` = 2xx.
4. `eql` for deep equality.
5. `setNextRequest` only in runs; `null` stops; last call wins; prefer IDs.
6. `skipRequest` only in pre-request.
7. CSV types unreliable (number-like → number, `007` → 7, booleans stay strings) — normalise; extra iterations reuse last row.
8. Newman exit 1 on failures; `--env-var` for secrets; `--export-environment` to persist.
9. Postman CLI: login with API key, IDs or files, `api lint`, results to Postman.
10. Mocks: examples, `x-mock-response-code/name/id`, `x-api-key` private.
11. Monitors: cloud, regions, alerts; can't reach private nets without allowlisting.
12. Vault: `{{vault:x}}`, `await pm.vault.get`, not in Newman/CLI/monitors.
13. Shared vs local values (old initial vs current).
14. Dynamic vars change per use → `replaceIn` once and store.
15. `pm.request.body.raw` is unresolved → `replaceIn` before `JSON.parse`.
16. Form-data: don't set Content-Type manually.
17. OAuth token endpoint uses x-www-form-urlencoded.
18. OAuth browser callback `https://oauth.pstmn.io/v1/callback`.
19. Interactive OAuth doesn't run in CI → script token fetch.
20. CORS doesn't apply to Postman desktop.
21. `ENOTFOUND {{var}}` = unresolved variable.
22. `ECONNREFUSED ::1` = IPv6 localhost vs IPv4 server.
23. `Unexpected token '<'` = HTML instead of JSON.
24. Corporate SSL errors → add CA cert, not disable verification.
25. GraphQL errors often 200 with `errors`.
26. gRPC codes: 0 OK, 3 INVALID_ARGUMENT, 5 NOT_FOUND, 7 PERMISSION_DENIED, 14 UNAVAILABLE, 16 UNAUTHENTICATED.
27. OWASP API 2023 #1 BOLA, #2 Broken Auth, #3 BOPLA, #5 BFLA, #7 SSRF.
28. Exported JSON: `listen: "prerequest"` / `listen: "test"`.
29. v11: Scripts tab (Pre-request/Post-response), Vault, Package Library, Spec Hub; v12: Native Git YAML, Agent Mode, API Catalog.
30. March 2026 plans: Free (1 user), Solo, Team, Enterprise.

➡️ Next: **Chapter 20 — Hands-On Labs & Capstone Project**.

Chapter 20 — Hands-On Labs & Capstone Project

> 🧒 Reading about swimming doesn't teach you to swim. **Do every lab.** Each lab tells you *what to do*, *what you should see*, and *what you learned*.

Practice APIs (free, public — availability can change; if one is down use another):

- `https://jsonplaceholder.typicode.com` — fake REST data (writes are simulated, not saved)
- `https://httpbin.org` — echoes requests, status codes, auth, delays, cookies
- `https://postman-echo.com` — Postman's echo service
- `https://restful-booker.herokuapp.com` — CRUD + auth for testers (data resets periodically)
- `https://petstore.swagger.io/v2` — Swagger Petstore (with OpenAPI spec at `/v2/swagger.json`)

Lab 1 — First requests (🟢 15 min)

1. `GET https://jsonplaceholder.typicode.com/todos/1` → 200, `{ userId:1, id:1, title:"delectus aut autem", completed:false }`.
2. `GET https://jsonplaceholder.typicode.com/posts?userId=2` → array of 10 posts.
3. `GET https://jsonplaceholder.typicode.com/posts/:id` with path variable `id=3`.
4. `GET https://httpbin.org/headers` → see headers Postman sent (User-Agent PostmanRuntime, Postman-Token). ✅ Learned: methods, params, path variables, auto headers.

Lab 2 — Bodies (🟢 20 min)

1. `POST https://httpbin.org/post` raw JSON `{"name":"Asha"}` → echoed under `json`.
2. Same with **x-www-form-urlencoded** → echoed under `form`.
3. **form-data** with a file → echoed under `files`.
4. Remove JSON selection (Text) → see `Content-Type: text/plain` echoed; understand 415 bugs. ✅ Learned: body types and Content-Type.

Lab 3 — Status codes & redirects (🟢 15 min)

1. `GET https://httpbin.org/status/404`, `/status/500`, `/status/429`.
2. `GET https://httpbin.org/redirect/3` → final 200; turn off "Automatically follow redirects" in request Settings → see 302 + `Location`.
3. `GET https://httpbin.org/delay/3` → time ≈ 3s; set request timeout 1000ms in Settings → timeout error. ✅ Learned: status classes, redirect settings, timeouts.

Lab 4 — Auth types (🟡 30 min)

1. Basic: `GET https://httpbin.org/basic-auth/user/passwd`, Auth → Basic user/passwd → 200; wrong password → 401.
2. Bearer: `GET https://httpbin.org/bearer` without token → 401; with any Bearer token → 200 + token echoed.
3. Digest: `GET https://httpbin.org/digest-auth/auth/user/passwd` with Digest Auth user/passwd → 200.
4. API key: `GET https://postman-echo.com/get` with API Key auth `x-api-key: test` in Header → echoed in headers; switch to Query Params → appears in `args`. ✅ Learned: auth tab types and where credentials go.

Lab 5 — Environments & variables (🟡 30 min)

1. Create env **Practice – JSONPH** with `baseUrl = https://jsonplaceholder.typicode.com`.
2. Create env **Practice – ECHO** with `baseUrl = https://postman-echo.com` (different API — notice how switching env changes behaviour).
3. Replace hosts with `{{baseUrl}}`. Switch envs and observe.
4. Add collection variable `baseUrl = https://example.invalid` → env value still wins (precedence).
5. In pre-request: `pm.variables.set("baseUrl", "https://httpbin.org/anything")` → local wins over everything.
6. Unselect environment → `{{baseUrl}}` resolves from collection → `ENOTFOUND` → understand the error. ✅ Learned: scopes, precedence, typical errors.

Lab 6 — Tests (🟡 40 min)

On `GET {{baseUrl}}/users` (JSONPlaceholder):

```
const users = pm.response.json();
pm.test("200", () => pm.response.to.have.status(200));
pm.test("10 users", () => pm.expect(users).to.have.lengthOf(10));
pm.test("Each has email & geo", () => users.forEach(u => {
    pm.expect(u.email).to.include("@");
    pm.expect(u).to.have.nested.property("address.geo.lat");
}));
pm.test("Unique usernames", () => pm.expect(new Set(users.map(u =>
u.username)).size).to.eql(10));
pm.test("Schema", () => pm.response.to.have.jsonSchema({
    type: "array", items: { type: "object", required: ["id","name","username","email"],
    properties: { id: {type:"integer"}, email: {type:"string"} } } }));
```

Break a test on purpose (`lengthOf(11)`) → read failure message. ✅ Learned: assertions, arrays, schema.

Lab 7 — Chaining & dynamic data (🟡 40 min) — Restful-Booker

1. `POST {{rbBase}}/auth` body `{"username":"admin","password":"password123"}` → post-response: `pm.environment.set("rbToken", pm.response.json().token)`.
2. `POST {{rbBase}}/booking` with `Accept: application/json`, body using `{{$randomFirstName}}`, `{{$randomLastName}}`, `{{$randomInt}}` → save `bookingid`:

```
const b = pm.response.json();
pm.test("Created", () => pm.response.to.have.status(200));
pm.collectionVariables.set("bookingId", b.bookingid);
pm.collectionVariables.set("firstname", b.booking.firstname);
```

3. `GET {{rbBase}}/booking/{{bookingId}}` → test firstname equals stored value.
4. `PATCH {{rbBase}}/booking/{{bookingId}}` header `Cookie: token={{rbToken}}`, body `{"firstname":"Asha"}` → 200.
5. `DELETE ...` with Cookie → **201** (API's documented behaviour).
6. `GET` again → 404. ✅ Learned: chaining, dynamic variables, "follow the contract".

Lab 8 — Runner & data-driven (🟡 30 min)

`login.csv`:

```
case,username,password,expectReason
valid,admin,password123,
bad password,admin,nope,Bad credentials
empty,,,Bad credentials
```

Request `POST {{rbBase}}/auth` body `{"username":"{{username}}","password":"{{password}}"}`. Test:

```
const reason = pm.iterationData.get("expectReason");
const body = pm.response.json();
pm.test(`[${pm.iterationData.get("case")}]`, () => {
    if (reason) pm.expect(body.reason).to.eql(reason);
    else pm.expect(body.token).to.be.a("string");
});
```

(Restful-Booker returns 200 with `{"reason":"Bad credentials"}` on failure — another real-world quirk worth reporting as a design issue.) Run in Runner with the file → 3 iterations. ✅ Learned: CSV DDT, iteration data, API quirks.

Lab 9 — Workflows (🔴 40 min)

1. `GET {{baseUrl}}/posts?userId=1` (JSONPlaceholder) → save the 10 post IDs.
2. `GET {{baseUrl}}/posts/{{postId}}/comments` → loop through all IDs with `setNextRequest` (Chapter 9 Pattern 2), assert each post has 5 comments.

3. Add a guard counter (max 20).
4. Run in Runner → see 10 executions of request 2. ✅ Learned: loops, setNextRequest by ID, guard counters.

Lab 10 — Pre-request magic (🔴 30 min)

1. Collection pre-request adds `X-Correlation-ID: {{$guid}}`.
2. On `POST https://postman-echo.com/post`, pre-request computes `X-Signature = HMAC-SHA256(body, "secret")`.
3. Post-response verifies echoed header equals your computed signature (`pm.response.json().headers["x-signature"]`). ✅ Learned: modifying requests, CryptoJS, verifying via echo.

Lab 11 — Mock server (🟡 30 min)

1. In "Orders (Design)" collection create `GET {{baseUrl}}/orders/:id` with examples: `200 OK` (body with `{{$randomPrice}}`), `404 Not found`.
2. Create mock → save mock URL as environment `baseUrl`.
3. Send → 200 example. Add header `x-mock-response-code: 404` → 404 example. ✅ Learned: examples drive mocks; control headers.

Lab 12 — Newman & CI (🔴 45 min)

1. Export your Restful-Booker collection (v2.1) and environment into a Git repo folder `postman/`.
2. `npx newman run postman/rb.postman_collection.json -e postman/rb.postman_environment.json -r cli,htmlextra --reporter-htmlextra-export out/report.html` (install `newman` and `newman-reporter-htmlextra` as dev dependencies).
3. Make a test fail → check exit code: `echo $?` → 1.
4. Add the GitHub Actions workflow from Chapter 11 → push → see run & artifact. ✅ Learned: CLI, reports, exit codes, CI.

Lab 13 — OpenAPI import (🟡 20 min)

Import `https://petstore.swagger.io/v2/swagger.json` → generated collection → run `GET /pet/findByStatus?status=available` → add schema test from the spec's `Pet` definition. ✅ Learned: spec → collection workflow.

Lab 14 — Other protocols (🟡 30 min)

- GraphQL: public demo endpoints exist (e.g., the Countries API `https://countries.trevorblades.com/`) — query `{ country(code:"IN") { name capital currency } }`.

- WebSocket: connect to a public echo server (e.g. `wss://echo.websocket.org`), send JSON, observe echo.
- SOAP: Calculator `Add` request from Chapter 13. ✅ Learned: request types beyond REST. (Public demo endpoints change; search for a current one if needed.)

Lab 15 — Debugging drills (🔴 30 min)

Create each failure deliberately and fix it using Chapter 16:

1. Typo variable `{{baseurl}}` → ENOTFOUND.
2. `http://localhost:9999` (nothing running) → ECONNREFUSED.
3. `https://expired.badssl.com/` → certificate expired error; `https://self-signed.badssl.com/` → self-signed error; observe toggling SSL verification.
4. `https://wrong.host.badssl.com/` → hostname mismatch.
5. Call `pm.response.json()` on `https://httpbin.org/html` → JSONError `<`.
6. `pm.execution.skipRequest()` in post-response → TypeError. ✅ Learned: real error messages and fixes.

🎓 Capstone Project — "Restful-Booker Regression Suite" (production-grade)

Goal

Build a collection that a company could actually run in CI.

Structure

```
📦 Restful-Booker — Regression
Collection auth: No Auth (API uses cookie token for writes)
Collection pre-request: correlation id, prod-write guard
Collection post-response: no 5xx, responds < 3s, JSON valid
├── 📁 00 Setup
│    ├── GET  Health check (/ping → 201)
│    └── POST Create token (/auth) → rbToken
├── 📁 01 Booking — Positive
│    ├── POST Create booking → bookingId, snapshot
│    ├── GET  Get booking by id → deep equals snapshot
│    ├── GET  List bookings filtered by firstname → contains bookingId
│    ├── PUT  Update booking (Cookie token) → all fields updated
│    └── PATCH Partial update → only firstname changed
├── 📁 02 Booking — Negative
│    ├── PUT  Update without token → 403
│    ├── GET  Non-existent id → 404
│    ├── POST Create with missing fields → 500 (document as defect: should be 400!)
│    └── POST Auth bad credentials (data-driven) → reason "Bad credentials"
├── 📁 03 Booking — Security
│    ├── DELETE with tampered token → 403
│    └── Check response headers (X-Powered-By present → report)
└── 📁 99 Teardown
```

```
      ├── DELETE booking → 201
      └── GET deleted booking → 404 ; unset variables
```

Requirements checklist

- [ ] `{{rbBase}}` from environment; DEV/QA style envs (both pointing to same public URL for practice).
- [ ] No hard-coded tokens; token set by script.
- [ ] Every request: status + at least one business assertion; positives have schema checks.
- [ ] Data-driven negative auth folder with CSV.
- [ ] Unique test data (`{{$randomFirstName}}` generated once and stored).
- [ ] Descriptions and examples for each request.
- [ ] Teardown runs even when tests fail (no `setNextRequest(null)` mid-run; jump to teardown instead).
- [ ] Newman run with JUnit + htmlextra; GitHub Actions workflow; artifacts uploaded.
- [ ] README in repo explaining how to run.
- [ ] Defects found documented as bug reports (Chapter 16.14 format).

Stretch goals

- Package Library helper `@me/rb-utils` for common checks.
- Performance tab run: 10 VUs, 2 min ramp-up; record p95.
- Monitor on `/ping` every hour with email alert.
- Visualizer: table of bookings.
- Convert to Native Git (v12) YAML layout and run with `postman collection run <path>`.

What to say in interviews about this project

"I built a regression suite with setup/teardown, chained auth, data-driven negatives, schema validation and security checks, runnable via Newman in GitHub Actions with JUnit/HTML reports. I found and documented API design defects such as 500 instead of 400 on validation errors and 200 with error body on failed auth."

🎉 **You've finished the book.** Go back to Chapter 18 regularly, do the labs twice, and build your own capstone on a real company API (with permission). That's how you become the Postman expert on your team.

➡️ Going further: **Chapters 21–26** (Native Git, Flows/FQL, Enterprise & governance, Pact, custom reporters, migration & career) and **Chapter 27** (verification report).

Chapter 21 — Native Git & the Postman v12 Workflow (Deep Dive)

> 🧒 Before v12, your Postman collections lived in a **cloud locker**, and Git was a separate **photo album** where you pasted pictures (exports) now and then. In v12, the collection **lives in the photo album itself**: each request is its own page (file). Git keeps the history, and the cloud copy is synced from it.

> ⚠️ This is the newest part of Postman (v12, 2026). The concepts below come from Postman's Native Git documentation and early public examples. Command flags and file layouts can still change between minor releases, so confirm with `postman --help` and the Learning Center for your version.

21.1 Why Native Git exists

Old pain (exports to Git)	Native Git answer
One giant `*.postman_collection.json` → unreadable diffs	Collection split into **many YAML files** (one per request), readable diffs
Merge conflicts in one huge JSON file	Conflicts only in the specific request files touched
"Which copy is latest: cloud or Git?"	**Local files are the source of truth**; the cloud is synced from them
API tests reviewed separately from code	Tests change in the **same pull request** as the code they test
Needs internet	Local-first: edit offline in **Local View**

21.2 What gets created on disk

When you connect a workspace to a local folder (a repo, or a service folder inside a monorepo), Postman creates two directories:

```
my-service/
├── .postman/                    ← workspace configuration
│   └── resources.yaml           ← manifest: maps local files ↔ Postman Cloud entities
├── postman/                     ← your API resources (synced by push/pull)
│   ├── collections/
│   │   └── Orders API/          ← a v3 collection = a directory tree
│   │       ├── .resources/      ← collection-level definition (variables, scripts, auth) in
public examples
│   │       ├── Get order.yaml   ← one file per request, scripts/tests inline
```

```
│   │       └── Create order.yaml
│   ├── environments/
│   ├── specs/
│   ├── flows/
│   ├── globals/
│   ├── mocks/
│   ├── documents/
│   └── sections.yaml
└── src/ ...                    ← your application code
```

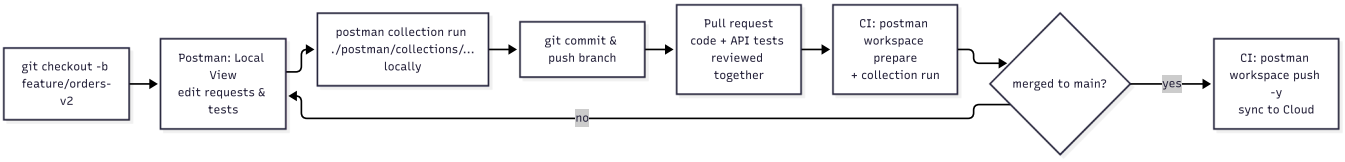
- **Collection format v3** is YAML, split into files. The classic v2.1 is a single JSON file.
- Check that `.postman/resources.yaml` exists after connecting. If it doesn't, the connection didn't complete.
- **Commit both `.postman/` and `postman/`.** Never commit secrets (Chapter 5).

21.3 Local View vs Cloud View

	Local View	Cloud View
Edits are saved to	YAML files in your folder	Postman cloud workspace
Reviewed through	Git pull requests	Postman forks/PRs (classic)
Safe to edit?	✅ Yes, the source of truth	⚠️ **No.** The docs warn that changes made directly in Cloud View are deleted on the next `postman workspace push`, because local files don't contain them

> 🏢 **Team rule:** In Native Git workspaces, *everyone edits in Local View on a branch*. Treat Cloud View as read-only, for sharing, monitors and mocks.

21.4 The daily workflow



1. **Branch** from `main`.
2. In Postman, switch to **Local View** and edit requests, scripts and environments. Changes land in `postman/…` YAML files.
3. **Run locally** by path with the Postman CLI.
4. **Commit and push** the branch, then open a **PR**. Reviewers see per-request diffs next to the code change.
5. **CI** validates and runs the tests.

6. **After merge to main**, CI runs `postman workspace push` so the cloud workspace (used by teammates, monitors, mocks, docs) matches `main`.

21.5 CLI commands you'll use

```
postman login --with-api-key "$POSTMAN_API_KEY"

# Validate local collections/environments before CI runs them:
# checks structure, resolves references, flags problems early
postman workspace prepare

# Run a v3 collection by path (Postman CLI only)
postman collection run ./postman/collections/"Orders API" -e ./postman/environments/qa.yaml

# Push local state to the cloud workspace (-y = no interactive confirmation; use in CI)
postman workspace push -y

# Non-default resource locations
postman workspace push -y --collections-dir ./api/postman/collections --environments-dir
./api/postman/environments
```

(Use the environment file name and extension that Postman created in your repo. Run `postman workspace --help` to see the exact flags for your version.)

> ❗ **Newman cannot run v3 (YAML) collections.** Only the **Postman CLI** can. If your CI uses Newman, either keep exporting v2.1 JSON or move CI to the Postman CLI. Early v12 docs also note that **only the `cli` reporter** is supported for some v3 request types. Check before relying on JUnit/HTML output.

GitHub Actions example (Native Git)

```
name: API tests (Native Git)
on:
  pull_request:
  push:
    branches: [main]
jobs:
  api:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install Postman CLI
        run: curl -o- "https://dl-cli.pstmn.io/install/linux64.sh" | sh
      - name: Login
        run: postman login --with-api-key "${{ secrets.POSTMAN_API_KEY }}"
      - name: Validate
        run: postman workspace prepare
      - name: Run tests
        run: postman collection run "./postman/collections/Orders API" --env-var
"clientSecret=${{ secrets.QA_CLIENT_SECRET }}"
      - name: Sync to cloud (main only)
```

```
        if: github.ref == 'refs/heads/main' && github.event_name == 'push'
        run: postman workspace push -y
```

21.6 Monorepos

- Create **one Postman workspace per service** and connect it to **that service's folder**, not the repo root.
- Run `postman workspace push` **from the service folder**, so the CLI reads that service's `.postman/resources.yaml` and syncs only that service.

```
monorepo/
├── services/orders/   (.postman/, postman/)  ↔ workspace "Commerce — Orders API"
├── services/payments/ (.postman/, postman/)  ↔ workspace "Commerce — Payments API"
└── .github/workflows/ (one job per service, working-directory: services/<name>)
```

21.7 Pull, conflicts, and mixed situations

- **Pulling from cloud** lets you choose which resources to bring down.
- If a local collection uses request types the cloud collection can't store, "Update existing" is disabled. You must choose **"Keep both"** or **"Rename and keep"**.
- **Git merge conflicts** are normal YAML conflicts. Resolve them in your editor, then run `postman workspace prepare` and the collection locally before pushing.
- Don't run classic forks/PRs in Postman cloud *and* Git PRs on the same collection. Pick Git as the review system.

21.8 Migrating an existing team to Native Git (playbook)

1. Pick one service and its **reference + test collections**.
2. Make sure the team is on v12 desktop and the Postman CLI.
3. Connect the workspace to the service folder, then commit `.postman/` and `postman/`.
4. Replace Newman CI jobs with Postman CLI jobs (or keep a parallel v2.1 export until reports are equivalent).
5. Announce **"edit in Local View only"**. Lock down Cloud View editing with roles where possible.
6. Add `postman workspace push -y` on merges to main.
7. After two or three sprints without issues, migrate the next service.

21.9 Interview questions on Native Git

Q. What changed in v12 for version control? Collections can live in your Git repo as YAML (Collection v3), split into many files. You edit locally (Local View) and review via Git PRs, and the Postman CLI syncs to the cloud with `postman workspace push`.

Q. Can Newman run v3 collections? No. Use the Postman CLI, or keep exporting v2.1 JSON for Newman.

Q. A teammate edited in Cloud View and their change disappeared. Why? In Native Git workspaces, local files are the source of truth. The next `workspace push` from CI removed cloud-only changes that weren't in the repo.

Q. How do you handle a monorepo? One workspace per service, connected to that service's folder, with push run from that folder so the right `.postman/resources.yaml` manifest is used.

➡️ Next: **Chapter 22 — Postman Flows & FQL Deep Dive**.

Chapter 22 — Postman Flows & FQL (Deep Dive)

> 🧒 A flow is a **marble run**. You drop data in at the start, it rolls through tubes (wires) and gadgets (blocks): one gadget calls an API, one sorts the marbles, one says "only red marbles go left". At the end the marbles land in a bucket (output, Slack message, chart).

22.1 When to use Flows (and when not)

Use Flows for	Prefer scripts/collections for
Visual API orchestration for demos, PMs, support teams	Regression test suites run in CI
Data transformation pipelines (read API A → transform → write API B)	Complex assertions and reporting
Scheduled automations (daily report → Slack)	Version-controlled, code-reviewed test logic (though v12 Native Git also stores flows in `postman/flows/`)
Exposing APIs as **AI agent tools** (MCP trigger)	
Prototyping integrations quickly	

22.2 Building blocks

Category	Blocks (names vary slightly by version)	Purpose
Triggers	Start (manual), **Request** trigger (HTTP), **Schedule**, **MCP**	How the flow starts. A deployable flow needs a trigger
Requests	HTTP/API Request (from a collection or inline), **GraphQL Request**	Call APIs
Logic	**If**, **Condition**, **Evaluate**	Branching and transformation. These accept **FQL or TypeScript**
Looping/collection	Repeat / For / Loop, **Collect**	Iterate and gather results
Data	Select, Template, Create (JSON/text/number/list), Record	Pick fields, build payloads
Timing	Delay	Wait between steps
Output	Display (table, chart, text), Log	Show results

Category	Blocks (names vary slightly by version)	Purpose
AI	**AI Request** (OpenAI, Anthropic, Google, OpenAI-compatible providers), agent-style blocks in newer releases	Use LLMs inside flows

Wires carry data from an output port to an input port. A **Send Request** block has **success** and **failure** outputs, so you can branch on errors.

22.3 FQL: Flows Query Language

FQL is how you **query and transform JSON** inside If, Condition and Evaluate blocks. Its syntax closely resembles **JSONata**.

> ✅ The expressions below were checked with the JSONata engine while writing this book, using the sample data shown, and they produce the results listed. Function availability in FQL can differ slightly, so use the FQL function reference in the Learning Center for your version.

Sample data (output of a "Get payments" request):

```
{
  "customer": { "name": "Asha Rao", "email": "asha@example.com" },
  "payments": [
    { "id": 1, "description": "recurring subscription", "amount": 499,  "status": "PAID",
"date": "2026-10-01" },
    { "id": 2, "description": "one-time",               "amount": 1200, "status": "FAILED",
"date": "2026-10-03" },
    { "id": 3, "description": "recurring subscription", "amount": 499,  "status": "PAID",
"date": "2026-10-08" }
  ]
}
```

Goal	FQL expression	Result
Filter records	`payments[description="recurring subscription"]`	the 2 matching objects
Filter, then pick a field	`payments[description="recurring subscription"].amount`	`[499, 499]`
Single match gives a value, not an array	`payments[id=2].status`	`"FAILED"`
Sum	`$sum(payments[status="PAID"].amount)`	`998`
Count	`$count(payments)`	`3`
Map	`$map(payments, function($p){ $p.amount * 2 })`	`[998, 2400, 998]`
Filter with a function	`$filter(payments, function($v, $i, $a){ $v.amount > 500 }).id`	`2`

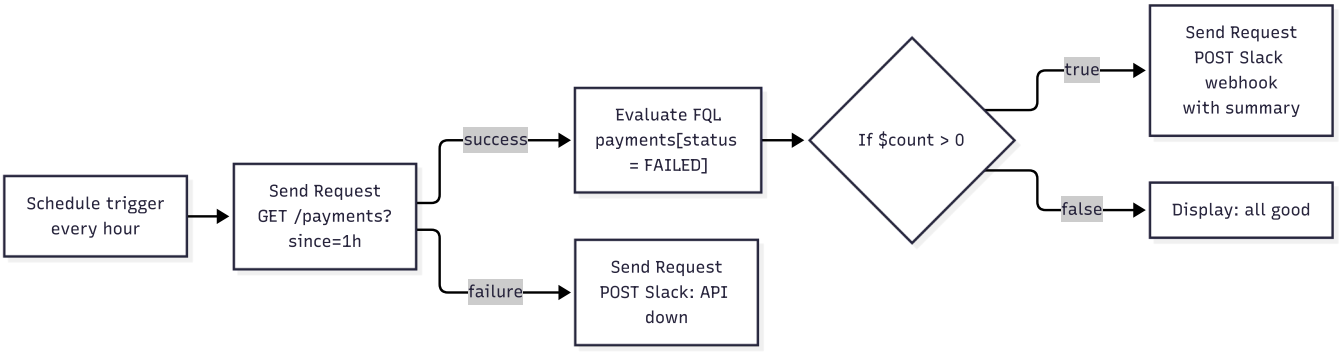
Goal	FQL expression	Result
Reduce	`$reduce(payments.amount, function($acc, $v){ $acc + $v }, 0)`	`2198`
Reshape each item	`payments.{ "paymentId": id, "rupees": amount, "ok": status = "PAID" }`	array of new objects
Build an object	`{ "name": customer.name, "total": $sum(payments.amount), "failed": $count(payments[status="FAILED"]) }`	`{"name":"Asha Rao","total":2198,"failed":1}`
Variables (`:=` , wrap in parentheses)	`( $paid := payments[status="PAID"]; { "paidCount": $count($paid), "withGst": $sum($paid.amount) * 1.18 } )`	`{"paidCount":2,"withGst":1177.64…}`
Conditional	`$count(payments[status="FAILED"]) > 0 ? "ALERT" : "OK"`	`"ALERT"`
Strings	`$uppercase($substring(customer.name, 0, 4)) & "-" & $join(["a","b"], "-")`	`"ASHA-a-b"`
Lookup a key	`$lookup(customer, "email")`	`"asha@example.com"`
Sort	`$sort(payments, function($l, $r){ $l.amount > $r.amount }).id`	`[1, 3, 2]`
Distinct	`$distinct(payments.description)`	2 unique values
Comments	`/* total */ $sum(payments.amount)`	`2198`
Today's date	`$fromMillis($toMillis($now()), "[Y0001]-[M01]-[D01]")`	e.g. `"2026-10-16"`

> ⚠️ **Floating point:** `withGst` came out as `1177.6399999999999`. Round money with `$round(x, 2)`.

TypeScript in Evaluate blocks

When logic gets complicated, switch the Evaluate block to **TypeScript** and write normal code against the block's inputs. Use FQL for queries and reshaping, and TypeScript for real algorithms.

22.4 Worked example: failed-payment alert flow



1. **Schedule** trigger: hourly.
2. **Send Request**: saved collection request `GET {{baseUrl}}/payments?since=1h`.
3. **Evaluate (FQL)**: `payments[status="FAILED"].{ "id": id, "amount": amount }`
4. **If**: `$count(input) > 0`
5. **Send Request**: Slack webhook body built with a Template block: `{"text": "⚠️ {{count}} failed payments in the last hour"}`
6. **Deploy** the flow, so it runs in the Postman cloud on schedule.

22.5 Flows as MCP servers (AI agents)

- Switch a flow's trigger to **MCP** and define a **schema of tools** the flow exposes (name, description, input parameters). Agent Mode can generate this schema.
- Once deployed, the flow has a **public URL** that MCP hosts (Claude Desktop, VS Code, Cursor) can add as a tool server.
- Example: a "get_order_status" tool that calls your internal Orders API with stored credentials and returns a clean JSON summary.
- **Security:** you are exposing API capabilities to AI agents. Expose only read-only, least-privilege operations first. Validate inputs. Don't leak secrets in outputs. Log usage.

22.6 Debugging flows

- Click a block to see its **input and output** data from the last run.
- Use **Display/Log** blocks as `console.log` equivalents.
- Test FQL on small sample data first. Remember that a filter with one match returns a **value, not an array**. This is the most common FQL surprise, because a following `$count` or loop then behaves unexpectedly. Wrap it to force an array: `[payments[id=2]]`.
- Failure outputs of request blocks must be wired, or errors silently stop that branch.

22.7 Interview questions on Flows

Q. What are Postman Flows? A visual, low-code tool for connecting blocks (requests, logic, loops, data, AI) to build API workflows. Flows can be deployed with HTTP, schedule or MCP triggers.

Q. What is FQL? Flows Query Language, a JSONata-like expression language for querying and transforming JSON in If, Condition and Evaluate blocks. TypeScript is the alternative for complex logic.

Q. Gotcha with FQL filters? One match returns a single object/value instead of an array, so wrap the expression in `[...]` when you need an array.

➡️ Next: **Chapter 23 — Enterprise Administration, Governance, Integrations & the VS Code Extension**.

Chapter 23 — Enterprise Administration, API Governance, Integrations & the VS Code Extension

> 🧒 In a small house, the parents decide everything by talking at dinner. In a **city**, you need **rules on paper**, **ID cards**, **security cameras** and **building inspectors**. Postman Enterprise features are the city's rules.

> ⚠️ Admin features and role names depend on plan (Team vs Enterprise) and version. Check your team's **Team Settings** and the Learning Center "Administration" section.

PART A — Identity & access

23.1 Single Sign-On (SSO)

- Enterprise teams can use **SAML 2.0 SSO** with most identity providers: Okta, OneLogin, Duo, Microsoft Entra ID (Azure AD), Google Workspace, Ping and others.
- Setup outline: Team Settings → **Authentication** → add SSO → exchange metadata (ACS URL, Entity ID, IdP certificate) with your IdP → test → (optionally) **enforce SSO** so password logins are disabled.
- Keep at least one **break-glass admin** who can sign in if the IdP is misconfigured.

23.2 SCIM provisioning

- **SCIM** lets your IdP **automatically create, update and deactivate** Postman users (and groups) when people join, change teams or leave.
- Benefit: no forgotten accounts after someone leaves (a common audit finding).
- Map IdP groups to Postman **groups**, then grant workspace access to groups instead of individuals.

23.3 Domain capture

- Claim your company email domains, so every Postman account created with `@yourcompany.com` is brought into your organisation's team, instead of employees using personal free accounts for company work (a data-leak risk).
- Works well with SCIM and auto-flex billing (seats added automatically as people join).

23.4 Roles (typical)

Level	Roles (names vary by plan/version)	What they control
Team	Admin, Billing, Developer (member), **Community Manager** (manages everything the team made public), Partner Manager (partner workspaces)	Team settings, security, billing, public presence
Workspace	Admin, Editor, Viewer	Members and content of a workspace
Element	Editor, Viewer (collections, environments, etc.)	Fine-grained control, e.g. view-only PROD environment

Least-privilege recipe:

- Most engineers: **Editor** on their team's workspaces, **Viewer** elsewhere.
- PROD environments: **Viewer** for most, **Editor** for a few owners. Secrets live in Vault/CI, not in the environment.
- Public workspaces: only Community Managers can publish.

23.5 Audit logs

- Track account changes: user management, team management, billing, security settings.
- Available in the UI and through the **Postman API**, so you can feed them into a **SIEM** (Splunk, Sentinel, Datadog…) for alerts such as "workspace made public" or "SSO disabled".

23.6 Other security settings (Team Settings)

- Restrict creation of **public workspaces** / public elements.
- **Secret scanner** alerts for exposed tokens.
- **Session/sign-in** policies, allowed authentication methods, custom domains for published docs, publisher profile.
- **Data residency** (EU region) where required by policy.

PART B — API governance (style guides as code)

23.7 How Postman governance works

- **Rules** check API definitions (OpenAPI/AsyncAPI) for style and security issues: missing descriptions, wrong naming, no auth scheme, inconsistent error formats.
- Violations show **inline in Spec Hub**, and in CI via `postman api lint` / `postman spec lint`.
- **Custom rules** are written in **Spectral** (Postman supports Spectral v6 rule syntax). A rule targets locations with a **JSON Path Plus** expression (`given`) and tests them with a **function** (`then`).

- Rulesets are applied to workspaces through **governance groups**. In v12 they're managed in a **Ruleset Library** (v11 calls this "Custom Rules").
- Gotchas from the docs: Postman accepts **only the Spectral properties it lists**, you can't create a rule that duplicates an existing one, and you must **turn rules on** after you create or import them.

23.8 Example custom rules (tested with Spectral CLI)

```yaml
rules:
  operation-must-have-summary:
    description: Every operation must have a summary.
    message: "{{path}} is missing a summary"
    severity: warn
    given: "$.paths[*][get,post,put,patch,delete]"
    then:
      field: summary
      function: truthy

  paths-kebab-case:
    description: Path segments must be kebab-case.
    severity: error
    given: "$.paths[*]~"
    then:
      function: pattern
      functionOptions:
        match: "^(/[a-z0-9-]+|/\\{[a-zA-Z0-9]+\\})+$"
```

Linting this spec:

```yaml
openapi: 3.0.3
info: { title: Orders API, version: 1.0.0 }
paths:
  /orders/{orderId}:
    get:
      summary: Get an order
      responses: { '200': { description: OK } }
  /orderItems:
    post:
      responses: { '201': { description: Created } }
```

gives (actual output while writing this book, using `npx spectral lint openapi.yaml --ruleset ruleset.yaml`):

```
 8:15    error  paths-kebab-case             Path segments must be kebab-case.
paths./orderItems
 9:10  warning  operation-must-have-summary  #/paths/~1orderItems/post is missing a summary
paths./orderItems.post
✖ 2 problems (1 error, 1 warning, 0 infos, 0 hints)
```

Useful built-in Spectral functions: `truthy`, `falsy`, `defined`, `undefined`, `pattern` (match/notMatch), `casing` (kebab, camel, pascal, snake…), `enumeration`, `length`, `schema`, `alphabetical`, `xor`.

23.9 Governance rollout playbook

1. Start with **warnings only** for a sprint (measure, don't block).
2. Agree on a style guide with API owners: naming, versioning, error format, pagination, auth.
3. Encode it as rules; publish the ruleset; apply it via governance groups.
4. Turn critical rules (security schemes, no secrets in examples) into **errors** that fail CI.
5. Track violation counts per API in the API Catalog over time.

PART C — Integrations

Integration type	Examples	Use
Source control	GitHub, GitLab, Bitbucket, Azure Repos	Spec/collection sync (v10/11 Git integration), Native Git (v12)
CI/CD	GitHub Actions, Jenkins, GitLab CI, Azure Pipelines, CircleCI, Bitbucket Pipelines	Run collections (Newman / Postman CLI)
Notifications	Slack, Microsoft Teams, email, webhooks	Monitor failures, collection changes
Incident/observability	PagerDuty, Opsgenie, Datadog, New Relic, Splunk	Monitor results and alerts, audit logs into SIEM
API gateways	AWS API Gateway, Azure API Management, Apigee	Import/sync API definitions
Secret managers (Vault integrations)	Azure Key Vault, HashiCorp Vault, AWS Secrets Manager, password managers	Reference company secrets from Postman Vault
Issue tracking	Jira (link issues in descriptions/comments)	Traceability

Find them under a workspace or team's **Integrations** area (the location varies by version). Availability depends on plan.

> 🏢 **Best practice:** send monitor alerts to the *owning team's* channel, not a shared #alerts graveyard, and include the environment name and request in the message.

PART D — The Postman VS Code extension

23.10 What it does

- Send HTTP (and other protocol) requests from inside the editor.
- Work with **collections, environments and workspaces** (synced with your Postman account); share and manage roles.
- **Import `.env` files** from your project as environment variables and keep them in sync.

- Run collections (Collection Runner) in versions that include it; check your version's feature list.
- Supported editors: Visual Studio Code, VS Code Insiders, VSCodium, and VS Code-based editors such as Cursor, Windsurf, Kiro and Antigravity.
- Note: the extension's variable model may still show **initial/current values**, even where the desktop app moved to shared/local values.

23.11 When developers prefer it

- Quick checks while coding, without context switching.
- Combined with **Native Git** (collections in the repo) and AI coding tools (via the Postman MCP server), API work stays next to the code.

23.12 Interview questions (admin & governance)

Q. How do you prevent ex-employees from keeping access to Postman data? SSO enforcement plus SCIM deprovisioning from the IdP, domain capture, audit logs, and rotating any shared credentials they knew.

Q. How would you enforce API design standards across 50 teams? Governance rules (Spectral) in a shared ruleset, applied via governance groups. Lint in Spec Hub and in CI (`postman spec lint` / `api lint`) as a blocking check for critical rules, with dashboards for trends.

Q. Someone made a workspace public by mistake. How do you prevent and detect it? Restrict public workspace creation to Community Managers, alert on audit-log events in the SIEM, use the secret scanner, and review publishing in onboarding training.

➡️ Next: **Chapter 24 — Contract Testing with Pact (and how it fits with Postman)**.

Chapter 24 — Contract Testing with Pact (and How It Fits with Postman)

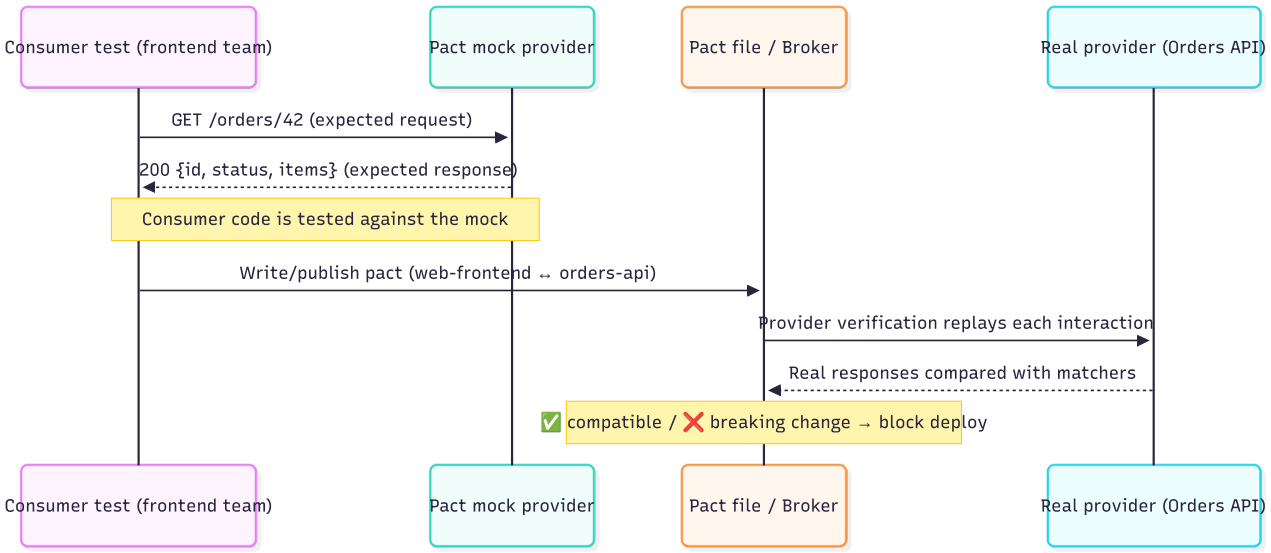
> 🧒 Two kids build a **toy train track** from opposite ends of the room. If they don't agree on *exactly* how the pieces connect, the tracks won't meet in the middle. A **contract** is the drawing of the connecting piece that both kids agree on **before** building.

24.1 Two styles of contract testing

	Provider-driven (spec-first)	Consumer-driven (Pact)
Source of truth	The provider's **OpenAPI spec**	**What each consumer actually uses**, recorded as "pacts"
Tooling	Postman schema tests, Spec Hub validation, governance linting, Prism	**Pact** (pact.io), Pact Broker / PactFlow
Catches	Provider responses that don't match the spec	Provider changes that break a **specific consumer**
Typical owner	API/provider team	Consumer and provider teams together
Postman role	Central	Complementary: Postman for exploration, E2E and spec tests; Pact for fine-grained compatibility

Interview line: "Schema tests in Postman verify the provider honours its published spec. Pact verifies the provider still satisfies what each consumer *actually relies on*, without both being deployed together."

24.2 How Pact works



1. **Consumer test**: the consumer defines the requests it makes and the minimal responses it needs, using **matchers** (types/patterns instead of exact values). It runs against Pact's mock server.
2. A **pact file** (JSON) is generated, and usually published to a **Pact Broker** / PactFlow.
3. **Provider verification**: the provider's CI replays every interaction against the real service, using **provider states** to set up data, and checks the responses.
4. **can-i-deploy**: the Broker tells you whether a version is safe to deploy against the versions currently in each environment.

24.3 Working example (Pact JS, verified)

> ✅ This code was **run successfully while writing this book** with `@pact-foundation/pact` 17.x on Node 22: the consumer test passed, a pact file was written, and the provider verification passed.

Install

```
npm install --save-dev @pact-foundation/pact axios
```

Consumer test — `consumer.pact.test.js`

```
const path = require("path");
const assert = require("assert");
const axios = require("axios");
const { PactV3, MatchersV3 } = require("@pact-foundation/pact");
const { like, integer, eachLike, regex } = MatchersV3;
```

```
const provider = new PactV3({
  consumer: "web-frontend",
  provider: "orders-api",
  dir: path.resolve(__dirname, "pacts"),
});

(async () => {
  provider
    .given("order 42 exists")                                  // provider state
    .uponReceiving("a request for order 42")
    .withRequest({ method: "GET", path: "/orders/42", headers: { Accept: "application/json" } })
    .willRespondWith({
      status: 200,
      headers: { "Content-Type": "application/json" },
      body: {
        id: integer(42),                                       // any integer
        status: regex("PENDING|PAID|SHIPPED", "PAID"),         // must match pattern
        items: eachLike({ sku: like("A1"), qty: integer(1) }) // array of ≥1 such objects
      },
    });

  await provider.executeTest(async (mockServer) => {
    // Call YOUR client code here; axios stands in for it
    const res = await axios.get(`${mockServer.url}/orders/42`, { headers: { Accept:
"application/json" } });
    assert.strictEqual(res.data.id, 42);
    assert.ok(["PENDING", "PAID", "SHIPPED"].includes(res.data.status));
  });
  console.log("Consumer contract OK; pact written to ./pacts");
})().catch((e) => { console.error(e); process.exit(1); });
```

Output: `pacts/web-frontend-orders-api.json`.

Provider verification — `provider.verify.js`

```
const path = require("path");
const express = require("express");
const { Verifier } = require("@pact-foundation/pact");

// Your real provider app (here a tiny Express stand-in)
const app = express();
app.get("/orders/:id", (req, res) =>
  res.json({ id: Number(req.params.id), status: "PAID", items: [{ sku: "B7", qty: 3 }] }));

const server = app.listen(4567, async () => {
  try {
    await new Verifier({
      providerBaseUrl: "http://127.0.0.1:4567",
      pactUrls: [path.resolve(__dirname, "pacts", "web-frontend-orders-api.json")],
      stateHandlers: {
        "order 42 exists": async () => { /* insert order 42 into the test DB */ },
      },
    }).verifyProvider();
    console.log("Provider honours the contract");
  } catch (e) {
    console.error("Contract broken:", e.message);
    process.exitCode = 1;
  } finally {
    server.close();
```

```
  }
});
```

Notice that the provider returned `sku: "B7", qty: 3`, values different from the consumer's examples, and verification still **passed**. **Matchers check shape and type, not exact values.** That makes contracts robust.

Real gotcha found during verification

In a corporate or CI network with an HTTP(S) proxy, the Pact verifier (a native core) may route even `127.0.0.1` calls through the proxy, which then returns **403** ("request blocked"). The symptom is the provider "failing" with a `text/plain` body. Fix: make sure `NO_PROXY` / `no_proxy` includes `localhost,127.0.0.1`, or unset the proxy variables **for that local verification step only**.

24.4 Pact concepts to know for interviews

Concept	Meaning
Consumer / Provider	API client / API server
Interaction	One request/response expectation
Pact file	JSON contract generated by consumer tests
Matchers	`like` (same type), `eachLike` (array of like), `integer`, `decimal`, `boolean`, `regex`, `datetime`, `uuid` ...
Provider states	Named preconditions ("order 42 exists") set up by the provider before replay
Pact Broker / PactFlow	Stores pacts and verification results; versioning, tags/branches, environments
can-i-deploy	CLI check: is this version compatible with what's deployed?
Pending pacts / WIP pacts	Let new consumer expectations not break provider builds immediately
Bi-directional contract testing (PactFlow)	Compare consumer pacts against the provider's **OpenAPI** spec. This is where Postman/OpenAPI-based provider testing and Pact meet

---

24.5 Where Postman fits in a contract-testing strategy

1. **Spec-first design** in Spec Hub, with governance linting.
2. **Postman provider tests**: generated collections plus JSON schema assertions against the spec, run in CI.
3. **Pact consumer tests** in each consumer repo, published to the Broker.
4. **Provider verification** in provider CI, plus `can-i-deploy` before release.
5. **Postman E2E/smoke** after deployment and **monitors** in production.

24.6 Interview questions

Q. Contract testing vs integration testing? Integration tests run real services together (slow, environment-dependent). Contract tests verify each side independently against a shared contract (fast, isolated), so they catch breaking changes earlier.

Q. Why use matchers instead of exact values? Providers return real, varying data. Matchers assert shape and types, so contracts don't break on harmless value changes.

Q. What are provider states? Named preconditions the provider sets up before replaying an interaction, e.g. seeding "order 42 exists".

Q. How does Postman relate to Pact? Postman covers spec-based provider validation, exploration, E2E and monitoring. Pact covers consumer-driven compatibility. Bi-directional contract testing connects them through OpenAPI.

➡️ Next: **Chapter 25 — Extending Newman: Custom Reporters & Programmatic Runs**.

Chapter 25 — Extending Newman: Custom Reporters & Programmatic Runs

> 🧒 Newman is a robot that runs your tests. A **reporter** is the robot's **mouth**. The default mouth talks in the terminal; others write JUnit or HTML. You can build your own mouth that speaks **exactly** the way your team wants: a Slack summary, a Markdown file, a dashboard update.

> ✅ The reporter in this chapter was **built and run successfully with Newman 6.2.3** while writing this book.

25.1 When you need a custom reporter

- A **one-line summary** for Slack/Teams with pass rate and top failures.
- **Markdown** output for PR comments.
- Sending results to an internal **dashboard/DB** (InfluxDB, Elasticsearch, Datadog).
- Company-specific formats (test-management tools like TestRail, Xray, Zephyr: many already have community reporters, so search npm for `newman-reporter-*` first).

25.2 How Newman finds a reporter

- `-r <name>` makes Newman look for an installed package named **`newman-reporter-<name>`** (globally if Newman is global, locally if Newman is local).
- Options on the command line, `--reporter-<name>-<option> value`, arrive in your reporter as `reporterOptions.<option>`.
- The package exports **one function**: `function (emitter, reporterOptions, collectionRunOptions)`.

25.3 A complete reporter (verified)

`newman-reporter-slacksummary/package.json`

```
{ "name": "newman-reporter-slacksummary", "version": "1.0.0", "main": "index.js" }
```

`newman-reporter-slacksummary/index.js`

```
const fs = require("fs");

function SlackSummaryReporter(emitter, reporterOptions, collectionRunOptions) {
  const failures = [];
  const slow = [];
  const threshold = Number(reporterOptions.slowMs || 1000);

  emitter.on("request", (err, args) => {
    if (err || !args.response) return;
    if (args.response.responseTime > threshold) {
      slow.push(`${args.item.name} (${args.response.responseTime} ms)`);
    }
  });

  emitter.on("assertion", (err, args) => {
    if (err) failures.push(`${args.item.name} → ${args.assertion}: ${err.message}`);
  });

  emitter.on("done", (err, summary) => {
    const s = summary.run.stats;
    const line = `${collectionRunOptions.collection.name}: ` +
      `${s.assertions.total - s.assertions.failed}/${s.assertions.total} assertions passed, ` +
      `${s.requests.total} requests, ${failures.length} failures`;
    console.log("[slacksummary] " + line);

    const md = [`# ${line}`, "", "## Failures", ...failures.map(f => `- ${f}`),
                "", `## Slower than ${threshold} ms`, ...slow.map(x => `- ${x}`)].join("\n");
    if (reporterOptions.export) fs.writeFileSync(reporterOptions.export, md);
    // To post to Slack: send `line` to process.env.SLACK_WEBHOOK_URL with https.request (not
shown)
  });
}

module.exports = SlackSummaryReporter;
```

Install and run:

```
npm install ./newman-reporter-slacksummary          # or publish it to your private npm registry
npx newman run collection.json -e env.json \
  -r cli,slacksummary \
  --reporter-slacksummary-export out/summary.md \
  --reporter-slacksummary-slowMs 500
```

Real output from the verification run:

```
[slacksummary] p3: 5/5 assertions passed, 3 requests, 0 failures
```

25.4 Useful emitter events & data

Event	`args` highlights
`start`	run starting

Event	`args` highlights
`beforeIteration` / `iteration`	`args.cursor.iteration`
`beforeItem` / `item`	`args.item.name`
`request`	`args.request`, `args.response` (`code`, `status`, `responseTime`, `stream`), `args.item`
`assertion`	`args.assertion` (test name), `err` set if the test failed, `args.skipped`
`console`	`args.level`, `args.messages` (script `console.log` output)
`exception`	script errors
`done`	`summary.run.stats` (iterations/requests/scripts/assertions: `total`, `failed`), `summary.run.failures`, `summary.run.timings`, `summary.run.executions`

Response body inside a reporter: `args.response.stream.toString()` (it's a Buffer).

25.5 Programmatic Newman patterns (recap and extensions)

```
const newman = require("newman");

function runCollection(collectionPath, envPath) {
  return new Promise((resolve, reject) => {
    newman.run({
      collection: collectionPath,
      environment: envPath,
      reporters: ["cli", "junit"],
      reporter: { junit: { export: `out/${collectionPath.split("/").pop()}.xml` } },
      envVar: [{ key: "clientSecret", value: process.env.CLIENT_SECRET }],
    }, (err, summary) => err ? reject(err) : resolve(summary));
  });
}

(async () => {
  const results = await Promise.all([
    runCollection("collections/users.json", "env/qa.json"),
    runCollection("collections/orders.json", "env/qa.json"),
  ]);
  const failed = results.reduce((n, s) => n + s.run.failures.length, 0);
  console.log(`Total failures: ${failed}`);
  process.exit(failed ? 1 : 0);
})();
```

Ideas:

- **Retry only failed collections** once (to separate flakes from real failures, and report both).
- **Shard** big collections by folder across CI workers (`folder: "Orders"`).

- **Gate on business metrics**, e.g. fail if p95 response time over all requests is above an SLA, computed from `request` events.

25.6 Interview questions

Q. How do you create a custom Newman reporter? Publish a package named `newman-reporter-<name>` exporting `function(emitter, reporterOptions, collectionRunOptions)`. Subscribe to events like `request`, `assertion` and `done`, and use it with `-r <name>`. Options come from `--reporter-<name>-<option>`.

Q. How do you get response bodies in a reporter? `args.response.stream` (a Buffer) in the `request` event.

Q. How would you run 10 collections in parallel and fail the build if any fail? Use the Newman Node API with `Promise.all` over `newman.run` wrappers, sum `summary.run.failures`, and exit 1 if the sum is non-zero. Make sure test data doesn't collide.

➡️ Next: **Chapter 26 — Migration Guide, Certifications & Career Growth**.

Chapter 26 — Migration Guides, Certifications & Career Growth

PART A — Migrating to (and from) Postman

26.1 General migration method (any tool)

1. **Inventory**: list suites, number of requests and tests, data sources, CI jobs, owners.
2. **Export** in the most portable format available: **OpenAPI**, **cURL**, **HAR**, or the tool's native export if Postman imports it.
3. **Import** into a *migration* workspace and fix base URLs, **variables**, **auth** and **environments** first.
4. **Rewrite assertions** in `pm.*`/Chai. Scripts never convert automatically between tools' languages.
5. **Run old and new side by side** for a few cycles and compare pass/fail and coverage.
6. **Switch CI** (Newman/Postman CLI), then **archive** the old suite (read-only) for a retention period.

26.2 SoapUI / ReadyAPI → Postman

SoapUI concept	Postman equivalent
Project	Workspace / collection
TestSuite / TestCase	Folder / sub-folder (one test case = ordered requests)
TestStep (SOAP/REST request)	Request
WSDL import	**Import → WSDL** (generates request per operation)
Assertions (XPath Match, Contains, Script/Groovy)	`pm.test` with `xml2Json` / `cheerio` (XML) or `pm.response.json()`
Property Transfer	Post-response script → `pm.collectionVariables.set`
Properties (project/suite/case)	Collection / environment / local variables
DataSource / DataSource Loop	Runner/Newman data file (CSV/JSON)
Groovy setup/teardown scripts	Collection/folder pre-request & post-response scripts, Setup/Teardown folders
Load tests	Postman performance runs (basic) or k6/JMeter
testrunner.sh in CI	`newman run` / `postman collection run`

XPath assertion example → Postman:

```
// SoapUI XPath:  //AddResult = 12
const $ = cheerio.load(pm.response.text(), { xmlMode: true });
pm.test("AddResult is 12", () => pm.expect($("AddResult").text()).to.eql("12"));
```

26.3 Insomnia → Postman

- Export from Insomnia (collection/workspace export) and use Postman **Import**. Recent Postman versions accept Insomnia exports; if yours doesn't, export **OpenAPI** or **HAR** instead.
- Insomnia environments (JSON, nested) → Postman environments (flat key/value). Flatten nested values.
- Insomnia template tags/plugins → Postman dynamic variables or pre-request scripts.

26.4 Thunder Client / Hoppscotch / Bruno → Postman

- Most can **export Postman-format collections** or OpenAPI. Import, then fix variables and auth.
- Bruno stores requests as `.bru` text files. Exporting to Postman collection format (from Bruno) or OpenAPI is the usual route.

26.5 JMeter → Postman (functional parts only)

- There's no direct import. Recreate the HTTP samplers as requests (cURL/HAR help).
- Map assertions (Response Assertion, JSON Assertion) → `pm.test`; CSV Data Set Config → data files; extractors (JSON/Regex) → post-response scripts.
- **Keep load testing in JMeter/k6.** Postman's performance runner is for lighter checks.

26.6 Postman → code frameworks (REST Assured, Playwright, pytest)

Teams move when they need complex logic, shared code with the app, or very large suites.

Postman	REST Assured (Java)	Playwright API (TS)	pytest + requests (Python)
`pm.response.to.have.status(200)`	`.then().statusCode(200)`	`expect(res.status()).toBe(200)`	`assert r.status_code == 200`
`pm.response.json().id`	`.extract().path("id")`	`(await res.json()).id`	`r.json()["id"]`
JSON schema test	`matchesJsonSchemaInClasspath("s.json")`	ajv in test	`jsonschema.validate(...)`
Environment	properties/profiles	`.env` + config	fixtures/env vars
Data file	TestNG DataProvider	`test.describe` loops	`@pytest.mark.parametrize`

Tip: keep the **Postman collection as living documentation and for exploratory work** even after moving automation to code.

26.7 Postman → Postman v12 Native Git

See Chapter 21.8 (playbook).

PART B — Certifications & learning paths

26.8 Official Postman credentials

- **Postman API Fundamentals Student Expert**: Postman's best-known badge, earned through the Postman Student program / Postman Academy by completing a hands-on training collection with practical tasks that's submitted for review. You get a digital badge/certificate to show on LinkedIn and your CV. (Despite the "Student" name, many working professionals complete it as an introduction.)
- **Postman Academy** also offers free courses and learning paths with completion badges (e.g. on API testing, Flows, Native Git and newer features). The catalogue changes, so check **academy.postman.com** for current courses and badges.

> ⚠️ Many "Postman certification" courses online are **third-party** (Udemy, training institutes). They can be useful for learning, but they aren't official Postman credentials. Be precise about this on your CV.

26.9 Complementary certifications employers value (API/QA roles)

- **ISTQB** Foundation, plus **Test Automation Engineer** / Advanced levels (testing fundamentals).
- **Cloud** basics (AWS/Azure/GCP associate level) for understanding gateways, IAM and serverless APIs.
- **Security**: OWASP knowledge; API security courses (e.g. APIsec University's free courses).
- **Programming**: JavaScript fundamentals (Postman scripting), plus Java/Python if moving to code frameworks.

PART C — Career growth with Postman skills

26.10 Skills matrix

Level	You can…
Beginner	Send requests, use params/headers/body, read status codes, save collections, basic tests
Intermediate	Environments and variables strategy, auth (OAuth 2.0), chaining, data-driven runs, schema tests, Newman in CI
Advanced	Framework design, Package Library, token caching, workflows (`setNextRequest`), mocks/monitors, security suites, troubleshooting anything

Level	You can…
Expert/Lead	Governance, Native Git adoption, contract-testing strategy (Spec + Pact), CI quality gates, coverage metrics, team standards, mentoring

26.11 Build a portfolio (gets interviews)

- A **public Postman workspace** (no secrets!) with a well-documented collection against public APIs (the Chapter 20 capstone): environments, tests, data-driven folder, visualizer, mock.
- A **GitHub repo** with the exported collection (or Native Git YAML), a GitHub Actions workflow running Newman/Postman CLI, HTML report artifacts, and a README.
- A short **write-up** (blog/LinkedIn): "How I built an API regression suite with Postman and CI."

26.12 Strong CV bullet examples

- "Built Postman regression suites (420 tests across 6 microservices) run by Newman in Jenkins; cut regression time from 2 days of manual testing to 18 minutes."
- "Introduced OAuth 2.0 token caching and Package Library helpers, reducing duplicated script code by ~70%."
- "Designed BOLA/IDOR security test matrix in Postman; found 3 high-severity authorization defects before release."
- "Migrated 900 SoapUI test steps to Postman/Newman with side-by-side parity runs over 2 sprints."
- "Led adoption of Spec Hub governance rules and contract tests; breaking API changes caught in PRs instead of production."

(Use **your real numbers**. Interviewers ask follow-up questions on every bullet.)

26.13 Roles that use these skills

QA Engineer, SDET / Automation Engineer, API Test Lead, Backend Developer, Integration Engineer, Solutions/Support Engineer (debugging customer API issues), Developer Advocate, API Product Manager, DevOps/SRE (monitors, health checks), Security Tester (API).

➡️ Back to the **README** for the full table of contents.

Chapter 27 — Verification Report: What Was Tested, and What Changed Because of It

> This chapter documents how the book's code was checked, so you know what is **verified by execution** and what is **documented but not executed**.

27.1 Environment

Item	Version
Node.js	22.22
Newman	6.2.3 (+ newman-reporter-htmlextra 1.23.1)
Pact JS	@pact-foundation/pact 17.1.4 (pact-core 20.2)
Spectral CLI	6.17.0
JSONata (to check FQL syntax)	2.x
Date	October 2026

Public practice APIs (JSONPlaceholder, httpbin, Restful-Booker) were **not reachable** from the build environment. So a **local stand-in API** (`verification/server.js`) reproduces the endpoints the book uses: users/posts, httpbin-style echo, status codes, cookies, HTML/XML, OAuth client-credentials token, pagination, async job polling, a flaky 503 endpoint, and Restful-Booker-style booking CRUD with cookie auth.

27.2 What was executed

Area	Result
Newman suite (`verification/build.js`): 57 requests, 100 test scripts, 71 pre-request scripts	**180 / 180 assertions passed**. 1 intentional request error (G9, unresolved `{{baseUrl}}` in `pm.sendRequest`)
Response assertions (`status` , `ok` , `success` , `accepted` , `notFound` , `rateLimited` , `header` , `jsonBody` , `jsonSchema` , `json` , `withBody`)	✅
Chai toolkit (`eql` , `deep.include` , `nested.property` , `all.keys` , `members` , `oneOf` , `closeTo` , `lengthOf.at.most` , `satisfy` , …)	✅

Area	Result
Sandbox libraries (lodash, moment, uuid, ajv, cheerio, xml2js, csv-parse, crypto-js, btoa/atob, tv4, buffer, url)	✅
Variable scopes & precedence, `replaceIn`, ~50 dynamic variables, data-file precedence	✅
Request modification (headers add/upsert/remove, query ops, body rewrite, method/URL change)	✅ (after fixes below)
HMAC signing, JWT decoding	✅
OAuth client-credentials token caching in a pre-request script with `pm.sendRequest`	✅ (fetched once, reused on next request)
Workflows: loop via `setNextRequest(requestId)`, pagination, polling, retry with backoff, `skipRequest`, `setNextRequest(null)`	✅
Cookies (`pm.cookies`, jar, Set-Cookie flags)	✅ (after fixes below)
Legacy syntax (`tests[]`, `responseBody`, `postman.setEnvironmentVariable`)	✅ works with deprecation warnings
htmlextra & JUnit reporters	✅
Custom Newman reporter (Ch 25) and Node API event fields	✅
Pact consumer test + provider verification (Ch 24)	✅
Spectral custom governance rules (Ch 23)	✅ real lint output included in the chapter
FQL expressions (Ch 22), evaluated with JSONata	✅
Newman CLI flags in Ch 11 checked against `newman run --help` and the reporters' READMEs	✅ (after fixes below)

27.3 Documented but NOT executed (needs your environment)

- Postman **desktop app**-only behaviour: UI labels, Vault UI, Package Library, Agent Mode/Postbot, Performance tab, mocks/monitors (cloud), Spec Hub UI.
- **Postman CLI** commands (requires a Postman account/API key): `login`, `collection run <id>`, `api lint`, `spec lint`, `workspace prepare/push`.
- CI YAML files (GitHub Actions, Jenkins, GitLab, Azure DevOps). They're syntactically standard but weren't run in those systems.
- gRPC, WebSocket, Socket.IO, MQTT and SOAP via Postman's dedicated request types (the XML/SOAP *assertions* were verified).
- Behaviour against the real public practice APIs (endpoints and quirks are described from their documentation, e.g. Restful-Booker returning 201 on DELETE).

27.4 Corrections made to the book because of verification

#	Original statement	What testing showed	Fixed in
1	"CSV values are strings"	Newman converts number-like values to numbers (`007` → `7` , `1e3` → `1000`); `true` / `false` stay strings	Ch 5, 8, 10, 18B, 19
2	`await pm.sendRequest` presented as generally available	Works only in recent Postman app versions. Newman 6 gives a SyntaxError for top-level `await` and `sendRequest` returns no promise. Portable Promise wrapper added (verified)	Ch 6, 7, 8, 18B, 19
3	`const _ = require("lodash")` in examples	Throws `Identifier '_' has already been declared` (`_` is a global)	Ch 7, 8
4	`pm.response.cookies.one(...)` for cookie-flag tests	Empty in Newman 6. Replaced with `Set-Cookie` header parsing + `pm.cookies` (verified)	Ch 7, 8
5	Pre-request `throw` as a guard; runner behaviour "may differ"	Newman **still sends the request** after a pre-request error. Recommend `skipRequest()`	Ch 5
6	`pm.request.body.update()` shown without caveat	Throws when the request has no saved body. Assigning a body object works (verified)	Ch 7
7	`headers.add` vs `upsert` not distinguished	`add` appends a duplicate header (`"1, 2"`)	Ch 7
8	Retry pattern note	Collection-level generic tests still fail on retried attempts. Added a mitigation	Ch 9
9	Error text "skipRequest isn't a function"	Exact text: `pm.execution.skipRequest is not a function`	Ch 7
10	htmlextra `--reporter-htmlextra-darkTheme`	Not in the reporter's documented options. Removed; real options listed	Ch 11
11	API key only via `?apikey=` URL	Newman 6 has `--postman-api-key` (safer). Added	Ch 11

27.5 Re-run it yourself

```
cd postman-book/verification
npm install
npm run server &          # local stand-in API on http://127.0.0.1:3555
npm test                  # Newman suite: expect 180/180 assertions, 1 intentional ENOTFOUND
request error
npm run fql               # FQL expressions via JSONata
npm run pact              # Pact consumer + provider (set NO_PROXY=127.0.0.1,localhost behind a
```

```
proxy)
npm run lint-spec         # Spectral governance rules: expect 1 error + 1 warning
```

To verify the **app-only** features, import `verify.postman_collection.json` (created by `npm run build`) into the Postman desktop app, select the generated environment, and run it with the Collection Runner. Compare results with Newman: differences are themselves useful knowledge (see 18D.4).